

**Draft Standard for
Local and metropolitan area networks —**

**Frame Replication and Elimination for
Reliability**

Prepared by the Maintenance Task Group of IEEE 802.1

**LAN MAN Standards Committee
of the
IEEE Computer Society**

Copyright © 2025 by the IEEE.

Three Park Avenue

New York, New York 10016-5997, USA

All rights reserved.

Important Notice

This document is an unapproved draft of a proposed IEEE Standard. IEEE hereby grants the named IEEE SA Working Group or Standards Committee Chair permission to distribute this document to participants in the receiving IEEE SA Working Group or Standards Committee, for purposes of review for IEEE standardization activities. No further use, reproduction, or distribution of this document is permitted without the express written permission of IEEE Standards Association (IEEE SA). Prior to any review or use of this draft standard, in part or in whole, by another standards development organization, permission must first be obtained from IEEE SA (stds-copyright@ieee.org). This page is included as the cover of this draft, and shall not be modified or deleted.

IEEE Standards Association
445 Hoes Lane
Piscataway, NJ 08854, USA

1 **Abstract:** This corrigendum to IEEE Std 802.1CB™-2017 provides technical and editorial
2 corrections.

3 **Keywords:** TSN, Time-Sensitive Networking, Redundancy, Bridging, Bridges, Frame Replication,
4 Frame Elimination, Stream identification, Extended Stream identification, Bridged Local Area
5 Networks, IEEE 802®, IEEE 802.1Q™, IEEE 802.1CB™, IEEE 802.1CBcv™, IEEE 802.1CBdb™,
6 local area networks (LANs), MAC Bridges, Virtual Bridged Local Area Networks (virtual LANs),
7 amendment.

1 Important Notices and Disclaimers Concerning IEEE Standards Documents

2 IEEE Standards documents are made available for use subject to important notices and legal disclaimers.
3 These notices and disclaimers, or a reference to this page (<https://standards.ieee.org/ipr/disclaimers.html>),
4 appear in all IEEE standards and may be found under the heading “Important Notices and Disclaimers Con-
5 cerning IEEE Standards Documents.”

6 7 Notice and Disclaimer of Liability Concerning the Use of IEEE Standards 8 Documents

9 IEEE Standards documents are developed within IEEE Societies and subcommittees of IEEE Standards
10 Association (IEEE SA) Board of Governors. IEEE develops its standards through an accredited consensus
11 development process, which brings together volunteers representing varied viewpoints and interests to
12 achieve the final product. IEEE Standards are documents developed by volunteers with scientific, academic,
13 and industry-based expertise in technical working groups. Volunteers involved in technical working groups
14 are not necessarily members of IEEE or IEEE SA and participate without compensation from IEEE. While
15 IEEE administers the process and establishes rules to promote fairness in the consensus development
16 process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the
17 soundness of any judgments contained in its standards.

18 IEEE makes no warranties or representations concerning its standards, and expressly disclaims all
19 warranties, express or implied, concerning all standards, including but not limited to the warranties of
20 merchantability, fitness for a particular purpose and non-infringement. IEEE Standards documents do not
21 guarantee safety, security, health, or environmental protection, or compliance with law, or guarantee against
22 interference with or from other devices or networks. In addition, IEEE does not warrant or represent that the
23 use of the material contained in its standards is free from patent infringement. IEEE standards documents are
24 supplied “AS IS” and “WITH ALL FAULTS.”

25 Use of an IEEE standard is wholly voluntary. The existence of an IEEE standard does not imply that there
26 are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to
27 the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and
28 issued is subject to change brought about through developments in the state of the art and comments
29 received from users of the standard.

30 In publishing and making its standards available, IEEE is not suggesting or rendering professional or other
31 services for, or on behalf of, any person or entity, nor is IEEE undertaking to perform any duty owed by any
32 other person or entity to another. Any person utilizing any IEEE Standards document should rely upon their
33 own independent judgment in the exercise of reasonable care in any given circumstances or, as appropriate,
34 seek the advice of a competent professional in determining the appropriateness of a given IEEE standard.

35 IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
36 EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO: THE
37 NEED TO PROCURE SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS;
38 OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
39 WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
40 OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE
41 UPON ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND
42 REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

43

44 Translations

45 The IEEE consensus balloting process involves the review of documents in English only. In the event that an
46 IEEE standard is translated, only the English language version published by IEEE is the approved IEEE
47 standard.

1 Use by artificial intelligence systems

2 In no event shall material in any IEEE Standards documents be used for the purpose of creating, training, en-
3 hancing, developing, maintaining, or contributing to any artificial intelligence systems without the express,
4 written consent of the IEEE SA in advance. “Artificial intelligence” refers to any software, application, or
5 other system that uses artificial intelligence, machine learning, or similar technologies, to analyze, train, pro-
6 cess, or generate content. Requests for consent can be submitted using the [Contact Us](#) form.¹

7 Official statements

8 A statement, written or oral, that is not processed in accordance with the IEEE SA Standards Board Operations
9 Manual is not, and shall not be considered or inferred to be, the official position of IEEE, IEEE SA, or any of
10 its committees, and is not, and shall not be considered to be or be relied upon as, a formal position of IEEE
11 or IEEE SA. At lectures, symposia, seminars, or educational courses, an individual presenting information on
12 IEEE standards shall make it clear that the presenter's views should be considered the personal views of that
13 individual rather than the formal position of IEEE, IEEE SA, the Standards Committee, or the Working
14 Group. Statements made by volunteers may not represent the formal position of their employer(s) or
15 affiliation(s). News releases about IEEE standards issued by entities other than IEEE SA should be
16 considered the view of the entity issuing the release rather than the formal position of IEEE or IEEE SA.

17

18 Comments on standards

19 Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of
20 membership affiliation with IEEE or IEEE SA. However, **IEEE does not provide interpretations,**
21 **consulting information, or advice pertaining to IEEE Standards documents.**

22 Suggestions for changes in documents should be in the form of a proposed change of text, together with
23 appropriate supporting comments. Since IEEE standards represent a consensus of concerned interests, it is
24 important that any responses to comments and questions also receive the concurrence of a balance of
25 interests. For this reason, IEEE and the members of its Societies and subcommittees of the IEEE SA Board
26 of Governors are not able to provide an instant response to comments or questions, except in those cases
27 where the matter has previously been addressed. For the same reason, IEEE does not respond to
28 interpretation requests. Any person who would like to participate in evaluating comments or revisions to an
29 IEEE standard is welcome to join the relevant IEEE SA working group. You can indicate interest in a
30 working group using the Interests tab in the Manage Profile & Interests area of the [IEEE SA myProject](#)
31 [system](#).²

32 Comments on standards should be submitted using the [Contact Us](#) form.

33

34 Laws and regulations

35 Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the
36 provisions of any IEEE Standards document does not constitute compliance to any applicable regulatory
37 requirements. Implementers of the standard are responsible for observing or referring to the applicable regu-
38 latory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not in
39 compliance with applicable laws, and these documents may not be construed as doing so.

40

41

42

¹ Available at: <https://standards.ieee.org/content/ieee-standards/en/about/contact/index.html>.

² Available at: <https://development.standards.ieee.org/myproject-web/public/view.html#landing>.

1 Data privacy

2 Users of IEEE Standards documents should evaluate the standards for considerations of data privacy and
3 data ownership in the context of assessing and using the standards in compliance with applicable laws and
4 regulations.

5

6

7 Copyrights

8 IEEE draft and approved standards are copyrighted by IEEE under U.S. and international copyright laws.
9 They are made available by IEEE and are adopted for a wide variety of both public and private uses. These
10 include both use by reference, in laws and regulations, and use in private self-regulation, standardization,
11 and the promotion of engineering practices and methods. By making these documents available for use and
12 adoption by public authorities and private users, neither IEEE nor its licensors waive any rights in copyright
13 to the documents.

14

15 Photocopies

16 Subject to payment of the appropriate licensing fees, IEEE will grant users a limited, non-exclusive license
17 to photocopy portions of any individual standard for company or organizational internal use or individual,
18 non-commercial use only. To arrange for payment of licensing fees, please contact Copyright Clearance
19 Center, Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400; [https://](https://www.copyright.com/)
20 www.copyright.com/. Permission to photocopy portions of any individual standard for educational
21 classroom use can also be obtained through the Copyright Clearance Center.

22

23 Updating of IEEE Standards documents

24

25 Users of IEEE Standards documents should be aware that these documents may be superseded at any time
26 by the issuance of new editions or may be amended from time to time through the issuance of amendments,
27 corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the
28 document together with any amendments, corrigenda, or errata then in effect.

29 Every IEEE standard is subjected to review at least every 10 years. When a document is more than 10 years
30 old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of
31 some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that
32 they have the latest edition of any IEEE standard.

33 In order to determine whether a given document is the current edition and whether it has been amended
34 through the issuance of amendments, corrigenda, or errata, visit [IEEE Xplore](#) or [contact IEEE](#).³ For more
35 information about the IEEE SA or IEEE's standards development process, visit the IEEE SA Website.

36

37 Errata

38 Errata, if any, for all IEEE standards can be accessed on the [IEEE SA Website](#).⁴ Search for standard number
39 and year of approval to access the web page of the published standard. Errata links are located under the
40 Additional Resources Details section. Errata are also available in [IEEE Xplore](#). Users are encouraged to
41 periodically check for errata.

42

³ Available at: <https://ieeexplore.ieee.org/browse/standards/collection/ieee>.

⁴ Available at: <https://standards.ieee.org/standard/index.html>.

1 Patents

2

3 IEEE standards are developed in compliance with the [IEEE SA Patent Policy](#).⁵

4 Attention is called to the possibility that implementation of this standard may require use of subject matter
5 covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the
6 existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has
7 filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the IEEE
8 SA Website at <https://standards.ieee.org/about/sasb/patcom/patents.html>. Letters of Assurance may indicate
9 whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or
10 under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair dis-
11 crimination to applicants desiring to obtain such licenses.

12 Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not
13 responsible for identifying Essential Patent Claims for which a license may be required, for conducting
14 inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or
15 conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing
16 agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that determi-
17 nation of the validity of any patent rights, and the risk of infringement of such rights, is entirely their own
18 responsibility. Further information may be obtained from the IEEE Standards Association.

19 IMPORTANT NOTICE

20 Technologies, application of technologies, and recommended procedures in various industries evolve over
21 time. The IEEE standards development process allows participants to review developments in industries,
22 technologies, and practices, and to determine what, if any, updates should be made to the IEEE standard.
23 During this evolution, the technologies and recommendations in IEEE standards may be implemented in
24 ways not foreseen during the standard's development. IEEE standards development activities consider
25 research and information presented to the standards development group in developing any safety recommen-
26 dations. Other information about safety practices, changes in technology or technology implementation, or
27 impact by peripheral systems also may be pertinent to safety considerations during implementation of the
28 standard. Implementers and users of IEEE Standards documents are responsible for determining and com-
29 plying with all appropriate safety, security, environmental, health, data privacy, and interference protection
30 practices and all applicable laws and regulations.

31

32

⁵ Available at: <https://standards.ieee.org/about/sasb/patcom/materials.html>.

1 **Participants**

2 The following is a list of participants in the IEEE 802.1 Working Group.

- 3 **Glenn Parsons, *Chair***
4 **Jessy Rouyer, *Vice-Chair***
5 **Mark Hantel, *Chair, Maintenance Task Group***
6 **Christophe Mangin, *Editor***
7

8 The following members of the individual balloting committee voted on this standard. Balloters may have
9 voted for approval, disapproval, or abstention.

10 When the IEEE-SA Standards Board approved this standard on XX Month 20XX, it had the following
11 membership:

- 12 **Name, *Chair***
13 **Name, *Vice Chair***
14 **Name, *Past Chair***

1 **Name, *Secretary***

SBMember1	SBMember9	SBMember17
SBMember2	SBMember10	SBMember18
SBMember3	SBMember11	SBMember19
SBMember4	SBMember12	SBMember20
SBMember5	SBMember13	SBMember21
SBMember6	SBMember14	SBMember22
SBMember7	SBMember15	SBMember23
SBMember8	SBMember16	

2 *Member Emeritus
3

4 Also included are the following nonvoting IEEE-SA Standards Board liaisons:

- 5
6 Name, *DOE Representative*
7 Name, *NIST Representative*
8
9
10 Name
11 IEEE Standards Program Manager, Document Development
12
13
14 Name
15 IEEE Standards Program Manager, Technical Program Development

1 Introduction

2

This introduction is not part of IEEE P802.1CB-2017-Rev-20XX, IEEE Draft Standard for Local and metropolitan area networks – Frame Replication and Elimination for Reliability

3 The first edition of IEEE Std 802.1CB™ was published in 2017. A first amendment IEEE Std 802.1CBv™-
4 2021 added the YANG data model that allows configuration and state reporting for all managed objects of
5 the base standard. A second amendment IEEE Std 802.1CBdb™-2021 added the mask-and-match Stream
6 identification function and provided technical and editorial corrections. A corrigendum IEEE, 802.1CB-
7 2017/Cor1-2025, implemented the changes necessitated by various maintenance items that have been noti-
8 fied to the IEEE 802.1 Working Group.

9 This revision incorporates all the changes introduced by the amendements, the corrigendum, the implemen-
10 tation of maintenance item #343, as well as technical and editorial corrections to existing functionality.

1 Editor's Foreword

2 <<Notes>>

3 <<Throughout this document, all notes such as this one, presented between angle braces, are temporary
4 notes inserted by the Editors for a variety of purposes; these notes and the Editors' Foreword will all be
5 removed prior to publication and are not part of the normative text.>>

6 <<Comments and participation in 802.1 standards development

7 Comments on this draft are encouraged. **PLEASE NOTE: All issues related to IEEE standards
8 presentation style, formatting, spelling, etc. are routinely handled between the 802.1 Editor and the
9 IEEE Staff Editors prior to publication, after balloting and the process of achieving agreement on the
10 technical content of the standard is complete.** Readers are urged to devote their valuable time and energy
11 only to comments that materially affect either the technical content of the document or the clarity of that
12 technical content. Comments should not simply state what is wrong, but also what might be done to fix the
13 problem.

14 Full participation in the development of this draft requires individual attendance at IEEE 802 meetings.
15 Information on 802.1 activities, working papers, and email distribution lists etc. can be found on the 802.1
16 website:

17 <https://1.ieee802.org/>

18 Use of the email distribution list is not presently restricted to 802.1 members, and the working group has had
19 a policy of considering ballot comments from all who are interested and willing to contribute to the
20 development of the draft. Individuals not attending meetings have helped to identify sources of
21 misunderstanding and ambiguity in past projects. Non-members are advised that the email lists exist primarily
22 to allow the members of the working group to develop standards, and are not a general forum.

23 All participants in IEEE standards development have responsibilities under the IEEE patent policy and should
24 familiarize themselves with that policy. See <http://standards.ieee.org/about/sasb/patcom/materials.html>.

25 Comments on this document may be sent to the 802.1 email exploder, to the editors, or to the Chairs of the
26 802.1 Working Group and Time-Sensitive Networking Task Group.

27 Christophe Mangin
28 Editor,
29 Email: c.mangin@fr.mercede.mee.com

30 Mark Hantel
31 Chair, 802.1 Maintenance Task Group
32 Email: mrhantel@ra.rockwell.com

Glenn Parsons
Chair, 802.1 Working Group
Email: glenn.parsons@ericsson.com

33 **PLEASE NOTE: Comments whose distribution is restricted in any way cannot be considered, and
34 may not be acknowledged. >>**

35 <<The draft text and accompanying information

36 This document currently comprises:

- 37 — A title page for the proposed standard including an Abstract and Keywords. This title page will be retained following working group approval of this draft, i.e. prior to sponsor ballot.

- 1 — The editors' forewords, including this text. These include an unofficial and informal appraisal of history and status, introductory notes to each draft that summarize the progress and focus of each successive draft, and requests for comments and contributions on major issues.
- 4 — IEEE boilerplate text.
- 5 — A record of participants (not included in early drafts but added prior to publication).
- 6 — The introduction to this standard.
- 7 — The proposed standard proper.

8 During the early stages of draft development, 802.1 editors have a responsibility to attempt to craft technically
9 coherent drafts from the resolutions of ballot comments and the other discussions that take place in the
10 working group meetings. Preparation of drafts often exposes inconsistencies in editors instructions or
11 exposes the need to make choices between approaches that were not fully apparent in the meeting. Choices
12 and requests by the editors' for contributions on specific issues will be found in the editors' introductory notes
13 to the current draft, at appropriate points in the draft, and in Annex Z. Significant discussion of more difficult
14 topics will be found in the last of these.

15 The ballot comments received on each draft, and the editors' proposed and final disposition of comments, are
16 part of the audit trail of the development of the standard and are available, along with all the revisions of the
17 draft on the 802.1 web site (for address see above).

18 >>

19 << Introductory notes to draft 0.2

20 Draft 0.1 was prepared for a second Task Group ballot. It incorporates the resolutions of the comments made
21 against Draft 0.1, resolved during the September interim meeting.

- 22 — Text color codes remain the same as for D0.1.
- 23 — Neither MIB nor YANG source file is provided. The changes made to the original 802.1CBcv YANG
24 modules and MIB are reflected in clauses 11 and 12 through the same color codes as for D0.1 (pine
25 green and violet). Clauses 11 and 12, rolled up from 802.1CBcv, are not magenta highlighted except
26 their clause header.
- 27 — So far, the changes made in the YANG modules have not been reflected in the MIB.
- 28 — Changes due to the resolution of comments against D0.1 on managed object definitions (clause 9)
29 and their associated YANG models (clause 12) are signaled using addition/~~deletion~~ revision marks.
- 30 — Changes made to the figures and text of clauses 6 and 8, as well as annex C (incorporated from
31 802.1CB-2017/Cor1 and maintenance item #343's implementation) are highlighted using addition/
32 ~~deletion~~ revision marks.

33 >>

34 << Introductory notes to draft 0.1

35 Draft 0.1 was prepared for a first Task Group ballot. The following color codes are used in this draft to ease
36 the visualization of changes made to IEEE 802.1CB-2017:

- 37 — all changes incorporated from IEEE 802.1CBcv™-2021 are magenta highlighted
- 38 — all changes incorporated from IEEE 802.1CBdb™-2021 are violet highlighted;
- 39 — all changes incorporated from IEEE 802.1CB™-2017/Cor1 are pine green highlighted.
- 40 — changes resulting from the implementation of maintenance item #343 are signaled using addition/
41 ~~deletion~~ revision marks

42

43 >>

1 << Project Authorization Request, Scope, Purpose, and Five Criteria

2 A PAR (Project Authorization Request) for P802.1CB-2017-Rev was approved by the IEEE Standards
3 Association on February 13, 2024. The following information is taken from the 802.1CB-2017-Rev PAR.

4 Need for the Project:

5 This revision project is needed to incorporate approved amendments and corrigenda, to incorporate technical
6 and editorial corrections to existing functionality, and to maintain consistency in the consolidated text.

7 Purpose of Proposed Project:

8 This document will not include a purpose clause.

Contents

2	1.	Overview	23
3	1.1	Scope	23
4	1.2	Rationale	23
5	1.3	State diagram conventions	23
6	1.4	Specification model	23
7	1.5	Specification precedence	24
8	1.6	Introduction	24
9	2.	Normative references	24
10	3.	Definitions	26
11	4.	Acronyms and abbreviations	28
12	5.	Conformance	30
13	5.1	Requirements terminology	30
14	5.2	Conformant components and equipment	30
15	5.3	Stream identification component required behaviors	30
16	5.4	Stream identification component recommended behavior	31
17	5.5	Stream identification component optional behaviors	31
18	5.6	Talker end system required behaviors	31
19	5.7	Talker end system recommended behaviors	31
20	5.8	Talker end system optional behaviors	31
21	5.9	Listener end system required behaviors	32
22	5.10	Listener end system recommended behavior	32
23	5.11	Listener end system optional behaviors	32
24	5.12	Relay system required behaviors	33
25	5.13	Relay system recommended behaviors	33
26	5.14	Relay system optional behaviors	33
27	5.15	FRER C-component required and optional behaviors	34
28	6.	Stream identification	35
29	6.1	Stream service subparameters	36
30	6.2	Stream identification function	37
31	6.3	Stream identification in systems	38
32	6.4	Null Stream identification	40
33	6.5	Source MAC and VLAN Stream identification	41
34	6.6	Active Destination MAC and VLAN Stream identification	41
35	6.7	IP Stream identification	42
36	6.8	Mask-and-match Stream identification	42
37	7.	Frame Replication and Elimination for Reliability	44
38	7.1	Overview of Frame Replication and Elimination for Reliability	44
39	7.1.1	Goals and objectives	44
40	7.2	Use of the term Stream	46
41	7.3	Frame Replication and Elimination for Reliability functions	46
42	7.4	Sequencing function	47
43	7.4.1	Sequence generation function	47
44	7.4.1.1	Events for sequence generation	48

1	7.4.1.2	Variables for sequence generation	48
2	7.4.1.2.1	GenSeqSpace	48
3	7.4.1.2.2	GenSeqNum	48
4	7.4.1.3	SequenceGenerationReset	48
5	7.4.1.4	SequenceGenerationAlgorithm	49
6	7.4.2	Sequence recovery function	49
7	7.4.3	Base recovery function	49
8	7.4.3.1	Events for sequence recovery	50
9	7.4.3.2	Variables for sequence recovery	50
10	7.4.3.2.1	RecovSeqSpace	50
11	7.4.3.2.2	SequenceHistory	51
12	7.4.3.2.3	RecovSeqNum	51
13	7.4.3.2.4	RemainingTicks	51
14	7.4.3.2.5	TicksPerSecond	51
15	7.4.3.2.6	TakeAny	51
16	7.4.3.3	SequenceRecoveryReset	51
17	7.4.3.4	VectorRecoveryAlgorithm	52
18	7.4.3.5	MatchRecoveryAlgorithm	54
19	7.4.3.6	ShiftSequenceHistory	55
20	7.4.4	Latent error detection function	56
21	7.4.4.1	Events for latent error detection	56
22	7.4.4.2	Variables for latent error detection	57
23	7.4.4.2.1	CurBaseDifference	57
24	7.4.4.3	LatentErrorReset	57
25	7.4.4.4	LatentErrorTest	57
26	7.5	Individual recovery function	58
27	7.6	Sequence encode/decode function	58
28	7.7	Stream splitting function	59
29	7.8	Redundancy tag	59
30	7.8.1	Redundancy tag EtherType	60
31	7.8.2	Redundancy tag information	60
32	7.9	HSR sequence tag	60
33	7.10	PRP sequence trailer	61
34	7.11	Autoconfiguration	62
35	7.11.1	Introduction to autoconfiguration	62
36	7.11.2	Creating autoconfigured Stream identity table entries	63
37	8.	Frame Replication and Elimination for Reliability in Bridges	67
38	8.1	Limiting options	67
39	8.2	FRER C-component input transformations	70
40	8.3	Frame Replication and Elimination for Reliability and VLAN tags	71
41	8.4	Configuring Frame Replication and Elimination for Reliability in Bridges	71
42	9.	Stream Identification Management	73
43	9.1	Stream identity table	73
44	9.1.1	tsnStreamIdEntry	73
45	9.1.1.1	tsnStreamIdHandle	73
46	9.1.1.2	tsnStreamIdInFacOutputPortList	73
47	9.1.1.3	tsnStreamIdOutFacOutputPortList	73
48	9.1.1.4	tsnStreamIdInFacInputPortList	74
49	9.1.1.5	tsnStreamIdOutFacInputPortList	74
50	9.1.1.6	tsnStreamIdIdentificationType	74
51	9.1.1.7	tsnStreamIdParameters	74

1	9.1.2	Managed objects for Null Stream identification	74
2	9.1.2.1	tsnCpeNullDownDestMac	75
3	9.1.2.2	tsnCpeNullDownTagged	75
4	9.1.2.3	tsnCpeNullDownVlan	75
5	9.1.3	Managed objects for Source MAC and VLAN Stream identification	75
6	9.1.3.1	tsnCpeSmacVlanDownSrcMac	75
7	9.1.3.2	tsnCpeSmacVlanDownTagged	75
8	9.1.3.3	tsnCpeSmacVlanDownVlan	75
9	9.1.4	Managed objects for Active Destination MAC and VLAN Stream identifications ..	75
10	9.1.4.1	tsnCpeDmacVlanDownDestMac	76
11	9.1.4.2	tsnCpeDmacVlanDownTagged	76
12	9.1.4.3	tsnCpeDmacVlanDownVlan	76
13	9.1.4.4	tsnCpeDmacVlanDownPriority	76
14	9.1.4.5	tsnCpeDmacVlanUpDestMac	76
15	9.1.4.6	tsnCpeDmacVlanUpTagged	76
16	9.1.4.7	tsnCpeDmacVlanUpVlan	77
17	9.1.4.8	tsnCpeDmacVlanUpPriority	77
18	9.1.5	Managed objects for IP Stream identification	77
19	9.1.5.1	tsnCpeIpIdDestMac	77
20	9.1.5.2	tsnCpeIpIdTagged	77
21	9.1.5.3	tsnCpeIpIdVlan	77
22	9.1.5.4	tsnCpeIpIdIpSource	78
23	9.1.5.5	tsnCpeIpIdIpDestination	78
24	9.1.5.6	tsnCpeIpIdDscp	78
25	9.1.5.7	tsnCpeIpIdNextProtocol	78
26	9.1.5.8	tsnCpeIpIdSourcePort	78
27	9.1.5.9	tsnCpeIpIdDestinationPort	78
28	9.1.6	Managed objects for Mask-and-match Stream identification	78
29	9.1.6.1	tsnCpeMmIdDestMacMask	78
30	9.1.6.2	tsnCpeMmIdDestMacMatch	79
31	9.1.6.3	tsnCpeMmIdSrcMacMask	79
32	9.1.6.4	tsnCpeMmIdSrcMacMatch	79
33	9.1.6.5	tsnCpeMmIdMsduMaskLength	79
34	9.1.6.6	tsnCpeMmIdMsduMask	79
35	9.1.6.7	tsnCpeMmIdMsduMatch	79
36	9.2	Operational per-port per-Stream Stream identification counters	80
37	9.2.1	tsnCpsSidInputPackets	80
38	9.2.2	tsnCpsSidOutputPackets	80
39	9.3	Operational per-port Stream identification counters	80
40	9.3.1	tsnCpSidInputPackets	80
41	9.3.2	tsnCpSidOutputPackets	80
42	9.4	Per-port managed object	80
43	9.4.1	tsnPpMmIdMsduMaskMaxLength	80
44	10.	Frame Replication and Elimination for Reliability management	81
45	10.1	Counter behavior	81
46	10.2	Additional tsnStreamIdEntry managed objects	81
47	10.2.1	tsnStreamIdAutoconfigured	82
48	10.2.2	tsnStreamIdLanPathId	82
49	10.3	Sequence generation table	82
50	10.3.1	frerSeqGenEntry	82
51	10.3.1.1	frerSeqGenStreamList	82
52	10.3.1.2	frerSeqGenDirection	82

1	10.3.1.3	frerSeqGenReset	82
2	10.4	Sequence recovery table	82
3	10.4.1	frerSeqRcvyEntry	82
4	10.4.1.1	frerSeqRcvyStreamList	83
5	10.4.1.2	frerSeqRcvyPortList	83
6	10.4.1.3	frerSeqRcvyDirection	83
7	10.4.1.4	frerSeqRcvyReset	83
8	10.4.1.5	frerSeqRcvyAlgorithm	83
9	10.4.1.6	frerSeqRcvyHistoryLength	83
10	10.4.1.7	frerSeqRcvyResetMSec	83
11	10.4.1.8	frerSeqRcvyInvalidSequenceValue	84
12	10.4.1.9	frerSeqRcvyTakeNoSequence	84
13	10.4.1.10	frerSeqRcvyIndividualRecovery	84
14	10.4.1.11	frerSeqRcvyLatentErrorDetection	84
15	10.4.1.12	Latent error detection managed objects	84
16	10.4.1.12.1	frerSeqRcvyLatentErrorDifference	84
17	10.4.1.12.2	frerSeqRcvyLatentErrorPeriod	84
18	10.4.1.12.3	frerSeqRcvyLatentErrorPaths	85
19	10.4.1.12.4	frerSeqRcvyLatentResetPeriod	85
20	10.5	Sequence identification table	85
21	10.5.1	frerSeqEncEntry	85
22	10.5.1.1	frerSeqEncStreamList	85
23	10.5.1.2	frerSeqEncPort	85
24	10.5.1.3	frerSeqEncDirection	85
25	10.5.1.4	frerSeqEncActive	85
26	10.5.1.5	frerSeqEncEncapsType	85
27	10.5.1.6	frerSeqEncPathIdLanId	85
28	10.6	Stream split table	86
29	10.6.1	frerSplitEntry	86
30	10.6.1.1	frerSplitPort	86
31	10.6.1.2	frerSplitDirection	86
32	10.6.1.3	frerSplitInputIdList	86
33	10.6.1.4	frerSplitOutputIdList	86
34	10.7	Managed objects for autoconfiguration	87
35	10.7.1	Sequence autoconfiguration table	87
36	10.7.1.1	frerAutSeqEntry	87
37	10.7.1.1.1	frerAutSeqSeqEncaps	87
38	10.7.1.1.2	frerAutSeqReceivePortList	87
39	10.7.1.1.3	frerAutSeqTagged	87
40	10.7.1.1.4	frerAutSeqVlan	87
41	10.7.1.1.5	frerAutSeqRecoveryPortList	87
42	10.7.1.1.6	frerAutSeqDestructMSec	87
43	10.7.1.1.7	frerAutSeqResetMSec	87
44	10.7.1.1.8	frerAutSeqAlgorithm	88
45	10.7.1.1.9	frerAutSeqHistoryLength	88
46	10.7.1.1.10	frerAutSeqCreateIndividual	88
47	10.7.1.1.11	frerAutSeqCreateRecovery	88
48	10.7.1.1.12	frerAutSeqLatErrDetection	88
49	10.7.1.1.13	frerAutSeqLatErrDifference	88
50	10.7.1.1.14	frerAutSeqLatErrPeriod	88
51	10.7.1.1.15	frerAutSeqLatErrResetPeriod	88
52	10.7.2	Output autoconfiguration table	88
53	10.7.2.1	frerAutOutEntry	88
54	10.7.2.1.1	frerAutOutPortList	89

1	10.7.2.1.2	frerAutOutEncaps	89
2	10.7.2.1.3	frerAutOutLanPathId	89
3	10.8	Operational per-port and per-Stream FRER counters	89
4	10.8.1	Per-Stream vs. per-Stream-per-port counters	89
5	10.8.2	frerCpsSeqGenResets	89
6	10.8.3	frerCpsSeqRcvyOutOfOrderPackets	90
7	10.8.4	frerCpsSeqRcvyRoguePackets	90
8	10.8.5	frerCpsSeqRcvyPassedPackets	90
9	10.8.6	frerCpsSeqRcvyDiscardedPackets	90
10	10.8.7	frerCpsSeqRcvyLostPackets	90
11	10.8.8	frerCpsSeqRcvyTaglessPackets	90
12	10.8.9	frerCpsSeqRcvyResets	90
13	10.8.10	frerCpsSeqRcvyLatentErrorResets	90
14	10.8.11	frerCpsSeqEncErroredPackets	90
15	10.9	Operational per-port FRER counters	91
16	10.9.1	frerCpSeqRcvyPassedPackets	91
17	10.9.2	frerCpSeqRcvyDiscardPackets	91
18	10.9.3	frerCpSeqEncErroredPackets	91
19	11.	Management Information Base (MIB)	92
20	11.1	Internet Standard Management Framework	92
21	11.2	Structure of the MIB	92
22	11.2.1	Structure of the IEEE8021-STREAM-IDENTIFICATION-MIB	94
23	11.2.2	Structure of the IEEE8021-FRER-MIB	96
24	11.3	Relationship to other MIBs	96
25	11.3.1	Relationship of the IEEE8021-STREAM-IDENTIFICATION-MIB to other MIBs	96
26	11.3.2	Relationship of the IEEE8021-FRER-MIB to other MIBs	101
27	11.4	Security considerations	101
28	11.4.1	Security considerations of the IEEE8021-STREAM-IDENTIFICATION-MIB	101
29	11.4.2	Security considerations of the IEEE8021-FRER-MIB	102
30	11.5	MIB modules,	102
31	11.5.1	Definitions for the IEEE8021-STREAM-IDENTIFICATION-MIB	102
32	11.5.2	Definitions for the IEEE8021-FRER-MIB	133
33	12.	YANG Data Model	170
34	12.1	YANG Framework	170
35	12.2	IEEE Std 802.1CB™ YANG model	171
36	12.2.1	Stream Identification model	171
37	12.2.2	Frame Replication and Elimination for Reliability model	173
38	12.3	Structure of the YANG model	176
39	12.3.1	Structure of the ieee802-dot1cb-stream-identification YANG module	177
40	12.3.2	Structure of the ieee802-dot1cb-mask-and-match YANG module	178
41	12.3.3	Structure of the ieee802-dot1cb-frer YANG module	179
42	12.4	Relationship to other YANG modules	180
43	12.4.1	IEEE 802 Types Module	180
44	12.4.2	IEEE 802.1Q Types Module	180
45	12.4.3	IETF Inet Types Module	180
46	12.4.4	IETF Interfaces YANG Module	180
47	12.4.5	IEEE 802.1CB Stream Identification Types YANG module	180
48	12.4.6	IEEE 802.1CB Stream Identification YANG module	180
49	12.4.7	IEEE 802.1CB Extended Stream identification module	180
50	12.4.8	IEEE 802.1CB FRER Types YANG module	181
51	12.5	Security Considerations	181

1	12.5.1	Security Considerations of the ieee802-dot1cb-stream-identification YANG module	
2		181	
3	12.5.2	Security Considerations of the ieee802-dot1cb-frer YANG module	181
4	12.6	Definition of IEEE Std 802.1CB™ YANG modules	181
5	12.6.1	YANG data scheme tree definitions	182
6	12.6.1.1	YANG data scheme definition for ieee802-dot1cb-stream-identification	
7		YANG module 182	
8	12.6.1.2	YANG data scheme definition for ieee802-dot1cb-mask-and-match	
9		YANG module 183	
10	12.6.1.3	YANG data scheme definition for ieee802-dot1cb-frer YANG module ...	
11		183	
12	12.6.2	YANG data module definitions	186
13	12.6.2.1	Definition for the ieee802-dot1cb-stream-identification-types YANG	
14		module 186	
15	12.6.2.2	Definition for the ieee802-dot1cb-stream-identification YANG module .	
16		188	
17	12.6.2.3	Definition for the ieee802-dot1cb-mask-and-match module	202
18	12.6.2.4	Definition for the ieee802-dot1cb-frer-types YANG module	207
19	12.6.2.5	Definition for the ieee802-dot1cb-frer YANG module	208
20			
21	Annex A (normative)	Protocol Implementation Conformance Statement (PICS) proforma	230
22	A.1	Introduction	230
23	A.1.1	Abbreviations and special symbols	230
24	A.1.2	Instructions for completing the PICS proforma	231
25	A.1.3	Additional information	231
26	A.1.4	Exceptional information	231
27	A.1.5	Conditional items	232
28	A.1.6	Identification	232
29	Implementation identification	232	
30	Protocol summary	232	
31	A.2	PICS proforma for Frame Replication and Elimination for Reliability	233
32	A.2.1	Major capabilities/options	233
33	A.2.2	Stream identification component	233
34	A.2.3	Talker end system	234
35	A.2.4	Listener end system	235
36	A.2.5	Relay system	237
37	A.2.6	FRER 802.1Q C-component	239
38	A.2.7	Common requirements	239
39	A.2.8	Management Information Base (MIB)	240
40	A.2.9	YANG	240
41	Annex B (informative)	Interoperability with other standards	241
42	B.1	Sequence number size	241
43	B.2	Per-Stream versus per-source sequencing	241
44	Annex C (informative)	Frame Replication and Elimination for Reliability in systems	242
45	C.1	Example 1: End-to-end FRER	242
46	C.2	Example 2: Various stack positions	243
47	C.3	Example 3: Ladder redundancy	247
48	C.4	Example 4: Multicast trees	248

1	C.5	Example 5: Protocol interworking	248
2	C.6	Example 6: Chained two-port end systems	249
3	C.7	Cautions	251
4	C.8	Balancing tag insertion and removal	251
5	C.9	FRER and reserved bandwidth	252
6	C.10	Use of the Individual recovery function	253
7	C.11	Use of autoconfiguration	253
8	C.11.1	Routing and labeling Member Streams	253
9	C.11.2	Recognizing packets that trigger autoconfiguration	255
10	C.11.3	Per-port packet decoding and encoding	256
11	C.11.4	Individual and Sequence recovery functions	256
12	Annex D (informative) Mask-and-match Stream identification function		257
13	D.1	Example 1: Generic Stream identification for industrial communication networks	257
14	D.2	Example 2: MPLS DetNet flow identification	257
15	Annex E (informative) Bibliography		259

1 List of figures

2	Figure 6-1—Stream identification service.....	35
3	Figure 6-2—A Stream with three Listeners.....	35
4	Figure 6-3—Stream identification function: single upper SAP.....	37
5	Figure 6-4—Stream identification function: array of upper SAPs	37
6	Figure 6-5—Stream functions in a relay system (three views of same system).....	38
7	Figure 6-5—Stream functions in a relay system (three views of same system).....	39
8	Figure 6-6—In- and out-facing functions	40
9	Figure 6-6—In- and out-facing functions	40
10	Figure 7-1—Compound Stream built from four Member Streams	44
11	Figure 7-2—Frame Replication and Elimination for Reliability functions.....	46
12	Figure 7-3—Sequence recovery functions and Individual recovery functions	58
13	Figure 7-4—R-TAG format.....	59
14	Figure 8-1—FRER functions in an FRER C-component	67
15	Figure 8-1—Split functions in an FRER C-component.....	68
16	Figure 8-2—Merge functions in an FRER C-component.....	68
17	Figure 8-3—Augmented Forwarding Process does sequence recovery	70
18	Figure 8-4—Example Ethernet frame format	71
19	Figure 11-1—MIB modules structure.....	93
20	Figure 12-1—General YANG hierarchy	170
21	Figure 12-2—YANG root hierarchy with IEEE Std 802.1CB™ YANG modules	171
22	Figure 12-3—Stream identification model	172
23	Figure 12-4—Stream identification counters model	173
24	Figure 12-5—FRER model.....	174
25	Figure 12-6—FRER Stream identification augmentation	175
26	Figure 12-7—FRER counters model	176
27	Figure C-1—Dual-homed end systems using Link Aggregation	242
28	Figure C-2—Protocol stack for End System B in Figure C-1	243
29	Figure C-3—Protocol stack for End System G in Figure C-1 and Figure C-4.....	243
30	Figure C-4—IEEE Std 802.1CB™ flexible positioning.....	244
31	Figure C-5—Protocol stack for relay system B, proxying for End System A, in Figure C-4	245
32	Figure C-5—Protocol stack for relay system B, proxying for End System A, in Figure C-4	245
33	Figure C-6—Protocol stack for relay system C in Figure C-4	246
34	Figure C-6—Protocol stack for relay system C in Figure C-4	246
35	Figure C-7—Protocol stack for relay system F in Figure C-4.....	247
36	Figure C-7—Protocol stack for relay system F in Figure C-4.....	247
37	Figure C-8—Ladder redundancy	248
38	Figure C-9—Multicast trees	248
39	Figure C-10—Protocol interworking.....	249
40	Figure C-10—Protocol interworking.....	249
41	Figure C-11—Dual-homed end systems using 3-port bridge	250
42	Figure C-12—Protocol stacks for Systems B and G in Figure C-11	250

1 Figure C-12—Protocol stacks for Systems B and G in Figure C-11 250

2 Figure C-13—Explicit path causing a loop 251

3 Figure C-14—Example of Long and short paths 252

4 Figure C-15—Autoconfiguration example 255

1 List of tables

2	Table 6-1—Stream identification functions	36
3	Table 7-1—R-TAG EtherType	60
4	Table 8-1—Managed objects for FRER in an FRER C-component	72
5	Table 9-1—Stream identification types	74
6	Table 10-1—Enumerated values for frerSeqRcvyAlgorithm	83
7	Table 10-2—Sequence Encode/Decode types	86
8	Table 11-1—Structure of the MIB modules	92
9	Table 11-2—IEEE8021-STREAM-IDENTIFICATION-MIB structure and	
10	relationship to this standard	94
11	Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard	97
12	Table 12-2—ieee802-dot1cb-stream-identification structure and relationship to this standard.....	177
13	Table 12-3—YANG module dependencies for the Stream identification model.....	177
14	Table 12-1—Description of the YANG modules	177
15	Table 12-4—ieee802-dot1cb-mask-and-match structure and relationship to this standard	178
16	Table 12-5—YANG module dependencies for the Extended Stream identification model	178
17	Table 12-6—ieee802-dot1cb-frer structure and relationship to this standard	179
18	Table 12-7—YANG module dependencies for the FRER model	179
19	Table D-1—Managed objects values for example 1.....	257
20	Table D-2—Managed objects values for example 2.....	258

1

IEEE Standard for Local and metropolitan area networks—

8 Frame Replication and Elimination for 9 Reliability

10 1. Overview

11 1.1 Scope

12 This standard specifies procedures, managed objects, and protocols for bridges and end systems that provide
13 identification and replication of packets for redundant transmission, identification of duplicate packets, and
14 elimination of duplicate packets. It is not concerned with the creation of the multiple paths over which the
15 duplicates are transmitted.

16 1.2 Rationale

17 The reason for Frame Replication and Elimination for Reliability (FRER) is to increase the probability that a
18 given packet will be delivered. It is expected that, in many applications, other means to increase the
19 probability of delivery are likely to be used as well. When FRER is used over paths that are fixed to a
20 specific topology, and that are protected against congestion loss (e.g., by using techniques described by
21 IEEE Std 802.1BA™ [B2]), FRER can substantially reduce the probability of packet loss due to equipment
22 failures.⁶

23 1.3 State diagram conventions

24 This document uses the programming language C (ISO/IEC 9899:2011) to document the operation of
25 conformant systems.⁷ C functions are distinguished with this special fixed-width font (e.g.,
26 7.4.3.3). Each C function is executed when a given event occurs, as described for that code segment or in the
27 accompanying text. Events are assumed to take place sequentially, not simultaneously, and code routines
28 execute instantaneously.

29 1.4 Specification model

30 The model of operation documented by this standard is simply a basis for describing the functionality of
31 compliant equipment. Implementations can adopt any internal model of operation compatible with the

⁶The numbers in brackets correspond to those of the bibliography in Annex E.

⁷Information on references can be found in Clause 2.

1 externally visible behavior that this standard specifies. Conformance of equipment to this standard is purely
2 in respect of observable protocol.

3 1.5 Specification precedence

4 If any conflict among parts of this standard become apparent, C functions (see 1.3) take precedence over
5 other parts of the standard, followed by information in normative tables, followed by that in normative text,
6 followed by that in normative figures. Non-normative tables, figures, and text are in annexes and are clearly
7 marked as such.

8 1.6 Introduction

9 This standard is one of a number of IEEE 802.1™ and other standards suitable for Time-Sensitive
10 Networking (TSN) that together have the overall goal of providing extremely low packet loss rates and
11 finite, low, and stable end-to-end latencies. TSN supports unicast and multicast Streams of packets that
12 implement a wide range of demanding real-time applications including audio/video studios, industrial
13 processes, and the control of machines and vehicles. The TSN goals are not achieved at the expense of
14 hampering the ability of the network to carry traffic for non-time-critical applications.

15 At the highest level, this standard posits the existence of one Talker end system and one or more Listener
16 end systems per Stream. A Stream is characterized by a maximum packet size and number of packets
17 transmitted per time interval. Because the Stream's maximum throughput is known, the resources, including
18 link bandwidth, buffer space, and control parameters, required at every hop along the Stream's path to
19 guarantee that Stream zero congestion loss and finite latency, can be provided (by other standards, e.g.,
20 Clause 35 of IEEE Std 802.1Q™-2022). This provisioned path carrying the Stream is called a *Reservation*.

21 On the assumption that the time required for a dynamic network control protocol to recover from an
22 equipment failure is unacceptable in certain applications, this standard defines Frame Replication and
23 Elimination for Reliability (FRER), which divides a Stream into one or more linked Member Streams, thus
24 making the original Stream a Compound Stream. It replicates the packets of the Stream, splitting the copies
25 into the multiple Member Streams, and then rejoins those Member Streams at one or more other points,
26 eliminates the replicates, and delivers the reconstituted Stream from those points.

27 In order to accommodate existing applications and to promote interoperability with similar standards, this
28 standard defines a number of schemes for identifying packets belonging to Streams and distinguishing them
29 from other packets.

2. Normative references

The following referenced documents are indispensable for the application of this document (i.e., they must be understood and used, so each referenced document is cited in text and its relationship to this document is explained). For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments or corrigenda) applies. Non-normative references (i.e., that provide additional information not required for the application of this document) are given in Annex E.

IEC 62439-3:2021, Industrial communication networks—High availability automation networks—Part 3: Parallel Redundancy Protocol (PRP) and High-availability Seamless Redundancy (HSR).⁸

IEEE Std 802[®], IEEE Standard for Local and metropolitan area networks: Overview and Architecture.^{9, 10}

IEEE Std 802.1AC[™], IEEE Standard for Local and metropolitan area networks—Media Access Control (MAC) Service Definition.

IEEE Std 802.1Q[™], IEEE Standard for Local and metropolitan area networks—Bridges and Bridged Networks.

IETF RFC 768, User Datagram Protocol, Postel, J., August 1980.¹¹

IETF RFC 791, Internet Protocol, Postel, J., Ed., September 1981.

IETF RFC 793, Transmission Control Protocol, Postel, J., Ed., September 1981.

IETF RFC 2460, Internet Protocol, Version 6 (IPv6) Specification, Deering, S. and R. Hinden, December 1998.

IETF RFC 2474, Definition of the Differentiated Services Field (DS Field) in the IPv4 and IPv6 Headers, Nichols, K., et al., December 1998.

IETF RFC 2578 (STD 58), Structure of Management Information Version 2 (SMIv2), McCloghrie, K., D. Perkins, and J. Schoenwaelder, Apr. 1999.

IETF RFC 2579 (STD 58), Textual Conventions for SMIv2, McCloghrie, K., D. Perkins, and J. Schoenwaelder, Apr. 1999.

IETF RFC 2580 (STD 58), Conformance Statements for SMIv2, McCloghrie, K., D. Perkins, and J. Schoenwaelder, Apr. 1999.

IETF RFC 3410 (informational), Introduction and Applicability Statements for Internet-Standard Management Framework, Case, J., R. Mundy, D. Partain, and B. Stewart, Dec. 2002.

IETF RFC 4960, Stream Control Transmission Protocol, Stewart, R., Ed., September 2007.

IETF RFC 8343 (proposed standard), A YANG Data Model for Interface Management, Bjorklund, M., Mar. 2018.

⁸IEC publications are available from the International Electrotechnical Commission (<http://www.iec.ch>) and the American National Standards Institute (<http://www.ansi.org/>).

⁹The IEEE standards or products referred to in Clause 2 are trademarks owned by The Institute of Electrical and Electronics Engineers, Incorporated.

¹⁰IEEE publications are available from The Institute of Electrical and Electronics Engineers (<http://standards.ieee.org>).

¹¹IETF RFCs are available from the Internet Engineering Task Force (<https://www.rfc-archive.org/>).

¹ ISO/IEC 9899:2011, Information technology—Programming languages—C.¹²

2

¹²ISO/IEC publications are available from the International Organization for Standardization (<http://www.iso.org/>) and the American National Standards Institute (<http://www.ansi.org/>).

3. Definitions

For the purposes of this document, the following terms and definitions apply. The *IEEE Standards Dictionary Online* should be consulted for terms not defined in this clause.¹³

This standard makes use of the following terms defined in IEEE Std 802®:

- End station
- Organizationally Unique Identifier (OUI)
- Company ID (CID)

bridge: A layer 2 interconnection device that conforms to IEEE Std 802.1Q™.

Company ID (CID): CIDs allow a general means of assuring unique identifiers for a number of purposes. See Clause 8 of IEEE Std 802-2014.

Compound Stream: A Compound Stream is a Stream composed of one or more Member Streams linked together via Frame Replication and Elimination for Reliability (FRER).

down: The direction through the protocol stack from a sublayer using a service provided by another sublayer to the sublayer whose services it uses; the direction of output packets.

end station: A device attached to a local area network (LAN) or metropolitan area network (MAN), which acts as a source of and/or destination for data traffic carried on the LAN or MAN. (From IEEE Std 802)

end system: A system attached to a network that is an initial source or a final destination of packets transmitted across that network.

NOTE—The term *end system* is often used in this document in places where the reader of IEEE 802 standards would expect the term *end station* in order to avoid confusion caused by standards relating to routers. For example, a router, as defined by IETF, is an *IEEE 802 end station*, but not an end system. Where this standard specifically refers to the use of IEEE 802 services, the term *end station* is used. Where it refers to more generalized instances of connectionless services, the term *end system* is used.¹⁴

in-facing: In a system that includes a Stream Transfer Function, the “in-facing” protocol functions are those functions that are below the Stream Transfer Function on the port that is not above the physical layer. On a system that does not include a Stream Transfer Function, there are no in-facing protocol functions.

NOTE—See Figure 6-6.

input: Input packets are those moving up the protocol stack, regardless of the position of receiving function relative to the physical media, for example, by the M_UNITDATA.indication primitive of the Internal Sublayer Service (ISS).

Internal local area network (LAN): An instance of a connectionless packet service with two Service Access Points (SAPs), both internal to a single system, that relays packets from one SAP to the other. It is not observable externally to that system.

Internal Sublayer Service (ISS): An augmented version of the MAC Service, defined in IEEE Std 802.1AC™

¹³IEEE Standards Dictionary Online is available at: <https://ieeexplore.ieee.org/browse/standards/dictionary>.

¹⁴Notes in text, tables, and figures of a standard are given for information only and do not contain requirements needed to implement this standard.

1 **Media Access Control (MAC):** The data link sublayer that is responsible for transferring data to and from
2 the Physical Layer.

3 **Member Stream:** A Stream that is linked with other Member Streams via IEEE Std 802.1CB™ (FRER) to
4 form a Compound Stream.

5 **Non-Stream Transfer Function (NSTF):** A two-port function that relays service indications on one port to
6 service requests on the other port.

7 **Organizationally Unique Identifier (OUI):** OUIs allow a general means of assuring unique identifiers for
8 a number of purposes. See Clause 9 and Clause 10 of IEEE Std 802-2014.

9 **out-facing:** In a system that includes a Stream Transfer Function, the “out-facing” protocol functions are
10 those functions that are below the Stream Transfer Function on the port that is above the physical layer. On a
11 system that does not include a Stream Transfer Function, all TSN functions are out-facing.

12 NOTE—See Figure 6-6.

13 **output:** Output packets are those moving down the protocol stack, regardless of the position of receiving
14 function relative to the physical media, for example, by the M_UNITDATA.request primitive of the Internal
15 Sublayer Service (ISS).

16 **packet:** A unit of data carried over a network, comprising, at least, one or more pairs of destination and
17 source addresses and a payload. A packet carried on an Ethernet local area network (LAN) is a frame.

18 NOTE—The term *packet* is often used in this document in places where the reader of IEEE 802 standards would expect
19 the term *frame*. Where the standard specifically refers to the use of IEEE 802 services, the term *frame* is used. Where the
20 standard refers to more generalized instances of connectionless services, the term *packet* is used.

21 **Quality of Service (QoS):** Overall performance of a packet Stream as it relates to packet loss probability,
22 latency, and latency variation.

23 **relay system:** A router or a bridge.

24 NOTE—The term *relay system* is often used in this document in places where the reader of IEEE 802 standards would
25 expect the term *bridge*. A relay system can, in theory, be a router, a bridge, or some other kind of forwarding device.
26 Where this standard specifically refers to one or the other, the terms *router* or *bridge* are used. Where it refers to more
27 generalized instances of connectionless services, the term *relay system* is used.

28 **Reservation:** The collection of state information and resources allocated, in a chain of end systems and
29 relay systems, for one or more Streams.

30 **router:** A packet forwarding device operating on Internet Protocol (IP) packets.

31 **Stream:** A unidirectional flow of time-sensitive data from one source to one or more destinations, and at the
32 highest level, one Talker end system to one or more Listener end systems.

33 **Stream Transfer Function:** A two-port function that relays service indications on one port to service
34 requests on the other port, where the service includes all TSN service subparameters.

35 **subparameter:** A service parameter encoded into the value of the connection_identifier parameter of the
36 Internal Sublayer Service (ISS).

37 **system:** An end system or a relay system.

38 **up:** The direction through the protocol stack from a sublayer offering a service to the sublayer making use of
39 that service; the direction of input packets.

4. Acronyms and abbreviations

This standard contains the following abbreviations:

CID	Company ID
DRNI	Distributed Resilient Network Interconnect
EISS	Enhanced Internal Sublayer Service
FRER	Frame Replication and Elimination for Reliability
IEC	International Electrotechnical Commission
IP	Internet Protocol
ISO	International Organization for Standardization
ISS	Internal Sublayer Service
LAN	Local Area Network
MAC	Medium Access Control
MIB	Management Information Base
NETCONF	Network Configuration Protocol
NSTF	Non-Stream Transfer Function
OUI	Organizationally Unique Identifier
PDU	Protocol Data Unit
PICS	Protocol Implementation Conformance Statement
QoS	Quality of Service
R-TAG	Redundancy tag
SAP	Service Access Point
SMI	Structure of Management Information
SNMP	Simple Network Management Protocol
TSN	Time-Sensitive Networking
UML	Unified Modeling Language
VLAN	Virtual Local Area Network
YANG	Yet Another Next Generation ¹⁵

¹⁵ YANG is best viewed as a name, not an acronym

5. Conformance

This clause specifies the mandatory and optional capabilities provided by conformant implementations of this standard.

5.1 Requirements terminology

For consistency with existing IEEE and IEEE 802.1 standards, requirements placed upon conformant implementations of this standard are expressed using the following terminology:

- a) *Shall* is used for mandatory requirements;
- b) *May* is used to describe implementation or administrative choices (“may” means “is permitted to,” and hence, “may” and “may not” mean precisely the same thing);
- c) *Should* is used for recommended choices (the behaviors described by “should” and “should not” are both permissible but not equally desirable choices).

The Protocol Implementation Conformance Statement (PICS) proformas (see Annex A) reflect the occurrences of the words “shall,” “may,” and “should” within the standard.

The standard avoids needless repetition and apparent duplication of its formal requirements by using *is*, *is not*, *are*, and *are not* for definitions and the logical consequences of conformant behavior. Behavior that is permitted but is neither always required nor directly controlled by an implementor or administrator, or whose conformance requirement is detailed elsewhere, is described by *can*. Behavior that never occurs in a conformant implementation or system of conformant implementations is described by *cannot*. The word *allow* is used as a replacement for the phrase “support the ability for,” and the word *capability* means “can be configured to.”

5.2 Conformant components and equipment

Conformance for Frame Replication and Elimination for Reliability (FRER) is defined for five types of components and equipment:

- a) Stream identification components (5.3, 5.4, 5.5), which provide a useful subset of the FRER capabilities for systems that are not talker end systems, listener end systems, or relay systems.
- b) Talker end systems (5.6, 5.7, 5.8), which originate Compound Streams.
- c) Listener end systems (5.9, 5.10, 5.11), which consume Compound Streams.
- d) Relay systems (5.12, 5.13, 5.14), which transfer or discard packets belonging to Compound Streams.
- e) One particular type of Relay system, an FRER C-component (5.15).

The Stream identification component can be useful in components or equipment that are not talker end systems, listener end systems, or relay systems as defined in this standard, e.g., bridges or routers that carry packets belonging to Compound Streams, and must identify them, but do not process FRER sequence numbers. Other useful systems can be created using the preceding components listed. For example, a two-port Talker could be modeled and implemented as two Talker end systems in a single chassis, as a single one-port Talker end system that uses IEEE 802.1AX Link Aggregation to combine two physical ports into a single logical port (C.1), or as a single-port Talker end system connected to a three-port Bridge (C.6).

5.3 Stream identification component required behaviors

A Stream identification component shall be able to instantiate the following out-facing functions on at least one port, for at least one Compound Stream:

- 1 a) Stream identification (Clause 6);
- 2 b) A Null Stream identification function (6.4); and
- 3 c) The managed objects in Clause 9.

4 **5.4 Stream identification component recommended behavior**

5 A Stream identification component should be able to instantiate the following out-facing functions on at
6 least one port, for at least one Compound Stream:

- 7 a) [A Source MAC and VLAN Stream identification function \(6.5\);](#)
- 8 b) An Active Destination MAC and VLAN Stream identification function (6.6).

9 **5.5 Stream identification component optional behaviors**

10 In addition to the requirements of 5.3 and 5.4, a Stream identification component may perform the following
11 functions:

- 12 a) The items in 5.3 and 5.4 on more than one port;
- 13 b) The items in 5.3 and 5.4 for some number of Compound Streams greater than 1;
- 14 c) An IP Stream identification function (6.7);
- 15 d) [A Mask-and-match Stream identification function \(6.8\); and/or](#)
- 16 e) Additional types of Stream identification functions.

17 **5.6 Talker end system required behaviors**

18 A Talker end system shall be able to instantiate the following out-facing functions on at least one port, for at
19 least one Compound Stream:

- 20 a) Stream identification (Clause 6);
- 21 b) A Null Stream identification function (6.4);
- 22 c) A Sequence generation function (7.4.1); and
- 23 d) The Redundancy tag Sequence encode/decode function specified in 7.6 and 7.8.
- 24 e) The managed objects in Clause 9 and Clause 10, excepting 10.7.

25 **5.7 Talker end system recommended behaviors**

26 A Talker end system should be able to instantiate the following out-facing functions on at least one port, for
27 at least one Compound Stream:

- 28 a) [A Source MAC and VLAN Stream identification function \(6.5\);](#)
- 29 b) An Active Destination MAC and VLAN Stream identification function (6.6); and
- 30 c) A Stream splitting function (7.7).

31 **5.8 Talker end system optional behaviors**

32 In addition to the requirements of 5.6, a Talker end system may perform the following functions:

- 33 a) The items in 5.6 and 5.7 on more than one port;
- 34 b) The items in 5.6 and 5.7 for some number of Compound Streams greater than 1;

- 1 c) An IP Stream identification function (6.7);
- 2 d) A Mask-and-match Stream identification function (6.8);
- 3 e) Additional types of Stream identification functions;
- 4 f) The HSR sequence tag (7.9);
- 5 g) The PRP sequence trailer (7.10);
- 6 h) Additional types of Sequence encode/decode functions;
- 7 i) Support SMIPv2 MIB modules for the management of talker end system functions (Clause 11) as
- 8 required by item e in 5.6; and/or
- 9 j) Support YANG modules for the management of talker end system functions (Clause 12) as required
- 10 by item e in 5.6.

11 5.9 Listener end system required behaviors

12 A Listener end system shall be able to instantiate the following out-facing functions on at least one port, for
13 at least one Compound Stream:

- 14 a) Stream identification (Clause 6);
- 15 b) A Null Stream identification function (6.4);
- 16 c) A Sequence recovery function (7.4.2) that supports the MatchRecoveryAlgorithm (7.4.3.5) and
- 17 supports the VectorRecoveryAlgorithm (7.4.3.4) with a value of at least 2 for the managed object
- 18 frerSeqRcvyHistoryLength (10.4.1.6);
- 19 d) At least two instances of Individual recovery functions (7.5), each using the
- 20 MatchRecoveryAlgorithm (7.4.3.5); and
- 21 e) The Redundancy tag Sequence encode/decode function specified in 7.6 and 7.8.
- 22 f) The managed objects in Clause 9 and Clause 10, excepting 10.7.

23 5.10 Listener end system recommended behavior

24 A Listener end system should be able to instantiate the following out-facing function on at least one port, for
25 at least one Compound Stream:

- 26 a) A Source MAC and VLAN Stream identification function (6.5);
- 27 b) An Active Destination MAC and VLAN Stream identification function (6.6).

28 5.11 Listener end system optional behaviors

29 In addition to the requirements of 5.9 and 5.10, a Listener end system may perform the following functions:

- 30 a) The items in 5.9 and 5.10 on more than one port;
- 31 b) The items in 5.9 and 5.10 for some number of Compound Streams greater than 1;
- 32 c) An IP Stream identification function (6.7);
- 33 d) A Mask-and-match Stream identification function (6.8);
- 34 e) Additional types of Stream identification functions;
- 35 f) The HSR sequence tag (7.9);
- 36 g) The PRP sequence trailer (7.10);
- 37 h) Additional types of Sequence encode/decode functions;
- 38 i) At least two instances of Individual recovery functions (7.5), each using the
- 39 VectorRecoveryAlgorithm (7.4.3.4);

- 1 j) Support SMiv2 MIB modules for the management of listener end system functions (Clause 11) as
2 required by item f in 5.9; and/or
- 3 k) Support YANG modules for the management of listener end system functions (Clause 12) as
4 required by item f in 5.9.

5.12 Relay system required behaviors

6 A relay system shall be able to instantiate the following in-facing functions on at least two ports, for both
7 transmit and receive, for at least one Stream:

- 8 a) Stream identification (Clause 6).
- 9 b) A Null Stream identification function (6.4);
- 10 c) A Sequence generation function (7.4.1);
- 11 d) The Redundancy tag Sequence encode/decode function specified in 7.6 and 7.8;
- 12 e) A Sequence recovery function (7.4.2) that supports the MatchRecoveryAlgorithm (7.4.3.5) and
13 supports the VectorRecoveryAlgorithm (7.4.3.4) with a value of at least 2 for the managed object
14 frerSeqRcvyHistoryLength (10.4.1.6);
- 15 f) At least two instances of Individual recovery functions (7.5), each using the
16 MatchRecoveryAlgorithm (7.4.3.5); and
- 17 g) The managed objects in Clause 9 and Clause 10, excepting 10.7.

5.13 Relay system recommended behaviors

19 A relay system should be able to instantiate the following in-facing functions on at least two ports, for both
20 transmit and receive, for at least one Stream:

- 21 a) A Source MAC and VLAN Stream identification function (6.5);
- 22 b) Active Destination MAC and VLAN Stream identification functions (6.6) for encoding and decod-
23 ing packets; and
- 24 c) either of the following:
 - 25 1) IP Stream identification functions (6.7);
 - 26 2) Mask-and-match Stream identification function (6.8) for identifying packets.

27 NOTE—IP Stream identification and Mask-and-match Stream identification enable a relay system to proxy for an
28 FRER-unaware end system.

5.14 Relay system optional behaviors

30 In addition to the requirements of 5.12 and 5.13, a relay system may perform the following functions:

- 31 a) The items in 5.12 or 5.13 on more than two ports;
- 32 b) The items in 5.12 or 5.13 for some number of Streams greater than 1;
- 33 c) Additional types of Stream identification functions;
- 34 d) The Stream splitting function (7.7);
- 35 e) The HSR sequence tag (7.9);
- 36 f) The PRP sequence trailer (7.10);
- 37 g) Additional types of Sequence encode/decode functions;
- 38 h) Some or all of the functions in 5.12 or 5.13 as both in- and out-facing functions;

- 1 i) At least two instances of Individual recovery functions (7.5), each using the
- 2 VectorRecoveryAlgorithm (7.4.3.4);
- 3 j) Autoconfiguration (7.11) and the associated managed objects (10.7);
- 4 k) Support SMIPv2 MIB modules for the management of relay systems functions (Clause 11) as
- 5 required by item g in 5.12; and/or
- 6 l) Support YANG modules for the management of relay systems functions (Clause 12) as required by
- 7 item g in 5.12.

8 5.15 FRER C-component required and optional behaviors

9 An FRER C-component is an IEEE 802.1Q C-VLAN component that is FRER-capable. An FRER
10 C-component:

- 11 a) Shall meet all of the required and any or none of the optional behaviors for an IEEE 802.1Q
- 12 C-VLAN component (5.5 of IEEE Std 802.1Q™-2022).
- 13 b) Shall meet all of the required behaviors for a relay system (5.12);
- 14 c) May meet any or none of the optional behaviors for a relay system (5.14); and
- 15 d) Shall conform to the requirements for configuring FRER C-components (8.4).

16

17

18

6. Stream identification

Clause 7 of IEEE Std 802.1AC™-2026 describes the IEEE 802.1 layering model, that Frame Replication and Elimination for Reliability (FRER) follows. Stream identification utilizes a single Service Access Point (SAP) to a connectionless packet service offered by the layer below it [e.g., the Intermediate Sublayer Service (ISS) of Clause 11 of IEEE Std 802.1AC™-2026, or the Enhanced Internal Sublayer Service (EISS) of 6.8 of IEEE Std 802.1Q™-2022], and offers an array of SAPs to the layers above it, corresponding to different Streams. The Stream identification model is illustrated in Figure 6-1.

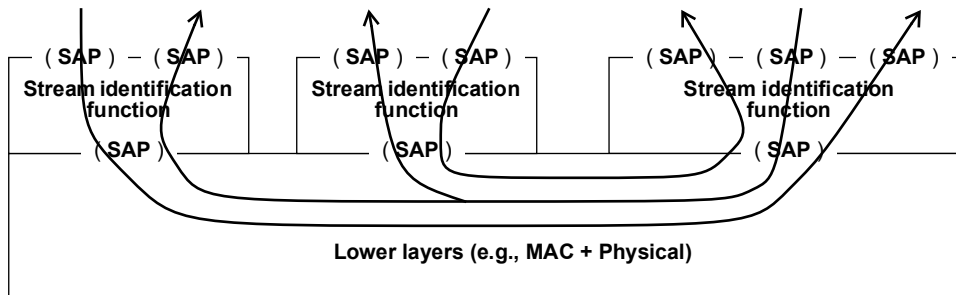


Figure 6-1—Stream identification service

Stream identification can be active (e.g., 6.6) or passive (e.g., 6.4). When accepting packets for transmission from the upper layers, an active Stream identification function (6.2) modifies the data parameters to encode the choice of SAPs, and offer the encapsulated packets to the lower layers. In the receive direction, the active Stream identification function accepts packets from the lower layers, decapsulates them, and passes them to the upper layers through the appropriate SAPs according to the Stream identification derived from the packet. A passive Stream identification function does nothing to packets passed down from the upper layers, but examines packets received from lower layers to identify the packet's Stream and determine to which SAP to pass it.

NOTE—In principle, any number of different methods for identifying and encoding Streams can be defined. Several required methods are specified in the following subclauses (6.4, 6.5, 6.6, 6.7, 6.8).

A Stream is the entity to which the Qualities of Service (QoSs) are offered. It is a sequence of packets, either unicast or multicast, from a Talker to one or more Listeners. Figure 6-2 illustrates a network carrying a single Stream.

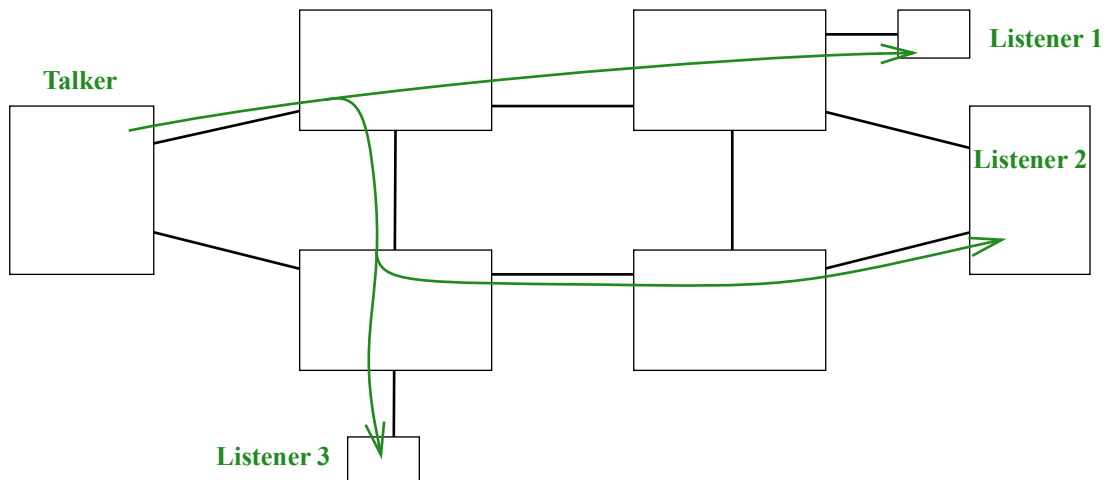


Figure 6-2—A Stream with three Listeners

1 Stream identification is described in the following subclauses as follows:

- 2 a) Additional service subparameters required by Stream identification are in 6.1.
- 3 b) The Stream identification function is described in 6.2, and its placement in the protocol stack of a
4 system in 6.3.
- 5 c) Five specific Stream identification functions are described: Null Stream identification (6.4), Source
6 MAC and VLAN Stream identification (6.5), Active Destination MAC and VLAN Stream
7 identification (6.6), IP Stream identification (6.7), and Mask-and-match Stream identification (6.8).

8 These Stream identification functions are summarized in Table 6-1.

Table 6-1—Stream identification functions

Stream identification function	Active/passive	Examines	Overwrites	Reference
Null Stream identification	Passive	destination_address, vlan_identifier	None	6.4, 9.1.2
Source MAC and VLAN Stream identification	Passive	source_address, vlan_identifier	None	6.5, 9.1.3
Active Destination MAC and VLAN Stream identification	Active	destination_address, vlan_identifier	destination_address, vlan_identifier, priority	6.6, 9.1.4
IP Stream identification	Passive	destination_address, vlan_identifier, IP source address, IP destination address, DSCP, IP next protocol, source port, destination port	None	6.7, 9.1.5
Mask-and-match Stream identification function	Passive	destination_address, source_address, mac_service_data_unit	None	6.8, 9.1.6

9 6.1 Stream service subparameters

10 The ISS defined in IEEE Std 802.1AC™ and the EISS defined in IEEE Std 802.1Q™ include a
11 connection_identifier parameter that is of local significance (to a system) only. The parameter is not carried
12 across the underlying service. Stream identification makes use of this parameter to carry parametrized
13 information. Stream identification has need for more than one subparameter, but an implementor can create
14 mathematical algorithms to combine those subparameters (and/or other subparameters for other layers) into
15 a single connection_identifier parameter, especially since the connection_identifier's values are undefined
16 outside the system implementing them. In this document, parameters that are assumed to be encoded in the
17 connection_identifier are deemed *subparameters*.

18 The parameters of the service offered by Stream identification include the following subparameters required
19 by the Stream identification function and other functions defined in this standard:

- 20 a) **stream_handle:** An integer identifying the Stream to which the packet belongs.

- 1 b) **sequence_number:** An unsigned integer identifying the order in which the packet was transmitted
2 relative to other packets in the same Compound Stream.

3 The numerical value of the stream_handle subparameter is local to a system; this subparameter is not to be
4 confused with, for example, a field in a Protocol Data Unit (PDU) with a similar function. The value of a
5 stream_handle is connected to protocol fields by means of protocol actions and network management. It can,
6 but does not necessarily, have a 1:1 relationship with an explicit field in the packet.

7 Not every sequence of packets of interest to FRER (Clause 7) requires the relay systems along its path to
8 identify to which Stream the packets belong. FRER does require the stream_handle and sequence_number
9 subparameters in systems where a Sequence recovery function (7.4.2) or Individual recovery function (7.5)
10 is configured for a Stream.

11 The numerical value of the sequence_number subparameter can be explicitly encoded in the packet by either
12 the Stream identification function (6.2) or the Sequence encode/decode function (7.6).

13 6.2 Stream identification function

14 As illustrated in Figure 6-3, the Stream identification function can be described as having two SAPs (see
15 IEEE Std 802.1AC™). One SAP connects Stream identification function to the upper layers. This SAP
16 includes a stream_handle subparameter and can include a sequence_number subparameter. The other SAP
17 connects to the lower layers. This SAP can, but typically does not, include the stream_handle or
18 sequence_number subparameters.

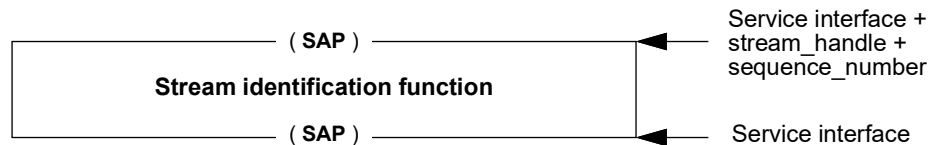


Figure 6-3—Stream identification function: single upper SAP

19 Equivalently, the Stream identification function can be described as having the same SAP to the lower
20 layers, but as having an array of SAPs to the upper layers. A unique SAP corresponds to each value of the
21 stream_handle subparameter, including one SAP for packets belonging to no known Stream. An SAP can
22 serve more than one stream_handle value. This is illustrated in Figure 6-4.

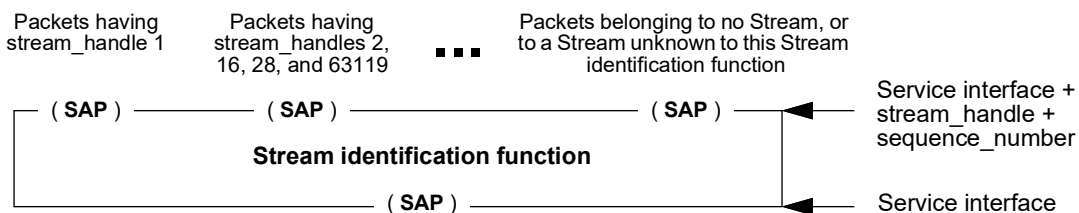


Figure 6-4—Stream identification function: array of upper SAPs

23 The Stream identification function performs two functions:

- 24 a) Packets received from the lower layer are examined by the Stream identification function to
25 determine a value for the stream_handle subparameter for that packet, i.e., to determine to which
26 Stream the packet belongs.
27 1) If the packet belongs to a Stream known to this Stream identification function the resultant
28 packet is passed, along with the stream_handle and any other subparameters (e.g.,

- sequence_number) extracted from the packet, to the upper layers. Depending on the particular Stream identification method (or methods) employed, parameters can be modified (e.g., a tag can be removed or an address changed) by the Stream identification function.
- 2) Otherwise (the packet belongs to no known Stream), the packet is passed unchanged to the next upper layer with null values for the stream_handle and sequence_number subparameters.
- b) For packets passed down from the upper layers for transmission, the Stream identification function uses the packet's stream_handle subparameter to determine how to handle the packet:
- 1) If the packet belongs to a Stream known to this Stream identification function the resultant packet is passed to the lower layers. Depending on the particular Stream identification method (or methods) employed, parameters can be modified (e.g., a tag can be added or an address changed) by the Stream identification function.
- 2) Otherwise (the packet belongs to no known Stream), the packet is passed unchanged to the next lower layer with all parameters intact.

The null-value is a value known from the management system as identifying the stream_handle assigned to packets that belong to no known Stream.

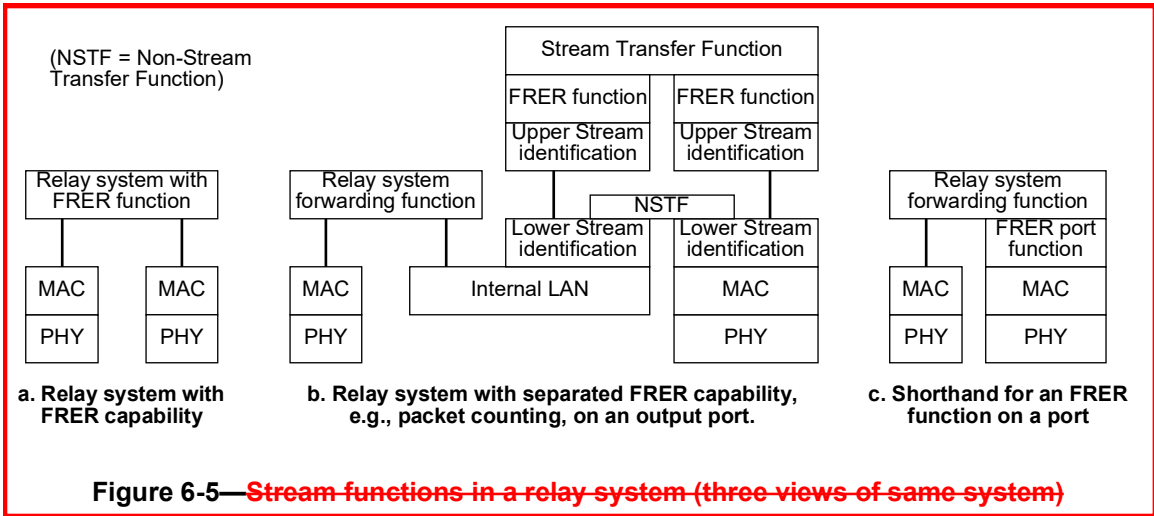
NOTE 1—The Stream identification method does not necessarily involve an actual transformation of the packet. See 6.4 and 6.6 for examples of both cases.

NOTE 2—The order of precedence of Stream identification functions and the invocation of their instances, when more than one matches the same frame, is out of scope of this standard.

6.3 Stream identification in systems

FRER (Clause 7) capabilities generally require Stream identification in order to function. How the Stream identification function (6.2) and the various FRER functions (7.4, 7.6) are arranged to accomplish a given task, and how they are described for the purposes of standards specification, can vary. Diagram a in Figure 6-5 illustrates a relay system (e.g., 8.6 of IEEE Std 802.1Q-2022) with two ports, that has its FRER capabilities embedded in its forwarding function, and not shown explicitly in the diagram. This formulation is the simplest, but does not make clear the ordering of operations, at least in the diagram.

Change Figure 6-5 as shown:



1 :

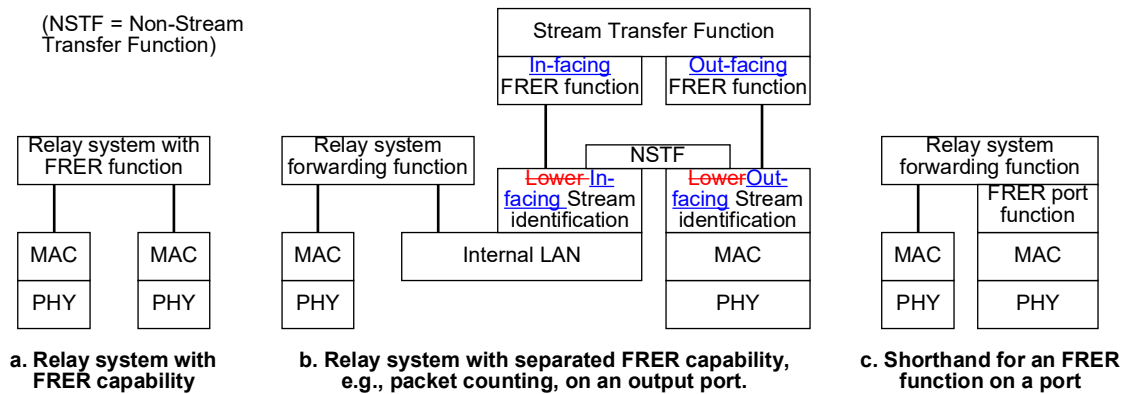


Figure 6-5—Stream functions in a relay system (three views of same system)

2 Change the text of the second paragraph as shown:

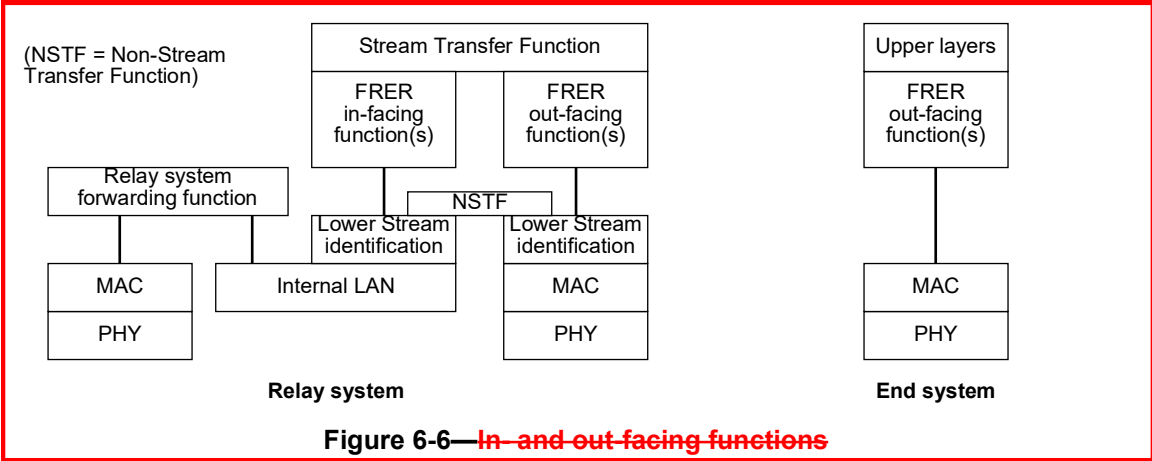
3 Diagram b in Figure 6-5 shows that same relay system, this time with the FRER functions placed explicitly
4 in one of its ports, the picture of which is greatly expanded. In Diagram b, the forwarding function has no
5 FRER capabilities. Stream identification functions extract Stream identification function subparameters in
6 order to enable an FRER function. [The connectivity between the Relay system's forwarding and the Stream
7 identification functions is modelled by an Internal LAN that provides transparent connectivity between ISS,
8 or EISS, SAPs supporting the Stream service subparameters \(6.1\).](#) The *Stream Transfer Function* acts as a
9 two-port packet relay, residing entirely inside the one port of the relay system, relaying packets belonging to
10 Streams. The *Non-Stream Transfer Function* (NSTF) does the same, but attaches to the “unknown” SAP of
11 Figure 6-4, and thus relays packets not recognized as belonging to Streams. The ~~Lower In-facing and Out-
12 facing~~ Stream identification functions separate FRER packets from non-FRER packets; the latter are relayed
13 across the NSTF as if the FRER capabilities were not present. ~~The Upper Stream identification functions
14 identify the FRER packets' Streams so that the other FRER functions can perform their tasks.~~ Each of the
15 two-port transfer functions have a pair of SAPs, and transfer all packet receive indications into packet
16 requests on the other SAP. This formulation illustrates exactly the peering relationships among the various
17 functions, but is too complex for many purposes.

18 Diagram c in Figure 6-5 shows a way of illustrating the function in diagram b in a more compact manner.
19 The operations to be performed can be more explicit than for diagram a, but the FRER sublayers are not at
20 the right peering level.

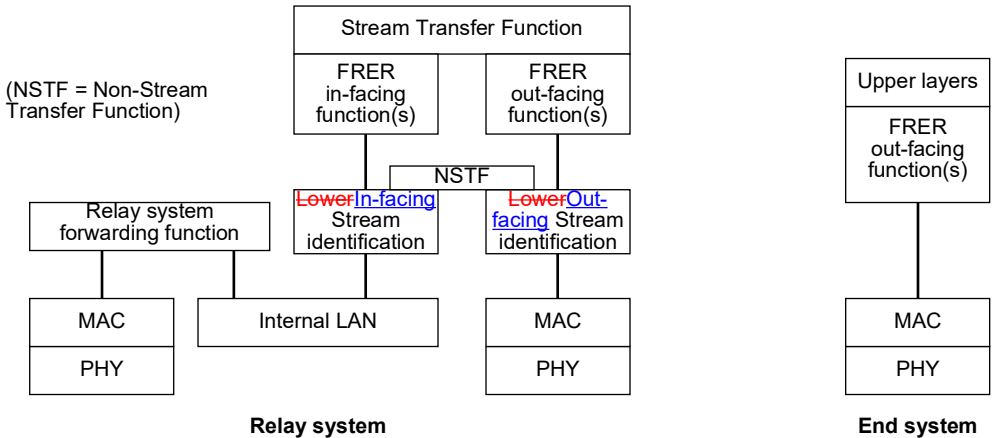
21 It is sometimes necessary to differentiate between functions that are on one side or the other of the expanded
22 port diagram (diagram b in Figure 6-5), Figure 6-6 illustrates a relay system with two ports, one of which
23 has in- and out-facing FRER functions, and an end system with one port, with out-facing FRER functions.
24 Every function communicates to its peers by sending and receiving packets through the real or virtual
25 service layers below it. Thus, in-facing functions are on the side of the Stream Transfer Function towards the
26 relay system's forwarding function, and out-facing functions are on either the side of the Stream Transfer
27 Function away from the forwarding function, or on a simple port, between the forwarding function and the
28 underlying real or virtual service layer. Typically, in-facing FRER functions are used for proxy functions
29 operating on behalf of end systems that are not FRER-capable. See Annex C for examples of the use of in-
30 and out-facing FRER functions. (The MAC and PHY layers are the Media Access Control and physical
31 layers, respectively.)

32 Change Figure 6-6 as shown:

1



2



3 6.4 Null Stream identification

4 *Change the text as shown:*

5 The Null Stream identification is a passive Stream identification function that operates at the frame level. It
6 can be defined using the Enhanced Internal Sublayer Service (EISS) described in 6.9 of
7 IEEE Std 802.1Q-2022, in which case it is enhanced with the extra stream_handle subparameter of the
8 connection_identifier, specified in 6.1 of the present standard. ~~It discards the stream_handle subparameter~~
9 ~~passed down the stack.~~ It generates a stream_handle subparameter on frames passed up the stack based on
10 the frame's destination MAC address and VLAN ID. It does not change any of a packet's other parameters.
11 It is suitable for applications in which all data packets to a particular {MAC address, VLAN} pair are
12 Stream packets. For example, AVB Streams (IEEE Std 802.1BA™-2011 [B2]) have a unique {destination
13 MAC address, VLAN} pair per Stream. In order to instantiate the Null Stream identification function, the
14 tsnStreamIdIdentificationType managed object (9.1.1.6) is encoded using the OUI (00-80-C2) and the type
15 values as shown in Table 9-1.

16 The managed objects for Null Stream identification are described in 9.1.2.

1 NOTE—The drop_eligible parameter is also present, along with the VLAN identifier and priority, in an IEEE 802.1Q
2 VLAN tag. FRER does not affect the use of this parameter. It passes through Null Stream identification unchanged, and
3 defaults to False when not present.

4 6.5 Source MAC and VLAN Stream identification

5 *Change the text as shown:*

6 The Source MAC and VLAN Stream identification is a passive Stream identification function that operates
7 at the frame level. It can be defined using the EISS described in 6.9 of IEEE Std 802.1Q™-2022, in which
8 case it is enhanced with the extra stream_handle subparameter of the connection_identifier, specified in 6.1
9 of the present standard. ~~It discards the stream_handle subparameter passed down the stack.~~ It generates a
10 stream_handle subparameter on frames passed up the stack based on the frame's source MAC address and
11 VLAN ID. It does not change any of a packet's parameters. It is suitable for applications in which all data
12 packets from a particular {source MAC address, VLAN} pair are Stream packets. In order to instantiate the
13 Source MAC and VLAN Stream identification function, the tsnStreamIdIdentificationType managed object
14 (9.1.1.6) is encoded using the OUI (00-80-C2) and the type values as shown in Table 9-1.

15 The managed objects for Source MAC and VLAN Stream identification are described in 9.1.3.

16 NOTE—The drop_eligible parameter is also present, along with the VLAN identifier and priority, in an IEEE 802.1Q
17 VLAN tag. FRER does not affect the use of this parameter. It passes through Source MAC and VLAN Stream
18 identification unchanged, and defaults to False when not present.

19 6.6 Active Destination MAC and VLAN Stream identification

20 The Active Destination MAC and VLAN Stream identification is an active Stream identification function
21 that operates at the frame level. It can be defined using the EISS described in 6.9 of IEEE Std 802.1Q-2022,
22 in which case it is enhanced with the extra stream_handle subparameter of the connection_identifier,
23 specified in 6.1 of the present standard. In order to instantiate the Active Destination MAC and VLAN
24 Stream identification function, the tsnStreamIdIdentificationType managed object (9.1.1.6) is encoded using
25 the OUI (00-80-C2) and the type values as shown in Table 9-1.

26 In the Active Destination MAC and VLAN Stream identification, the destination_address, vlan_identifier,
27 and priority parameters of the frame passed down the stack from the upper layers or up the stack from the
28 lower layers are replaced with alternate values. The replacement values for frames transmitted down the
29 stack to the Active Destination MAC and VLAN Stream identification, and used to recognize frames passed
30 up the stack to the Active Destination MAC and VLAN Stream identification function, ~~are those listed in~~
31 ~~9.1.2 as well as the replacement values for frames passed up the stack (not including the priority parameter)~~
32 ~~are described in 9.1.4. The replacement values for frames transmitted down the stack to the Active~~
33 ~~Destination MAC and VLAN Stream identification function are determined by the stream_handle~~
34 ~~subparameter passed down in the EISS request's connection_identifier.~~

35 Active Destination MAC and VLAN Stream identification is useful for translating a particular Stream,
36 within a Talker, to use a particular {MAC address, VLAN} pair to identify the Stream to IEEE 802.1Q
37 Bridges in the network, and within a Listener, to recover the original addressing information before passing
38 the packet up the protocol stack. It is also useful in a relay system that is providing that restoration as a proxy
39 service for a Listener.

40 The managed objects for Active Destination MAC and VLAN Stream identification are described in 9.1.4.

1 NOTE 1—The drop_eligible parameter is also present, along with the VLAN identifier and priority, in an IEEE 802.1Q
2 VLAN tag. FRER does not affect the use of this parameter. It passes through Active Destination MAC and VLAN
3 Stream identification unchanged, and defaults to False when not present.

4 NOTE 2—Changing the destination MAC address and/or VLAN must be done carefully, if the receiver is to recognize
5 the packet. For example, if Active Destination MAC and VLAN Stream identification is used along with IP Stream iden-
6 tification (6.7), the user can configure Active Destination MAC and VLAN Stream identification at the receiving end to
7 restore the original destination MAC address and VLAN before delivery up the protocol stack.

8 6.7 IP Stream identification

9 *Change the text as shown:*

10 The IP Stream identification is a passive Stream identification function that operates at the transport layer
11 and Internet Protocol (IP) interface layer. It can be defined using the union of the IP address primitives listed
12 in 9.1.5 and parameters defined by the EISS described in 6.9 of IEEE Std 802.1Q™-2022, in which case it is
13 enhanced with the stream_handle subparameter of the connection_identifier, specified in 6.1 of the present
14 standard. In IP Stream identification, the IP and higher layer address parameters and the EISS parameters are
15 used to determine the stream_handle subparameter of packets passed up the stack, ~~and discards the~~
16 ~~stream_handle subparameter for packets passed down the stack.~~ It does not change any of a packet's other
17 parameters. In order to instantiate the IP Stream identification function, the tsnStreamIdIdentificationType
18 managed object (9.1.1.6) is encoded using the OUI (00-80-C2) and the type values as shown in Table 9-1.

19 IP Stream identification can be coupled, for example, with Active Destination MAC and VLAN Stream
20 identification (6.6) to assign a particular {MAC address, VLAN, priority} triple to packets belonging to a
21 particular unicast IP Stream, as shown in Figure 8-1, Port A, where IP Stream identification would be in the
22 box labeled “Passive ~~Upper~~ Out-facing Stream identification functions (6-26.3).”

23 Practically, the IP Stream identification function requires locating the beginning of the IP packet
24 encapsulated in the frame, preceded by an IP EtherType field, e.g., 0x0800 or 0x86DD. The location of the
25 IP EtherType field within the frame is dependent on the combination of tag(s) following the addresses in a
26 frame. The method used to determine this location is beyond the scope of this standard.

27 The managed objects for IP Stream identification are described in 9.1.5.

28 NOTE—The drop_eligible parameter is also present, along with the VLAN identifier and priority, in an IEEE 802.1Q
29 VLAN tag. FRER does not affect the use of this parameter. It passes through IP Stream identification unchanged, and
30 defaults to False when not present.

31 6.8 Mask-and-match Stream identification

32 The Mask-and-match Stream identification is a passive Stream identification that operates at the frame level.
33 It can be defined using the Internal Sublayer Service (ISS) described in IEEE Std 802.1AC™, in which case
34 it is enhanced with the extra stream_handle subparameter of the connection_identifier, specified in 6.1 of the
35 present standard.

36 *Change the text as shown:*

37 ~~It discards the stream_handle subparameter passed down the stack.~~ It generates a stream_handle
38 subparameter on frames passed up the stack based on information extracted from a subset of the frame's ISS
39 parameters. It does not change any of a packet's parameters, other than the connection_identifier.

40 Input information for the Mask-and-match Stream identification function is obtained by masking one or
41 more of the following ISS parameters:

- 1 — source_address,
- 2 — destination_address, and
- 3 — the first N octets of mac_service_data_unit, N being a system-dependent variable.

4 Stream identification is then done by comparing these masked ISS parameters with specific match values.

5 The Mask-and-match Stream identification function supports Stream identification schemes ranging from
6 very simple, using a single mask applied to one of its input ISS parameters, to complex schemes involving
7 masks applied to all input ISS parameters, and in particular a specific mac_service_data_unit mask to
8 identify communication flows supported by various higher-layer protocols.

9 In order to instantiate the Mask-and-match Stream identification function, the
10 `tsnStreamIdIdentificationType` managed object (9.1.1.6) is encoded using the OUI (00-80-C2) and the type
11 values as shown in Table 9-1.

12 The managed objects for Mask-and-match Stream identification are described in 9.1.6.

13 ***Change the text as shown:***

14 Mask-and-match Stream identification can be coupled, for example, with Active Destination MAC and
15 VLAN Stream identification (6.6) to assign a particular {MAC address, VLAN, priority} triplet to packets
16 belonging to a particular unicast application Stream identified by the combination of the values of particular
17 fields in the MAC service data unit, as shown in Figure 8-1, Port A, where Mask-and-match Stream identifi-
18 cation would be in the box labeled “Passive ~~Upper~~Out-facing Stream identification functions (~~6-26.3~~).”

19

7. Frame Replication and Elimination for Reliability

7.1 Overview of Frame Replication and Elimination for Reliability

7.1.1 Goals and objectives

FRER, as specified in this clause, provides increased reliability (reduced packet loss rates) for a Stream by sequence numbering and replicating every packet, in the source end system and/or in relay systems in the network, and eliminating those replicates in the destination end system and/or in other relay systems.

Figure 7-1 illustrates an example of a Compound Stream with four component Member Streams. In this example, a sequence_number subparameter (6.1) is generated and encoded into each packet in the leftmost box. Sequence recovery functions (7.4.2) eliminate duplicate packets, and the non-duplicate packets copied as a new Member Stream (with sequence numbers unchanged), at two intermediate points. The final two Member Streams are brought together and the duplicates eliminated at the destination at right. This configuration protects against all 7 possible one-link failures, and against 16 of 21 possible two-link failures.

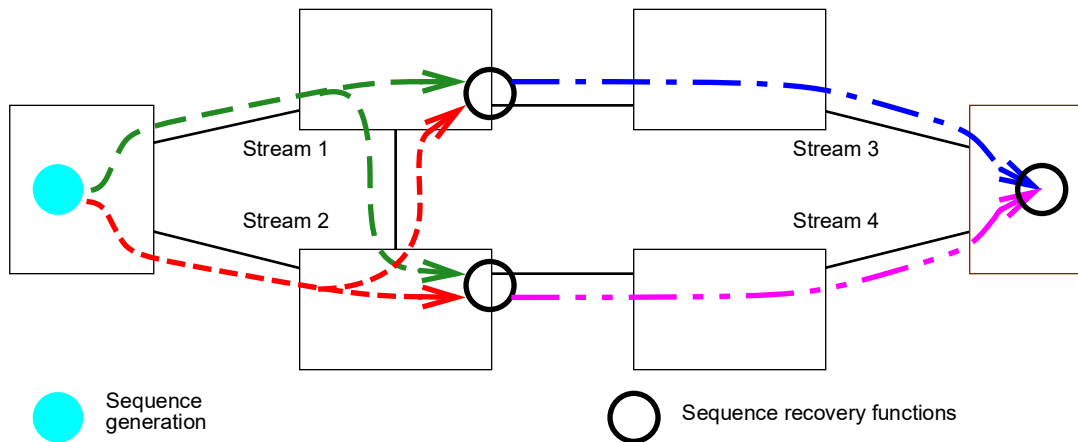


Figure 7-1—Compound Stream built from four Member Streams

FRER has the following goals and objectives:

- a) **Packet replication:** By replicating packets, sending them on separate paths, and then using the sequence_number to eliminate replicates, the effective probability of packet loss is reduced.

NOTE 1—The means by which the multiple paths are created is the subject of other standards, including IEEE Std 802.1Q™, IEEE Std 802.1Qca™, and IEEE P802.1Qcc™.¹⁶

- b) **Multicast or unicast:** A path on which a Stream is sent can be a point-to-point path or a point-to-multipoint tree.
- c) **Intermittent Streams:** A Stream such that there can be no more than one packet in flight on any given path than on any other path.
- d) **Bulk Streams:** A Stream such that there can be many more packets in flight on any given path than on any other path.

NOTE 2—An equivalent distinction between bulk and intermittent Streams is that, at the point at which the Streams are combined, the difference between the sequence numbers of an intermittent Stream along the two paths can be no greater than 1. Bulk Streams, for which the difference can be greater than 1, require a somewhat more complex replicate deletion capability than intermittent Streams.

¹⁶Numbers preceded by P are IEEE authorized standards projects that were not approved by the IEEE-SA Standards Board at the time this publication went to press. For information about obtaining drafts, contact the IEEE.

- 1 e) **Flexible positioning:** The FRER functions have to be usable in a number of configurations, as
2 explained in Annex C, including (but not limited to):
 - 3 1) In an end system's protocol stack.
 - 4 2) In a relay system's protocol stack (e.g., systems C and F in Figure C-4).
 - 5 3) In a relay system's protocol stack, where the FRER functions are acting in proxy for an
6 adjacent end system that lacks an FRER capability (e.g., system B in Figure C-4).
- 7 f) **Latent error detection:** When replicating a Stream, if a component along one path fails, it is not
8 immediately apparent to the receiver that something is wrong, since the other path continues to
9 deliver data. Given the higher reliability achievable with FRER, some means of detecting a failure to
10 actually deliver copies of each packet is required at the point that the replicated packets are
11 discarded.
- 12 g) **Interoperability:** There exist protocols antecedent to this standard that provide a similar function,
13 but have details that are different from this standard that would make it difficult or impossible to
14 design an interworking function that translates between the different standards' packet formats. This
15 standard defines a small number of controls that make such interoperation possible (see Annex B).
- 16 h) **Backward compatibility:** To increase the number of uses available for FRER, it is desirable for a
17 set of end systems conforming to this standard to obtain most or all of the benefits of FRER when
18 connected to a network that is not aware of FRER, and for a network of conformant relay systems to
19 offer these benefits to unaware end systems.
- 20 i) **Dynamic capability:** Some applications will need to turn FRER on or off on an operational Stream,
21 rather than only when the Stream is quiescent.
- 22 j) **Robustness:** At packet loss rates targeted by FRER, certain classes of errors, e.g., a stuck transmit-
23 ter repeatedly sending the same packet, become more important than, for example, simple packet
24 loss events. FRER has to be proof against such errors.
- 25 k) **Zero congestion loss:** Various techniques (e.g., IEEE 802.1BA-2021 [B2], Audio Video Bridging)
26 can be used to provide a Stream with zero (or very low) packet loss due to congestion. FRER has to
27 be compatible with such techniques.

28 NOTE 3—There are additional QoS features provided by Time-Sensitive Networking (TSN). FRER offers the most
29 benefit when used in combination with other features, including especially zero congestion loss.

- 30 l) **Ease of use:** It must be possible to employ FRER without per-Stream configuration in each relay
31 system.

32 In addition to the above goals, there are other possible capabilities that are explicitly not goals of this
33 standard:

- 34 m) **In-order delivery:** FRER can cause packets belonging to a Compound Stream to be delivered in a
35 sequence other than that in which they were transmitted. Rectifying this is beyond the scope of this
36 standard.

37 If the paths of the Member Streams comprising a Compound Stream have significantly different lengths, and
38 if the Streams are bulk Streams (item d), then a lost packet on the shorter path can easily give rise to an out-
39 of-order delivery when its duplicate arrives via the longer path, even though in-order delivery is maintained
40 for each Member Stream along its own path. Delivering packets in their original order for the Compound
41 Stream, as indicated by the sequence numbers, would require buffering the packets. Because of this
42 buffering requirement, in-order delivery for a Compound Stream is explicitly not a goal of FRER. Users of
43 Bridged Virtual LANs, as defined in IEEE Std 802.1Q™, need to understand that the usual expectations of
44 in-order delivery can be compromised by FRER.

45 FRER can be configured to apply recursively, so that a Compound Stream composed of Member Streams
46 can itself be a Member Stream to an enclosing Compound Stream.

7.2 Use of the term *Stream*

Readers of this standard who are familiar with IEEE Std 802.1Q™ will recognize that the distinction between a Member Stream and a Compound Stream is required by the present standard. For example, in IEEE Std 802.1Q, the Stream is both the entire Talker-to-Listener path and that for which a bandwidth reservation must be made before the Stream can obtain the desired QoS. It is expected that bandwidth reservations using the mechanisms of IEEE Std 802.1Q™ could be made for each of the four Member Streams illustrated in Figure 7-1, while the end-to-end Talker-to-Listener path is the whole Compound Stream. (See C.9.)

In addition, IEEE Std 802.1Q™ defines a StreamID that is used to identify a stream between a Talker and one or more Listener(s). In contrast, the present standard defines a stream_handle subparameter that is used internally to identify a Stream.

Some end systems sequence number their packets per system, rather than per-Stream. Such an end system can transmit any number of separate Streams (in the sense of data flows), each with different destination_mac_address, IP address, TCP port numbers, VLANs, etc., each with its own entries in a relay system's forwarding database and/or each with its own bandwidth reservation. In receiving end systems or in relay systems, Source MAC and VLAN Stream identification (6.5) is useful for organizing Individual recovery functions and Sequence recovery functions, because the sequence_number parameter is per-source MAC address, not per-destination MAC address; each state machine can operate on all of the individual Streams transmitted by a single source as a single Stream. Therefore, any number of individual Streams (in the bandwidth reservation sense) can be considered a single Stream (in the source MAC address sense) by different systems in a network. See also B.2.

7.3 Frame Replication and Elimination for Reliability functions

FRER provides five of the functions illustrated in Figure 7-2. Not all functions are required in every protocol stack.

Upper layer
Sequencing function (7.4)
Stream splitting function (7.7)
Individual recovery function (7.5)
Sequence encode/decode function (7.6)
Stream identification (Clause 6)
Lower layer

Figure 7-2—Frame Replication and Elimination for Reliability functions

- a) **Sequencing:** This function (see 7.4):
- Supplies a sequential value for the sequence_number subparameter for packets passed down from the upper layer for transmission (7.4.1);
 - Examines the sequence numbers of received packets passed up to it (from multiple Streams), and discards packets whose sequence_number subparameter indicates that it is a duplicate of a packet received previously (7.4.2); and
 - Monitors counter variables to detect latent errors of Streams passed up to it (7.4.4).

- 1 b) **Stream splitting:** This function (see 7.7):
 - 2 1) Replicates each packet passed down to it, assigning each replicate a different stream_handle, at
 - 3 most one of which can be the same as the original.
 - 4 2) Passes packets unchanged up the stack.
- 5 c) **Individual recovery:** This function (see 7.5):
 - 6 1) Examines the sequence numbers of received packets passed up to it (belonging to Member
 - 7 Streams), and discards packets whose sequence_number subparameter indicates that it is a
 - 8 duplicate of a packet received previously.
- 9 d) **Sequence Encode/Decode:** This function (see 7.6):
 - 10 1) Encodes the sequence_number subparameter into the packet in a manner such that it can be
 - 11 decoded by its peer Sequencing function(s) by altering the other packet parameters, typically
 - 12 by encoding the sequence_number in the packet data by some means, e.g., an R-TAG (7.8).
 - 13 2) Extracts the sequence_number from a received packet passed up to it. Depending on the Stream
 - 14 identification function used, the Sequencing function can remove the sequence_number
 - 15 encapsulation from the packet.
 - 16 The Sequence encode/decode function is not required if the Stream identification function used in
 - 17 the stack encodes all of the required subparameters.

18 FRER also requires the Stream identification function (Clause 6):

- 19 e) **Stream identification:** This function (see Clause 6):
 - 20 1) Passes each packet down the stack and, if required by the particular encapsulation method
 - 21 configured, uses the stream_handle to alter the packet; and
 - 22 2) Derives a stream_handle from a received packet and passes it up the stack, altering the packet if
 - 23 required by the encapsulation method.

24 7.4 Sequencing function

25 The Sequencing function makes use of the stream_handle and sequence_number service subparameters of
 26 6.1. The Sequencing function has two kinds of component functions. Sequence generation functions (7.4.1)
 27 operate on packets passed down the protocol stack towards the physical layer in Figure 7-2 and generate a
 28 value for the sequence_number subparameter. Sequence recovery functions (7.4.2) operate on packets
 29 passed up the stack towards the higher layer functions and use the sequence_number subparameter to decide
 30 which packets to pass and which to discard.

31 On any given port, zero or more instances of the Sequence generation function, and/or Sequence recovery
 32 function can be instantiated. These functions can be instantiated as in-facing or out-facing functions. For
 33 both kinds of functions, the stream_handle subparameter of a packet determines through which function, if
 34 any, the packet is passed. Each instance of each function has its own set of state variables.

35 7.4.1 Sequence generation function

36 The sequence generation function resides in the Sequencing function (7.4). It generates a sequence_number
 37 subparameter for each packet of a Stream passed down to the lower layers. There is at most one sequence
 38 generation function per port per stream_handle value per direction (in-facing or out-facing), as specified
 39 through managed objects (10.3).

40 The sequence generation function is described in terms of the following:

- 1 a) Managed objects that control the sequence generation function and that are affected by the sequence
2 generation function (10.3, 10.8);
- 3 b) Events, the occurrence of which trigger the execution of code routines, and that can be triggered by
4 the code routines (7.4.1.1);
- 5 c) Variables that are manipulated by the code routines and maintain the state of the sequence
6 generation function between routine executions (7.4.1.2); and
- 7 d) Two code routines that interact with the managed objects, events, and variables (7.4.1.4, 7.4.1.3).

8 7.4.1.1 Events for sequence generation

9 There are two events that can affect the sequence generation function, and one that it produces.

- 10 a) **BEGIN:** The global event that resets all functions, including the sequence generation function. This
11 event triggers the execution of the SequenceGenerationReset routine (7.4.1.3).
- 12 b) **DATA_REQUEST:** The event that presents a packet from the upper layers to the sequence
13 generation function for transmission to the lower layers, e.g., the M_UNITDATA.request primitive
14 of the ISS of IEEE Std 802.1AC™. This event triggers the execution of the
15 SequenceGenerationAlgorithm routine (7.4.1.4).
- 16 c) **SEND_DATA:** The event that presents a packet down the stack from the sequence generation
17 function to the lower layers, e.g., the M_UNITDATA.request primitive of the ISS of
18 IEEE Std 802.1AC.

19 7.4.1.2 Variables for sequence generation

20 Each instance of the sequence generation function has its own set of variables, independent from any other
21 instance.

22 7.4.1.2.1 GenSeqSpace

23 GenSeqSpace specifies the range of values for the sequence_number subparameter and other variables that
24 depend on it. It is a constant with the value 65 536.

25 7.4.1.2.2 GenSeqNum

26 The GenSeqNum variable contains the value that the sequence generation function increments after passing
27 down to the lower layer as the sequence_number subparameter. The variable is an unsigned integer in the
28 range 0 to (GenSeqSpace – 1). GenSeqSpace is defined in 7.4.1.2.1. GenSeqNum is initialized to 0
29 whenever the function is reset (7.4.1.3), and incremented by 1 after its value is copied to the
30 sequence_number subparameter. When incremented past its maximum value, the new value is 0.

31 7.4.1.3 SequenceGenerationReset

32 The SequenceGenerationReset function is called whenever the BEGIN event occurs (item a in 7.4.1.1) or
33 the value True is written to the **frerSeqGenReset** managed object (10.3.1.3). It resets GenSeqNum (7.4.1.2.2)
34 and increments frerCpsSeqGenResets (10.8.2).

```
35 void SequenceGenerationReset () {
36     GenSeqNum = 0;
37     frerCpsSeqGenResets = frerCpsSeqGenResets + 1;
38 }
```

39 **NOTE** —SequenceGenerationReset can cause packet drop.

1 7.4.1.4 SequenceGenerationAlgorithm

2 The SequenceGenerationAlgorithm function is called whenever the DATA_REQUEST (item b in 7.4.1.1)
 3 event occurs. It copies GenSeqNum (7.4.1.2.2) to the sequence_number subparameter (item b in 6.1) and
 4 increments GenSeqNum. It then triggers the SEND_DATA primitive (item c in 7.4.1.1) to pass the packet
 5 on to the next lower layer.

```

6 void SequenceGenerationAlgorithm () {
7     unsigned int sequence_number = GenSeqNum;
8     if (GenSeqNum >= (GenSeqSpace - 1))
9         GenSeqNum = 0;
10    else
11        GenSeqNum = GenSeqNum + 1;
12    SEND_DATA; // Pass packet & sequence_number subparameter down stack
13 }

```

14 7.4.2 Sequence recovery function

15 The Sequence recovery function operates on a merged set of Member Streams originally marked with
 16 sequence_number (6.1) values from a single instance of the Sequence generation function (7.4.1). It is not to
 17 be confused with the Individual recovery function (7.5) that applies to only one Member Stream, although
 18 both use the same basic algorithms.

19 An instantiation of the Sequence recovery function consists of the following:

- 20 a) An instantiation of the Base recovery function (7.4.3), either the VectorRecoveryAlgorithm (7.4.3.4)
 21 or the MatchRecoveryAlgorithm (7.4.3.5), with its frerSeqRevyIndividualRecovery object
 22 (10.4.1.10) set to False, configured to apply to one or more values of the sequence_number
 23 subparameter; and
- 24 b) An instantiation of the Latent error detection function (7.4.4).

25 Managed objects (Clause 9 and Clause 10) are used to instantiate these functions on ports of a system.

26 NOTE—The managed objects in Clause 9 and Clause 10 create an instantiation of the Sequence recovery function per
 27 port. In fact, a relay system can choose to create a single instance of the Sequence recovery function as part of its
 28 forwarding function, or one instance per line card, or in some other, distributed fashion, without violating the externally
 29 visible behaviors specified by this standard. The per-instantiation variables are internal to the system, so their definitions
 30 are not affected by this choice. See 10.8.1 for a discussion of what this choice means for the counters in Clause 9 and
 31 Clause 10 that are referenced by instantiations of the Sequence recovery function.

32 7.4.3 Base recovery function

33 A Base recovery function resides in the Sequencing function (7.4). It evaluates the sequence_number
 34 subparameter of a packet of one or more Member Streams passed up from the lower layers, in order to
 35 discard duplicated packets. A given instantiation of a Base recovery function can function as either a
 36 Sequence recovery function (7.4.2) or an Individual recovery function (7.5). This choice, the selection of
 37 stream_handle subparameter value(s) to which an instantiation applies, the port and direction (in-facing or
 38 out-facing) on which it resides, and other operational parameters are specified through managed objects
 39 (10.4).

40 The sequence recovery function is described in terms of the following:

- 41 a) Managed objects that control the sequence recovery function and that are affected by the sequence
 42 recovery function (10.4, 10.8);

- 1 b) Events, the occurrence of which trigger the execution of code routines and which can be triggered by
- 2 the code routines (7.4.3.1);
- 3 c) Variables that are manipulated by the code routines and maintain the state of the sequence recovery
- 4 function between routine executions (7.4.3.2); and
- 5 d) Four code routines that interact with the managed objects, events, and variables (7.4.3.3, 7.4.3.4,
- 6 7.4.3.5, 7.4.3.6).

7 The sequence recovery function has two sequence recovery algorithms (7.4.3.4, 7.4.3.5) so that it can be
8 used either for Intermittent Streams (item c in 7.1.1) or for Bulk Streams (item d in 7.1.1). As long as the
9 maximum difference in the number of packets in flight among all of the paths taken by the Member Streams
10 served by a given sequence recovery function does not exceed the size of the SequenceHistory variable
11 (7.4.3.2.2, 10.4.1.6), the packets are delivered without duplication, but can be delivered out-of-order. ~~If the~~
12 ~~difference exceeds this value, then duplicate packets can be delivered~~If the difference exceeds this value,
13 then packets can be discarded.

14 NOTE 1—The requirements for conformance to this standard, particularly item c in 5.9 and item e in 5.12, specify that
15 the sequence recovery function need only support a minimum difference in path length for the correct elimination of
16 duplicates in Bulk Streams. ~~If the actual path difference in a given network exceeds the capability of a sequence~~
17 ~~recovery function, then duplicate packets will be delivered~~If the actual path difference in a given network exceeds the
18 capability of a sequence recovery function, then packets will be discarded. It is up to the user to see that the capabilities
19 of the systems purchased match the needs of the particular network in which they are employed.

20 An FRER recovery function shall not process a packet unless the lower-level packet validity checks (e.g.,
21 IEEE Std 802.3™ [B4], Frame Check Sequence) have been completed successfully.

22 NOTE 2—This restriction has the effect of disallowing cut through forwarding on a port on which the FRER recovery
23 function is discarding packets.

24 7.4.3.1 Events for sequence recovery

25 There are three events that can affect the sequence recovery function, and one that the Base recovery
26 function can trigger.

- 27 a) **BEGIN:** The global event that resets all functions, including the sequence recovery function. This
- 28 event triggers the execution of the SequenceRecoveryReset routine (7.4.3.3).
- 29 b) **DATA_INDICATION:** The event that presents a packet up the stack to the sequence recovery
- 30 function from the lower layers, e.g., the M_UNITDATA.indication primitive of the ISS of
- 31 IEEE Std 802.1AC.
- 32 c) **RECOVERY_TIMEOUT:** The event that occurs when the RemainingTicks (7.4.3.2.4) variable
- 33 reaches 0. It triggers the execution of the SequenceRecoveryReset routine (7.4.3.3).
- 34 d) **PRESENT_DATA:** The event that presents a packet up the stack from the sequence recovery func-
- 35 tion to the upper layers, e.g., the M_UNITDATA.indication primitive of the ISS of
- 36 IEEE Std 802.1AC. This event can be triggered by the VectorRecoveryAlgorithm routine (7.4.3.4)
- 37 and the MatchRecoveryAlgorithm routine (7.4.3.5).

38 7.4.3.2 Variables for sequence recovery

39 Each instance of the sequence recovery function has its own set of variables, independent from any other
40 instance.

41 7.4.3.2.1 RecovSeqSpace

42 RecovSeqSpace specifies the range of values for the sequence_number subparameter and other variables
43 that depend on it. It is a constant with the value 65 536.

1 7.4.3.2.2 SequenceHistory

2 The SequenceHistory variable maintains a history of the sequence_number subparameters of recently
 3 received packets. The SequenceHistory variable is a bit vector, with one bit for each value from 0 to
 4 ($\text{frerSeqRcvyHistoryLength} - 1$), corresponding to sequence_numbers in the range RecovSeqNum
 5 (7.4.3.2.3) through ($\text{RecovSeqNum} - \text{frerSeqRcvyHistoryLength} + 1$), inclusive, with the arithmetic in both
 6 expressions performed modulo $\text{frerSeqRcvyHistoryLength}$. A 1 in the SequenceHistory indicates that the
 7 corresponding sequence_number has been received, and a 0 that it has not.

8 7.4.3.2.3 RecovSeqNum

9 The RecovSeqNum variable holds the highest sequence_number value received (modulo RecovSeqSpace),
 10 or the value ($\text{RecovSeqSpace} - 1$), if none have been received since the sequence recovery function was
 11 reset. The variable is an unsigned integer in the range 0 to ($\text{RecovSeqSpace} - 1$). RecovSeqSpace is defined
 12 in 7.4.3.2.1. RecovSeqNum is initialized to ($\text{RecovSeqSpace} - 1$) whenever the function is reset (7.4.3.3).
 13 When incremented past its maximum value, the new value is 0.

14 7.4.3.2.4 RemainingTicks

15 An unsigned integer variable that holds the number of clock ticks remaining until a
 16 RECOVERY_TIMEOUT event (item c in 7.4.3.1) occurs. It is decremented regularly, at the rate indicated
 17 by TicksPerSecond (7.4.3.2.5), and can be reset by the VectorRecoveryAlgorithm (7.4.3.4) and the
 18 MatchRecoveryAlgorithm (7.4.3.5). When RemainingTicks decrements from 1 to 0, a
 19 RECOVERY_TIMEOUT event occurs, and RemainingTicks remains at 0.

20 7.4.3.2.5 TicksPerSecond

21 An unsigned integer value, supplied by the implementation, that indicates the number of times per second
 22 that the variable RemainingTicks (7.4.3.2.4) is decremented. The value of TicksPerSecond shall be 100 or
 23 greater.

24 7.4.3.2.6 TakeAny

25 A Boolean value indicating whether the recovery algorithm (7.4.3.4 or 7.4.3.5) is to accept the next packet,
 26 no matter what the value of its sequence_number subparameter.

27 7.4.3.3 SequenceRecoveryReset

28 SequenceRecoveryReset is called whenever the BEGIN event (item a in 7.4.3.1) or the
 29 RECOVERY_TIMEOUT event (item c in 7.4.3.1) occurs. It resets the RecovSeqNum (7.4.3.2.3) and
 30 SequenceHistory (7.4.3.2.2) variables to their initial states, increments $\text{frerCpsSeqRcvyResets}$ (10.8.9), and
 31 sets TakeAny (7.4.3.2.6). Note that RecovSeqNum and SequenceHistory are reset only if the
 32 VectorRecoveryAlgorithm (7.4.3.4) is configured.

```

33 void SequenceRecoveryReset (
34     if (frerSeqRcvyAlgorithm == Vector_Alg) {
35         int i;
36         RecovSeqNum = RecovSeqSpace - 1;
37         for (i = 0; i < frerSeqRcvyHistoryLength; i = i + 1)
38             SequenceHistory[i] = 0; // Set all bits 0 (packet not seen)
39     }
40     frerCpsSeqRcvyResets = frerCpsSeqRcvyResets + 1;
41     TakeAny               = true;
42 }

```

1 NOTE—Reset of Sequence recovery function may cause temporary duplicate delivery.

2 7.4.3.4 VectorRecoveryAlgorithm

3 VectorRecoveryAlgorithm is called whenever the DATA_INDICATION event occurs (item b in 7.4.3.1)
4 and the Sequence recovery function is configured to use the VectorRecoveryAlgorithm
5 (frerSeqRcvyAlgorithm = **Vector_Alg**, 10.4.1.5). If it terminates without triggering the PRESENT_DATA
6 event (item d in 7.4.3.1), then the packet is effectively discarded. Each instance of the function is controlled
7 by the constant RecovSeqSpace (7.4.3.2.1) and the managed object frerSeqRcvyHistoryLength (10.4.1.6).

8 ~~One can observe that if the managed object frerSeqRcvyHistoryLength contains the value 1, then this
9 algorithm is suitable for Intermittent Streams (item c in 7.1.1). In that case, the SequenceHistory variable
10 (7.4.3.2.2) merely records whether the RecovSeqNum (7.4.3.2.3) does or does not record the
11 sequence_number subparameter of a received packet. [It does not immediately after
12 SequenceRecoveryReset (7.4.3.3) is called.] If frerSeqRcvyHistoryLength contains a value greater than one,
13 VectorRecoveryAlgorithm serves as the more complex algorithm suitable for Bulk Streams (item d in
14 7.1.1).~~

15 Immediately after SequenceRecoveryReset (7.4.3.3) is called, the VectorRecoveryAlgorithm accepts the
16 first packet received as valid. After the first packet has been accepted, ~~all subsequent packets that are in the
17 window last packet number accepted + frerSeqRcvyHistoryLength are accepted, and all subsequent packets
18 that are in the recovery window (i.e., last packet number accepted - frerSeqRcvyHistoryLength + 1 to last
19 packet number accepted + frerSeqRcvyHistoryLength) are accepted, and those packets with
20 sequence_number values outside that range are discarded.~~ Each packet accepted and passed up the stack
21 resets the timer variable RemainingTicks (7.4.3.2.4). If that variable ticks down to 0, meaning that no packet
22 has been accepted in frerSeqRcvyResetMSec milliseconds (10.4.1.7), then SequenceRecoveryReset again
23 resets the algorithm, and the next packet received is accepted.

24 This timeout mechanism means that:

- 25 a) ~~“Rogue” packets, meaning packets outside the frerSeqRcvyHistoryLength window, are discarded as~~
26 ~~invalid~~ “Rogue” packets, meaning packets outside the recovery window, are discarded as invalid.
- 27 b) If a Base recovery function somehow gets out of step with its corresponding Sequence generation
28 function, then after frerSeqRcvyResetMSec milliseconds, the Base recovery function will be reset
29 and data will again be passed.
- 30 c) If a Sequence generation function is reset, perhaps by rebooting a system, then Base recovery
31 functions that have not been reset are likely to discard packets until the timeout has occurred.
- 32 d) If a Talker ~~or a relay system~~ fails in such a way as to repeatedly transmit packets with the same
33 sequence_number subparameter (perhaps repeating exactly the same packet), ~~those packets will~~
34 ~~continue to be discarded, at least until the sequence_number wraps around~~ then in case of Individual
35 Recovery (7.5), all repeated packets are discarded. If only Sequence Recovery (7.4.2) is used, then
36 all except one repeated packets are discarded per RemainingTicks (7.4.3.2.4) interval. If a relay
37 system fails the same way, then in case of Individual Recovery (7.5), all packets are discarded. If
38 only Sequence Recovery (7.4.2) is used and if there is a correct member stream, then the failed
39 member stream pollutes the correct member stream.

40 NOTE—Individual recovery functions (7.5) can be configured to discard repeated sequence_number values (or
41 sequences of values) and prevent even this wrap around problem.

42 As a result of item d, it is advisable for a Listener or relay system to be configured to discard packets in a
43 single Member Stream. See 7.5 and C.10.

```
44 void VectorRecoveryAlgorithm () {
45     // Check that sequence number is present in the packet
46     unsigned int sequence_number;
```

```

1  if (sequence_number == frerSeqRcvyInvalidSequenceValue) {
2      frerCpsSeqRcvyTaglessPackets = frerCpsSeqRcvyTaglessPackets + 1;
3      if (frerSeqRcvyTakeNoSequence) {
4          frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
5          frerCpSeqRcvyPassedPackets = frerCpSeqRcvyPassedPackets + 1;
6          RemainingTicks =
7              ((frerSeqRcvyResetMSec*TicksPerSecond)+999)/1000;
8          PRESENT_DATA;
9      } else {
10         frerCpsSeqRcvyDiscardedPackets =
11             frerCpsSeqRcvyDiscardedPackets + 1;
12         frerCpSeqRcvyDiscardPackets = frerCpSeqRcvyDiscardPackets + 1;
13     }
14     return;
15 }
16 // Compute signed difference modulo RecovSeqSpace.
17 int delta = (sequence_number-RecovSeqNum) & (RecovSeqSpace - 1);
18 if (0 != (delta & (RecovSeqSpace/2)))
19     delta = delta - RecovSeqSpace;
20 // Here, -(RecovSeqSpace/2)<=delta<=((RecovSeqSpace/2)-1)
21 // After reset, accept any packet
22 if (TakeAny) {
23     TakeAny = false;
24     SequenceHistory[0] = 1; // Shift, adding a "seen" bit
25     RecovSeqNum = sequence_number;
26     frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
27     frerCpSeqRcvyPassedPackets = frerCpSeqRcvyPassedPackets + 1;
28     RemainingTicks =
29         ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
30     PRESENT_DATA;
31 } else if (delta > frerSeqRcvyHistoryLength ||
32     delta <= -frerSeqRcvyHistoryLength )
33 {
34     // Packet is out-of-range. Count and discard it.
35     frerCpsSeqRcvyRoguePackets = frerCpsSeqRcvyRoguePackets + 1;
36     frerCpSeqRcvyDiscardPackets = frerCpSeqRcvyDiscardPackets + 1;
37     // Reset timer if working on an individual Stream
38     if (frerSeqRcvyIndividualRecovery)
39         RemainingTicks =
40             ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
41 } else if (delta <= 0) {
42     // Packet is old and in SequenceHistory; have we seen it before?
43     if (0 == SequenceHistory[-delta]) {
44         // Packet has not been seen. Take it.
45         SequenceHistory[-delta] = 1;
46         frerCpsSeqRcvyOutOfOrderPackets =
47             frerCpsSeqRcvyOutOfOrderPackets + 1;
48         frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
49         frerCpSeqRcvyPassedPackets = frerCpSeqRcvyPassedPackets + 1;
50         RemainingTicks =
51             ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
52         PRESENT_DATA;
53     } else {
54         // Packet has been seen. Do not forward. Count the discard.

```

```

1      frerCpsSeqRcvyDiscardedPackets =
2          frerCpsSeqRcvyDiscardedPackets + 1;
3      frerCpSeqRcvyDiscardPackets = frerCpSeqRcvyDiscardPackets + 1;
4      // Reset timer if working on an individual Stream
5      if (frerSeqRcvyIndividualRecovery)
6          RemainingTicks =
7              ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
8      }
9  } else {
10     // Packet is not too far ahead of the one we want.
11     // Packet is out-of-order unless it directly follows RecovSeqNum
12     if (delta != 1)
13         frerCpsSeqRcvyOutOfOrderPackets =
14             frerCpsSeqRcvyOutOfOrderPackets + 1;
15     // Shift the history until bit 0 refers to sequence_number.
16     while (0 != (delta = delta - 1))
17         ShiftSequenceHistory(0); // Shift, adding a "not seen" bit
18         ShiftSequenceHistory(1); // Shift, adding a "seen" bit
19     RecovSeqNum = sequence_number;
20     frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
21     frerCpSeqRcvyPassedPackets = frerCpSeqRcvyPassedPackets + 1;
22     RemainingTicks =
23         ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
24     PRESENT_DATA;
25 }
26 }

```

27 7.4.3.5 MatchRecoveryAlgorithm

28 MatchRecoveryAlgorithm is called whenever the DATA_INDICATION event occurs (item b in 7.4.3.1)
29 and the Sequence recovery function is configured to use the MatchRecoveryAlgorithm
30 (frerSeqRcvyAlgorithm = **Match_Alg**, 10.4.1.5). If it terminates without triggering the PRESENT_DATA
31 event (item d in 7.4.3.1), then the packet is effectively discarded.

32 Immediately after SequenceRecoveryReset (7.4.3.3) is called, the MatchRecoveryAlgorithm accepts the
33 first packet received as valid. After the first packet has been accepted, all subsequent packets either match
34 the last packet number accepted, and are therefore discarded, or do not, in which case they are accepted.
35 Each packet accepted and passed up the stack resets the timer variable RemainingTicks (7.4.3.2.4). **Dropped**
36 **packets also reset the timer variable RemainingTicks (7.4.3.2.4) in case of Individual recovery.** If that
37 variable ticks down to 0, meaning that no packet has been accepted in frerSeqRcvyResetMSec milliseconds
38 (10.4.1.7), then SequenceRecoveryReset again resets the algorithm, and the next packet received is
39 accepted.

40 **The timer mechanism prevents the MatchRecoveryAlgorithm from getting stuck forever, blocking packet 1,**
41 **in case a Talker fails or is reset soon after initialization.**

42 If a Talker or a relay system fails in such a way as to repeatedly transmit packets with the same
43 sequence_number subparameter (perhaps repeating exactly the same packet), those packets will continue to
44 be discarded, at least until the sequence_number wraps around. At that point, either the stuck packet or the
45 right packet could be passed. Therefore, it is advisable for a Listener or relay system to be configured to
46 discard packets in a single Member Stream. See 7.5 and C.10.

47 NOTE—Individual recovery functions (7.5) can be configured to discard repeated sequence_number values (or
48 sequences of values) and prevent even this wrap around problem.

```

1 void MatchRecoveryAlgorithm () {
2     // Check that sequence number is present in the packet
3     unsigned int sequence_number;
4     if (sequence_number == frerSeqRcvyInvalidSequenceValue) {
5         frerCpsSeqRcvyTaglessPackets = frerCpsSeqRcvyTaglessPackets + 1;
6         frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
7         frerCpSeqRcvyPassedPackets = frerCpSeqRcvyPassedPackets + 1;
8         PRESENT_DATA;
9         return;
10    }
11    // After reset, accept any packet
12    if (TakeAny) {
13        TakeAny = false;
14        RecovSeqNum = sequence_number;
15        frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
16        frerCpSeqRcvyPassedPackets = frerCpSeqRcvyPassedPackets + 1;
17        RemainingTicks =
18            ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
19        PRESENT_DATA;
20        return;
21    }
22    // Compute signed difference modulo RecovSeqSpace.
23    int delta = (sequence_number - RecovSeqNum) & (RecovSeqSpace - 1);
24    if (delta == 0) {
25        // Packet has been seen. Do not forward. Count the discard.
26        frerCpsSeqRcvyDiscardedPackets =
27            frerCpsSeqRcvyDiscardedPackets + 1;
28        frerCpSeqRcvyDiscardPackets = frerCpSeqRcvyDiscardPackets + 1;
29        // Reset timer if working on an individual Stream
30        if (frerSeqRcvyIndividualRecovery)
31            RemainingTicks =
32                ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
33    } else {
34        // Packet has not been seen; accept it.
35        // Packet is out-of-order unless it directly follows RecovSeqNum
36        if (delta != 1)
37            frerCpsSeqRcvyOutOfOrderPackets =
38                frerCpsSeqRcvyOutOfOrderPackets + 1;
39        RecovSeqNum = sequence_number;
40        frerCpsSeqRcvyPassedPackets = frerCpsSeqRcvyPassedPackets + 1;
41        frerCpSeqRcvyPassedPackets = frerCpSeqRcvyPassedPackets + 1;
42        RemainingTicks =
43            ((frerSeqRcvyResetMSec * TicksPerSecond) + 999) / 1000;
44        PRESENT_DATA;
45    }
46 }

```

47 7.4.3.6 ShiftSequenceHistory

48 This routine is called by the VectorRecoveryAlgorithm routine (7.4.3.4) to advance the SequenceHistory bit
49 array (7.4.3.2.2) and to count lost packets (frerCpsSeqRcvyLostPackets, 10.8.7). ShiftSequenceHistory
50 takes one parameter, which is the new value for index 0 in the SequenceHistory bit array.

```

51 void ShiftSequenceHistory (int newZeroValue) {

```



```

1  int i;
2  if (0 == SequenceHistory[frerSeqRcvyHistoryLength - 1])
3      frerCpsSeqRcvyLostPackets = frerCpsSeqRcvyLostPackets + 1;
4  for (i = frerSeqRcvyHistoryLength - 1; i != 0; i = i - 1)
5      SequenceHistory[i] = SequenceHistory[i - 1];
6  SequenceHistory[0] = newZeroValue;
7 }

```

8 7.4.4 Latent error detection function

9 Each instance of a Latent error detection function is part of a Sequence recovery function (7.4.2). It monitors
10 the managed objects associated with a single instance of the Base recovery function (7.4.3), in order to
11 detect the condition that relatively few packets are being discarded by that function. Latent error detection
12 operates on the assumption that, in a properly functioning Compound Stream employing n paths into the
13 current system, there will be $n - 1$ packets discarded for every packet passed through the Base recovery
14 function (7.4.3). The latent error detection function issues a `SIGNAL_LATENT_ERROR` (item d in 7.4.4.1)
15 when that assumption is violated.

16 There is at most one latent error detection function per port per stream_handle value per direction (in-facing
17 or out-facing), as specified through the managed objects (10.4) that control both functions. For every Base
18 recovery function configured, there can be only zero or one associated latent error detection function. Any
19 implementation that supports the Base recovery function shall support the Latent error detection function.

20 Latent error detection is accomplished by two periodic functions. The first (`LatentErrorTest`, 7.4.4.4)
21 examines the number of packets passed and discarded, and reports a latent error if the differences among
22 those counters exceed a set threshold. The other periodic function (`LatentErrorReset`, 7.4.4.3) resets the
23 variables used by the first, so that occasional random packet losses do not accumulate forever. These
24 functions are driven by timers, not by the receipt or transmission of packets.

25 The latent error detection function is described in terms of the following:

- 26 a) Managed objects that control the latent error detection function and that are affected by the latent
27 error detection function (10.4, 10.8);
- 28 b) Events, the occurrence of which trigger the execution of code routines, and that can be triggered by
29 the code routines (7.4.4.1);
- 30 c) Variables that are manipulated by the code routines, and maintain the state of the latent error
31 detection function between routine executions (7.4.4.2); and
- 32 d) Two code routines that interact with the managed objects, events, and variables (7.4.4.3, 7.4.4.4).

33 NOTE—The latent error detection algorithm (7.4.4.3, 7.4.4.4) cannot catch all errors. For example, if one path
34 erroneously produces twice the proper number of copies and the other path fails, `LatentErrorTest` will generate no error.
35 For this reason, the `LatentErrorTest` should be used in conjunction with Individual recovery functions (7.5).

36 7.4.4.1 Events for latent error detection

37 There are three events that can affect the latent error detection function:

- 38 a) **BEGIN:** The global event that resets all functions, including the latent error detection function. This
39 event triggers the execution of the `LatentErrorReset` routine (7.4.4.3).
- 40 b) **RESET_LATENT_ERROR:** This event is generated periodically and is controlled by
41 `frerSeqRcvyLatentResetPeriod` (10.4.1.12.4). It triggers the execution of the `LatentErrorReset`
42 routine (7.4.4.3).

1 c) **TRIGGER_LATENT_ERROR_TEST:** This event is generated periodically and is controlled by
2 frerSeqRcvyLatentErrorPeriod (10.4.1.12.2). It triggers the execution of the LatentErrorTest routine
3 (7.4.4.4).

4 The latent error detection function can trigger one event:

5 d) **SIGNAL_LATENT_ERROR:** The event generated by the latent error detection function that
6 indicates to the upper layers a possible error condition.

7 7.4.4.2 Variables for latent error detection

8 Each instance of the latent error detection function has its own set of variables, independent from any other
9 instance.

10 7.4.4.2.1 CurBaseDifference

11 The CurBaseDifference variable is an unsigned integer that is the same size as the counters (e.g.,
12 frerCpsSeqRcvyPassedPackets, 10.8.5) from which its value is computed. It contains the offset between the
13 expected and actual number of discarded packets as it was the last time that the LatentErrorReset function
14 (7.4.4.3) was called.

15 7.4.4.3 LatentErrorReset

16 LatentErrorReset is called whenever the BEGIN event (item a in 7.4.4.1) or the RESET_LATENT_ERROR
17 (item b in 7.4.4.1) event occur. It recomputes the CurBaseDifference (7.4.4.2.1) variable based on
18 frerCpsSeqRcvyPassedPackets (10.8.5), frerSeqRcvyLatentErrorPaths (10.4.1.12.3), and
19 frerCpsSeqRcvyDiscardedPackets (10.8.6), and increments frerCpsSeqRcvyLatentErrorResets (10.8.10).

```
20 void LatentErrorReset () {
21     CurBaseDifference = (frerCpsSeqRcvyPassedPackets *
22                         (frerSeqRcvyLatentErrorPaths - 1)) -
23                         frerCpsSeqRcvyDiscardedPackets;
24     frerCpsSeqRcvyLatentErrorResets = frerCpsSeqRcvyLatentErrorResets + 1;
25 }
```

26 7.4.4.4 LatentErrorTest

27 LatentErrorTest is called whenever the TRIGGER_LATENT_ERROR_TEST event occurs (item c in
28 7.4.4.1). It tests to see whether the number of packets discarded by the sequence recovery function is
29 approximately the expected number, and triggers an event if not, using CurBaseDifference
30 (7.4.4.2.1), frerCpsSeqRcvyPassedPackets (10.8.5), frerSeqRcvyLatentErrorPaths
31 (10.4.1.12.3), frerCpsSeqRcvyDiscardedPackets (10.8.6), frerSeqRcvyLatentErrorPeriod
32 (10.4.1.12.2), and frerSeqRcvyLatentErrorDifference (10.4.1.12.1).

```
33 void LatentErrorTest () {
34     int diff = CurBaseDifference - ((frerCpsSeqRcvyPassedPackets *
35                                     (frerSeqRcvyLatentErrorPaths - 1)) -
36                                     frerCpsSeqRcvyDiscardedPackets);
37     if (frerSeqRcvyLatentErrorPaths > 1 && // There are multiple paths
38         frerSeqRcvyLatentErrorPeriod > 0 ) // LE detection is turned on
39     {
40         if (diff < 0)
41             diff = - diff;
42         if (diff > frerSeqRcvyLatentErrorDifference) {
```

```

1      SIGNAL_LATENT_ERROR;
2    }
3  }
4 }

```

5 NOTE—This algorithm has, in effect, a quantization error of 2. That is, if the user is unlucky in the timing of
6 LatentErrorTest and LatentErrorReset, it may take a `diff` of $2 * \text{frerSeqRcvyLatentErrorDifference}$ to trigger a fault.

7.5 Individual recovery function

8 The Individual recovery function is defined in order to meet the Robustness goal (item j in 7.1.1). It
9 accomplishes this by removing repeating sequence numbered packets received from a stuck transmitter. An
10 instantiation of the Individual recovery function consists of an instantiation of the Base recovery function
11 (7.4.3) with its `frerSeqRcvyIndividualRecovery` object (10.4.1.10) set to True, configured to apply to a
12 single Member Stream. [A Sequence recovery function (7.4.2) operates on all Member Streams of a
13 Compound Stream.] An instantiation of the Individual recovery function does not include an instance of the
14 Latent error detection function (7.4.4). An instantiation of the Individual recovery function can employ
15 either the VectorRecoveryAlgorithm (7.4.3.4) or the MatchRecoveryAlgorithm (7.4.3.5).

16 By applying the Individual recovery function (7.5) to Member Streams, whether before or without the
17 application of the Sequence recovery function (7.4.2) to Compound Streams, errored Streams can be
18 discovered early, and pollution of the merged Stream avoided. Figure 7-3 shows the example network of
19 Figure 7-1, to which instances of the Individual recovery function have been added. See also C.10.

20 Unlike the Sequence recovery function, which is modeled as being instantiated on a specific port, a single
21 instantiation of an Individual recovery function can be applied to any number of ports. See C.11.

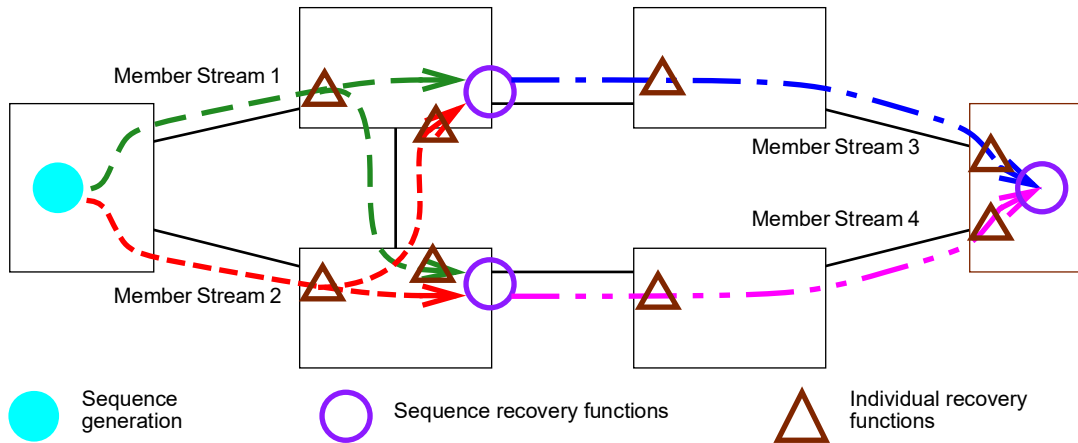


Figure 7-3—Sequence recovery functions and Individual recovery functions

7.6 Sequence encode/decode function

23 The Sequence encode/decode function is responsible for inserting the `sequence_number` subparameter (item
24 b in 6.1) into the packet, and extracting it from the packet. If the Stream identification function (6.2) also
25 encodes and decodes the `sequence_number` subparameter for a given Stream on a particular port and
26 direction, then no Sequence encode/decode sublayer is needed. Instances of the Sequence encode/decode
27 function are instantiated via managed objects (10.5).

28 The only Sequence encode/decode format specified as required, separately from any Stream identification
29 function, is the Redundancy tag (R-TAG, 7.8). In addition, two optional formats (7.9, 7.10) are defined.

1 **7.7 Stream splitting function**

2 The Stream splitting function accepts a packet from the upper layers with a stream_handle subparameter
3 (item a in 6.1), makes zero or more copies of that packet, each with a stream_handle subparameter that can
4 be different from the original stream_handle, and passes those packets to the next-lower layer. This
5 effectively creates Member Stream(s) from a Compound Stream. Packets passing up the stack are
6 unchanged by the Stream splitting function.

7 Instances of the Stream splitting function are created to process specific Streams by means of the Stream
8 split table (10.6) managed object. A packet passed down from the upper layers is acted upon by a Stream
9 splitting function on a particular port (10.6.1.1) and direction (10.6.1.2) only if its stream_handle
10 subparameter is in the frerSplitInputIdList (10.6.1.3) configured for that port and direction in some entry in
11 the Stream split table (10.6). If the stream_handle matches any of the items in that list, then one copy of the
12 packet is generated for each item in that same frerSplitEntry's frerSplitOutputIdList (10.6.1.4), each copy
13 with one of those values for its stream_handle subparameter.

14 **7.8 Redundancy tag**

15 The Redundancy tag (R-TAG) is an example of a Sequence encode/decode function (7.6). It operates at the
16 frame level and can be defined using the ISS defined by IEEE Std 802.1ACTM, or the EISS described in 6.9
17 of IEEE Std 802.1QTM-2022, enhanced with the extra stream_handle and sequence_number subparameters
18 specified in 6.1. In order to instantiate the Sequence encode/decode function using the R-TAG, the
19 frerSeqEncEncapsType managed object (10.5.1.5) is encoded using the OUI (00-80-C2) and the type values
20 as shown in Table 10-2. The R-TAG is illustrated in Figure 7-4. The value for the EtherType for the R-TAG
21 is given in Table 7-1.

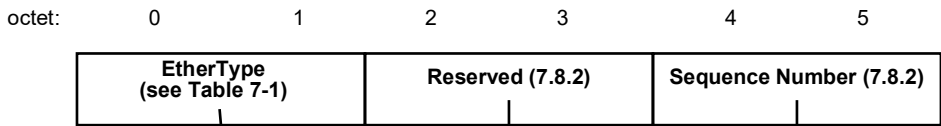


Figure 7-4—R-TAG format

22 When presented by the upper layers with a service request (ISS M_UNITDATA.request or EISS
23 EM_UNITDATA.request), the Redundancy tag Sequence encode/decode function:

- 24 a) Creates an Redundancy tag information (7.8.2) word by copying the low-order 16 bits from the
25 sequence_number subparameter to the Sequence Number field of the Redundancy tag information,
26 and fills the Reserved field with zeros. The most-significant octet of the low-order 16 bits of the
27 sequence_number goes in the first octet (octet 4 in Figure 7-4).

28 NOTE 1—Any high-order bits above the low-order 16 bits of the sequence_number subparameter are ignored.

- 29 b) Inserts an EtherType (7.8.1) and the Redundancy tag information as the first octets of the
30 mac_service_data_unit parameter, thus increasing the size of the mac_service_data_unit parameter
31 by 6 octets.

32 When presented by the lower layers with a service indication (ISS M_UNITDATA.indication or EISS
33 EM_UNITDATA.indication), the Redundancy tag Sequence encode/decode function:

- 34 c) Examines the first two octets of the mac_service_data_unit parameter for equality with the
35 Redundancy tag EtherType specified in 7.8.1. If it is equal, and if the mac_service_data_unit is at
36 least 6 octets in length, then the Redundancy tag Sequence encode/decode function:
37 1) Removes the first 6 octets of the mac_service_data_unit, shortening it by 6 octets; and

- 1 2) Copies the Sequence Number field of the Redundancy tag information to the sequence_number
2 subparameter, ignoring the contents of the Reserved field.
- 3 d) If the first two octets of the mac_service_data_unit parameter are not equal to the Redundancy tag
4 EtherType, or if the mac_service_data_unit is less than 6 octets in length, then the Redundancy tag
5 Sequence encode/decode function:
- 6 1) Sets the sequence_number subparameter to frerSeqRcvyInvalidSequenceValue (10.4.1.8).
- 7 2) Increments the frerCpsSeqEncErroredPackets and frerCpSeqEncErroredPackets managed
8 objects (10.8.11, 10.9.3).

9 NOTE 2—The position of the R-TAG in a frame relative to other tags depends on the number and relative position of the
10 Sequence encode/decode functions and other functions in the protocol stacks of the various components of a network.
11 The managed objects in Clause 9 and Clause 10 support placing the Sequence encode/decode functions in any position
12 to suit the needs of an application. See Clause 8 for a typical application of FRER to an IEEE 802.1Q Bridge.

13 **7.8.1 Redundancy tag EtherType**

14 The Tag Protocol Identifier (TPID) of the R-TAG shall use the value shown in Table 7-1.

Table 7-1—R-TAG EtherType

Purpose	EtherType
Redundancy tag (R-TAG)	F1-C1

15 **7.8.2 Redundancy tag information**

16 The R-TAG information consists of two fields:

- 17 a) The first two octets of the R-TAG information is a 16-bit Reserved field. This field shall be
18 transmitted with all zeros and shall be ignored on receipt. It is intended that future revisions of this
19 standard can use the most-significant bits of the Reserved field for sub-typing purposes, as described
20 in 9.2.1 of IEEE Std 802-2014.
- 21 b) The last two octets of the R-TAG information are a 16-bit value, the Sequence Number field.

22 **7.9 HSR sequence tag**

23 The optional High-availability Seamless Redundancy (HSR) sequence tag is an example of a Sequence
24 encode/decode function (7.6). It operates at the frame level and can be defined using the ISS defined by
25 IEEE Std 802.1AC™, or the EISS described in 6.9 of IEEE Std 802.1Q™-2022, enhanced with the extra
26 stream_handle and sequence_number subparameters specified in 6.1. In order to instantiate the Sequence
27 encode/decode function using the HSR sequence tag, the frerSeqEncEncapsType managed object (10.5.1.5)
28 is encoded using the OUI (00-80-C2) and the type values as shown in Table 10-2. The HSR sequence tag is
29 described in 5.7.1 of IEC 62439-3:2016.

30 When presented by the upper layers with a service request (ISS M_UNITDATA.request or EISS
31 EM_UNITDATA.request), the HSR sequence tag Sequence encode/decode function:

- 32 a) Creates an HSR sequence tag by:
- 33 1) Copying the low-order 16 bits from the sequence_number subparameter to the SeqNr field of
34 the HSR sequence tag;
- 35 2) Setting the LSDU Size field in accordance with 5.7.1 of IEC 62439-3:2016;
- 36 3) Setting the PathId field according to the per-port per-Stream managed object
37 frerSeqEncPathIdLanId (10.5.1.6); and

4) Setting the EtherType field according to 5.7.1 of IEC 62439-3:2016.

NOTE 1—Any high-order bits above the low-order 16 bits of the sequence_number subparameter are ignored.

b) Inserts the HSR sequence tag as the first octets of the mac_service_data_unit parameter, thus increasing the size of the mac_service_data_unit parameter by 6 octets.

When presented by the lower layers with a service indication (ISS M_UNITDATA.indication or EISS EM_UNITDATA.indication), the HSR sequence tag Sequence encode/decode function:

c) Examines the first two octets of the mac_service_data_unit parameter for equality with the HSR sequence tag EtherType specified in 5.7.1 of IEC 62439-3:2016. If it is equal, and if the mac_service_data_unit is at least 6 octets in length, then the HSR sequence tag Sequence encode/decode function:

1) Removes the first 6 octets of the mac_service_data_unit, shortening it by 6 octets; and

2) Copies the SeqNr field to the sequence_number subparameter.

d) If the first two octets of the mac_service_data_unit parameter are not equal to the HSR sequence tag EtherType, or if the mac_service_data_unit is less than 6 octets in length, then the HSR sequence tag Sequence encode/decode function:

1) Sets the sequence_number subparameter to frerSeqRevyInvalidSequenceValue (10.4.1.8).

2) Increments the frerCpsSeqEncErroredPackets and frerCpSeqEncErroredPackets managed objects (10.8.11, 10.9.3).

NOTE 2—No part of this standard is to be construed in a manner so as to alter the specifications in, or the intent of, IEC 62439-3:2016, or to restrict its future development in any way. The purpose of 7.9 is to enable the creation of interwork- ing functions between end systems employing the R-TAG (7.8) and the HSR sequence tag.

NOTE 3—See B.2 for additional considerations when interworking between FRER and HSR.

NOTE 4—The position of the Sequence encode/decode function in the protocol stack does not need to change depending on whether the R-TAG, the HSR tag, or the PRP trailer is used.

7.10 PRP sequence trailer

The optional Parallel Redundancy Protocol (PRP) sequence trailer is an example of a Sequence encode/ decode function (7.6). It operates at the frame level and can be defined using the ISS defined by IEEE Std 802.1AC, or the EISS described in 6.9 of IEEE Std 802.1Q™-2022, enhanced with the extra stream_handle and sequence_number subparameters specified in 6.1. In order to instantiate the Sequence encode/decode function using the PRP sequence trailer, the frerSeqEncEncapsType managed object (10.5.1.5) is encoded using the OUI (00-80-C2) and the type values as shown in Table 10-2. The PRP sequence trailer is described in 4.2.7.3 of IEC 62439-3:2016.

When presented by the upper layers with a service request (ISS M_UNITDATA.request or EISS EM_UNITDATA.request), the PRP sequence trailer Sequence encode/decode function:

a) Creates an PRP sequence trailer by:

1) Copying the low-order 16 bits from the sequence_number subparameter to the SeqNr field of the PRP sequence trailer;

2) Setting the LSDU Size field in accordance with 4.2.7.3 of IEC 62439-3:2016;

3) Setting the LanId field according to the per-port per-Stream managed object frerSeqEncPathIdLanId (10.5.1.6); and

4) Setting the PRPsuffix field according to 4.2.7.3 of IEC 62439-3:2016.

NOTE 1—Any high-order bits above the low-order 16 bits of the sequence_number subparameter are ignored.

- 1 b) Inserts the PRP sequence trailer as the final octets of the `mac_service_data_unit` parameter, thus
2 increasing the size of the `mac_service_data_unit` parameter by 6 octets.

3 When presented by the lower layers with a service indication (ISS `M_UNITDATA.indication` or EISS
4 `EM_UNITDATA.indication`), the PRP sequence trailer Sequence encode/decode function:

- 5 c) Examines the last two octets of the `mac_service_data_unit` parameter for equality with the PRP
6 sequence trailer `PRPsuffix` field specified in 4.2.7.3 of IEC 62439-3:2016. If it is equal, and if the
7 `mac_service_data_unit` is at least 8 octets in length, then the PRP sequence trailer Sequence encode/
8 decode function:
 - 9 1) Removes the last 6 octets of the `mac_service_data_unit`, shortening it by 6 octets; and
 - 10 2) Copies the `SeqNr` field to the `sequence_number` subparameter.
- 11 d) If the last two octets of the `mac_service_data_unit` parameter are not equal to the PRP sequence
12 trailer `EtherType`, or if the `mac_service_data_unit` is less than 8 octets in length, then the PRP
13 sequence trailer Sequence encode/decode function:
 - 14 1) Sets the `sequence_number` subparameter to `frerSeqRcvyInvalidSequenceValue` (10.4.1.8).
 - 15 2) Increments the `frerCpsSeqEncErroredPackets` and `frerCpSeqEncErroredPackets` managed
16 objects (10.8.11, 10.9.3).

17 NOTE 2—No part of this standard is to be construed in a manner so as to alter the specifications in, or the intent of, IEC
18 62439-3:2016, or to restrict its future development in any way. The purpose of 7.10 is to enable the creation of
19 interworking functions between end systems employing the R-TAG (7.8) and the PRP sequence trailer.

20 NOTE 3—See B.2 for additional considerations when interworking between FRER and PRP.

21 7.11 Autoconfiguration

22 7.11.1 Introduction to autoconfiguration

23 In order to satisfy the goal of Ease of use (item 1 in 7.1.1), there are two methods by which Stream
24 identification functions (6.2), Individual recovery functions (7.5), Sequence recovery functions (7.4.2), and
25 Sequence encode/decode functions (7.6) can be configured in a system:

- 26 a) Explicit configuration of entries in the Stream identity table (9.1), Sequence recovery table
27 (10.4), Sequence identification table (10.5), and Sequence identification table (10.5); and
- 28 b) Autoconfiguration of entries in these tables using the Sequence autoconfiguration table (10.7.1) and
29 Output autoconfiguration table (10.7.2).

30 Autoconfiguration can only be used with Source MAC and VLAN Stream identification (6.5) and the
31 `MatchRecoveryAlgorithm` (7.4.3.5).

32 Whenever a packet is received that has a `sequence_number` subparameter encoded in a manner identified in
33 an entry in the Sequence autoconfiguration table, but for which there is no `tsnStreamIdEntry` (9.1.1) in the
34 Stream identity table, a new entry in the Stream identity table is created for the packet's Stream, and
35 optionally, a new entry in the Sequence recovery table for an Individual recovery function for that Stream. If
36 a second or subsequent such Stream identity table entry is created that shares all the same key parameters as
37 another autoconfigured `tsnStreamIdEntry`, except for port number
38 (`tsnStreamIdOutFacInputPortList`, 9.1.1.5) and/or `LanId/PathId` (`tsnStreamIdLanPathId`, 10.2.2), then
39 another entry in the Sequence recovery table, this time for a Sequence recovery function, is created (for the
40 second Member Stream) or is expanded (for subsequent Member Streams) for what is now known to be a
41 Compound Stream.

1 The Sequence autoconfiguration table controls whether Individual recovery functions, Sequence recovery
2 functions, neither, or both, are created automatically. Stream identification functions are always created. If
3 the R-TAG (7.8) is used to encode the sequence_number, entries autoconfigured in the Sequence
4 identification table, and thus the Individual recovery functions (7.5) created, are keyed to the individual
5 ports and VLANs on which the first packet of each Member Stream is received. If the HSR sequence tag
6 (7.9) or PRP sequence trailer (7.10) is used, entries autoconfigured in the Sequence identification table and
7 the autocreated Individual recovery functions are keyed to the PathId or LanId of the first-received packet of
8 each Member Stream, and span all autoconfigurable ports.

9 In order to support the goals of Ease of use (item l in 7.1.1) and Interoperability (item g in 7.1.1) among
10 systems using the HSR sequence tag (7.9), the PRP sequence trailer (7.10), and the R-TAG (7.8), passive
11 (packet decoding) entries are also automatically created in the Sequence identification table to instantiate
12 Sequence encode/decode functions as governed by the Sequence autoconfiguration table (10.7.1). Typically,
13 the Output autoconfiguration table (10.7.2) is configured to construct active (packet encoding) Sequence
14 encode/decode functions on every port on which a Stream might be output. See C.11.

15 7.11.2 Creating autoconfigured Stream identity table entries

16 NOTE 1—The following description, if implemented naïvely, might well result in a relay system delaying the
17 transmission of the first-received packet of a Member Stream for an unacceptable length of time. The purpose of the
18 following description is to describe the externally visible behavior of a system solely in terms of the managed objects in
19 Clause 9 and Clause 10, not to constrain an implementation. It is expected that an actual implementation is likely to
20 construct databases in software, firmware, and/or hardware that combine elements of the managed objects in Clause 9
21 and Clause 10, and to perform up-front preliminary configuration, so as to optimize the reaction time when a packet is
22 actually received.

23 Autoconfiguration is triggered by the receipt of a packet that matches an entry in the Sequence
24 autoconfiguration table (10.7.1) as follows:

- 25 a) If a packet received on an out-facing port matches an entry in the Stream identity table (9.1), it is
26 processed according to the managed objects in 10.3, 10.4, 10.5, and 10.6.

27 NOTE 2—It does not matter whether the Stream identity table entry was configured explicitly, or was created via
28 Autoconfiguration.

- 29 b) If the packet does not match any entry in the Stream identity table at all, the packet is examined to
30 see if it matches an entry in the Sequence autoconfiguration table. A match between the packet and a
31 frerAutSeqEntry (10.7.1.1) in the Sequence autoconfiguration table occurs if all of the following
32 match conditions are met:

- 33 1) The packet has a sequence_number encapsulated in the manner specified by
34 frerAutSeqSeqEncaps (10.7.1.1.1).
- 35 2) The packet was received on a port in frerAutSeqReceivePortList (10.7.1.1.2).
- 36 3) The packet's VLAN tag (or lack thereof) matches frerAutSeqTagged (10.7.1.1.3).
- 37 4) Either the vlan_identifier parameter is in the list in frerAutSeqVlan (10.7.1.1.4), or that list is
38 empty.

- 39 c) If the packet does not match an entry in the Sequence autoconfiguration table, no further
40 Autoconfiguration processing takes place. Otherwise (the packet did not match any
41 tsnStreamIdEntry, but does match a frerAutSeqEntry), Autoconfiguration processing proceeds.

- 42 d) The system creates (autoconfigures) new entries in the Stream identity table, the Sequence recovery
43 table (10.4), and the Sequence identification table (10.5) using the matched frerAutSeqEntry as
44 follows:

- 45 1) One new tsnStreamIdEntry (9.1.1) is created in the Stream identity table as follows:
46 i) tsnStreamIdHandle (9.1.1.1) = an unused stream_handle value chosen by the
47 implementation.

- ii) `tsnStreamIdInFacOutputPortList` (9.1.1.2) = empty.
 - iii) `tsnStreamIdOutFacOutputPortList` (9.1.1.3) = empty.
 - iv) `tsnStreamIdInFacInputPortList` (9.1.1.4) = empty.
 - v) If `frerAutSeqSeqEncaps` equals either HSR or PRP, `tsnStreamIdOutFacInputPortList` (9.1.1.5) = `frerAutSeqReceivePortList` (10.7.1.1.2). Otherwise, `frerSeqRcvyPortList` (10.4.1.2) = only the port on which the packet was received.
 - vi) `tsnStreamIdIdentificationType` (9.1.1.6) = 00-80-C2, 2 = Source MAC and VLAN Stream identification (6.5).
 - vii) `tsnStreamIdAutoconfigured` (10.2.1) = True.
 - viii) If `frerAutSeqSeqEncaps` equals either HSR or PRP, `tsnStreamIdLanPathId` (10.2.2) = the `PathId` or `LanId` field from the packet. Otherwise, `tsnStreamIdLanPathId` is not used.
 - ix) `tsnCpeSmacVlanDownSrcMac` (9.1.3.1) = the `source_mac_address` parameter of the received packet.
 - x) `tsnCpeSmacVlanDownTagged` (9.1.3.2) = `frerAutSeqTagged` (10.7.1.1.3).
 - xi) `tsnCpeSmacVlanDownVlan` (9.1.3.3) = the `vlan_identifier` parameter of the received packet.
- 2) If and only if `frerAutSeqCreateIndividual` (10.7.1.1.10) is True, a `frerSeqRcvyEntry` (10.4.1) is created in the Sequence recovery table (10.4) as follows:
- i) `frerSeqRcvyStreamList` (10.4.1.1) = the `stream_handle` from item d:2:i, above.
 - ii) `frerSeqRcvyPortList` (10.4.1.2) = the same list as `tsnStreamIdOutFacInputPortList` (9.1.1.5), described in item d:2:v, above. (See C.11.)
 - iii) `frerSeqRcvyDirection` (10.4.1.3) = True (out-facing).
 - iv) `frerSeqRcvyReset` (10.4.1.4) = True. (The state machine is reset when created.)
 - v) `frerSeqRcvyAlgorithm` (10.4.1.5) = `Match_Alg` (00-80-C2, 1, see Table 10-1).
 - vi) `frerSeqRcvyHistoryLength` (10.4.1.6) is not used.
 - vii) `frerSeqRcvyResetMSec` (10.4.1.7) = `frerAutSeqResetMSec` (10.7.1.1.7).
 - viii) `frerSeqRcvyInvalidSequenceValue` (10.4.1.8) is a read-only value chosen by the system.
 - ix) `frerSeqRcvyTakeNoSequence` (10.4.1.9) = True.
 - x) `frerSeqRcvyIndividualRecovery` (10.4.1.10) = True.
 - xi) `frerSeqRcvyLatentErrorDetection` (10.4.1.11) = False.
- 3) One new passive `frerSeqEncEntry` (10.5.1) is created in the Sequence identification table to decode further received packets as follows:
- i) `frerSeqEncStreamList` (10.5.1.1) = the `stream_handle` from item d:2:i, above.
 - ii) `frerSeqEncPort` (10.5.1.2) = `frerAutSeqReceivePortList` (10.7.1.1.2).
 - iii) `frerSeqEncDirection` (10.5.1.3) = True (out-facing).
 - iv) `frerSeqEncActive` (10.5.1.4) = False (passive).
 - v) `frerSeqEncEncapsType` (10.5.1.5) = `frerAutSeqSeqEncaps` (10.7.1.1.1).
 - vi) If `frerAutSeqSeqEncaps` = 00-80-C2, 2 (HSR), `frerSeqEncPathIdLanId` (10.5.1.6) = the `PathId` field of the HSR sequence tag (7.9) found in the packet. If `frerAutSeqSeqEncaps` = 00-80-C2, 3 (PRP), `frerSeqEncPathIdLanId` = the `LanId` field of the PRP sequence trailer (7.10) found in the packet. Otherwise, `frerSeqEncPathIdLanId` is not used.
- If a `frerSeqEncEntry` with the same port list, direction, and encapsulation type is present, a new entry need not be created; the packet's `stream_handle` can be added to that `frerSeqEncEntry`'s `frerSeqEncStreamList`.
- 4) One new active `frerSeqEncEntry` (10.5.1) is created in the Sequence identification table for each `frerAutOutEntry` (10.7.2.1) in the Output autoconfiguration table (10.7.2) to format transmitted packets for this packet's Stream as follows:
- i) `frerSeqEncStreamList` (10.5.1.1) = the `stream_handle` from item d:2:i, above.
 - ii) `frerSeqEncPort` (10.5.1.2) = `frerAutOutPortList` (10.7.2.1.1).
 - iii) `frerSeqEncDirection` (10.5.1.3) = True (out-facing).
 - iv) `frerSeqEncEncapsType` (10.5.1.5) = `frerAutOutEncaps` (10.7.2.1.2).
 - v) `frerSeqEncPathIdLanId` (10.5.1.6) = `frerAutOutLanPathId` (10.7.2.1.3).

- 1 If a `frerSeqEncEntry` with the same port list, direction, encapsulation type, and `PathId/LanId` is
2 present, a new entry need not be created; the packet's `stream_handle` can be added to that
3 `frerSeqEncEntry`'s `frerSeqEncStreamList`.
- 4 e) If none of the ports to which the received packet is to be output are listed in the `frerAutSeqEntry`'s
5 `frerAutSeqRecoveryPortList` (10.7.1.1.5), or if `frerAutSeqCreateRecovery` (10.7.1.1.11) is false,
6 Autoconfiguration processing is terminated. No Sequence recovery function need be instantiated.
- 7 f) The system next scans the Stream identity table looking for a `tsnStreamIdEntry` that nearly matches
8 the received packet, in order to discover whether there are other Streams similar to the received
9 packet's Stream such that they are all Member Streams of a Compound Stream. "Nearly matches"
10 means that all of the following conditions are met:
- 11 1) `tsnStreamIdIdentificationType` (9.1.1.6) == 00-80-C2, 2 == Source MAC and VLAN Stream
12 identification (6.5).
 - 13 2) The packet's `stream_handle` is not equal to `tsnStreamIdHandle` (9.1.1.1).
 - 14 3) The `source_mac_address` parameter of the packet == `tsnCpeSmacVlanDownSrcMac` (9.1.3.1).
 - 15 4) Either `tsnCpeSmacVlanDownVlan` (9.1.3.3) is in the `frerAutSeqVlan` (10.7.1.1.4) list, or that
16 list is empty.
- 17 g) If no `tsnStreamIdEntry` is found that meets these criteria, then Autoconfiguration processing is
18 terminated. No Sequence recovery function need be instantiated.
- 19 h) If a `tsnStreamIdEntry` is found that does meet these criteria, then either Sequence recovery functions
20 need to be instantiated, or existing instantiations need to be modified to include the received
21 packet's Member Stream. To determine which, the system searches the Sequence recovery table for
22 a `frerSeqRcvyEntry` that meets all of the following criteria:
- 23 1) `frerSeqRcvyStreamList` contains the `stream_handle` of the found `tsnStreamIdEntry`.
 - 24 2) `frerSeqRcvyIndividualRecovery` (10.4.1.10) == False == Sequence recovery function.
 - 25 3) `frerSeqRcvyPortList` (10.4.1.2) == `frerAutSeqRecoveryPortList` (10.7.1.1.5).
 - 26 4) For a relay system, `frerSeqRcvyDirection` (10.4.1.3) is False (in-facing). For an end system,
27 True (out-facing).
- 28 i) If the Sequence recovery table search (item h, above) is found, then the following steps are taken:
- 29 1) The packet's `stream_handle` is added to the `frerSeqRcvyEntry`'s `frerSeqRcvyStreamList`
30 (10.4.1.1).
 - 31 2) If `frerAutSeqLatErrDetection` (10.7.1.1.12) is True, then the following steps are taken:
 - 32 i) `frerSeqRcvyLatentErrorPaths` (10.4.1.12.3) is incremented by 1.
 - 33 ii) The `LatentErrorReset` function (7.4.4.3) is called for the instantiation of the Latent error
34 detection function (7.4.4) on every port in the `frerSeqRcvyEntry`'s `frerSeqRcvyPortList`
35 (10.4.1.2).
- 36 j) Otherwise (the Sequence recovery table search in item h, above, failed to find a `frerSeqRcvyEntry`),
37 a new `frerSeqRcvyEntry` is created as follows:
- 38 1) `frerSeqRcvyStreamList` (10.4.1.1) = two `stream_handle` values, the `stream_handle` from the
39 received packet (item d:2:i, above) and the `tsnStreamIdHandle` (9.1.1.1) from the
40 `tsnStreamIdEntry` found in item f, above.
 - 41 2) `frerSeqRcvyPortList` (10.4.1.2) = `frerAutSeqRecoveryPortList` (10.7.1.1.5).
 - 42 3) `frerSeqRcvyDirection` (10.4.1.3) = False (in-facing) for a relay system, or True (out-facing) for
43 an end system.
 - 44 4) `frerSeqRcvyReset` (10.4.1.4) = True. (The state machine is reset when created.)
 - 45 5) `frerSeqRcvyAlgorithm` (10.4.1.5) = `frerAutSeqAlgorithm` (10.7.1.1.8).
 - 46 6) `frerSeqRcvyHistoryLength` (10.4.1.6) = `frerAutSeqAlgorithm` (10.7.1.1.8). This value is not
47 used if `frerSeqRcvyAlgorithm` (10.4.1.5) == `Match_Alg` (00-80-C2, 1, see Table 10-1).
 - 48 7) `frerSeqRcvyResetMSec` (10.4.1.7) = `frerAutSeqResetMSec` (10.7.1.1.7).

- 1 8) `frerSeqRcvyInvalidSequenceValue` (10.4.1.8) is a read-only value chosen by the system.
- 2 9) `frerSeqRcvyTakeNoSequence` (10.4.1.9) = True.
- 3 10) `frerSeqRcvyIndividualRecovery` (10.4.1.10) = False.
- 4 11) `frerSeqRcvyLatentErrorDetection` (10.4.1.11) = `frerAutSeqLatErrDetection` (10.7.1.1.12).
- 5 12) if `frerAutSeqLatErrDetection` is True, then the following assignments are made:
 - 6 i) `frerSeqRcvyLatentErrorDifference` (10.4.1.12.1) = `frerAutSeqLatErrDifference`
7 (10.7.1.1.13).
 - 8 ii) `frerSeqRcvyLatentErrorPeriod` (10.4.1.12.2) = `frerAutSeqLatErrPeriod` (10.7.1.1.14).
 - 9 iii) `frerSeqRcvyLatentErrorPaths` (10.4.1.12.3) = 2.
 - 10 iv) `frerSeqRcvyLatentResetPeriod` (10.4.1.12.4) = `frerAutSeqLatErrResetPeriod`
11 (10.7.1.1.15).
- 12 k) The packet is then processed according to the newly created entries.
- 13 l) After the reception of the packet that triggers Autoconfiguration for a `stream_handle`, if none of the
14 Individual recovery functions (7.5) or Sequence recovery functions (7.4.2) instantiated by a given
15 `frerAutSeqEntry` (10.7.1.1) increment any packet counters (10.8) for longer than the period specified
16 by the `frerAutSeqEntry`'s `frerAutSeqDestructMSec` (10.7.1.1.6) object, the system can destroy the
17 state machines created by that `frerAutSeqEntry`.

18 NOTE 3—As Member Streams are found, instances of the Sequence recovery function are created, and the number of
19 Member Flows increased to enable Latent Error Detection. If the destruct timer (`frerAutSeqDestructMSec`, 10.7.1.1.6) is
20 not much larger than the normal reset period for the state machines (`frerSeqRcvyResetMSec`, 10.4.1.7), there is a risk
21 that the state machines will be destroyed due to a temporary service interruption, that one or more of the Member
22 Streams will fail to be restored after the interruption, and the purpose of Latent Error Detection will be defeated.

8. Frame Replication and Elimination for Reliability in Bridges

8.1 Limiting options

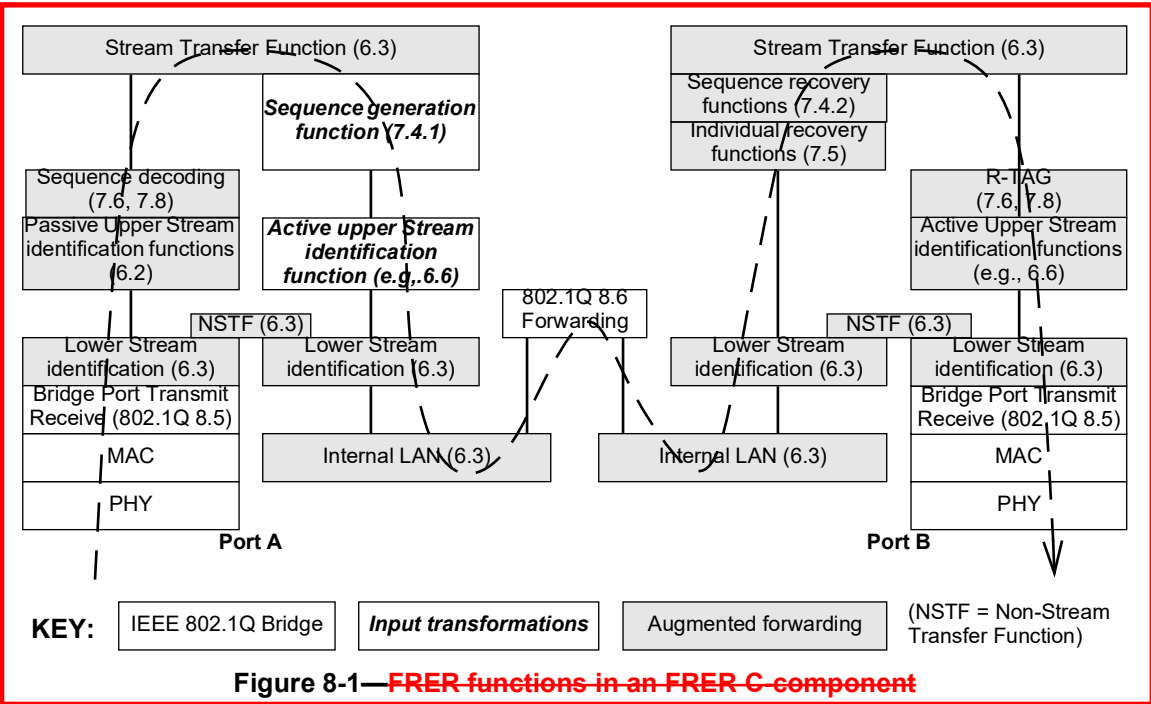
The arrangements of the various components of Time-Sensitive Networking (TSN) and Frame Replication and Elimination for Reliability (FRER) in Annex C show that the model described in Clause 6 and Clause 7 has the advantages that proper layering principles are observed, that the descriptions of the individual functions are simple, and most importantly, that the definition of a relay system’s forwarding function does not have to be modified in order to add TSN or FRER functions; all of the TSN and FRER capabilities can be provided as add-ons in the ports. However, this model offers a multitude of options for the placement of functions in an actual system. Experience shows that such flexibility is often not necessary, and if implemented naively, can increase the complexity of the system both for the implementor and the user.

Change the text as shown:

Most of the flexibility of the model described in Clause 6 and Clause 7 can be achieved in an IEEE 802.1Q Bridge, if we limit the Bridge to two stages of activity:

- 1) **Input transformations:** An expanded port (as in, e.g., Figure C-5) that includes Sequence generation functions (7.4.1) and active **upperIn-facing** Stream identification functions (6.6), shown in white boxes with boldface type in Figure 8-1.
- 2) **Augmented forwarding:** The Bridge Forwarding Process, described in 8.6 of IEEE Std 802.1Q-2022 (shown in white boxes in Figure 8-1 [and Figure 8-2](#)), augmented with Stream identification functions (6.2, both incoming and outgoing), Sequence encode/decode functions (7.6, both incoming and outgoing), Sequence recovery functions (7.4.2), Individual recovery functions (7.4.2), and their associated infrastructure, all shown in shaded boxes in Figure 8-1 [and Figure 8-2](#).

Replace Figure 8-1 by Figures 8-1 and 8-2 as shown:



1

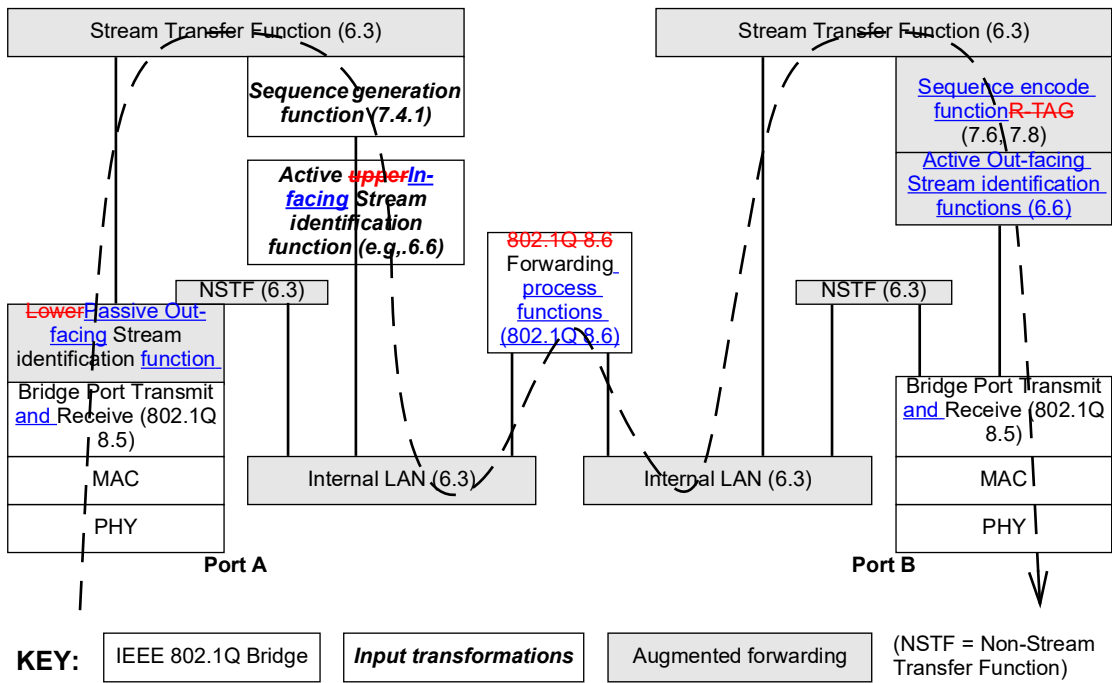


Figure 8-1—Split functions in an FRER C-component

2

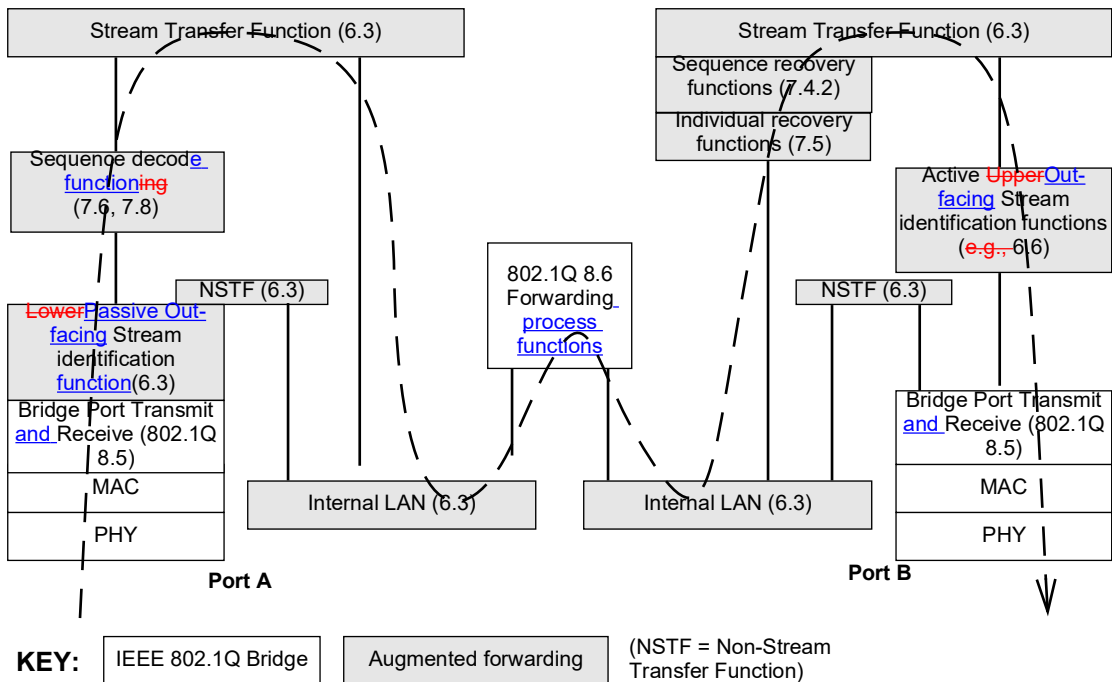


Figure 8-2—Merge functions in an FRER C-component

3 Item 1, Input transformations, allows the Bridge to:

- 1 a) Proxy for FRER-unaware Talkers; and
- 2 b) Perform Stream identification or Sequence encapsulation transformations on Streams entering the
- 3 Bridge.

4 Item 2, Augmented forwarding, allows the Bridge to:

- 5 c) Serve as an intermediate sequence recovery point (e.g., system F in Figure C-4) in a network
- 6 providing for multiple failures;
- 7 d) Proxy for FRER-unaware Listeners; and
- 8 e) Perform Stream identification or Sequence encapsulation transformations on Streams exiting the
- 9 Bridge.

10 ***Change the text as shown:***

11 As shown in [Figure 8-1](#) [Figure 8-2](#), the paradigm is that a given Stream is recognized on input, meaning that
12 the `stream_handle` and `sequence_number` subparameters are extracted. The frame is then forwarded
13 normally through the Bridge. On the output port, the `stream_handle` and `sequence_number` subparameters
14 found on the input port are used to drive the Individual recovery functions, Sequence recovery functions,
15 Sequence encoding, and Stream identification functions.

16 Figure 8-1 [and Figure 8-2](#) ~~does~~ not imply that the Augmented forwarding functions have to be placed in
17 expanded ports; they can be integrated with the Bridge forwarding function (8.6 of IEEE 802.1Q-2022) as
18 shown in [Figure 8-3](#) [Figure 8-3](#). The ordering of ~~this~~ [these](#) functions with respect to 8.6 of IEEE 802.1Q-2022
19 can be important. In particular:

- 20 f) As modeled, the `stream_handle` subparameter is extracted only once, on ingress. A frame is not re-
- 21 identified, even if Input transformations modifies the frame.
- 22 g) As modeled, the `sequence_number` subparameter is not encoded into the frame during Input
- 23 transformations; the final output encoding determines the frame format.

24 NOTE—Whether an implementation encodes the `stream_handle` and `sequence_number` subparameters into the frame's
25 `mac_service_data_unit` parameter and decodes them again after forwarding, or carries them through the forwarding
26 process as separate subparameters, does not matter as long as the externally visible behavior conforms to that of the
27 model.

- 28 h) If Input transformations are performed, the frame is forwarded (8.6 of IEEE Std 802.1Q™-2022)
- 29 according to its post-transformation parameters.
- 30 i) Flow metering (8.6.5 of IEEE Std 802.1Q™-2022) is placed after the passive Stream identification
- 31 function (6.2) and before the Individual recovery functions (7.5). This makes the `stream_handle`
- 32 subparameter available for Flow metering, and means that Flow metering can be applied to the
- 33 individual Streams feeding an instance of the Sequence recovery function. As a result, Flow
- 34 metering can be applied to frames that will be discarded by the Individual recovery functions or
- 35 Sequence recovery functions.
- 36 j) Output transformations following the Sequence recovery function take place after all forwarding
- 37 (except Queuing frames, 8.6.6 of IEEE Std 802.1Q™-2022). Thus, output transformations can
- 38 appear to cause violations of the normal forwarding rules, e.g., Egress filtering (8.6.4 of
- 39 IEEE Std 802.1Q-2022).

40 With the model of Figure 8-3, no explicit Stream splitting function (7.7) is required. Frames in a single
41 Compound Stream can be replicated using the normal multicast mechanisms, and active Stream
42 identification functions (6.2) on different output ports can cause these to be recognized as different Member
43 Streams at the next hop. Assigning a single Sequence encode/decode function and Stream identification
44 function to more than one `stream_handle` value can cause multiple Member Streams to have the same exact

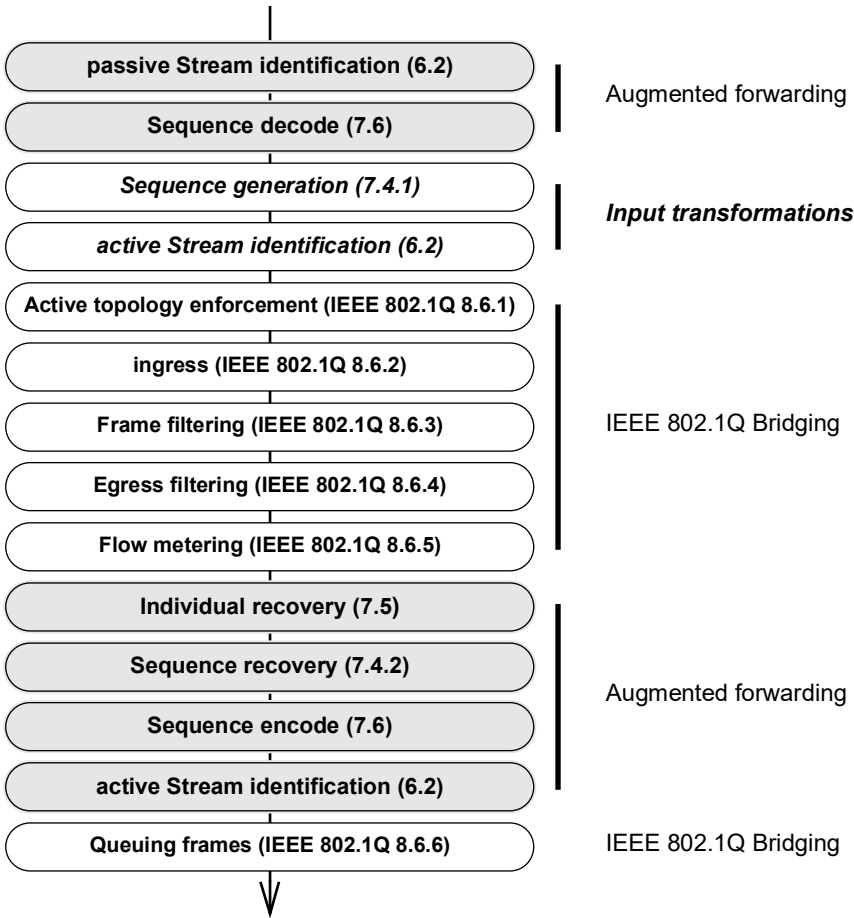


Figure 8-3—Augmented Forwarding Process does sequence recovery

1 Stream identification method on output, and thus be merged into a single Stream, as seen by the next
2 receiver.

3 In theory, if using the model of Figure 8-3, there exist instantiations of Individual recovery functions (7.5)
4 and Sequence recovery function (7.4.2) for each distinct set of ports for which an identical set of Streams is
5 to be output. The vector of output ports that, in principle, accompanies the packet through the IEEE 802.1Q
6 Forwarding Process, guides the allocation of frames to these functions. After this stage, sets of Sequence
7 encode/decode functions (7.6), and/or Stream identification functions (6.2) can be configured to further
8 transform the packet(s).

9 **8.2 FRER C-component input transformations**

10 The Input transformations, marked with white boxed **with and** boldface type in Figure 8-1, enable a Bridge
11 to proxy for a non-FRER-capable end system. The expanded input port identifies packets belonging to a
12 Stream, e.g., using IP Stream identification (6.7) or Mask-and-match Stream identification (6.8), serializes
13 the packets with a Sequence generation function (7.4.1), encodes the sequence number with an R-TAG
14 (7.8), and then gives the packets belonging to this Stream a {vlan_identifier, destination_mac_address} pair
15 that is unique, at least inside this Bridge, using Active Destination MAC and VLAN Stream identification
16 (6.6). The IEEE 802.1Q Forwarding Process, enhanced with the Individual recovery function (7.5) and
17 Sequence recovery function (7.4.2), then forwards the frame.

1 **8.3 Frame Replication and Elimination for Reliability and VLAN tags**

2 *Change the text as shown:*

3 As illustrated in Figure 8-1, [Figure 8-2](#) and Figure 8-4, FRER in an IEEE 802.1Q C-VLAN Component is
4 above the Bridge Port Transmit and Receive function (8.5 of IEEE Std 802.1Q™-2022) in the protocol
5 stack. As a consequence of this placement, a frame containing both an IEEE 802.1Q™ C-VLAN tag and an
6 R-TAG but no other IEEE 802.1Q™ tags, would be as illustrated in ~~Figure 8-4~~[Figure 8-4](#).

Field	Offset	Length
Destination MAC address	0	6
Source MAC address	6	6
C-TAG EtherType	12	2
Priority, DE, VLAN ID	14	2
R-TAG EtherType	16	2
Reserved	18	2
Sequence number	20	2
Payload Length/EtherType	22	2
Data	24	<i>n</i>
Frame Check Sequence	24+ <i>n</i>	4

Figure 8-4—Example Ethernet frame format

7 **8.4 Configuring Frame Replication and Elimination for Reliability in Bridges**

8 *Change the text as shown:*

9 As described in 8.1, ~~we can describe~~ the output transformations, including Individual recovery functions
10 (7.5), Sequence recovery functions (7.4.2), Sequence encode/decode functions (7.6), and Stream
11 identification functions (6.2), [can be considered](#) as being embedded in the IEEE 802.1Q Forwarding
12 Process. However, since different activities can be required by an application on different output ports, the
13 managed objects in Clause 9 and Clause 10 can be used to configure an FRER C-component. Figure 8-1 [and](#)
14 [Figure 8-2](#) illustrates an allowed arrangement of FRER functions on an output Bridge port as perceived by
15 the managed objects in Clause 9 and Clause 10.

16 An FRER C-component shall implement all of the managed objects in Clause 9 and Clause 10 that are
17 needed to meet the conformance requirements of 5.12 and 5.15 with the exceptions noted in Table 8-1.

Table 8-1—Managed objects for FRER in an FRER C-component

Variable	Reference	Limitation	Reason
tsnStreamIdInFacOutputPortList	9.1.1.2	Only Active Destination MAC and VLAN Stream identification (or nothing) can be configured.	Only needed for Input transformations.
tsnStreamIdOutFacOutputPortList	9.1.1.3	Only Active Destination MAC and VLAN Stream identification (or nothing) can be configured.	A Bridge can modify the <code>vlan_identifier</code> and <code>destination_mac_address</code> of packets belonging to a particular Stream
tsnStreamIdInFacInputPortList	9.1.1.4	Not allowed	There is no need to re-identify Streams on their way out of the Bridge.
tsnStreamIdOutFacInputPortList	9.1.1.5	No limitations	—
tsnCpeDmacVlanDownTagged	9.1.4.2	Not required	The decision whether to tag an outgoing frame is determined, in an IEEE 802.1Q Bridge, by other managed objects.
frerSeqGenDirection	10.3.1.2	Must be False (in-facing)	Sequence numbers can be added only to incoming packets.
frerSeqRcvDirection	10.4.1.3	Must be False (in-facing)	Duplicate packets can be discarded only on outgoing packets.
frerSeqEncDirection	10.5.1.3	Must be True (out-facing)	Packets' <code>sequence_numbers</code> are encoded/decoded as they leave.
Stream split table	10.6	Not required	—

9. Stream Identification Management

The description of the managed objects that control Stream identification are described in the following subclauses:

- a) The Stream identity table (9.1) assigns packets a `stream_handle` (6.1);
- b) The per-port, per-Stream packet counters that are kept by Stream identification functions for inspection by network management entities are described in 9.2, and the per-port (totaled over all Streams) counters in 9.3.

The general requirements on the behavior of the Stream Identification counters in 9.2 and 9.3 are described in 10.1.

9.1 Stream identity table

The Stream identity table consists of a set of `tsnStreamIdEntry` objects (9.1.1), each relating to a single Stream, specifying the points in the system where Stream identification functions (6.2) are to be instantiated. Each entry in the Stream identity table has a `tsnStreamIdHandle` object (9.1.1.1) specifying a `stream_handle` value and one or more `tsnStreamIdEntry` objects (9.1.1) describing one identification method for that Stream. If a single Stream has multiple identification methods, perhaps (but not necessarily) on different ports, then there can be multiple `tsnStreamIdEntry` objects with the same value for the `tsnStreamIdHandle`. If the HSR or PRP method or the Sequence encode/decode function is applied to a packet, then the `LanId` or `PathId` fields are also used to identify the Stream to which the packet belongs.

9.1.1 `tsnStreamIdEntry`

A set of managed objects, all applying to the Stream specified by `tsnStreamIdHandle` (9.1.1.1), and all using the same Stream identification types and parameters (9.1.1.6, 9.1.1.7).

See 10.2 for additional managed objects that are present in the `tsnStreamIdEntry` only if Autoconfiguration (7.11) is used.

9.1.1.1 `tsnStreamIdHandle`

The objects in a given entry of the Stream identity table are used to control packets whose `stream_handle` subparameter is equal to the entry's `tsnStreamIdHandle` object. The specific values used in the `tsnStreamIdHandle` object are not necessarily used in the system; they are used only to relate the various management objects in Clause 9 and Clause 10.

NOTE—It is also possible to assign a packet that was identified as belonging to a known stream (i.e., a specific `tsnStreamIdEntry`) a `stream_handle` equal 0.

9.1.1.2 `tsnStreamIdInFacOutputPortList`

The list of ports on which an in-facing Stream identification function (6.2) using this identification method (9.1.1.6, 9.1.1.7) is to be placed for this Stream (9.1.1.1) in the output (towards the system forwarding function) direction. At most one `tsnStreamIdEntry` can list a given port for a given `tsnStreamIdHandle` in its `tsnStreamIdInFacOutputPortList`.

9.1.1.3 `tsnStreamIdOutFacOutputPortList`

The list of ports on which an out-facing Stream identification function (6.2) using this identification method (9.1.1.6, 9.1.1.7) is to be placed for this Stream (9.1.1.1) in the output (towards the physical interface)

1 direction. At most one `tsnStreamIdEntry` can list a given port for a given `tsnStreamIdHandle` in its
2 `tsnStreamIdOutFacOutputPortList`.

3 9.1.1.4 `tsnStreamIdInFacInputPortList`

4 The list of ports on which an in-facing Stream identification function (6.2) using this identification method
5 (9.1.1.6, 9.1.1.7) is to be placed for this Stream (9.1.1.1) in the input (coming from the system forwarding
6 function) direction. Any number of `tsnStreamIdEntry` objects can list the same port for the same
7 `tsnStreamIdHandle` in its `tsnStreamIdInFacInputPortList`.

8 9.1.1.5 `tsnStreamIdOutFacInputPortList`

9 The list of ports on which an out-facing Stream identification function (6.2) using this identification method
10 (9.1.1.6, 9.1.1.7) is to be placed for this Stream (9.1.1.1) in the input (coming from the physical interface)
11 direction. Any number of `tsnStreamIdEntry` objects can list the same port for the same `tsnStreamIdHandle` in
12 its `tsnStreamIdOutFacInputPortList`.

13 9.1.1.6 `tsnStreamIdIdentificationType`

14 An enumerated value indicating the method used to identify packets belonging to the Stream. The
15 enumeration includes an Organizationally Unique Identifier (OUI) or Company ID (CID) to identify the
16 organization defining the enumerated type. The values defined by this standard are shown in Table 9-1.

Table 9-1—Stream identification types

OUI/CID	Type number	Stream identification function	Controlling parameters
00-80-C2	0	Reserved	—
00-80-C2	1	Null Stream identification (6.4)	9.1.2
00-80-C2	2	Source MAC and VLAN Stream identification (6.5)	9.1.3
00-80-C2	3	Active Destination MAC and VLAN Stream identification (6.6)	9.1.4
00-80-C2	4	IP Stream identification (6.7)	9.1.5
00-80-C2	5	Mask-and-match Stream identification (6.8)	9.1.6
00-80-C2	6–255	Reserved	—
other	—	Defined by entity owning the OUI or CID	—

17 9.1.1.7 `tsnStreamIdParameters`

18 The number of controlling parameters for a Stream identification method, their types and values, are specific
19 to the `tsnStreamIdIdentificationType` (9.1.1.6) and are referenced in Table 9-1.

20 9.1.2 Managed objects for Null Stream identification

21 When instantiating an instance of the Null Stream identification function (6.4) for a particular input Stream,
22 the managed objects in the following subclauses serve as the `tsnStreamIdParameters` managed object
23 (9.1.1.7).

1 9.1.2.1 tsnCpeNullDownDestMac

2 Specifies the destination_address that identifies a packet in an EISS indication primitive, to the Null Stream
3 identification function.

4 9.1.2.2 tsnCpeNullDownTagged

5 An enumerated value indicating whether a packet in an EISS indication primitive to the Null Stream
6 identification function is permitted to have a VLAN tag. It can take the following values:

- 7 1) **tagged:** A frame must have a VLAN tag to be recognized as belonging to the Stream.
- 9 2) **priority:** A frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be
11 recognized as belonging to the Stream.
- 12 3) **all:** A frame is recognized as belonging to the Stream whether tagged or not.

14 9.1.2.3 tsnCpeNullDownVlan

15 Specifies the vlan_identifier parameter that identifies a packet in an EISS indication primitive to the Null
16 Stream identification function. ~~A value of 0 indicates that the vlan_identifier parameter is ignored on EISS-~~
17 ~~indication primitives.~~

18 9.1.3 Managed objects for Source MAC and VLAN Stream identification

19 When instantiating an instance of the Source MAC and VLAN Stream identification function (6.5) for a
20 particular input Stream, the managed objects in the following subclauses serve as the tsnStreamIdParameters
21 managed object (9.1.1.7).

22 9.1.3.1 tsnCpeSmacVlanDownSrcMac

23 Specifies the source_address that identifies a packet in an EISS indication primitive, to the Source MAC and
24 VLAN Stream identification function.

25 9.1.3.2 tsnCpeSmacVlanDownTagged

26 An enumerated value indicating whether a packet in an EISS indication primitive to the Source MAC and
27 VLAN Stream identification function is permitted to have a VLAN tag. It can take the following values:

- 28 1) **tagged:** A frame must have a VLAN tag to be recognized as belonging to the Stream.
- 30 2) **priority:** A frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be
32 recognized as belonging to the Stream.
- 33 3) **all:** A frame is recognized as belonging to the Stream whether tagged or not.

35 9.1.3.3 tsnCpeSmacVlanDownVlan

36 Specifies the vlan_identifier parameter that identifies a packet in an EISS indication primitive to the Source
37 MAC and VLAN Stream identification function. ~~A value of 0 indicates that the vlan_identifier parameter is~~
38 ~~ignored on EISS indication primitives.~~

39 9.1.4 Managed objects for Active Destination MAC and VLAN Stream identifications

40 When instantiating an instance of the Active Destination MAC and VLAN Stream identification function
41 (6.6) for a particular output Stream, the managed objects in the following subclauses, ~~along with those listed~~
42 ~~in 9.1.2,~~ serve as the tsnStreamIdParameters managed object (9.1.1.7).

1 9.1.4.1 tsnCpeDmacVlanDownDestMac

2 Specifies the destination_address parameter to use in the EISS request primitive for output packets sent to
3 lower layers by the Active Destination MAC and VLAN Stream identification function, and the
4 destination_address that identifies an input packet in an EISS indication primitive to the Active Destination
5 MAC and VLAN Stream identification function.

6 9.1.4.2 tsnCpeDmacVlanDownTagged

7 An enumerated value indicating whether a packet in an EISS indication or request primitive between the
8 Active Destination MAC and VLAN Stream identification function and the lower layers is to have a VLAN
9 tag. It can take the following values:

- 10 1) **tagged:** An input frame must have a VLAN tag to be recognized as belonging to the Stream.
12 An output frame receives a VLAN tag.
- 13 2) **priority:** An input frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be
15 recognized as belonging to the Stream. An output frame is marked with a VLAN tag with
16 VLAN ID = 0.
- 17 3) **all:** A frame is recognized as belonging to the Stream whether tagged or not. An output frame is
19 to be untagged.

20 This variable is not used in an FRER C-component. See 8.4.

21 9.1.4.3 tsnCpeDmacVlanDownVlan

22 Specifies the vlan_identifier parameter to use in the EISS request primitive for output packets sent to lower
23 layers by the Active Destination MAC and VLAN Stream identification function, and the vlan_identifier
24 that identifies an input packet in an EISS indication primitive to the Active Destination MAC and VLAN
25 Stream identification function. ~~A value of 0 indicates that the vlan_identifier parameter is ignored on EISS-~~
26 ~~indication primitives.~~

27 9.1.4.4 tsnCpeDmacVlanDownPriority

28 Specifies the priority parameter to use in the EISS request primitive for output packets sent to lower layers
29 by the Active Destination MAC and VLAN Stream identification function for all packets in a particular
30 Stream.

31 9.1.4.5 tsnCpeDmacVlanUpDestMac

32 Specifies the destination_address parameter to use in the EISS indication primitive for input packets offered
33 to upper layers by the Active Destination MAC and VLAN Stream identification layer. This address
34 replaces the address that was used to identify the packet (tsnCpeDmacVlanDownDestMac, 9.1.4.1).

35 9.1.4.6 tsnCpeDmacVlanUpTagged

36 An enumerated value indicating whether a packet in an EISS indication or request primitive between the
37 Active Destination MAC and VLAN Stream identification function and the upper layers is to have a VLAN
38 tag. It can take the following values:

- 39 1) **tagged:** An output frame must have a VLAN tag to be recognized as belonging to the Stream.
41 An input frame receives a VLAN tag.

- 1 2) **priority:** An output frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be
3 recognized as belonging to the Stream. An input frame is marked with a VLAN tag with VLAN
4 ID = 0.
- 5 3) **all:** A frame is recognized as belonging to the Stream whether tagged or not. An input frame is
7 to be untagged.

8 This variable is used only by an end system and not by a relay system. See 8.4.

9 **9.1.4.7 tsnCpeDmacVlanUpVlan**

10 Specifies the vlan_identifier parameter to use in the EISS indication primitive for packets offered to upper
11 layers, or the VLAN ID field for an IEEE 802.1Q tag in an ISS mac_service_data_unit. This address
12 replaces the VLAN ID that was used to identify the packet (tsnCpeDmacVlanDownVlan, 9.1.4.3).

13 **9.1.4.8 tsnCpeDmacVlanUpPriority**

14 Specifies the priority parameter to use in the EISS indication primitive for packets offered to upper layers.

15 **9.1.5 Managed objects for IP Stream identification**

16 When instantiating an instance of the IP Stream identification function (6.7), the parameters in the following
17 subclauses replace the tsnStreamIdParameters managed object (9.1.1.7).

18 **9.1.5.1 tsnCpeIpIdDestMac**

19 ~~Specifies the destination_address parameter that identifies a packet in an EISS indication primitive. The~~
20 ~~tsnCpeIpIdDestMac parameter can either be a MAC address or a wildcard value. An address of all 0~~
21 ~~indicates that the MAC destination address is to be ignored on packets received from lower layers. If it is a~~
22 ~~MAC address, it specifies the destination_address parameter that identifies a packet in an EISS indication~~
23 ~~primitive. If it is a wildcard, it indicates the destination_address parameter from the EISS indication~~
24 ~~primitive is to be ignored on packets received from lower layers;~~

25 **9.1.5.2 tsnCpeIpIdTagged**

26 An enumerated value indicating whether a packet in an EISS indication or request primitive to the IP Stream
27 identification function is to have a VLAN tag. It can take the following values:

- 28 1) **tagged:** An input frame must have a VLAN tag to be recognized as belonging to the Stream.
30 An output frame receives a VLAN tag.
- 31 2) **priority:** An input frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be
33 recognized as belonging to the Stream. An output frame is marked with a VLAN tag with
34 VLAN ID = 0.
- 35 3) **all:** A frame is recognized as belonging to the Stream whether tagged or not. An output frame is
37 to be untagged.

38 **9.1.5.3 tsnCpeIpIdVlan**

39 Specifies the vlan_identifier parameter that identifies a packet in an EISS indication primitive. ~~A value of 0~~
40 ~~indicates that the frame is not to have a VLAN tag.~~

1 9.1.5.4 tsnCpeIpIdIpSource

2 ~~Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) source address parameter that must be matched to identify~~
3 ~~packets coming up from lower layers~~ Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) source address
4 parameter or a wildcard value. An address of all 0 indicates that the IP source address is to be ignored on
5 ~~packets received from lower layers~~ If it is an IP address, it specifies the source IP address parameter that
6 must be matched to identify packets coming up from lower layers. If it is a wildcard, it indicates the source
7 IP address is to be ignored on packets received from lower layers.

8 9.1.5.5 tsnCpeIpIdIpDestination

9 Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) destination address parameter that must be matched to
10 identify packets coming up from lower layers.

11 9.1.5.6 tsnCpeIpIdDscp

12 Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) differentiated services codepoint (DSCP, RFC 2474) that
13 must be matched to identify packets coming up from the lower layers. A value of 64 decimal indicates that
14 the DSCP is to be ignored on packets received from lower layers.

15 9.1.5.7 tsnCpeIpIdNextProtocol

16 Specifies the IP next protocol parameter that must be matched to identify packets coming up from lower
17 layers. The value of this parameter must specify either none, UDP (RFC 768), TCP (RFC 793), or SCTP
18 (RFC 4960). If “none,” then the tsnCpeIpIdSourcePort (9.1.5.8) and tsnCpeIpIdDestinationPort (9.1.5.9)
19 managed objects are not used.

20 9.1.5.8 tsnCpeIpIdSourcePort

21 Specifies the TCP or UDP Source Port parameter that must be matched to identify packets coming up from
22 lower layers. A value of 0 indicates that the Source Port number of the packet is to be ignored on packets
23 received from lower layers.

24 9.1.5.9 tsnCpeIpIdDestinationPort

25 Specifies the TCP or UDP Destination Port parameter that must be matched to identify packets coming up
26 from lower layers. A value of 0 indicates that the Destination Port number of the packet is to be ignored on
27 packets received from lower layers.

28 9.1.6 Managed objects for Mask-and-match Stream identification

29 When instantiating an instance of the Mask-and-match Stream identification function (6.8) for a particular
30 input Stream, the managed objects in the following subclauses serve as the tsnStreamIdParameters managed
31 object (9.1.1.7).

32 9.1.6.1 tsnCpeMmIdDestMacMask

33 Specifies a 48-bit mask. A bitwise AND operation is performed between tsnCpeMmIdDestMacMask and
34 the destination_address parameter passed by the ISS indication primitive to the Mask-and-match Stream
35 identification function. The resulting 48-bit information is the masked destination_address that is used as
36 input for the instance of the Mask-and-match Stream identification function. If tsnCpeMmIdDestMacMask
37 has a value of 0, the destination_address parameter is ignored.

1 9.1.6.2 tsnCpeMmIdDestMacMatch

2 Specifies the 48-bit value of the masked destination_address, as defined in 9.1.6.1, to be matched by the
3 instance of the Mask-and-match Stream identification function.

4 9.1.6.3 tsnCpeMmIdSrcMacMask

5 Specifies a 48-bit mask. A bitwise AND operation is performed between tsnCpeMmIdSrcMacMask and the
6 source_address parameter passed by the ISS indication primitive to the Mask-and-match Stream
7 identification function. The resulting 48-bit information is the masked source_address that is used as input
8 for the instance of the Mask-and-match Stream identification function. If tsnCpeMmIdSrcMacMask has a
9 value of 0, the source_address parameter is ignored.

10 9.1.6.4 tsnCpeMmIdSrcMacMatch

11 Specifies the 48-bit value of the masked source_address, as defined in 9.1.6.3, to be matched by the instance
12 of the Mask-and-match Stream identification function.

13 9.1.6.5 tsnCpeMmIdMsduMaskLength

14 Is an integer value that specifies the length, in octets, of tsnCpeMmIdMsduMask (9.1.6.6) and
15 tsnCpeMmIdMsduMatch (9.1.6.7). The behavior of the Mask-and-match Stream identification function is
16 undefined on any port where the value of tsnCpeMmIdMsduMaskLength exceeds the per-port
17 mac_service_data_unit mask maximum length (9.4.1). tsnCpeMmIdMsduMaskLength has a minimum
18 value of 2. If the length of the frame's mac_service_data_unit is shorter than
19 tsnCpeMmIdMsduMaskLength, then the identification fails.

20 9.1.6.6 tsnCpeMmIdMsduMask

21 Specifies a mask of tsnCpeMmIdMsduMaskLength octets. A bitwise AND operation is performed between
22 tsnCpeMmIdMsduMask and the tsnCpeMmIdMsduMaskLength (9.1.6.5) first octets of the
23 mac_service_data_unit parameter passed by the ISS indication primitive to the Mask-and-match Stream
24 identification function. The resulting tsnCpeMmIdMsduMaskLength octets of information are the masked
25 mac_service_data_unit that is used as input for the instance of the Mask-and-match Stream identification
26 function. If tsnCpeMmIdMsduMask has a value of 0, the mac_service_data_unit parameter is ignored.

27 9.1.6.7 tsnCpeMmIdMsduMatch

28 Specifies the value of the masked mac_service_data_unit, as defined in 9.1.6.5 and 9.1.6.6, to be matched by
29 the instance of the Mask-and-match Stream identification function.

30 NOTE—The Mask-and-match Stream identification function takes as input the ISS indication primitive parameters,
31 which include the mac_service_data_unit parameter. If specific part(s) of the mac_service_data_unit are required for the
32 identification of a Stream, the ad-hoc tsnCpeMmIdMsduMask and tsnCpeMmIdMsduMatch have to be defined. For
33 instance, when a VLAN ID is required for Stream identification, it is to be masked within the mac_service_data_unit by
34 setting 16 bits to 1 at the offset where the tag is expected (to isolate the EtherType), followed by 4 bits set to 0 (to ignore
35 the PCP and DEI fields), and ending with 12 bits set to 1 (to mask the full VID) in tsnCpeMmIdMasduMask. In a Cus-
36 tomer Bridged Network, the C-VLAN Tag is expected at an offset of 0, and the tsnCpeMmIdMsduMatch value used in
37 the Mask-and-match Stream identification function instance contains, at the same offset, a pattern matching the C-
38 VLAN EtherType (81-00), followed by 4 don't-care bits (set to 0), followed by 12 bits representing the VID value to be
39 matched.

9.2 Operational per-port per-Stream Stream identification counters

The following counters in this subclause are instantiated for each port on which the Stream identification function (6.2) is configured. They counters are indexed by port number, facing (in-facing or out-facing), and stream_handle value (tsnStreamIdHandle, 9.1.1.1). ~~All counters are unsigned integers. If used on links faster than 650 000 000 bits per second, they shall be 64 bits in length to ensure against excessively short wrap times~~Encoded as unsigned integers, they shall be 64 bits in length to insure against excessively short wrap times when used on links on which transmission rate exceeds 650 000 000 bits per second.

9.2.1 tsnCpsSidInputPackets

The tsnCpsSidInputPackets counter is incremented once for each packet identified by the Stream identification function (6.2).

9.2.2 tsnCpsSidOutputPackets

The tsnCpsSidOutputPackets counter is incremented once for each packet passed down the stack by the Stream identification function (6.2).

9.3 Operational per-port Stream identification counters

The counters in this subclause are instantiated for each port on which the Stream identification function (6.2) is configured. They are indexed by port number. Encoded as unsigned integers, they shall be 64 bits in length to insure against excessively short wrap times when used on links on which transmission rate exceeds 650 000 000 bits per second.

9.3.1 tsnCpSidInputPackets

The tsnCpSidInputPackets counter is incremented once for each packet identified by any Stream identification function (6.2) on this port. Its value equals the sum (modulo the size of the counters) of all of the tsnCpsSidInputPackets (9.2.1) counters on this same port.

9.3.2 tsnCpSidOutputPackets

The tsnCpSidOutputPackets counter is incremented once for each packet passed down the stack by any Stream identification function (6.2) on this port. Its value equals the sum (modulo the size of the counters) of all of the tsnCpsSidOutputPackets (9.2.2) counters on this same port.

9.4 Per-port managed object

9.4.1 tsnPpMmIdMsduMaskMaxLength

tsnPpMmIdMsduMaskMaxLength is a read-only variable defined for each port the Mask-and-match Stream identification function is to be run on. It defines the maximum length a mac_service_data_unit mask can have. It is an integer value, in octets, with a minimum value of 2 and a maximum value of 1984 (maximum ISS mac_service_data_unit length for an IEEE 802.3 frame).

10. Frame Replication and Elimination for Reliability management

The managed objects that control Stream identification are described in Clause 9. The managed objects that control FRER are described in this Clause 10 as follows:

- a) General requirements on the behavior of counters are in 10.1.
- b) The various tables of managed objects that can manage, in detail, each individual Stream, are described in five subclauses, including:
 - 1) Additions (10.2) to the Stream identity table (9.1) required for Autoconfiguration (7.11, 10.7).
 - 2) The Sequence generation table (10.3) that configures instances of the Sequence generation function (7.4.1);
 - 3) The Sequence recovery table (10.4) that configures instances of the Individual recovery function (7.5), the Sequence recovery function (7.4.2), and the Latent error detection function (7.4.4);
 - 4) The Sequence identification table (10.5) that configures instances of the Sequence encode/decode function (7.6); and
 - 5) The Stream split table (10.6) that configures instances of the Stream splitting function (7.7).
- c) The managed objects that support the automatic configuration, upon receipt of a packet, of entries in the first four of the preceding tables (10.2 through 10.5), are described in the subclause on Autoconfiguration (10.7).
- d) The per-port, per-Stream packet counters that are kept by FRER functions for inspection by network management entities are described in 10.8, and the per-port (totaled over all Streams) counters in 10.9.

The managed objects in the subclauses under 9.1 make it possible to configure more than one encapsulation for the same stream_handle subparameter on the same port. Similarly, the managed objects in the subclauses under 10.3 and 10.4 make it possible to configure more than one Sequence encode/decode function (7.6) or more than one Sequence generation function (7.4.1) for the same stream_handle subparameter. [The same value of stream_handle can be in the frerSeqGenStreamList (10.3.1.1) of more than one frerSeqGenEntry (10.3.1) or in the frerSeqRcvyStreamList (10.4.1.1) of more than one frerSeqRcvyEntry (10.4.1).] A system shall return an error if an attempt is made to configure conflicting requirements upon that system.

10.1 Counter behavior

All counters defined by this standard (e.g., frerCpsSeqEncErroredPackets, 10.8.11) when incremented past the maximum value representable shall roll over to 0 and continue. There is no provision for resetting any counter to any specific value, e.g., 0. A counter can be set to any value when the system is reset. This standard assumes the following:

- a) Any management entity will read any counters often enough that at most one rollover can occur between examinations;
- b) There exists a “time of last counter reset” or “time since last counter reset” or an equivalent managed object in the system, that the management entity can examine to determine whether the counters have been reset since the last time they were examined.
- c) A counter managed object is large enough to take at least one minute to roll over from 0 to 0.

10.2 Additional tsnStreamIdEntry managed objects

Two managed objects augment each tsnStreamIdEntry (9.1.1) in the Stream identity table (9.1) when Managed objects for autoconfiguration (7.11, 10.7) is implemented.

1 10.2.1 **tsnStreamIdAutoconfigured**

2 A read-only Boolean value, supplied by the system, specifying whether this entry was created explicitly
3 (False) or via the Sequence autoconfiguration table (10.7.1, True).

4 10.2.2 **tsnStreamIdLanPathId**

5 An integer specifying a path or LAN. If and only if a packet matches an entry in the Sequence identification
6 table (10.5) that specifies HSR or PRP in its `frerSeqEncEncapsType` (10.5.1.5) object,
7 `tsnStreamIdLanPathId` specifies the `LanId` or `PathId` value that must be matched for this `tsnStreamIdEntry` to
8 apply. A value of `-1` indicates that the `LanId` or `PathId` are to be ignored.

9 10.3 **Sequence generation table**

10 There is one Sequence generation table in a system, and one entry in the Sequence generation table for each
11 Sequence generation function (7.4.1).

12 10.3.1 **frerSeqGenEntry**

13 Each `frerSeqGenEntry` lists the Streams (10.3.1.1) and direction (10.3.1.2) for which a single instance of the
14 Sequence generation function (7.4.1) is to be placed.

15 10.3.1.1 **frerSeqGenStreamList**

16 A list of `stream_handle` values, corresponding to the values of the `tsnStreamIdHandle` objects (9.1.1.1) in the
17 Stream identity table (9.1), on which this instance of the Sequence generation function (7.4.1) is to operate.
18 The single instance of the Sequence generation function created by this `frerSeqGenEntry` operates every
19 packet belonging to this Stream, regardless of the port on which it is received.

20 10.3.1.2 **frerSeqGenDirection**

21 A Boolean object indicating whether the Sequence generation function (7.4.1) is to be placed on the
22 out-facing (True) or in-facing (False) side of the port (Figure 6-6).

23 10.3.1.3 **frerSeqGenReset**

24 A Boolean object indicating that the Sequence generation function (7.4.1) is to be reset by calling its
25 corresponding `SequenceGenerationReset` function (7.4.1.3). Writing the value `True` to `frerSeqGenReset`
26 triggers a reset; writing the value `False` has no effect. When read, `frerSeqGenReset` always returns the value
27 `False`.

28 10.4 **Sequence recovery table**

29 There is one Sequence recovery table in a system, and one entry in the Sequence recovery table for each
30 Sequence recovery function (7.4.2) or Individual recovery function (7.5) that can also be present. The entry
31 describes a set of managed objects for the single instance of a Base recovery function (7.4.3) and Latent
32 error detection function (7.4.4) included in the Sequence recovery function or Individual recovery function.

33 10.4.1 **frerSeqRcvyEntry**

34 Each `frerSeqRcvyEntry` lists the Streams (10.4.1.1), ports (10.4.1.2), and direction (10.4.1.3) for which
35 instances of a Sequence recovery function (7.4.2) or Individual recovery function (7.5) are to be instantiated.

1 10.4.1.1 frerSeqRcvyStreamList

2 A list of the stream_handle values, corresponding to the values of the tsnStreamIdHandle objects (9.1.1.1) in
3 the Stream identity table (9.1), to which the system is to apply the instance of the Sequence recovery
4 function (7.4.2) or Individual recovery function (7.5).

5 10.4.1.2 frerSeqRcvyPortList

6 The list of ports on each of which the system is to instantiate the Sequence recovery function (7.4.2), or from
7 which received packets are to be fed to a single instance of the Individual recovery function (7.5).

8 10.4.1.3 frerSeqRcvyDirection

9 A Boolean object indicating whether the Sequence recovery function (7.4.2) or Individual recovery function
10 (7.5) is to be placed on the out-facing (True) or in-facing (False) side of the port (Figure 6-6).

11 10.4.1.4 frerSeqRcvyReset

12 A Boolean object indicating that the Sequence recovery function (7.4.2) or Individual recovery function
13 (7.5) is to be reset by calling its corresponding [SequenceGenerationResetSequenceRecoveryReset](#) function
14 (7.4.3.3). Writing the value True to frerSeqRcvyReset triggers a reset; writing the value False has no effect.
15 When read, frerSeqRcvyReset always returns the value False.

16 10.4.1.5 frerSeqRcvyAlgorithm

17 This object is an enumerated value specifying which sequence recovery algorithm is to be used for this
18 instance of the Sequence recovery function (7.4.2). The enumeration uses an OUI or CID as shown in
19 Table 10-1. The default value for frerSeqRcvyAlgorithm is **Vector_Alg** (00-80-C2, 0).

Table 10-1—Enumerated values for frerSeqRcvyAlgorithm

Enumeration	OUI/CID	Type number	Sequence recovery function algorithm
Vector_Alg	00-80-C2	0	VectorRecoveryAlgorithm (7.4.3.4)
Match_Alg	00-80-C2	1	MatchRecoveryAlgorithm (7.4.3.5)
—	00-80-C2	2–255	Reserved
—	Other	—	Defined by entity owning the OUI or CID

20 10.4.1.6 frerSeqRcvyHistoryLength

21 An integer specifying how many bits of the SequenceHistory variable (7.4.3.2.2) are to be used. The
22 minimum and the default value is 2, maximum is the maximum allowed by the implementation. [Not used if
23 frerSeqRcvyAlgorithm (10.4.1.5) = **Match_Alg** (00-80-C2, 1).]

24 10.4.1.7 frerSeqRcvyResetMSec

25 An unsigned integer specifying the timeout period in milliseconds for the RECOVERY_TIMEOUT event
26 (item c in 7.4.3.1).

1 10.4.1.8 frerSeqRcvyInvalidSequenceValue

2 A read-only unsigned integer value that cannot be encoded in a packet as a value for the sequence_number
3 subparameter (item b in 6.1), i.e., frerSeqRcvyInvalidSequenceValue is larger than or equal to
4 RecovSeqSpace (7.4.3.2.1).

5 10.4.1.9 frerSeqRcvyTakeNoSequence

6 A Boolean value specifying whether packets with no sequence_number subparameter are to be accepted
7 (True) or not (False). Default value False. See item i in 7.1.1.

8 10.4.1.10 frerSeqRcvyIndividualRecovery

9 A Boolean value specifying whether this entry describes a Sequence recovery function (7.4.2) or Individual
10 recovery function (7.5).

11 a) **True:** The entry describes an Individual recovery function (7.5). ~~Packets discarded by the~~
12 ~~SequenceGenerationAlgorithm (7.4.1.4) will cause the variable RemainingTicks (7.4.3.2.4) to be~~
13 ~~reset~~ Packets discarded by the VectorRecoveryAlgorithm (7.4.3.4) or the MatchRecoveryAlgorithm
14 (7.4.3.5) will cause the variable RemainingTicks (7.4.3.2.4) to be reset. There is no Latent error
15 detection function (7.4.4) associated with this entry, so frerSeqRcvyLatentErrorDetection
16 (10.4.1.11) cannot also be True.

17 b) **False:** The entry describes a Sequence recovery function (7.4.2). ~~Packets discarded by the~~
18 ~~SequenceGenerationAlgorithm (7.4.1.4) will not cause~~ Packets discarded by the VectorRecoveryAl-
19 ~~gorithm (7.4.3.4) or the MatchRecoveryAlgorithm (7.4.3.5) will not cause~~ the variable Remaining-
20 Ticks (7.4.3.2.4) to be reset.

21 10.4.1.11 frerSeqRcvyLatentErrorDetection

22 A Boolean value indicating whether an instance of the Latent error detection function (7.4.4) is to be
23 instantiated along with the Base recovery function (7.4.3) in this Sequence recovery function (7.4.2) or
24 Individual recovery function (7.5). frerSeqRcvyLatentErrorDetection cannot be set True if
25 frerSeqRcvyIndividualRecovery (10.4.1.10) is also True; an Individual recovery function does not include a
26 Latent error detection function.

27 10.4.1.12 Latent error detection managed objects

28 The objects in the following subclauses are present if and only if frerSeqRcvyIndividualRecovery
29 (10.4.1.10) is False.

30 10.4.1.12.1 frerSeqRcvyLatentErrorDifference

31 An integer specifying the maximum difference between frerCpsSeqRcvyDiscardedPackets (10.8.6), and the
32 product of frerCpsSeqRcvyPassedPackets (10.8.5) and (frerSeqRcvyLatentErrorPaths – 1) (10.4.1.12.3) that
33 is allowed. Any larger difference will trigger the detection of a latent error by the LatentErrorTest function
34 (7.4.4.4).

35 10.4.1.12.2 frerSeqRcvyLatentErrorPeriod

36 The integer number of milliseconds that are to elapse between instances of running the LatentErrorTest
37 function (7.4.4.4). An implementation can have a minimum value for frerSeqRcvyLatentErrorPeriod, below
38 which it cannot be set, but this minimum shall be no larger than 1000 ms (1 s). Default value 2000 (2 s).

1 **10.4.1.12.3 frerSeqRcvyLatentErrorPaths**

2 The integer number of paths over which FRER is operating for this instance of the Base recovery function
3 (7.4.3) and Latent error detection function (7.4.4).

4 **10.4.1.12.4 frerSeqRcvyLatentResetPeriod**

5 The integer number of milliseconds that are to elapse between instances of running the LatentErrorReset
6 function (7.4.4.3). An implementation can have a minimum value for LatentErrorReset, below which it
7 cannot be set, but this minimum shall be no larger than 1000 ms (1 s). Default value 30000 (30 s).

8 **10.5 Sequence identification table**

9 There is one Sequence identification table per system, and one entry in the Sequence identification table for
10 each port and direction for which an instance of the Sequence encode/decode function (7.6) is to be created.

11 **10.5.1 frerSeqEncEntry**

12 Each entry in the Sequence identification table specifies a port (10.5.1.2) and direction (10.5.1.3) on which
13 an instance of the Sequence encode/decode function is to be instantiated for a list of Streams (10.5.1.1).

14 **10.5.1.1 frerSeqEncStreamList**

15 A list of stream_handles, corresponding to the values of the tsnStreamIdHandle objects (9.1.1.1) in the
16 Stream identity table (9.1), for which the system is to use the same encapsulation (10.5.1.5) for the Sequence
17 encode/decode function.

18 **10.5.1.2 frerSeqEncPort**

19 The port on which the system is to place an instance of the Sequence encode/decode function (7.6).

20 **10.5.1.3 frerSeqEncDirection**

21 A Boolean object indicating whether the Sequence encode/decode function (7.6) is to be placed on the
22 out-facing (True) or in-facing (False) side of the port (Figure 6-6).

23 **10.5.1.4 frerSeqEncActive**

24 A Boolean value specifying whether this frerSeqEncEntry is passive (False), and therefore is used only to
25 decode (extract information from) input packets passing up the protocol stack, or active (True), and
26 therefore is used both for recognizing input packets and for encoding output packets being passed down the
27 protocol stack.

28 **10.5.1.5 frerSeqEncEncapsType**

29 An enumerated value indicating the type of encapsulation used for this instance of the Sequence encode/
30 decode function (7.6). The type includes an OUI or CID. The values defined by this standard are shown in
31 Table 10-2.

32 **10.5.1.6 frerSeqEncPathIdLanId**

33 A 4-bit integer value to be placed in the PathId field of an HSR sequence tag (7.9) or the LanId field of a
34 PRP sequence trailer (7.10) added to an output packet. This managed object is used only if:

Table 10-2—Sequence Encode/Decode types

OUI/CID	Type number	Sequence encode/decode method
00-80-C2	0	Reserved
00-80-C2	1	R-TAG (7.8)
00-80-C2	2	HSR sequence tag (7.9)
00-80-C2	3	PRP sequence trailer (7.10)
00-80-C2	4–255	Reserved
Other	—	Defined by entity owning the OUI or CID

- 1 a) The HSR sequence tag or the PRP sequence trailer is selected by the `frerSeqEncEncapsType` object (10.5.1.5); and
- 2
- 3 b) `frerSeqEncActive` (10.5.1.4) is False (passive)

4 10.6 Stream split table

5 There is one Stream split table per system, with one `frerSplitEntry` (10.6.1) per Stream splitting function (7.7) per set of `stream_handle` values.

7 10.6.1 `frerSplitEntry`

8 Each entry in the Stream split table specifies a port (10.6.1.1) and direction (10.6.1.2) on which an instance (9 of the Stream splitting function (7.7) is to be instantiated, and the list of `stream_handles` specifying its (10 operation.

11 10.6.1.1 `frerSplitPort`

12 The port on which the system is to place an instance of the Stream splitting function (7.7) performing the (13 `stream_handle` translations specified by `frerSplitInputIdList` and `frerSplitOutputIdList` (10.6.1.3, 10.6.1.4) is (14 to be placed.

15 10.6.1.2 `frerSplitDirection`

16 A Boolean object indicating whether the instance of the Stream splitting function (7.7) performing the (17 `stream_handle` translations specified by `frerSplitInputIdList` and `frerSplitOutputIdList` (10.6.1.3, 10.6.1.4) is (18 to be placed on the out-facing (True) or in-facing (False) side of the port (Figure 6-6).

19 10.6.1.3 `frerSplitInputIdList`

20 A list of `stream_handles` (`tsnStreamIdHandle` values, 9.1.1.1) that are to be split.

21 10.6.1.4 `frerSplitOutputIdList`

22 A list of `stream_handles` (`tsnStreamIdHandle` values, 9.1.1.1) into which the input packet is to be split, one (23 copy per item in the `frerSplitOutputIdList`.

1 10.7 Managed objects for autoconfiguration

2 10.7.1 Sequence autoconfiguration table

3 There is one Sequence autoconfiguration table per system. It contains any number of table entries (10.7.1.1).
4 No two (or more) entries in the Sequence autoconfiguration table can have the same values for
5 frerAutSeqSeqEncaps (10.7.1.1.1), frerAutSeqTagged (10.7.1.1.3), and frerAutSeqVlan (10.7.1.1.4) on any
6 given port (10.7.1.1.2).

7 10.7.1.1 frerAutSeqEntry

8 Each frerAutSeqEntry objects (10.7.1.1) relates to a single class of Streams, and specifies how entries are
9 created (and destroyed) in the Stream identity table (9.1), the Sequence recovery table (10.4), and the
10 Sequence identification table (10.5).

11 10.7.1.1.1 frerAutSeqSeqEncaps

12 An enumerated value from Table 10-2, specifying which Sequence encode/decode function, and therefore,
13 which type sequence_number encoding, is to be recognized for the purposes of Autoconfiguration.

14 10.7.1.1.2 frerAutSeqReceivePortList

15 The list of ports to which this frerAutSeqEntry applies, and on which Stream identification functions
16 (6.2), Sequence encode/decode functions (7.6), and Individual recovery functions (7.5) are to be
17 autocreated.

18 10.7.1.1.3 frerAutSeqTagged

19 An enumerated value indicating whether packets to be matched by this frerAutSeqEntry are permitted to
20 have a VLAN tag. It can take the following values:

- 21 1) **tagged:** A frame must have a VLAN tag to be matched.
- 23 2) **priority:** A frame must be untagged, or have a VLAN tag with a VLAN ID = 0 to be matched.
- 25 3) **all:** A frame is matched whether tagged or not.

27 10.7.1.1.4 frerAutSeqVlan

28 A list of vlan_identifiers for the packet to match. A null list matches all vlan_identifiers.

29 10.7.1.1.5 frerAutSeqRecoveryPortList

30 The list of ports on which Sequence recovery functions (7.4.2) are to be autocreated by this
31 frerAutSeqEntry.

32 10.7.1.1.6 frerAutSeqDestructMSec

33 An integer number of milliseconds after which an idle set of functions created by this frerAutSeqEntry can
34 be destroyed. A value of 0 indicates that idle autoconfigured functions are not to be destroyed. Default value
35 is 86 400 000 decimal (one day).

36 10.7.1.1.7 frerAutSeqResetMSec

37 The value used to fill frerSeqRcvyResetMSec (10.4.1.7) when autoconfiguring entries in the Sequence
38 recovery table.

1 **10.7.1.1.8 frerAutSeqAlgorithm**

2 The value used to fill frerSeqRcvyAlgorithm (10.4.1.5) when autoconfiguring entries in the Sequence
3 recovery table.

4 **10.7.1.1.9 frerAutSeqHistoryLength**

5 The value used to fill frerSeqRcvyHistoryLength (10.4.1.6) when autoconfiguring entries in the Sequence
6 recovery table.

7 **10.7.1.1.10 frerAutSeqCreateIndividual**

8 A Boolean value. If True, the receipt of a packet that triggers the autoconfiguration of a new
9 tsnStreamIdEntry also triggers the instantiation of a frerSeqRcvyEntry for an Individual recovery function.

10 **10.7.1.1.11 frerAutSeqCreateRecovery**

11 A Boolean value. If True, the receipt of a packet that triggers the autoconfiguration of a new
12 tsnStreamIdEntry can also trigger the instantiation of a frerSeqRcvyEntry for a Sequence recovery function.

13 **10.7.1.1.12 frerAutSeqLatErrDetection**

14 A Boolean value. If True, the autoconfiguration of a new Sequence recovery function also creates an
15 associated Latent Error Detection function.

16 **10.7.1.1.13 frerAutSeqLatErrDifference**

17 The value used to fill frerSeqRcvyLatentErrorDifference (10.4.1.12.1) when autoconfiguring entries in the
18 Sequence recovery table.

19 **10.7.1.1.14 frerAutSeqLatErrPeriod**

20 The value used to fill frerSeqRcvyLatentErrorPeriod (10.4.1.12.2) when autoconfiguring entries in the
21 Sequence recovery table.

22 **10.7.1.1.15 frerAutSeqLatErrResetPeriod**

23 The value used to fill frerSeqRcvyLatentResetPeriod (10.4.1.12.4) when autoconfiguring entries in the
24 Sequence recovery table.

25 **10.7.2 Output autoconfiguration table**

26 There is one Output autoconfiguration table per system. It contains any number of frerAutOutEntry objects
27 (10.7.2.1), each relating to a single class of Streams specifying how active entries are created in the
28 Sequence identification table (10.5).

29 **10.7.2.1 frerAutOutEntry**

30 No two (or more) entries in the Output autoconfiguration table can include the same port in their
31 frerAutSeqReceivePortList objects (10.7.1.1.2).

1 **10.7.2.1.1 frerAutOutPortList**

2 The list of ports to which this frerAutOutEntry applies, and on which active Sequence encode/decode
3 functions (7.6) are to be autocreated.

4 **10.7.2.1.2 frerAutOutEncaps**

5 An enumerated value from Table 10-2, specifying which Sequence encode/decode function, and therefore,
6 which type sequence_number encoding, is to be used for autoconfigured Streams on the ports in
7 frerAutSeqReceivePortList (10.7.1.1.2).

8 **10.7.2.1.3 frerAutOutLanPathId**

9 An integer specifying a path or LAN. If and only if frerAutOutEncaps (10.7.2.1.2) specifies HSR or PRP
10 frerAutOutLanPathId specifies the LanId or PathId value to be inserted into the HSR sequence tag or PRP
11 sequence trailer of autoconfigured packets transmitted on the ports in frerAutSeqReceivePortList
12 (10.7.1.1.2).

13 **10.8 Operational per-port and per-Stream FRER counters**

14 The following counters are instantiated for each port on which any of the Stream identification function
15 (6.2), Sequencing function (7.4), or Sequence encode/decode function (7.6) is configured. The counters are
16 indexed by port number, facing (in-facing or out-facing), and stream_handle value
17 (tsnStreamIdHandle, 9.1.1.1). All counters are unsigned integers. If used on links faster than
18 650 000 000 bits per second, they shall be 64 bits in length to insure against excessively short wrap times.

19 A Stream identification component (5.3) shall implement the first two counters tsnCpsSidInputPackets
20 (9.2.1) and tsnCpsSidOutputPackets (9.2.2); the remainder of the counters in 10.8 are optional for such a
21 system.

22 **10.8.1 Per-Stream vs. per-Stream-per-port counters**

23 The preceding managed objects create an instantiation of the Sequence recovery function (7.4.2) per port. In
24 fact, a relay system can choose to create a single instance of the Sequence recovery function as part of its
25 forwarding function, or one instance per line card, or in some other, distributed fashion, without violating
26 the externally visible behaviors specified by this standard. The per-instantiation variables defined in
27 Clause 7 are internal to the system, so their definitions are not affected by this choice.

28 However, when the algorithms of Clause 7 [e.g., the MatchRecoveryAlgorithm (7.4.3.5)] reference counters
29 [e.g., frerCpsSeqRcvyPassedPackets (10.8.5)] defined as follows, this choice becomes important. Whenever
30 an algorithm in Clause 7 increments a counter in the following subclauses, it increments each of the relevant
31 counters, according to the ports on which the packet is (or would have been) output, no matter how many
32 instantiations of the Sequence recovery function actually exist. Thus, if a duplicate packet that would have
33 been output on ports 1, 6, and 27 is discarded by a single instantiation of the Sequence recovery function
34 residing in a relay system's forwarding function, three instances of the frerCpsSeqRcvyDiscardedPackets
35 (10.8.6) counter are incremented, one for each of those ports.

36 **10.8.2 frerCpsSeqGenResets**

37 The frerCpsSeqGenResets counter is incremented each time the SequenceGenerationReset function
38 (7.4.1.3) is called.

1 **10.8.3 frerCpsSeqRcvyOutOfOrderPackets**

2 The frerCpsSeqRcvyOutOfOrderPackets counter is incremented once for each packet accepted out-of-order
3 by the VectorRecoveryAlgorithm (7.4.3.4) or MatchRecoveryAlgorithm (7.4.3.5). Out-of-order means that
4 the packet's sequence number is not one more than the previous packet received. (See item m in 7.1.1.)

5 **10.8.4 frerCpsSeqRcvyRoguePackets**

6 The frerCpsSeqRcvyRoguePackets counter is incremented once for each packet discarded by the
7 VectorRecoveryAlgorithm (7.4.3.4) because its sequence_number subparameter is more than
8 frerSeqRcvyHistoryLength (10.4.1.6) from RecvSeqNum (7.4.3.2.3).

9 **10.8.5 frerCpsSeqRcvyPassedPackets**

10 The frerCpsSeqRcvyPassedPackets counter is incremented once for each packet passed up the stack by the
11 VectorRecoveryAlgorithm (7.4.3.4) or MatchRecoveryAlgorithm (7.4.3.5).

12 **10.8.6 frerCpsSeqRcvyDiscardedPackets**

13 The frerCpsSeqRcvyDiscardedPackets counter is incremented once for each packet discarded due to a
14 duplicate sequence number by the VectorRecoveryAlgorithm (7.4.3.4) or MatchRecoveryAlgorithm
15 (7.4.3.5).

16 **10.8.7 frerCpsSeqRcvyLostPackets**

17 The frerCpsSeqRcvyLostPackets counter is incremented once for each packet lost by the
18 VectorRecoveryAlgorithm (7.4.3.4). A packet is counted as lost if its sequence number is not received on
19 any ingress port.

20 NOTE—If per-source sequence numbering is used, frerCpsSeqRcvyLostPackets can count, as lost, packets that were
21 sent to another destination, but not lost. See B.2.

22 **10.8.8 frerCpsSeqRcvyTaglessPackets**

23 The frerCpsSeqRcvyTaglessPackets counter is incremented once for each packet received by the
24 VectorRecoveryAlgorithm (7.4.3.4) or the MatchRecoveryAlgorithm (7.4.3.5) that has no
25 sequence_number subparameter (item b in 6.1).

26 **10.8.9 frerCpsSeqRcvyResets**

27 The frerCpsSeqRcvyResets counter is incremented once each time the SequenceRecoveryReset function
28 (7.4.3.3) is called.

29 **10.8.10 frerCpsSeqRcvyLatentErrorResets**

30 The frerCpsSeqRcvyLatentErrorResets counter is incremented once each time the LatentErrorReset
31 function (7.4.4.3) is called.

32 **10.8.11 frerCpsSeqEncErroredPackets**

33 The frerCpsSeqEncErroredPackets counter is incremented once each time the Sequence encode/decode
34 function (7.6) receives a packet that it is unable to decode successfully.

10.9 Operational per-port FRER counters

The following counters are instantiated for each port on which any of the Stream identification function (6.2), Sequencing function (7.4), or Sequence encode/decode function (7.6) is configured. The counters are indexed by port number and facing (in-facing or out-facing). All counters are unsigned integers. If used on links faster than 650 000 000 bits per second, they shall be 64 bits in length to ensure against excessively short wrap times. Encoded as unsigned integers, they shall be 64 bits in length to insure against excessively short wrap times when used on links on which transmission rate exceeds 650 000 000 bits per second. A Stream identification component (5.3) shall implement the first two counters `tsnCpSidInputPackets` (9.3.1) and `tsnCpSidOutputPackets` (9.3.2); the remainder of the counters in 10.9 are optional for such a system.

A Stream identification component (5.3) shall implement the first two counters `tsnCpSidInputPackets` (9.3.1) and `tsnCpSidOutputPackets` (9.3.2); the remainder of the counters in 10.9 are optional for such a system.

10.9.1 `frerCpSeqRcvyPassedPackets`

The `frerCpSeqRcvyPassedPackets` counter is incremented once for each packet passed up the stack by the `VectorRecoveryAlgorithm` (7.4.3.4) or `MatchRecoveryAlgorithm` (7.4.3.5). Its value equals the sum (modulo the size of the counters) of all of the `frerCpsSeqRcvyPassedPackets` (10.8.5) counters on this same port.

10.9.2 `frerCpSeqRcvyDiscardPackets`

The `frerCpSeqRcvyDiscardPackets` counter is incremented once for each packet discarded due to a duplicate sequence number or for being a rogue packet by any `VectorRecoveryAlgorithm` (7.4.3.4) or `MatchRecoveryAlgorithm` (7.4.3.5) on this port. Its value equals the sum (modulo the size of the counters) of all of the `frerCpsSeqRcvyRoguePackets` (10.8.4) and `frerCpsSeqRcvyDiscardedPackets` (10.8.6) counters on this same port.

10.9.3 `frerCpSeqEncErroredPackets`

The `frerCpSeqEncErroredPackets` counter is incremented once each time the Sequence encode/decode function (7.6) receives a packet that it is unable to decode successfully. Its value equals the sum (modulo the size of the counters) of all of the `frerCpsSeqEncErroredPackets` (10.8.11) counters on this same port.

1 **11. Management Information Base (MIB)**

2 This clause contains a complete SMIV2 MIB set for all features of this standard.

3 **11.1 Internet Standard Management Framework**

4 The MIBs in this standard are designed to operate in a manner consistent with the principles of the Internet
5 Standard Management Framework, which describes the separation of a data modeling language (for
6 example, SMIV2) from content-specific data models (for example, the LLDP remote systems MIB), and
7 from messages and protocol operations used to manipulate the data (for example, SNMPv3).

8 For a detailed overview of the documents that describe the current Internet Standard Management
9 Framework, please refer to section 7 of IETF RFC 3410 (2002).

10 Managed objects are accessed via a virtual information store, termed the Management Information Base or
11 MIB. MIB objects are generally accessed through the Simple Network Management Protocol (SNMP).
12 Objects in the MIB are defined using the mechanisms defined in the Structure of Management Information
13 (SMI). This clause specifies a MIB module that is compliant to the SMIV2, which is described in IETF RFC
14 2578 (STD 58), IETF RFC 2579 (STD 58), and IETF RFC 2580 (STD 58).

15 **11.2 Structure of the MIB**

16 The IEEE Std 802.1CB™ MIB is divided into a number of MIB modules. A summary of the modules
17 contained in this clause is represented in Table 11-1. The general MIB table structures for the MIB modules
18 contained in this clause are illustrated in Figure 11-1.

Table 11-1—Structure of the MIB modules

Module	Subclause	References	Notes
IEEE8021-STREAM-IDENTIFICATION-MIB	11.5.1	Clause 9	MIB model for Stream identification.
IEEE8021-FRER-MIB	11.5.2	Clause 10	MIB model for FRER.

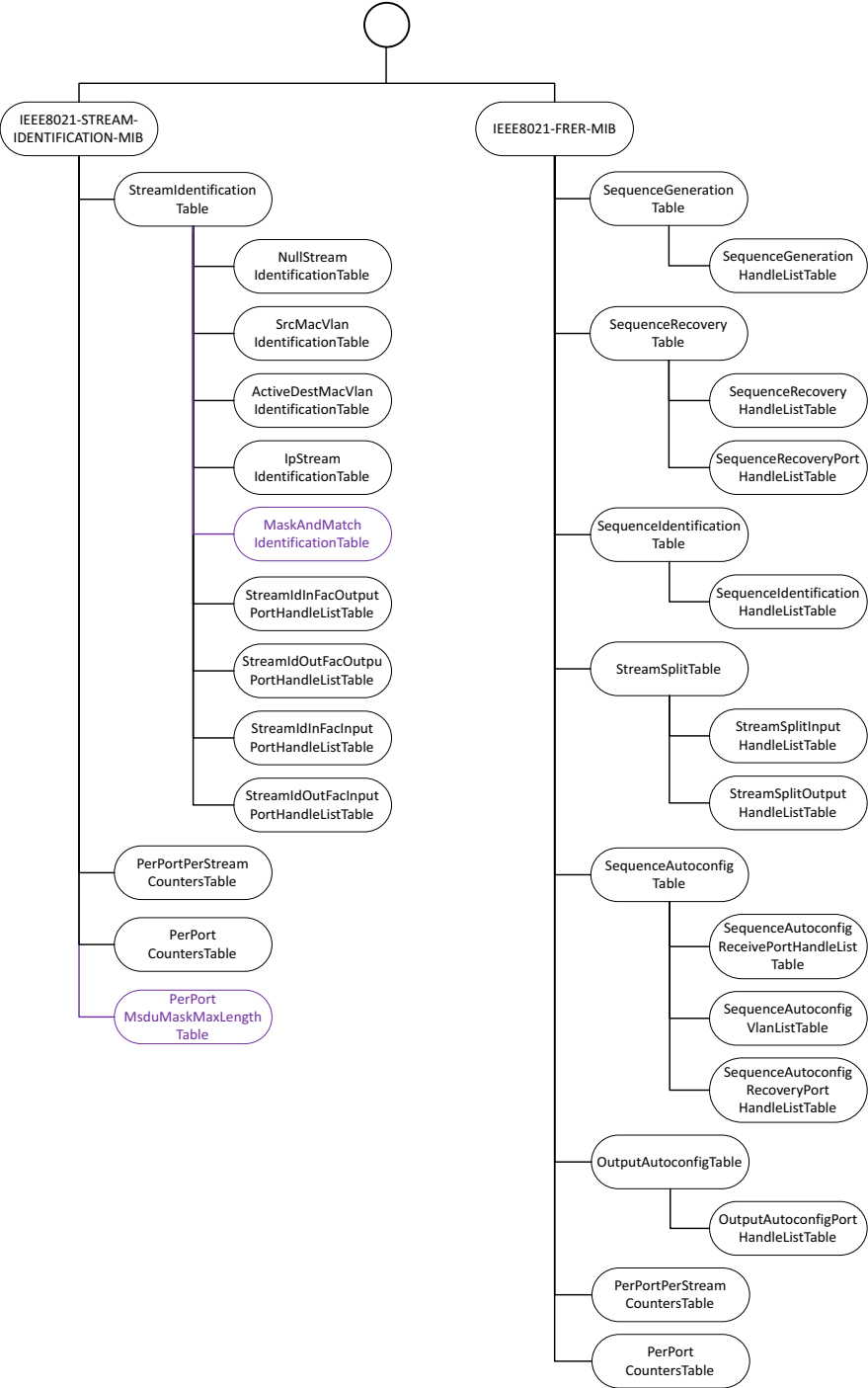


Figure 11-1—MIB modules structure

1 11.2.1 Structure of the IEEE8021-STREAM-IDENTIFICATION-MIB

2 The IEEE8021-STREAM-IDENTIFICATION-MIB module is divided into a number of branches
3 (e.g., subtrees). Each subtree is organized as a set of related objects. Where appropriate, the corresponding
4 Clause 9 management reference is also included.

5 Table 11-2 indicates the structure of the IEEE8021-STREAM-IDENTIFICATION-MIB module.

Table 11-2—IEEE8021-STREAM-IDENTIFICATION-MIB structure and relationship to this standard

Clause 11 MIB table/object	Reference
ieee8021StreamIdStreamIdentificationTable	9.1, 9.1.1
ieee8021StreamIdStreamIdentificationIndex	—
ieee8021StreamIdStreamIdIdentificationType	—
ieee8021StreamIdStreamIdIdentificationTypeOUI	—
ieee8021StreamIdStreamIdIdentificationCustomType	—
ieee8021StreamIdStreamIdIdentificationCustomTypeOUI	—
ieee8021StreamIdStreamIdIdentificationTypeSelect	—
ieee8021StreamIdStreamIdIdentificationTypeSelect	—
ieee8021StreamIdStreamIdInFacOutputPortList	—
ieee8021StreamIdStreamIdOutFacOutputPortList	—
ieee8021StreamIdStreamIdInFacInputPortList	—
ieee8021StreamIdStreamIdOutFacInputPortList	—
ieee8021StreamIdAutoConfigured	—
ieee8021StreamIdLanPathId	—
ieee8021StreamIdStatus	—
ieee8021StreamIdNullStreamIdentificationTable	9.1.2
ieee8021StreamIdCpeNullDownDestMac	—
ieee8021StreamIdCPENullDownTagged	—
ieee8021StreamIdCpeNullDownVlan	—
ieee8021StreamIdSrcMacVlanIdentificationTable	9.1.3
ieee8021StreamIdCpeSmacVlanDownSrcMac	—
ieee8021StreamIdCpeSmacVlanDownTagged	—
ieee8021StreamIdCpeSmacVlanDownVlan	—
ieee8021StreamIdActiveDestMacVlanIdentificationTable	9.1.4
ieee8021StreamIdCpeDmacVlanDownDestMac	—
ieee8021StreamIdCpeDmacVlanDownTagged	—

Table 11-2—IEEE8021-STREAM-IDENTIFICATION-MIB structure and relationship to this standard (*continued*)

Clause 11 MIB table/object	Reference
ieee8021StreamIdCpeDmacVlanDownVlan	—
ieee8021StreamIdCpeDmacVlanDownPriority	—
ieee8021StreamIdCpeDmacVlanUpDestMac	—
ieee8021StreamIdCpeDmacVlanUpTagged	—
ieee8021StreamIdCpeDmacVlanUpVlan	—
ieee8021StreamIdCpeDmacVlanUpPriority	—
ieee8021StreamIdIpStreamIdentificationTable	9.1.5
ieee8021StreamIdCpeIpIdDestMac	—
ieee8021StreamIdCpeIpIdTagged	—
ieee8021StreamIdCpeIpIdVlan	—
ieee8021StreamIdCpeIpIdIpSourceType	—
ieee8021StreamIdCpeIpIdIpSource	—
ieee8021StreamIdCpeIpIdIpDestinationType	—
ieee8021StreamIdCpeIpIdIpDestination	—
ieee8021StreamIdCpeIpIdDscp	—
ieee8021StreamIdCpeIpIdNextProtocol	—
ieee8021StreamIdCpeIpIdSourcePort	—
ieee8021StreamIdCpeIpIdDestinationPort	—
ieee8021StreamIdMaskAndMatchIdentificationTable	9.1.6
ieee8021StreamIdCpeMmIdDestMacMask	—
ieee8021StreamIdCpeMmIdDestMacMatch	—
ieee8021StreamIdCpeMmIdSrcMacMask	—
ieee8021StreamIdCpeMmIdSrcMacMatch	—
ieee8021StreamIdCpeMmIdMsduMaskLength	—
ieee8021StreamIdCpeMmIdMsduMask	—
ieee8021StreamIdCpeMmIdMsduMatch	—
ieee8021StreamIdStreamIdInFacOutputPortHandleListTable	9.1.1.2
ieee8021StreamIdStreamIdInFacOutputPortHandleListIndex	—
ieee8021StreamIdStreamIdInFacOutputPortHandle	—
ieee8021StreamIdStreamIdInFacOutputPortHandleListStatus	—
ieee8021StreamIdStreamIdOutFacOutputPortHandleListTable	9.1.1.3
ieee8021StreamIdStreamIdOutFacOutputPortHandleListIndex	—

Table 11-2—IEEE8021-STREAM-IDENTIFICATION-MIB structure and relationship to this standard (*continued*)

Clause 11 MIB table/object	Reference
ieee8021StreamIdStreamIdOutFacOutputPortHandle	—
ieee8021StreamIdStreamIdOutFacOutputPortHandleListStatus	—
ieee8021StreamIdStreamIdInFacInputPortHandleListTable	9.1.1.2
ieee8021StreamIdStreamIdInFacInputPortHandleListIndex	—
ieee8021StreamIdStreamIdInFacInputPortHandle	—
ieee8021StreamIdStreamIdInFacInputPortHandleListStatus	—
ieee8021StreamIdStreamIdOutFacInputPortHandleListTable	9.1.1.5
ieee8021StreamIdStreamIdOutFacInputPortHandleListIndex	—
ieee8021StreamIdStreamIdOutFacInputPortHandle	—
ieee8021StreamIdStreamIdOutFacInputPortHandleListStatus	—
ieee8021StreamIdPerPortPerStreamCountersTable	9.2
ieee8021StreamIdPerPortPerStreamDirection	—
ieee8021StreamIdPerPortPerStreamInputPackets	—
ieee8021StreamIdPerPortPerStreamOutputPackets	—
ieee8021StreamIdPerPortCountersTable	9.3
ieee8021StreamIdPerPortInputPackets	—
ieee8021StreamIdPerPortOutputPackets	—
ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthTable	9.4.1
ieee8021StreamIdMaskAndMatchMsduMaskMaxLength	—

1 11.2.2 Structure of the IEEE8021-FRER-MIB

2 The IEEE8021-FRER-MIB module is divided into a number of branches (e.g., subtrees). Each subtree is
3 organized as a set of related objects. Where appropriate, the corresponding Clause 9 management reference
4 is also included.

5 Table 11-3 indicates the structure of the IEEE8021-FRER-MIB module.

6 11.3 Relationship to other MIBs

7 In order to facilitate interoperable management of the features defined in this standard by remote means,
8 11.5 contains a complete set of SMiv2 MIB modules. This subclause describes how the MIB modules
9 defined in this standard are related to the MIB modules that are imported.

10 11.3.1 Relationship of the IEEE8021-STREAM-IDENTIFICATION-MIB to other MIBs

11 The IEEE8021-STREAM-IDENTIFICATION-MIB identifies the interfaces for the counters provided by
12 this standard by an ifIndex and therefore imports from the IF-MIB.

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard

Clause 11 MIB table/object	Reference
ieee8021FrerSequenceGenerationTable	10.3
ieee8021FrerSequenceGenerationIndex	—
ieee8021FrerSequenceGenerationStreamList	—
ieee8021FrerSequenceGenerationDirection	—
ieee8021FrerSequenceGenerationReset	—
ieee8021FrerSequenceGenerationHandleListTable	10.3.1.1
ieee8021FrerSequenceGenerationHandleListIndex	—
ieee8021FrerSequenceGenerationStreamHandle	—
ieee8021FrerSequenceGenerationHandleListStatus	—
ieee8021FrerSequenceRecoveryTable	10.4
ieee8021FrerSequenceRecoveryIndex	—
ieee8021FrerSequenceRecoveryStreamList	—
ieee8021FrerSequenceRecoveryPortList	—
ieee8021FrerSequenceRecoveryDirection	—
ieee8021FrerSequenceRecoveryReset	—
ieee8021FrerSequenceRecoveryAlgorithm	—
ieee8021FrerSequenceRecoveryAlgorithmOUI	—
ieee8021FrerSequenceCustomRecoveryAlgorithm	—
ieee8021FrerSequenceCustomRecoveryAlgorithmOUI	—
ieee8021FrerSequenceRecoveryHistoryLength	—
ieee8021FrerSequenceRecoveryResetMSec	—
ieee8021FrerSequenceRecoveryInvalidSequenceValue	—
ieee8021FrerSequenceRecoveryTakeNoSequence	—
ieee8021FrerSequenceRecoveryIndividualRecovery	—
ieee8021FrerSequenceRecoveryLatentErrorDetection	—
ieee8021FrerSequenceRecoveryLatentErrorDifference	—
ieee8021FrerSequenceRecoveryLatentErrorPeriod	—
ieee8021FrerSequenceRecoveryLatentErrorPaths	—
ieee8021FrerSequenceRecoveryLatentResetPeriod	—
ieee8021FrerSequenceRecoveryStatus	—
ieee8021FrerSequenceRecoveryHandleListTable	10.4.1.1
ieee8021FrerSequenceRecoveryHandleListIndex	—
ieee8021FrerSequenceRecoveryStreamHandle	—

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard (*continued*)

Clause 11 MIB table/object	Reference
ieee8021FrerSequenceRecoveryHandleListStatus	—
ieee8021FrerSequenceRecoveryPortHandleListTable	10.4.1.2
ieee8021FrerSequenceRecoveryPortHandleListIndex	—
ieee8021FrerSequenceRecoveryPortHandle	—
ieee8021FrerSequenceRecoveryPortHandleListStatus	—
ieee8021FrerSequenceIdentificationTable	10.5
ieee8021FrerSequenceIdentificationEncodePort	—
ieee8021FrerSequenceIdentificationEncodeDirection	—
ieee8021FrerSequenceIdentificationStreamList	—
ieee8021FrerSequenceIdentificationEncodeActive	—
ieee8021FrerSequenceIdentificationEncodeEncapsulationType	—
ieee8021FrerSequenceIdentificationEncodeEncapsulationOUI	—
ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType	—
ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI	—
ieee8021FrerSequenceIdentificationEncodePathIdLanId	—
ieee8021FrerSequenceIdentificationStatus	—
ieee8021FrerSequenceIdentificationHandleListTable	10.5.1.1
ieee8021FrerSequenceIdentificationHandleListIndex	—
ieee8021FrerSequenceIdentificationStreamHandle	—
ieee8021FrerSequenceIdentificationHandleListStatus	—
ieee8021FrerStreamSplitTable	10.6
ieee8021FrerStreamSplitIndex	—
ieee8021FrerStreamSplitPort	—
ieee8021FrerStreamSplitDirection	—
ieee8021FrerStreamSplitInputIdList	—
ieee8021FrerStreamSplitOutputIdList	—
ieee8021FrerStreamSplitInputHandleListTable	10.6.1.3
ieee8021FrerStreamSplitInputHandleListIndex	—
ieee8021FrerStreamSplitInputIdHandle	—
ieee8021FrerStreamSplitInputHandleListStatus	—
ieee8021FrerStreamSplitOutputHandleListTable	10.6.1.4
ieee8021FrerStreamSplitOutputHandleListIndex	—
ieee8021FrerStreamSplitOutputIdHandle	—

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard (*continued*)

Clause 11 MIB table/object	Reference
ieee8021FrerStreamSplitOutputHandleListStatus	—
ieee8021FrerSequenceAutoconfigTable	10.7, 10.7.1
ieee8021FrerSequenceAutoconfigIndex	—
ieee8021FrerSequenceAutoconfigSequenceEncapsulation	—
ieee8021FrerSequenceAutoconfigSequenceEncapsulationOUI	—
ieee8021FrerSequenceAutoconfigCustomSequenceEncapsulation	—
ieee8021FrerSequenceAutoconfigCustomSequenceEncOUI	—
ieee8021FrerSequenceAutoconfigReceivePortList	—
ieee8021FrerSequenceAutoconfigTagged	—
ieee8021FrerSequenceAutoconfigVlanList	—
ieee8021FrerSequenceAutoconfigRecoveryPortList	—
ieee8021FrerSequenceAutoconfigDestructMSec	—
ieee8021FrerSequenceAutoconfigResetMSec	—
ieee8021FrerSequenceAutoconfigAlgorithm	—
ieee8021FrerSequenceAutoconfigAlgorithmOUI	—
ieee8021FrerSequenceAutoconfigCustomAlgorithm	—
ieee8021FrerSequenceAutoconfigCustomAlgorithmOUI	—
ieee8021FrerSequenceAutoconfigHistoryLength	—
ieee8021FrerSequenceAutoconfigCreateIndividual	—
ieee8021FrerSequenceAutoconfigCreateRecovery	—
ieee8021FrerSequenceAutoconfigLatentErrorDetection	—
ieee8021FrerSequenceAutoconfigLatentErrorDifference	—
ieee8021FrerSequenceAutoconfigLatentErrorPeriod	—
ieee8021FrerSequenceAutoconfigLatentErrorResetPeriod	—
ieee8021FrerSequenceAutoconfigStatus	—
ieee8021FrerSequenceAutoconfigReceivePortHandleListTable	10.7.1.1.2
ieee8021FrerSequenceAutoconfigReceivePortHandleListIndex	—
ieee8021FrerSequenceAutoconfigReceivePortHandle	—
ieee8021FrerSequenceAutoconfigReceivePortHandleListStatus	—
ieee8021FrerSequenceAutoconfigVlanListTable	10.7.1.1.4
ieee8021FrerSequenceAutoconfigVlanListIndex	—
ieee8021FrerSequenceAutoconfigVlan	—

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard (*continued*)

Clause 11 MIB table/object	Reference
ieee8021FrerSequenceAutoconfigVlanListStatus	—
ieee8021FrerSequenceAutoconfigRecoveryPortHandleListTable	10.7.1.1.5
ieee8021FrerSequenceAutoconfigRecoveryPortHandleListIndex	—
ieee8021FrerSequenceAutoconfigRecoveryPortHandle	—
ieee8021FrerSequenceAutoconfigRecoveryPortHandleListStatus	—
ieee8021FrerOutputAutoconfigTable	10.7, 10.7.2
ieee8021FrerOutputAutoconfigIndex	—
ieee8021FrerOutputAutoconfigPortList	—
ieee8021FrerOutputAutoconfigEncapsulation	—
ieee8021FrerOutputAutoconfigEncapsulationOUI	—
ieee8021FrerOutputAutoconfigCustomEncapsulation	—
ieee8021FrerOutputAutoconfigCustomEncapsulationOUI	—
ieee8021FrerOutputAutoconfigLanPathId	—
ieee8021FrerOutputAutoconfigStatus	—
ieee8021FrerOutputAutoconfigPortHandleListTable	10.7.2.1.1
ieee8021FrerOutputAutoconfigPortHandleListIndex	—
ieee8021FrerOutputAutoconfigPortHandle	—
ieee8021FrerOutputAutoconfigPortHandleListStatus	—
ieee8021FrerPerPortPerStreamCountersTable	10.8
ieee8021FrerPerPortPerStreamDirection	—
ieee8021FrerPerPortPerStreamSeqGenResets	—
ieee8021FrerPerPortPerStreamSeqRecoveryOutOfOrderPackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets	—

Table 11-3—IEEE8021-FRER-MIB structure and relationship to this standard (*continued*)

Clause 11 MIB table/object	Reference
ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryLostPackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryTaglessPackets	—
ieee8021FrerPerPortPerStreamSeqRecoveryResets	—
ieee8021FrerPerPortPerStreamSeqRecoveryLatentErrorResets	—
ieee8021FrerPerPortPerStreamSeqEncErroredPackets	—
ieee8021FrerPerPortCountersTable	10.9
ieee8021FrerPerPortSeqRecoveryPassedPackets	—
ieee8021FrerPerPortfrerCpSeqRecoveryDiscardPackets	—
ieee8021FrerPerPortfrerCpSeqEncErroredPackets	—

1 It also imports items from the SNMPv2-SMI, SNMPv2-TC, INET-ADDRESS-MIB, and
2 IEEE8021-TC-MIB and requires SNMPv2-CONF for conformance.

3 11.3.2 Relationship of the IEEE8021-FRER-MIB to other MIBs

4 The IEEE8021-FRER-MIB identifies the interfaces for the counters provided by this standard by an ifIndex
5 and therefore imports from the IF-MIB.

6 It also imports items from the IEEE8021-STREAM-IDENTIFICATION-MIB, SNMPv2-SMI,
7 SNMPv2-TC, and IEEE8021-TC-MIB and requires SNMPv2-CONF for conformance.

8 11.4 Security considerations

9 SNMP versions prior to SNMPv3 did not include adequate security. Even if the network itself is secure (for
10 example, by using IPsec), there is no control about who on the secure network is allowed to access and GET/
11 SET (read/change/create/delete) the objects in these MIB modules.

12 It is recommended that implementers consider the security features as provided by the SNMPv3 framework
13 (see IETF RFC 3410 (2002), section 8), including full support for the SNMPv3 cryptographic mechanisms
14 (for authentication and privacy).

15 Further, deployment of SNMP versions prior to SNMPv3 is not recommended. Instead, it is recommended
16 to deploy SNMPv3 and to enable cryptographic security. It is then a customer/operator responsibility to
17 ensure that the SNMP entity giving access to an instance of these MIB modules is properly configured to
18 give access to the objects only to those principals (users) that have legitimate rights to indeed GET or SET
19 (change/create/delete) them.

20 11.4.1 Security considerations of the IEEE8021-STREAM-IDENTIFICATION-MIB

21 There are a number of management objects defined in the IEEE8021-STREAM-IDENTIFICATION-MIB
22 module that have a MAX-ACCESS clause of read-write and/or read-create. Such objects may be considered

1 sensitive or vulnerable in some network environments. The support for SET operations in a nonsecure
2 environment without proper protection can have a negative effect on network operations.

3 Some of the readable objects in this MIB module (i.e., objects with a MAX-ACCESS other than
4 not-accessible) may be considered sensitive or vulnerable in some network environments. It is thus
5 important to control all types of access (including GET and/or NOTIFY) to these objects and possibly to
6 even encrypt the values of these objects when sending them over the network via SNMP.

7 **11.4.2 Security considerations of the IEEE8021-FRER-MIB**

8 There are a number of management objects defined in the IEEE8021-FRER-MIB module that have a MAX-
9 ACCESS clause of read-write and/or read-create. Such objects may be considered sensitive or vulnerable in
10 some network environments. The support for SET operations in a nonsecure environment without proper
11 protection can have a negative effect on network operations.

12 Some of the readable objects in this MIB module (i.e., objects with a MAX-ACCESS other than
13 not-accessible) may be considered sensitive or vulnerable in some network environments. It is thus
14 important to control all types of access (including GET and/or NOTIFY) to these objects and possibly to
15 even encrypt the values of these objects when sending them over the network via SNMP.

16 **11.5 MIB modules^{17, 18}**

17 In this subclause, certain terms (e.g., “SHOULD” and “MUST”) denote requirements terminology according
18 to the usage specified in IETF RFC 2119, as is customary for IETF MIB module definitions. In the MIB
19 modules definitions below, if any discrepancy between the DESCRIPTION text and the corresponding
20 definition in any other part of this standard occur, the definitions outside this subclause take precedence.

21 **11.5.1 Definitions for the IEEE8021-STREAM-IDENTIFICATION-MIB**

```
22 IEEE8021-STREAM-IDENTIFICATION-MIB DEFINITIONS ::= BEGIN
23
24 -- =====
25 -- MIB for support of stream identification of 802.1CB
26 -- Frame Replication and Elimination for Reliability
27 -- =====
28
29 IMPORTS
30     OBJECT-GROUP,
31     MODULE-COMPLIANCE
32     FROM SNMPv2-CONF
33     MODULE-IDENTITY,
34     OBJECT-TYPE,
35     OBJECT-IDENTITY,
36     Counter64,
37     Unsigned32,
38     Integer32
39     FROM SNMPv2-SMI
40     TEXTUAL-CONVENTION,
41     TruthValue,
```

¹⁷ Copyright release for MIBs: Users of this standard may freely reproduce the MIBs contained in this subclause so that they can be used for their intended purpose.

¹⁸ An ASCII version of the MIB modules are attached to the PDF version of this standard, and can be obtained by Web browser from the IEEE 802.1 Website at <https://1.ieee802.org/mib-modules/>.

```

1   RowStatus,
2   MacAddress,
3   AutonomousType,
4   VariablePointer
5       FROM SNMPv2-TC
6   ifIndex
7       FROM IF-MIB
8   InetAddressType,
9   InetAddress,
10  InetPortNumber
11      FROM INET-ADDRESS-MIB
12  IEEE8021PriorityValue,
13  ieee802dot1mibs
14      FROM IEEE8021-TC-MIB
15  ;
16
17 ieee8021StreamIdMib MODULE-IDENTITY
18     LAST-UPDATED "202107220000Z" -- July 22, 2021
19     ORGANIZATION "IEEE 802.1 Working Group"
20     CONTACT-INFO
21         "WG-URL: http://ieee802.org/1/
22         WG-EMail: stds-802-1-1@ieee.org
23
24     Contact: IEEE 802.1 Working Group Chair
25     Postal: C/O IEEE 802.1 Working Group
26             IEEE Standards Association
27             445 Hoes Lane
28             Piscataway, NJ 08854
29             USA
30
31     E-mail: stds-802-1-chairs@ieee.org"
32
33     DESCRIPTION
34         "The Management Information Base module for IEEE 802.1CB stream
35         identification. Unless otherwise indicated, the references in
36         this MIB module are to IEEE Std 802.1CB-2017 as amended by
37         IEEE Std 802.1CBcv-2021 and IEEE Std 802.1CBdb-2021. Copyright
38         (C) IEEE (2021).
39         This version of this MIB module is part of IEEE Std 802.1CB-2021;
40         see the standard itself for full legal notices."
41
42     REVISION "202107220000Z" -- July 22, 2021
43     DESCRIPTION
44         "Published as part of IEEE Std 802.1CBdb-2021. Adds extended
45         Stream identification function objects."
46
47     REVISION "202107210000Z" -- July 21, 2021
48     DESCRIPTION
49         "Initial revision published as part of IEEE Std 802.1CBcv-2021."
50
51 ::= { ieee802dot1mibs 34 }
52
53 -- =====
54 -- Textual Conventions
55 -- =====
56
57 IEEE8021CBStreamIdentificationType ::= TEXTUAL-CONVENTION
58     STATUS          current
59     DESCRIPTION

```



```

1      "An enumerated value indicating the method used to identify
2      packets belonging to the Stream. The values 0-255 are reserved
3      for use by IEEE 802."
4  REFERENCE      "9.1.1.6 and Table 9-1"
5  SYNTAX         INTEGER {
6                  reserved(0),
7                  nullStreamIdentification(1),
8                  srcMacVlanStreamIdentification(2),
9                  activeDstMacVlanStreamIdentification(3),
10                 ipStreamStreamIdentification(4),
11                 maskAndMatchStreamIdentification(5),
12                 nonIEEESpecified(256)
13             }
14
15 Ieee8021CBTaggedType ::= TEXTUAL-CONVENTION
16     STATUS          current
17     DESCRIPTION
18         "An enumerated value indicating whether a packet in an EISS
19         indication primitive to a stream identification function is
20         permitted to have a VLAN tag."
21     REFERENCE      "9.1.2.2, 9.1.3.2, 9.1.4.2, 9.1.4.6, 9.1.5.2, and
22                     10.7.1.1.3"
23     SYNTAX         INTEGER {
24                 tagged(1),
25                 priority(2),
26                 all(3)
27             }
28
29 Ieee8021CBVlanIdentifier ::= TEXTUAL-CONVENTION
30     DISPLAY-HINT   "d"
31     STATUS          current
32     DESCRIPTION
33         "Specifies the vlan_identifier. A value of 0 indicates a special
34         treatment, depending on the use of Ieee8021CBVlanIdentifier."
35     SYNTAX         Integer32 (0|1..4094)
36
37 Ieee8021CBIpDscpType ::= TEXTUAL-CONVENTION
38     DISPLAY-HINT   "d"
39     STATUS          current
40     DESCRIPTION
41         "The code point used for discriminating a traffic
42         stream."
43     SYNTAX         Integer32 (-1| 0..63)
44
45 Ieee8021CBIdNextProtocolType ::= TEXTUAL-CONVENTION
46     STATUS          current
47     DESCRIPTION
48         "An enumerated value indicating an IP next protocol."
49     SYNTAX         INTEGER {
50                 none(1),
51                 udp(2),
52                 tcp(3),
53                 sctp(4)
54             }
55
56 Ieee8021CBLanPathIdType ::= TEXTUAL-CONVENTION
57     DISPLAY-HINT   "d"
58     STATUS          current
59     DESCRIPTION

```

```

1      "An integer specifying a path or LAN. If and only if a packet
2      matches an entry in the Sequence identification table that
3      specifies HSR or PRP in its frerSeqEncEncapsType object,
4      tsnStreamIdLanPathId specifies the LanId or PathId value that
5      must be matched for this tsnStreamIdEntry to apply. A value
6      of -1 indicates that the LanId or PathId are to be ignored."
7  REFERENCE      "10.22"
8  SYNTAX         Integer32 (-1 | 0..15)
9
10 -- =====
11 -- subtrees in the stream identification MIB
12 -- =====
13
14 ieee8021StreamIdNotifications
15     OBJECT IDENTIFIER ::= { ieee8021StreamIdMib 0 }
16
17 ieee8021StreamIdObjects
18     OBJECT IDENTIFIER ::= { ieee8021StreamIdMib 1 }
19
20 ieee8021StreamIdConformance
21     OBJECT IDENTIFIER ::= { ieee8021StreamIdMib 2 }
22
23 ieee8021StreamIdStreamIdentification
24     OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 1 }
25
26 ieee8021StreamIdStreamIdInFacOutputPortHandleList
27     OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 2 }
28
29 ieee8021StreamIdStreamIdOutFacOutputPortHandleList
30     OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 3 }
31
32 ieee8021StreamIdStreamIdInFacInputPortHandleList
33     OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 4 }
34
35 ieee8021StreamIdStreamIdOutFacInputPortHandleList
36     OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 5 }
37
38 ieee8021StreamIdPerPortPerStreamCounters
39     OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 6 }
40
41 ieee8021StreamIdPerPortCounters
42     OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 7 }
43
44 ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLength
45     OBJECT IDENTIFIER ::= { ieee8021StreamIdObjects 8 }
46
47 -- =====
48 -- the ieee8021StreamIdStreamIdentification table
49 -- =====
50
51 -- =====
52 -- the ieee8021StreamIdStreamIdentificationTypes
53 -- =====
54
55 ieee8021StreamIdStreamIdentificationTypes
56     OBJECT IDENTIFIER ::= { ieee8021StreamIdStreamIdentification 1 }
57
58 ieee8021StreamIdNullStream OBJECT-IDENTITY
59     STATUS current

```

```

1      DESCRIPTION
2          "The stream identification type for the Null Stream
3          identification."
4      ::= { ieee8021StreamIdStreamIdentificationTypes 1 }
5
6 ieee8021StreamIdSrcMacVlan OBJECT-IDENTITY
7     STATUS current
8     DESCRIPTION
9         "The stream identification type for the Source MAC and VLAN
10        Stream identification."
11     ::= { ieee8021StreamIdStreamIdentificationTypes 2 }
12
13 ieee8021StreamIdActiveDestMacVlan OBJECT-IDENTITY
14     STATUS current
15     DESCRIPTION
16         "The stream identification type for the Active Destination
17         and VLAN Stream identification."
18     ::= { ieee8021StreamIdStreamIdentificationTypes 3 }
19
20 ieee8021StreamIdIpStream OBJECT-IDENTITY
21     STATUS current
22     DESCRIPTION
23         "The stream identification type for the IP Stream
24         identification."
25     ::= { ieee8021StreamIdStreamIdentificationTypes 4 }
26
27 ieee8021StreamIdMaskAndMatch OBJECT-IDENTITY
28     STATUS current
29     DESCRIPTION
30         "The stream identification type for the Mask-and-match
31         Stream identification."
32     ::= { ieee8021StreamIdStreamIdentificationTypes 5 }
33
34 -- =====
35 -- the ieee8021StreamIdStreamIdentification
36 -- =====
37
38 ieee8021StreamIdStreamIdentificationTable OBJECT-TYPE
39     SYNTAX      SEQUENCE OF Ieee8021StreamIdStreamIdentificationEntry
40     MAX-ACCESS  not-accessible
41     STATUS      current
42     DESCRIPTION
43         "A table containing a set of tsNStreamIdEntry objects each
44         relating to a single Stream, specifying the points in the
45         system where Stream identification functions are to be
46         instantiated."
47     REFERENCE   "9.1"
48     ::= { ieee8021StreamIdStreamIdentification 2 }
49
50 ieee8021StreamIdStreamIdentificationEntry OBJECT-TYPE
51     SYNTAX      Ieee8021StreamIdStreamIdentificationEntry
52     MAX-ACCESS  not-accessible
53     STATUS      current
54     DESCRIPTION
55         "A set of managed objects, all applying to the Stream specified
56         by tsNStreamIdHandle, and all using the same Stream
57         identification types and parameters."
58     REFERENCE   "9.1.1"
59     INDEX       { ieee8021StreamIdStreamIdentificationIndex }

```

```

1      ::= { ieee8021StreamIdStreamIdentificationTable 1 }
2
3 Ieee8021StreamIdStreamIdentificationEntry ::=
4     SEQUENCE {
5         ieee8021StreamIdStreamIdentificationIndex
6             Unsigned32,
7         ieee8021StreamIdStreamIdIdentificationType
8             Ieee8021CBStreamIdentificationType,
9         ieee8021StreamIdStreamIdIdentificationTypeOUI
10            OCTET STRING,
11         ieee8021StreamIdStreamIdIdentificationCustomType
12            Integer32,
13         ieee8021StreamIdStreamIdIdentificationCustomTypeOUI
14            OCTET STRING,
15         ieee8021StreamIdStreamIdIdentificationTypeSelect
16            AutonomousType,
17         ieee8021StreamIdStreamIdHandle
18            Unsigned32,
19         ieee8021StreamIdStreamIdInFacOutputPortList
20            AutonomousType,
21         ieee8021StreamIdStreamIdOutFacOutputPortList
22            AutonomousType,
23         ieee8021StreamIdStreamIdInFacInputPortList
24            AutonomousType,
25         ieee8021StreamIdStreamIdOutFacInputPortList
26            AutonomousType,
27         ieee8021StreamIdAutoConfigured
28            TruthValue,
29         ieee8021StreamIdLanPathId
30            Ieee8021CBLanPathIdType,
31         ieee8021StreamIdStatus
32            RowStatus
33     }
34
35 ieee8021StreamIdStreamIdentificationIndex OBJECT-TYPE
36     SYNTAX      Unsigned32
37     MAX-ACCESS  not-accessible
38     STATUS      current
39     DESCRIPTION
40         "Index for the Stream Identity table."
41     ::= { ieee8021StreamIdStreamIdentificationEntry 1 }
42
43 ieee8021StreamIdStreamIdIdentificationType OBJECT-TYPE
44     SYNTAX      Ieee8021CBStreamIdentificationType
45     MAX-ACCESS  read-create
46     STATUS      current
47     DESCRIPTION
48         "This managed object is an enumerated value indicating the
49         method used to identify packets belonging to the Stream.
50         If the value used is nonIEEESpecified(256), the managed objects
51         ieee8021StreamIdStreamIdIdentificationCustomType and
52         ieee8021StreamIdStreamIdIdentificationCustomTypeOUI are used to
53         specify OUI/CID and the stream identification method defined by
54         the entity owning the OUI/CID."
55     REFERENCE   "9.1.1.6"
56     DEFVAL      { nullStreamIdentification }
57     ::= { ieee8021StreamIdStreamIdentificationEntry 2 }
58
59 ieee8021StreamIdStreamIdIdentificationTypeOUI OBJECT-TYPE

```

```

1  SYNTAX      OCTET STRING (SIZE (3))
2  MAX-ACCESS  read-only
3  STATUS      current
4  DESCRIPTION
5      "This managed object specifies the OUI/CID used together with
6      the stream identification method specified in
7      ieee8021StreamIdStreamIdIdentificationType.
8      It is used if and only if the value for
9      ieee8021StreamIdStreamIdIdentificationType is in the range of
10     0-255. In this case it always takes the value 00-80-C2."
11  REFERENCE   "9.1.1.6"
12  ::= { ieee8021StreamIdStreamIdentificationEntry 3 }
13
14 ieee8021StreamIdStreamIdIdentificationCustomType OBJECT-TYPE
15  SYNTAX      Integer32 (256..2147483647)
16  MAX-ACCESS  read-create
17  STATUS      current
18  DESCRIPTION
19      "This managed object is an enumerated value indicating the
20      method used to identify packets belonging to the Stream. It is
21      used if and only if the value for
22      ieee8021StreamIdStreamIdIdentificationType is 256."
23  REFERENCE   "9.1.1.6"
24  ::= { ieee8021StreamIdStreamIdentificationEntry 4 }
25
26 ieee8021StreamIdStreamIdIdentificationCustomTypeOUI OBJECT-TYPE
27  SYNTAX      OCTET STRING (SIZE (3))
28  MAX-ACCESS  read-create
29  STATUS      current
30  DESCRIPTION
31      "This managed object specifies the OUI/CID used together with
32      the stream identification method specified in
33      ieee8021StreamIdStreamIdIdentificationType. It is used if and
34      only if the value for
35      ieee8021StreamIdStreamIdIdentificationType is 256."
36  REFERENCE   "9.1.1.6"
37  ::= { ieee8021StreamIdStreamIdentificationEntry 5 }
38
39 ieee8021StreamIdStreamIdIdentificationTypeSelect OBJECT-TYPE
40  SYNTAX      AutonomousType
41  MAX-ACCESS  read-only
42  STATUS      current
43  DESCRIPTION
44      "An indication of the type of stream identification method that
45      is used.
46
47      If this value is
48      'ieee8021StreamIdNullStream ieee8021StreamIdObjects 1' then
49      an entry exists in the
50      ieee8021StreamIdStreamIdentificationTable
51      which corresponds to the Null Stream identification.
52
53      If this value is
54      'ieee8021StreamIdSrcMacVlan ieee8021StreamIdObjects 2' then
55      an entry exists in the
56      ieee8021StreamIdStreamIdentificationTable
57      which corresponds to the Source MAC and VLAN Stream
58      identification.
59

```

```

1      If this value is
2      'ieee8021StreamIdActiveDestMacVlan  ieee8021StreamIdObjects 3'
3      then an entry exists in the
4      ieee8021StreamIdStreamIdentificationTable
5      which corresponds to the Active Destination MAC and VLAN Stream
6      identification.
7
8      If this value is
9      'ieee8021StreamIdIpStream  ieee8021StreamIdObjects 4' then
10     an entry exists in the
11     ieee8021StreamIdStreamIdentificationTable
12     which corresponds to the IP Stream identification.
13
14     If this value is
15     'ieee8021StreamIdMaskAndMatch  ieee8021StreamIdObjects 5' then
16     an entry exists in the
17     ieee8021StreamIdStreamIdentificationTable
18     which corresponds to the Mask-and-match Stream
19     identification."
20     ::= { ieee8021StreamIdStreamIdentificationEntry 6 }
21
22 ieee8021StreamIdStreamIdHandle OBJECT-TYPE
23     SYNTAX      Unsigned32
24     MAX-ACCESS  read-create
25     STATUS      current
26     DESCRIPTION
27         "The objects in a given entry of the Stream identity table are
28         used to control packets whose stream_handle subparameter is
29         equal to the entry's tsnStreamIdHandle object."
30     REFERENCE   "9.1.1.1"
31     ::= { ieee8021StreamIdStreamIdentificationEntry 7 }
32
33 ieee8021StreamIdStreamIdInFacOutputPortList OBJECT-TYPE
34     SYNTAX      AutonomousType
35     MAX-ACCESS  read-only
36     STATUS      current
37     DESCRIPTION
38         "The list of ports on which an in-facing Stream identification
39         function using this identification method is to be placed for
40         this Stream in the output (towards the system forwarding
41         function) direction.
42         For each port in this list, there exists an entry in the
43         ieee8021StreamIdStreamIdInFacOutputPortHandleListTable."
44     REFERENCE   "9.1.1.2"
45     ::= { ieee8021StreamIdStreamIdentificationEntry 8 }
46
47 ieee8021StreamIdStreamIdOutFacOutputPortList OBJECT-TYPE
48     SYNTAX      AutonomousType
49     MAX-ACCESS  read-only
50     STATUS      current
51     DESCRIPTION
52         "The list of ports on which an out-facing Stream identification
53         function using this identification method is to be placed for
54         this Stream in the output (towards the physical interface)
55         direction.
56         For each port in this list, there exists an entry in the
57         ieee8021StreamIdStreamIdOutFacOutputPortHandleListTable."
58     REFERENCE   "9.1.1.3"
59     ::= { ieee8021StreamIdStreamIdentificationEntry 9 }

```

```

1
2 ieee8021StreamIdStreamIdInFacInputPortList OBJECT-TYPE
3     SYNTAX      AutonomousType
4     MAX-ACCESS  read-only
5     STATUS      current
6     DESCRIPTION
7         "The list of ports on which an in-facing Stream identification
8         function using this identification method is to be placed for
9         this Stream in the input (coming from the system forwarding
10        function) direction.
11        For each port in this list, there exists an entry in the
12        ieee8021StreamIdStreamIdInFacInputPortHandleListTable."
13     REFERENCE   "9.1.1.4"
14     ::= { ieee8021StreamIdStreamIdentificationEntry 10 }
15
16 ieee8021StreamIdStreamIdOutFacInputPortList OBJECT-TYPE
17     SYNTAX      AutonomousType
18     MAX-ACCESS  read-only
19     STATUS      current
20     DESCRIPTION
21         "The list of ports on which an out-facing Stream identification
22         function using this identification method is to be placed for
23         this Stream in the input (coming from the physical interface)
24         direction.
25         For each port in this list, there exists an entry in the
26         ieee8021StreamIdStreamIdOutFacInputPortHandleListTable."
27     REFERENCE   "9.1.1.5"
28     ::= { ieee8021StreamIdStreamIdentificationEntry 11 }
29
30 ieee8021StreamIdAutoConfigured OBJECT-TYPE
31     SYNTAX      TruthValue
32     MAX-ACCESS  read-only
33     STATUS      current
34     DESCRIPTION
35         "A read-only Boolean value, supplied by the system, specifying
36         whether this entry was created explicitly (False) or via the
37         Sequence autoconfiguration table (True)."
38     REFERENCE   "10.2.1"
39     ::= { ieee8021StreamIdStreamIdentificationEntry 12 }
40
41 ieee8021StreamIdLanPathId OBJECT-TYPE
42     SYNTAX      Ieee8021CBLanPathIdType
43     MAX-ACCESS  read-only
44     STATUS      current
45     DESCRIPTION
46         "An integer specifying a path or LAN. If and only if a packet
47         matches an entry in the Sequence identification table that
48         specifies HSR or PRP in its frerSeqEncEncapsType object,
49         tsNStreamIdLanPathId specifies the LanId or PathId value that
50         must be matched for this tsNStreamIdEntry to apply. A value
51         of -1 indicates that the LanId or PathId are to be ignored."
52     REFERENCE   "10.2.2"
53     ::= { ieee8021StreamIdStreamIdentificationEntry 13 }
54
55 ieee8021StreamIdStatus OBJECT-TYPE
56     SYNTAX      RowStatus
57     MAX-ACCESS  read-create
58     STATUS      current
59     DESCRIPTION

```

```

1      "Row-Status for the Stream Identity table."
2      ::= { ieee8021StreamIdStreamIdentificationEntry 14 }
3
4-- =====
5-- the ieee8021StreamIdNullStreamIdentificationTable
6-- =====
7
8ieee8021StreamIdNullStreamIdentificationTable OBJECT-TYPE
9    SYNTAX      SEQUENCE OF Ieee8021StreamIdNullStreamIdentificationEntry
10   MAX-ACCESS  not-accessible
11   STATUS      current
12   DESCRIPTION
13     "A table containing a set of controlling parameters for the
14     Null Stream identification method."
15   REFERENCE   "9.1.2"
16   ::= { ieee8021StreamIdStreamIdentification 3 }
17
18ieee8021StreamIdNullStreamIdentificationEntry OBJECT-TYPE
19   SYNTAX      Ieee8021StreamIdNullStreamIdentificationEntry
20   MAX-ACCESS  not-accessible
21   STATUS      current
22   DESCRIPTION
23     "A set of managed objects that serve as the Stream
24     identification parameters when using the Null Stream
25     identification method."
26   REFERENCE   "9.1.2"
27   INDEX { ieee8021StreamIdStreamIdentificationIndex }
28   ::= { ieee8021StreamIdNullStreamIdentificationTable 1 }
29
30 Ieee8021StreamIdNullStreamIdentificationEntry ::=
31   SEQUENCE {
32     ieee8021StreamIdCpeNullDownDestMac
33       MacAddress,
34     ieee8021StreamIdCPENullDownTagged
35       Ieee8021CBTaggedType,
36     ieee8021StreamIdCpeNullDownVlan
37       Ieee8021CBVlanIdentifier
38   }
39
40 ieee8021StreamIdCpeNullDownDestMac OBJECT-TYPE
41   SYNTAX      MacAddress
42   MAX-ACCESS  read-write
43   STATUS      current
44   DESCRIPTION
45     "Specifies the destination_address that identifies a packet in
46     an EISS indication primitive, to the Null Stream identification
47     function."
48   REFERENCE   "9.1.2.1"
49   ::= { ieee8021StreamIdNullStreamIdentificationEntry 1 }
50
51 ieee8021StreamIdCPENullDownTagged OBJECT-TYPE
52   SYNTAX      Ieee8021CBTaggedType
53   MAX-ACCESS  read-write
54   STATUS      current
55   DESCRIPTION
56     "An enumerated value indicating whether a packet in an EISS
57     indication primitive to the Null Stream identification function
58     is permitted to have a VLAN tag."
59   REFERENCE   "9.1.2.2"

```



```

1      ::= { ieee8021StreamIdNullStreamIdentificationEntry 2 }
2
3 ieee8021StreamIdCpeNullDownVlan OBJECT-TYPE
4     SYNTAX      Ieee8021CBVlanIdentifier
5     MAX-ACCESS  read-write
6     STATUS      current
7     DESCRIPTION
8         "Specifies the vlan_identifier parameter that identifies a
9         packet in an EISS indication primitive to the Null Stream
10        identification function. A value of 0 indicates that the
11        vlan_identifier parameter is ignored on EISS indication
12        primitives."
13     REFERENCE   "9.1.2.3"
14     ::= { ieee8021StreamIdNullStreamIdentificationEntry 3 }
15
16 -- =====
17 -- the ieee8021StreamIdSrcMacVlanIdentificationTable
18 -- =====
19
20 ieee8021StreamIdSrcMacVlanIdentificationTable OBJECT-TYPE
21     SYNTAX      SEQUENCE OF Ieee8021StreamIdSrcMacVlanIdentificationEntry
22     MAX-ACCESS  not-accessible
23     STATUS      current
24     DESCRIPTION
25         "A table containing a set of controlling parameters for the
26         Source MAC and VLAN Stream identification method."
27     REFERENCE   "9.1.3"
28     ::= { ieee8021StreamIdStreamIdentification 4 }
29
30 ieee8021StreamIdSrcMacVlanIdentificationEntry OBJECT-TYPE
31     SYNTAX      Ieee8021StreamIdSrcMacVlanIdentificationEntry
32     MAX-ACCESS  not-accessible
33     STATUS      current
34     DESCRIPTION
35         "A set of managed objects that serve as the Stream
36         identification parameters when using the Source MAC and VLAN
37         Stream identification method."
38     REFERENCE   "9.1.3"
39     INDEX { ieee8021StreamIdStreamIdentificationIndex }
40     ::= { ieee8021StreamIdSrcMacVlanIdentificationTable 1 }
41
42 Ieee8021StreamIdSrcMacVlanIdentificationEntry ::=
43     SEQUENCE {
44         ieee8021StreamIdCpeSmacVlanDownSrcMac
45             MacAddress,
46         ieee8021StreamIdCpeSmacVlanDownTagged
47             Ieee8021CBTaggedType,
48         ieee8021StreamIdCpeSmacVlanDownVlan
49             Ieee8021CBVlanIdentifier
50     }
51
52 ieee8021StreamIdCpeSmacVlanDownSrcMac OBJECT-TYPE
53     SYNTAX      MacAddress
54     MAX-ACCESS  read-write
55     STATUS      current
56     DESCRIPTION
57         "Specifies the source_address that identifies a packet in an
58         EISS indication primitive, to the Source MAC and VLAN Stream
59         identification function."

```

```

1  REFERENCE    "9.1.3.1"
2  ::= { ieee8021StreamIdSrcMacVlanIdentificationEntry 1 }
3
4  ieee8021StreamIdCpeSmacVlanDownTagged OBJECT-TYPE
5  SYNTAX      Ieee8021CBTaggedType
6  MAX-ACCESS  read-write
7  STATUS      current
8  DESCRIPTION
9      "An enumerated value indicating whether a packet in an EISS
10     indication primitive to the Source MAC and VLAN Stream
11     identification function is permitted to have a VLAN tag."
12  REFERENCE    "9.1.3.2"
13  ::= { ieee8021StreamIdSrcMacVlanIdentificationEntry 2 }
14
15  ieee8021StreamIdCpeSmacVlanDownVlan OBJECT-TYPE
16  SYNTAX      Ieee8021CBVlanIdentifier
17  MAX-ACCESS  read-write
18  STATUS      current
19  DESCRIPTION
20      "Specifies the vlan_identifier parameter that identifies a
21      packet in an EISS indication primitive to the Source MAC and
22      VLAN Stream identification function. A value of 0 indicates
23      that the vlan_identifier parameter is ignored on EISS
24      indication primitives."
25  REFERENCE    "9.1.3.3"
26  ::= { ieee8021StreamIdSrcMacVlanIdentificationEntry 3 }
27
28  -- =====
29  -- the ieee8021StreamIdActiveDestMacVlanIdentificationTable
30  -- =====
31
32  ieee8021StreamIdActiveDestMacVlanIdentificationTable OBJECT-TYPE
33  SYNTAX      SEQUENCE OF Ieee8021StreamIdActiveDestMacVlanIdentificationEntry
34  MAX-ACCESS  not-accessible
35  STATUS      current
36  DESCRIPTION
37      "A table containing a set of controlling parameters for the
38      Active Destination MAC and VLAN Stream identification method."
39  REFERENCE    "9.1.4"
40  ::= { ieee8021StreamIdStreamIdentification 5 }
41
42  ieee8021StreamIdActiveDestMacVlanIdentificationEntry OBJECT-TYPE
43  SYNTAX      Ieee8021StreamIdActiveDestMacVlanIdentificationEntry
44  MAX-ACCESS  not-accessible
45  STATUS      current
46  DESCRIPTION
47      "A set of managed objects that serve as the Stream
48      identification parameters when using the Active Destination
49      MAC and VLAN Stream identification method."
50  REFERENCE    "9.1.4"
51  INDEX { ieee8021StreamIdStreamIdentificationIndex }
52  ::= { ieee8021StreamIdActiveDestMacVlanIdentificationTable 1 }
53
54  Ieee8021StreamIdActiveDestMacVlanIdentificationEntry ::=
55  SEQUENCE {
56      ieee8021StreamIdCpeDmacVlanDownDestMac
57          MacAddress,
58      ieee8021StreamIdCpeDmacVlanDownTagged
59          Ieee8021CBTaggedType,

```

```

1      ieee8021StreamIdCpeDmacVlanDownVlan
2          Ieee8021CBVlanIdentifier,
3      ieee8021StreamIdCpeDmacVlanDownPriority
4          IEEE8021PriorityValue,
5      ieee8021StreamIdCpeDmacVlanUpDestMac
6          MacAddress,
7      ieee8021StreamIdCpeDmacVlanUpTagged
8          Ieee8021CBTaggedType,
9      ieee8021StreamIdCpeDmacVlanUpVlan
10         Ieee8021CBVlanIdentifier,
11         ieee8021StreamIdCpeDmacVlanUpPriority
12         IEEE8021PriorityValue
13     }
14
15 ieee8021StreamIdCpeDmacVlanDownDestMac OBJECT-TYPE
16     SYNTAX      MacAddress
17     MAX-ACCESS  read-write
18     STATUS      current
19     DESCRIPTION
20         "Specifies the destination_address parameter to use in the
21         EISS request primitive for output packets sent to lower layers
22         by the Active Destination MAC and VLAN Stream identification
23         function, and the destination_address that identifies an input
24         packet in an EISS indication primitive to the Active
25         Destination MAC and VLAN Stream identification function."
26     REFERENCE   "9.1.4.1"
27     ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 1 }
28
29 ieee8021StreamIdCpeDmacVlanDownTagged OBJECT-TYPE
30     SYNTAX      Ieee8021CBTaggedType
31     MAX-ACCESS  read-write
32     STATUS      current
33     DESCRIPTION
34         "An enumerated value indicating whether a packet in an EISS
35         indication or request primitive between the Active Destination
36         MAC and VLAN Stream identification function and the lower
37         layers is to have a VLAN tag.
38         This variable is not used in an FRER C-component."
39     REFERENCE   "9.1.4.2"
40     ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 2 }
41
42 ieee8021StreamIdCpeDmacVlanDownVlan OBJECT-TYPE
43     SYNTAX      Ieee8021CBVlanIdentifier
44     MAX-ACCESS  read-write
45     STATUS      current
46     DESCRIPTION
47         "Specifies the vlan_identifier parameter to use in the EISS
48         request primitive for output packets sent to lower layers by
49         the Active Destination MAC and VLAN Stream identification
50         function, and the vlan_identifier that identifies an input
51         packet in an EISS indication primitive to the Active
52         Destination MAC and VLAN Stream identification function.
53         A value of 0 indicates that the vlan_identifier parameter is
54         ignored on EISS indication primitives."
55     REFERENCE   "9.1.4.3"
56     ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 3 }
57
58 ieee8021StreamIdCpeDmacVlanDownPriority OBJECT-TYPE
59     SYNTAX      IEEE8021PriorityValue

```

```

1    MAX-ACCESS    read-write
2    STATUS        current
3    DESCRIPTION
4        "Specifies the priority parameter to use in the EISS request
5        primitive for output packets sent to lower layers by the Active
6        Destination MAC and VLAN Stream identification function for all
7        packets in a particular Stream."
8    REFERENCE     "9.1.4.4"
9    ::= { ieee8021StreamIdActiveDestMacVlanIdentificationEntry 4 }
10
11 ieee8021StreamIdCpeDmacVlanUpDestMac OBJECT-TYPE
12     SYNTAX        MacAddress
13     MAX-ACCESS    read-write
14     STATUS        current
15     DESCRIPTION
16         "Specifies the destination_address parameter to use in the EISS
17         indication primitive for input packets offered to upper layers
18         by the Active Destination MAC and VLAN Stream identification
19         layer. This address replaces the address that was used to
20         identify the packet (tsnCpeDmacVlanDownDestMac)."

```

```

1
2 -- =====
3 -- the ieee8021StreamIdIpStreamIdentificationTable
4 -- =====
5
6 ieee8021StreamIdIpStreamIdentificationTable OBJECT-TYPE
7     SYNTAX      SEQUENCE OF Ieee8021StreamIdIpStreamIdentificationEntry
8     MAX-ACCESS  not-accessible
9     STATUS      current
10    DESCRIPTION
11        "A table containing a set of controlling parameters for the
12         IP Stream identification method."
13    REFERENCE   "9.1.5"
14    ::= { ieee8021StreamIdStreamIdentification 6 }
15
16 ieee8021StreamIdIpStreamIdentificationEntry OBJECT-TYPE
17     SYNTAX      Ieee8021StreamIdIpStreamIdentificationEntry
18     MAX-ACCESS  not-accessible
19     STATUS      current
20     DESCRIPTION
21        "A set of managed objects that serve as the Stream
22         identification parameters when using the IP Stream
23         identification method."
24     REFERENCE   "9.1.5"
25     INDEX { ieee8021StreamIdStreamIdentificationIndex }
26     ::= { ieee8021StreamIdIpStreamIdentificationTable 1 }
27
28 Ieee8021StreamIdIpStreamIdentificationEntry ::=
29     SEQUENCE {
30         ieee8021StreamIdCpeIpIdDestMac
31             MacAddress,
32         ieee8021StreamIdCpeIpIdTagged
33             Ieee8021CBTaggedType,
34         ieee8021StreamIdCpeIpIdVlan
35             Ieee8021CBVlanIdentifier,
36         ieee8021StreamIdCpeIpIdIpSourceType
37             InetAddressType,
38         ieee8021StreamIdCpeIpIdIpSource
39             InetAddress,
40         ieee8021StreamIdCpeIpIdIpDestinationType
41             InetAddressType,
42         ieee8021StreamIdCpeIpIdIpDestination
43             InetAddress,
44         ieee8021StreamIdCpeIpIdDscp
45             Ieee8021CBIPDscpType,
46         ieee8021StreamIdCpeIpIdNextProtocol
47             Ieee8021CBIdNextProtocolType,
48         ieee8021StreamIdCpeIpIdSourcePort
49             InetPortNumber,
50         ieee8021StreamIdCpeIpIdDestinationPort
51             InetPortNumber
52     }
53
54 ieee8021StreamIdCpeIpIdDestMac OBJECT-TYPE
55     SYNTAX      MacAddress
56     MAX-ACCESS  read-write
57     STATUS      current
58     DESCRIPTION
59        "Specifies the destination_address parameter that identifies a

```

```

1      packet in an EISS indication primitive."
2  REFERENCE    "9.1.5.1"
3  ::= { ieee8021StreamIdIpStreamIdentificationEntry 1 }
4
5  ieee8021StreamIdCpeIpIdTagged OBJECT-TYPE
6      SYNTAX      Ieee8021CBTaggedType
7      MAX-ACCESS  read-write
8      STATUS      current
9      DESCRIPTION
10         "An enumerated value indicating whether a packet in an EISS
11         indication or request primitive to the IP Stream identification
12         function is to have a VLAN tag."
13  REFERENCE    "9.1.5.2"
14  ::= { ieee8021StreamIdIpStreamIdentificationEntry 2 }
15
16  ieee8021StreamIdCpeIpIdVlan OBJECT-TYPE
17      SYNTAX      Ieee8021CBVlanIdentifier
18      MAX-ACCESS  read-write
19      STATUS      current
20      DESCRIPTION
21         "Specifies the vlan_identifier parameter that identifies a
22         packet in an EISS indication primitive. A value of 0 indicates
23         that the frame is not to have a VLAN tag."
24  REFERENCE    "9.1.5.3"
25  ::= { ieee8021StreamIdIpStreamIdentificationEntry 3 }
26
27  ieee8021StreamIdCpeIpIdIpSourceType OBJECT-TYPE
28      SYNTAX      InetAddressType
29      MAX-ACCESS  read-write
30      STATUS      current
31      DESCRIPTION
32         "Specifies the type of the source address parameter supplied
33         in ieee8021StreamIdCpeIpIdIpSource."
34  REFERENCE    "9.1.5.4"
35  ::= { ieee8021StreamIdIpStreamIdentificationEntry 4 }
36
37  ieee8021StreamIdCpeIpIdIpSource OBJECT-TYPE
38      SYNTAX      InetAddress
39      MAX-ACCESS  read-write
40      STATUS      current
41      DESCRIPTION
42         "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) source address
43         parameter that must be matched to identify packets coming up
44         from lower layers. An address of all 0 indicates that the
45         IP source address is to be ignored on packets received from
46         lower layers."
47  REFERENCE    "9.1.5.4"
48  ::= { ieee8021StreamIdIpStreamIdentificationEntry 5 }
49
50  ieee8021StreamIdCpeIpIdIpDestinationType OBJECT-TYPE
51      SYNTAX      InetAddressType
52      MAX-ACCESS  read-write
53      STATUS      current
54      DESCRIPTION
55         "Specifies the type of the destination address parameter
56         supplied in ieee8021StreamIdCpeIpIdIpDestination."
57  REFERENCE    "9.1.5.4"
58  ::= { ieee8021StreamIdIpStreamIdentificationEntry 6 }
59

```

```

1 ieee8021StreamIdCpeIpIdIpDestination OBJECT-TYPE
2     SYNTAX      InetAddress
3     MAX-ACCESS  read-write
4     STATUS      current
5     DESCRIPTION
6         "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) destination
7         address parameter that must be matched to identify packets
8         coming up from lower layers."
9     REFERENCE   "9.1.5.5"
10    ::= { ieee8021StreamIdIpStreamIdentificationEntry 7 }
11
12 ieee8021StreamIdCpeIpIdDscp OBJECT-TYPE
13     SYNTAX      Ieee8021CBIdDscpType
14     MAX-ACCESS  read-write
15     STATUS      current
16     DESCRIPTION
17         "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) differentiated
18         services codepoint (DSCP, RFC 2474) that must be matched to
19         identify packets coming up from the lower layers. A value of
20         64 decimal indicates that the DSCP is to be ignored on packets
21         received from lower layers."
22     REFERENCE   "9.1.5.6"
23    ::= { ieee8021StreamIdIpStreamIdentificationEntry 8 }
24
25 ieee8021StreamIdCpeIpIdNextProtocol OBJECT-TYPE
26     SYNTAX      Ieee8021CBIdNextProtocolType
27     MAX-ACCESS  read-write
28     STATUS      current
29     DESCRIPTION
30         "Specifies the IP next protocol parameter that must be matched
31         to identify packets coming up from lower layers. The value of
32         this parameter must specify either none, UDP (RFC 768),
33         TCP (RFC 793), or SCTP (RFC 4960). If 'none', then the
34         tsnCpeIpIdSourcePort and tsnCpeIpIdDestinationPort managed
35         objects are not used."
36     REFERENCE   "9.1.5.7"
37    ::= { ieee8021StreamIdIpStreamIdentificationEntry 9 }
38
39 ieee8021StreamIdCpeIpIdSourcePort OBJECT-TYPE
40     SYNTAX      InetPortNumber
41     MAX-ACCESS  read-write
42     STATUS      current
43     DESCRIPTION
44         "Specifies the TCP or UDP Source Port parameter that must be
45         matched to identify packets coming up from lower layers.
46         A value of 0 indicates that the Source Port number of the
47         packet is to be ignored on packets received from lower layers."
48     REFERENCE   "9.1.5.8"
49    ::= { ieee8021StreamIdIpStreamIdentificationEntry 10 }
50
51 ieee8021StreamIdCpeIpIdDestinationPort OBJECT-TYPE
52     SYNTAX      InetPortNumber
53     MAX-ACCESS  read-write
54     STATUS      current
55     DESCRIPTION
56         "Specifies the TCP or UDP Destination Port parameter that must
57         be matched to identify packets coming up from lower layers.
58         A value of 0 indicates that the Destination Port number of the
59         packet is to be ignored on packets received from lower layers."

```

```

1  REFERENCE    "9.1.5.9"
2  ::= { ieee8021StreamIdIpStreamIdentificationEntry 11 }
3
4  -- =====
5  -- the ieee8021StreamIdMaskAndMatchIdentificationTable
6  -- =====
7
8  ieee8021StreamIdMaskAndMatchIdentificationTable OBJECT-TYPE
9      SYNTAX      SEQUENCE OF Ieee8021StreamIdMaskAndMatchIdentificationEntry
10     MAX-ACCESS   not-accessible
11     STATUS       current
12     DESCRIPTION
13         "A table containing a set of controlling parameters for the
14         Mask-and-match Stream identification method."
15     REFERENCE    "9.1.6"
16     ::= { ieee8021StreamIdStreamIdentification 7 }
17
18  ieee8021StreamIdMaskAndMatchIdentificationEntry OBJECT-TYPE
19      SYNTAX      Ieee8021StreamIdMaskAndMatchIdentificationEntry
20      MAX-ACCESS   not-accessible
21      STATUS       current
22      DESCRIPTION
23          "A set of managed objects that serve as the Stream
24          identification parameters when using the Mask-and-match Stream
25          identification method."
26      REFERENCE    "9.1.6"
27      INDEX { ieee8021StreamIdStreamIdentificationIndex }
28      ::= { ieee8021StreamIdMaskAndMatchIdentificationTable 1 }
29
30  Ieee8021StreamIdMaskAndMatchIdentificationEntry ::=
31      SEQUENCE {
32          ieee8021StreamIdCpeMmIdDestMacMask
33              MacAddress,
34          ieee8021StreamIdCpeMmIdDestMacMatch
35              MacAddress,
36          ieee8021StreamIdCpeMmIdSrcMacMask
37              MacAddress,
38          ieee8021StreamIdCpeMmIdSrcMacMatch
39              MacAddress,
40          ieee8021StreamIdCpeMmIdMsduMaskLength
41              Integer32,
42          ieee8021StreamIdCpeMmIdMsduMask
43              OCTET STRING,
44          ieee8021StreamIdCpeMmIdMsduMatch
45              OCTET STRING
46      }
47
48  ieee8021StreamIdCpeMmIdDestMacMask OBJECT-TYPE
49      SYNTAX      MacAddress
50      MAX-ACCESS   read-write
51      STATUS       current
52      DESCRIPTION
53          "Specifies a 48-bit mask. A bitwise AND operation is performed
54          between MmIdDestMacMask and the destination_address parameter
55          passed by the ISS indication primitive to the Mask-and-match
56          Stream identification function. The resulting 48-bit
57          information is the masked destination_address that is used as
58          input for the instance of the Mask-and-match Stream
59          identification function. If MmIdDestMacMask has a value of 0,
```



```

1         the destination_address parameter is ignored."
2     REFERENCE    "9.1.6.1"
3     ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 1 }
4
5 ieee8021StreamIdCpeMmIdDestMacMatch OBJECT-TYPE
6     SYNTAX      MacAddress
7     MAX-ACCESS  read-write
8     STATUS      current
9     DESCRIPTION
10        "Specifies the 48-bit value of the masked destination_address,
11        as defined in 9.1.6.1, to be matched by the instance of the
12        Mask-and-match Stream identification function."
13     REFERENCE    "9.1.6.2"
14     ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 2 }
15
16 ieee8021StreamIdCpeMmIdSrcMacMask OBJECT-TYPE
17     SYNTAX      MacAddress
18     MAX-ACCESS  read-write
19     STATUS      current
20     DESCRIPTION
21        "Specifies a 48-bit mask. A bitwise AND operation is performed
22        between CpeMmIdSrcMacMask and the source_address parameter
23        passed by the ISS indication primitive to the Mask-and-match
24        Stream identification function. The resulting 48-bit
25        information is the masked source_address that is used as input
26        for the instance of the Mask-and-match Stream identification
27        function. If CpeMmIdSrcMacMask has a value of 0, the
28        source_address parameter is ignored."
29     REFERENCE    "9.1.6.3"
30     ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 3 }
31
32 ieee8021StreamIdCpeMmIdSrcMacMatch OBJECT-TYPE
33     SYNTAX      MacAddress
34     MAX-ACCESS  read-write
35     STATUS      current
36     DESCRIPTION
37        "Specifies the 48-bit value of the masked source_address, as
38        defined in 9.1.6.3, to be matched by the instance of the
39        Mask-and-match Stream identification function."
40     REFERENCE    "9.1.6.4"
41     ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 4 }
42
43 ieee8021StreamIdCpeMmIdMsduMaskLength OBJECT-TYPE
44     SYNTAX      Integer32(2..1984)
45     MAX-ACCESS  read-write
46     STATUS      current
47     DESCRIPTION
48        "Is an integer value that specifies the length, in octets, of
49        CpeMmIdMsduMask (9.1.6.6) and CpeMmIdMsduMatch (9.1.6.7). The
50        behavior of the Mask-and-match Stream identification function
51        is undefined on any port where the value of
52        CpeMmIdMsduMaskLength exceeds the per-port
53        mac_service_data_unit mask maximum length(9.4.1). If the
54        length of the frame's mac_service_data_unit is shorter than
55        CpeMmIdMsduMaskLength, then the identification fails."
56     REFERENCE    "9.1.6.5"
57     ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 5 }
58
59 ieee8021StreamIdCpeMmIdMsduMask OBJECT-TYPE

```

```

1  SYNTAX      OCTET STRING (SIZE(2..1984))
2  MAX-ACCESS  read-write
3  STATUS      current
4  DESCRIPTION
5      "Specifies a mask of CpeMmIdMsduMaskLength octets. A bitwise
6      AND operation is performed between CpeMmIdMsduMask and the
7      CpeMmIdMsduMaskLength (9.1.6.6) first octets of the
8      mac_service_data_unit parameter passed by the ISS indication
9      primitive to the Mask-and-match Stream identification
10     function. The resulting CpeMmIdMsduMaskLength octets of
11     information are the masked mac_service_data_unit that is used
12     as input for the instance of the Mask-and-match Stream
13     identification function. If CpeMmIdMsduMask has a value of 0,
14     the mac_service_data_unit parameter is ignored."
15  REFERENCE   "9.1.6.6"
16  ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 6 }
17
18 ieee8021StreamIdCpeMmIdMsduMatch OBJECT-TYPE
19  SYNTAX      OCTET STRING (SIZE(2..1984))
20  MAX-ACCESS  read-write
21  STATUS      current
22  DESCRIPTION
23      "Specifies the value of the masked mac_service_data_unit, as
24      defined in 9.1.6.5 and 9.1.6.6, to be matched by the instance
25      of the Mask-and-match Stream identification function."
26  REFERENCE   "9.1.6.7"
27  ::= { ieee8021StreamIdMaskAndMatchIdentificationEntry 7 }
28
29 -- =====
30 -- the ieee8021StreamIdStreamIdInFacOutputPortHandleList
31 -- table
32 -- =====
33
34 ieee8021StreamIdStreamIdInFacOutputPortHandleListTable OBJECT-TYPE
35  SYNTAX      SEQUENCE OF
36  Ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry
37  MAX-ACCESS  not-accessible
38  STATUS      current
39  DESCRIPTION
40      "A table containing a list of ports on which an in-facing Stream
41      identification function using this identification method is to
42      be placed for this Stream in the output (towards the system
43      forwarding function) direction, referenced in
44      ieee8021StreamIdStreamIdentificationIndex."
45  REFERENCE   "9.1.1.2"
46  ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleList 2 }
47
48 ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry OBJECT-TYPE
49  SYNTAX      Ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry
50  MAX-ACCESS  not-accessible
51  STATUS      current
52  DESCRIPTION
53      "A set of managed objects providing the ports on which an
54      in-facing Stream identification function using this
55      identification method is to be placed for this Stream in the
56      output (towards the system forwarding function) direction,
57      referenced in ieee8021StreamIdStreamIdentificationIndex."
58  REFERENCE   "9.1.1.2"
59  INDEX      { ieee8021StreamIdStreamIdentificationIndex,

```

```

1         ieee8021StreamIdStreamIdInFacOutputPortHandleListIndex
2     }
3     ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleListTable 1 }
4
5 Ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry ::=
6     SEQUENCE {
7         ieee8021StreamIdStreamIdInFacOutputPortHandleListIndex
8         Unsigned32,
9         ieee8021StreamIdStreamIdInFacOutputPortHandle
10        VariablePointer,
11        ieee8021StreamIdStreamIdInFacOutputPortHandleListStatus
12        RowStatus
13    }
14
15 ieee8021StreamIdStreamIdInFacOutputPortHandleListIndex OBJECT-TYPE
16     SYNTAX      Unsigned32
17     MAX-ACCESS  not-accessible
18     STATUS      current
19     DESCRIPTION
20         "Index for the In-Facing Output Port handle list table."
21     ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry 1 }
22
23 ieee8021StreamIdStreamIdInFacOutputPortHandle OBJECT-TYPE
24     SYNTAX      VariablePointer
25     MAX-ACCESS  read-create
26     STATUS      current
27     DESCRIPTION
28         "A pointer to an interface that is an element of the
29         ieee8021StreamIdStreamIdInFacOutputPortList instance in the
30         ieee8021StreamIdStreamIdentificationTable."
31     REFERENCE   "9.1.1.2"
32     ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry 2 }
33
34 ieee8021StreamIdStreamIdInFacOutputPortHandleListStatus OBJECT-TYPE
35     SYNTAX      RowStatus
36     MAX-ACCESS  read-create
37     STATUS      current
38     DESCRIPTION
39         "Row-Status for the In-Facing Output Port handle list table."
40     ::= { ieee8021StreamIdStreamIdInFacOutputPortHandleListEntry 3 }
41
42 -- =====
43 -- the ieee8021StreamIdStreamIdOutFacOutputPortList
44 -- table
45 -- =====
46
47 ieee8021StreamIdStreamIdOutFacOutputPortHandleListTable OBJECT-TYPE
48     SYNTAX      SEQUENCE OF
49 Ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry
50     MAX-ACCESS  not-accessible
51     STATUS      current
52     DESCRIPTION
53         "A table containing a list of ports on which an out-facing
54         Stream identification function using this identification method
55         is to be placed for this Stream in the output (towards the
56         system forwarding function) direction, referenced in
57         ieee8021StreamIdStreamIdentificationIndex."
58     REFERENCE   "9.1.1.3"
59     ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleList 3 }

```

```

1
2 ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry OBJECT-TYPE
3     SYNTAX      Ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry
4     MAX-ACCESS  not-accessible
5     STATUS      current
6     DESCRIPTION
7         "A set of managed objects providing the ports on which an
8         out-facing Stream identification function using this
9         identification method is to be placed for this Stream in the
10        output (towards the system forwarding function) direction,
11        referenced in ieee8021StreamIdStreamIdentificationIndex."
12    REFERENCE    "9.1.1.3"
13    INDEX        { ieee8021StreamIdStreamIdentificationIndex,
14                  ieee8021StreamIdStreamIdOutFacOutputPortHandleListIndex
15                }
16    ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleListTable 1 }
17
18 Ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry ::=
19     SEQUENCE {
20         ieee8021StreamIdStreamIdOutFacOutputPortHandleListIndex
21         Unsigned32,
22         ieee8021StreamIdStreamIdOutFacOutputPortHandle
23         VariablePointer,
24         ieee8021StreamIdStreamIdOutFacOutputPortHandleListStatus
25         RowStatus
26     }
27
28 ieee8021StreamIdStreamIdOutFacOutputPortHandleListIndex OBJECT-TYPE
29     SYNTAX      Unsigned32
30     MAX-ACCESS  not-accessible
31     STATUS      current
32     DESCRIPTION
33         "Index for the Out-Facing Output Port handle list table."
34     ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry 1 }
35
36 ieee8021StreamIdStreamIdOutFacOutputPortHandle OBJECT-TYPE
37     SYNTAX      VariablePointer
38     MAX-ACCESS  read-create
39     STATUS      current
40     DESCRIPTION
41         "A pointer to an interface that is an element of the
42         ieee8021StreamIdStreamIdOutFacOutputPortList instance in the
43         ieee8021StreamIdStreamIdentificationTable."
44     REFERENCE    "9.1.1.3"
45     ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry 2 }
46
47 ieee8021StreamIdStreamIdOutFacOutputPortHandleListStatus OBJECT-TYPE
48     SYNTAX      RowStatus
49     MAX-ACCESS  read-create
50     STATUS      current
51     DESCRIPTION
52         "Row-Status for the Out-Facing Output Port handle list table."
53     ::= { ieee8021StreamIdStreamIdOutFacOutputPortHandleListEntry 3 }
54
55 -- =====
56 -- the ieee8021StreamIdStreamIdInFacInputPortList
57 -- table
58 -- =====
59

```

```

1 ieee8021StreamIdStreamIdInFacInputPortHandleListTable OBJECT-TYPE
2   SYNTAX      SEQUENCE OF Ieee8021StreamIdStreamIdInFacInputPortHandleListEntry
3   MAX-ACCESS  not-accessible
4   STATUS      current
5   DESCRIPTION
6       "A table containing a list of ports on which an in-facing Stream
7       identification function using this identification method is to
8       be placed for this Stream in the input (coming from the system
9       forwarding function) direction, referenced in
10      ieee8021StreamIdStreamIdentificationIndex."
11   REFERENCE   "9.1.1.4"
12   ::= { ieee8021StreamIdStreamIdInFacInputPortHandleList 4 }
13
14 ieee8021StreamIdStreamIdInFacInputPortHandleListEntry OBJECT-TYPE
15   SYNTAX      Ieee8021StreamIdStreamIdInFacInputPortHandleListEntry
16   MAX-ACCESS  not-accessible
17   STATUS      current
18   DESCRIPTION
19       "A set of managed objects providing the ports on which an
20       in-facing Stream identification function using this
21       identification method is to be placed for this Stream in the
22       input (coming from the system forwarding function) direction,
23       referenced in ieee8021StreamIdStreamIdentificationIndex."
24   REFERENCE   "9.1.1.4"
25   INDEX       { ieee8021StreamIdStreamIdentificationIndex,
26                 ieee8021StreamIdStreamIdInFacInputPortHandleListIndex
27               }
28   ::= { ieee8021StreamIdStreamIdInFacInputPortHandleListTable 1 }
29
30 Ieee8021StreamIdStreamIdInFacInputPortHandleListEntry ::=
31   SEQUENCE {
32       ieee8021StreamIdStreamIdInFacInputPortHandleListIndex
33       Unsigned32,
34       ieee8021StreamIdStreamIdInFacInputPortHandle
35       VariablePointer,
36       ieee8021StreamIdStreamIdInFacInputPortHandleListStatus
37       RowStatus
38   }
39
40 ieee8021StreamIdStreamIdInFacInputPortHandleListIndex OBJECT-TYPE
41   SYNTAX      Unsigned32
42   MAX-ACCESS  not-accessible
43   STATUS      current
44   DESCRIPTION
45       "Index for the In-Facing Input Port handle list table."
46   ::= { ieee8021StreamIdStreamIdInFacInputPortHandleListEntry 1 }
47
48 ieee8021StreamIdStreamIdInFacInputPortHandle OBJECT-TYPE
49   SYNTAX      VariablePointer
50   MAX-ACCESS  read-create
51   STATUS      current
52   DESCRIPTION
53       "A pointer to an interface that is an element of the
54       ieee8021StreamIdStreamIdOutFacInputPortList instance in the
55       ieee8021StreamIdStreamIdentificationTable."
56   REFERENCE   "9.1.1.4"
57   ::= { ieee8021StreamIdStreamIdInFacInputPortHandleListEntry 2 }
58
59 ieee8021StreamIdStreamIdInFacInputPortHandleListStatus OBJECT-TYPE

```

```

1  SYNTAX      RowStatus
2  MAX-ACCESS  read-create
3  STATUS      current
4  DESCRIPTION
5      "Row-Status for the In-Facing Input Port handle list table."
6  ::= { ieee8021StreamIdStreamIdInFacInputPortHandleListEntry 3 }
7
8 -- =====
9 -- the ieee8021StreamIdStreamIdOutFacInputPortList
10 -- table
11 -- =====
12
13 ieee8021StreamIdStreamIdOutFacInputPortHandleListTable OBJECT-TYPE
14     SYNTAX          SEQUENCE OF
15     Ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry
16     MAX-ACCESS      not-accessible
17     STATUS          current
18     DESCRIPTION
19         "A table containing a list of ports on which an out-facing
20         Stream identification function using this identification method
21         is to be placed for this Stream in the input (coming from the
22         physical interface) direction, referenced in
23         ieee8021StreamIdStreamIdentificationIndex."
24     REFERENCE       "9.1.1.5"
25     ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleList 5 }
26
27 ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry OBJECT-TYPE
28     SYNTAX          Ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry
29     MAX-ACCESS      not-accessible
30     STATUS          current
31     DESCRIPTION
32         "A set of managed objects providing the ports on which an
33         out-facing Stream identification function using this
34         identification method is to be placed for this Stream in the
35         input (coming from the physical interface) direction,
36         referenced in ieee8021StreamIdStreamIdentificationIndex."
37     REFERENCE       "9.1.1.5"
38     INDEX           { ieee8021StreamIdStreamIdentificationIndex,
39                       ieee8021StreamIdStreamIdOutFacInputPortHandleListIndex
40                     }
41     ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleListTable 1 }
42
43 Ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry ::=
44     SEQUENCE {
45         ieee8021StreamIdStreamIdOutFacInputPortHandleListIndex
46         Unsigned32,
47         ieee8021StreamIdStreamIdOutFacInputPortHandle
48         VariablePointer,
49         ieee8021StreamIdStreamIdOutFacInputPortHandleListStatus
50         RowStatus
51     }
52
53 ieee8021StreamIdStreamIdOutFacInputPortHandleListIndex OBJECT-TYPE
54     SYNTAX          Unsigned32
55     MAX-ACCESS      not-accessible
56     STATUS          current
57     DESCRIPTION
58         "Index for the Out-Facing Input Port handle list table."
59     ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry 1 }

```

```

1
2 ieee8021StreamIdStreamIdOutFacInputPortHandle OBJECT-TYPE
3     SYNTAX      VariablePointer
4     MAX-ACCESS  read-create
5     STATUS      current
6     DESCRIPTION
7         "A pointer to an interface that is an element of the
8         ieee8021StreamIdStreamIdOutFacInputPortList instance in the
9         ieee8021StreamIdStreamIdentificationTable."
10    REFERENCE   "9.1.1.5"
11    ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry 2 }
12
13 ieee8021StreamIdStreamIdOutFacInputPortHandleListStatus OBJECT-TYPE
14     SYNTAX      RowStatus
15     MAX-ACCESS  read-create
16     STATUS      current
17     DESCRIPTION
18         "Row-Status for the Out-Facing Input Port handle list table."
19     ::= { ieee8021StreamIdStreamIdOutFacInputPortHandleListEntry 3 }
20
21
22 -- =====
23 -- the ieee8021StreamIdPerPortPerStreamCountersTable
24 -- =====
25
26 ieee8021StreamIdPerPortPerStreamCountersTable OBJECT-TYPE
27     SYNTAX      SEQUENCE OF Ieee8021StreamIdPerPortPerStreamCountersEntry
28     MAX-ACCESS  not-accessible
29     STATUS      current
30     DESCRIPTION
31         "A table containing a set of counters for stream
32         identification that are instantiated per port, stream and
33         direction."
34     REFERENCE   "9.2"
35     ::= { ieee8021StreamIdPerPortPerStreamCounters 6 }
36
37 ieee8021StreamIdPerPortPerStreamCountersEntry OBJECT-TYPE
38     SYNTAX      Ieee8021StreamIdPerPortPerStreamCountersEntry
39     MAX-ACCESS  not-accessible
40     STATUS      current
41     DESCRIPTION
42         "A set of managed objects for stream identification
43         counters that are instantiated per port, stream and
44         direction."
45     REFERENCE   "9.2"
46     INDEX { ifIndex,
47             ieee8021StreamIdStreamIdHandle,
48             ieee8021StreamIdPerPortPerStreamDirection
49           }
50     ::= { ieee8021StreamIdPerPortPerStreamCountersTable 1 }
51
52 Ieee8021StreamIdPerPortPerStreamCountersEntry ::=
53     SEQUENCE {
54         ieee8021StreamIdPerPortPerStreamDirection
55         TruthValue,
56         ieee8021StreamIdPerPortPerStreamInputPackets
57         Counter64,
58         ieee8021StreamIdPerPortPerStreamOutputPackets
59         Counter64

```

```

1      }
2
3 ieee8021StreamIdPerPortPerStreamDirection OBJECT-TYPE
4     SYNTAX      TruthValue
5     MAX-ACCESS  not-accessible
6     STATUS      current
7     DESCRIPTION
8         "A managed object specifying the facing of a given stream
9         on a port. The stream can be either in-facing (False) or
10        out-facing (True)."
```

11 REFERENCE "9.2"

12 ::= { ieee8021StreamIdPerPortPerStreamCountersEntry 1 }

13

```

14 ieee8021StreamIdPerPortPerStreamInputPackets OBJECT-TYPE
15     SYNTAX      Counter64
16     MAX-ACCESS  read-only
17     STATUS      current
18     DESCRIPTION
19         "A managed object serving as a counter that is incremented once
20         for each packet identified by the Stream identification
21         function."
```

22 REFERENCE "9.2.1"

23 ::= { ieee8021StreamIdPerPortPerStreamCountersEntry 2 }

24

```

25 ieee8021StreamIdPerPortPerStreamOutputPackets OBJECT-TYPE
26     SYNTAX      Counter64
27     MAX-ACCESS  read-only
28     STATUS      current
29     DESCRIPTION
30         "A managed object serving as a counter that is incremented once
31         for each packet passed down the stack by the Stream
32         identification function."
```

33 REFERENCE "9.2.2"

34 ::= { ieee8021StreamIdPerPortPerStreamCountersEntry 3 }

35

36 -- =====

37 -- the ieee8021StreamIdPerPortCountersTable

38 -- =====

39

```

40 ieee8021StreamIdPerPortCountersTable OBJECT-TYPE
41     SYNTAX      SEQUENCE OF Ieee8021StreamIdPerPortCountersEntry
42     MAX-ACCESS  not-accessible
43     STATUS      current
44     DESCRIPTION
45         "A table containing a set of counters for stream
46         identification that are instantiated per port and
47         direction."
```

48 REFERENCE "9.3"

49 ::= { ieee8021StreamIdPerPortCounters 7 }

50

```

51 ieee8021StreamIdPerPortCountersEntry OBJECT-TYPE
52     SYNTAX      Ieee8021StreamIdPerPortCountersEntry
53     MAX-ACCESS  not-accessible
54     STATUS      current
55     DESCRIPTION
56         "A set of managed objects for stream identification
57         counters that are instantiated per port and direction."
```

58 REFERENCE "9.3"

59 INDEX { ifIndex }


```

1      ::= { ieee8021StreamIdPerPortCountersTable 1 }
2
3 Ieee8021StreamIdPerPortCountersEntry ::=
4     SEQUENCE {
5         ieee8021StreamIdPerPortInputPackets
6             Counter64,
7         ieee8021StreamIdPerPortOutputPackets
8             Counter64
9     }
10
11 ieee8021StreamIdPerPortInputPackets OBJECT-TYPE
12     SYNTAX      Counter64
13     MAX-ACCESS  read-only
14     STATUS      current
15     DESCRIPTION
16         "A managed object serving as a counter that is incremented once
17         for each packet identified by any Stream identification
18         function on this port. Its value equals the sum (modulo the
19         size of the counters) of all of the
20         ieee8021StreamIdPerPortPerStreamInputPackets counters on this
21         same port."
22     REFERENCE   "9.3.1"
23     ::= { ieee8021StreamIdPerPortCountersEntry 1 }
24
25 ieee8021StreamIdPerPortOutputPackets OBJECT-TYPE
26     SYNTAX      Counter64
27     MAX-ACCESS  read-only
28     STATUS      current
29     DESCRIPTION
30         "A managed object serving as a counter that is incremented once
31         for each packet passed down the stack by any Stream
32         identification function on this port. Its value equals the sum
33         (modulo the size of the counters) of all of the
34         ieee8021StreamIdPerPortPerStreamOutputPackets counters on this
35         same port."
36     REFERENCE   "9.3.2"
37     ::= { ieee8021StreamIdPerPortCountersEntry 2 }
38
39 -- =====
40 -- the ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthTable
41 -- =====
42
43 ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthTable OBJECT-TYPE
44     SYNTAX      SEQUENCE OF
45 Ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry
46     MAX-ACCESS  not-accessible
47     STATUS      current
48     DESCRIPTION
49         "A table containing a set of maximum msdu mask length
50         parameters that are defined per port."
51     REFERENCE   "9.4.1"
52     ::= { ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLength 8 }
53
54 ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry OBJECT-TYPE
55     SYNTAX      Ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry
56     MAX-ACCESS  not-accessible
57     STATUS      current
58     DESCRIPTION
59         "A a set of managed objects specifying the maximum msdu

```

```

1      mask length per port, applicable to all the instances
2      of Mask-And-Match Stream identification functions"
3  REFERENCE    "9.4.1"
4  INDEX { ifIndex }
5  ::= { ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthTable 1 }
6
7  Ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry ::=
8      SEQUENCE {
9          ieee8021StreamIdMaskAndMatchMsduMaskMaxLength
10         Integer32
11     }
12
13  ieee8021StreamIdMaskAndMatchMsduMaskMaxLength OBJECT-TYPE
14      SYNTAX      Integer32(2..1984)
15      MAX-ACCESS  read-write
16      STATUS      current
17      DESCRIPTION
18          "A managed object specifying the maximum msdu mask length for
19          a given port."
20      REFERENCE    "9.4.1"
21      ::= { ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthEntry 1 }
22
23 -- =====
24 -- IEEE802 STREAM IDENTIFICATION MIB - Conformance Information
25 -- =====
26
27  ieee8021StreamIdCompliances
28      OBJECT IDENTIFIER ::= { ieee8021StreamIdConformance 1 }
29  ieee8021StreamIdGroups
30      OBJECT IDENTIFIER ::= { ieee8021StreamIdConformance 2 }
31
32 -- =====
33 -- units of conformance
34 -- =====
35
36 -- =====
37 -- the ieee8021StreamIdStreamIdentification group
38 -- =====
39
40  ieee8021StreamIdStreamIdentificationGroup OBJECT-GROUP
41      OBJECTS {
42          ieee8021StreamIdStreamIdIdentificationTypeSelect,
43          ieee8021StreamIdStreamIdIdentificationType,
44          ieee8021StreamIdStreamIdIdentificationTypeOUI,
45          ieee8021StreamIdStreamIdIdentificationCustomType,
46          ieee8021StreamIdStreamIdIdentificationCustomTypeOUI,
47          ieee8021StreamIdStreamIdHandle,
48          ieee8021StreamIdStreamIdInFacOutputPortList,
49          ieee8021StreamIdStreamIdOutFacOutputPortList,
50          ieee8021StreamIdStreamIdInFacInputPortList,
51          ieee8021StreamIdStreamIdOutFacInputPortList,
52          ieee8021StreamIdStreamIdInFacOutputPortHandle,
53          ieee8021StreamIdStreamIdInFacOutputPortHandleListStatus,
54          ieee8021StreamIdStreamIdOutFacOutputPortHandle,
55          ieee8021StreamIdStreamIdOutFacOutputPortHandleListStatus,
56          ieee8021StreamIdStreamIdInFacInputPortHandle,
57          ieee8021StreamIdStreamIdInFacInputPortHandleListStatus,
58          ieee8021StreamIdStreamIdOutFacInputPortHandle,
59          ieee8021StreamIdStreamIdOutFacInputPortHandleListStatus

```

```

1      }
2      STATUS          current
3      DESCRIPTION
4          "Objects that are part of the stream identification."
5      ::= { ieee8021StreamIdGroups 1 }
6
7 -- =====
8 -- the ieee8021StreamIdNullStreamIdentification group
9 -- =====
10
11 ieee8021StreamIdNullStreamIdentificationGroup OBJECT-GROUP
12     OBJECTS {
13         ieee8021StreamIdCpeNullDownDestMac,
14         ieee8021StreamIdCPENullDownTagged,
15         ieee8021StreamIdCpeNullDownVlan
16     }
17     STATUS          current
18     DESCRIPTION
19         "Objects that are part of the null stream identification
20         method."
21     ::= { ieee8021StreamIdGroups 2 }
22
23 -- =====
24 -- the ieee8021StreamIdSrcMacVlanIdentification group
25 -- =====
26
27 ieee8021StreamIdSrcMacVlanIdentificationGroup OBJECT-GROUP
28     OBJECTS {
29         ieee8021StreamIdCpeSmacVlanDownSrcMac,
30         ieee8021StreamIdCpeSmacVlanDownTagged,
31         ieee8021StreamIdCpeSmacVlanDownVlan
32     }
33     STATUS          current
34     DESCRIPTION
35         "Objects that are part of the Source MAC and VLAN stream
36         identification method."
37     ::= { ieee8021StreamIdGroups 3 }
38
39 -- =====
40 -- the ieee8021StreamIdActiveDestMacVlanIdentification group
41 -- =====
42
43 ieee8021StreamIdActiveDestMacVlanIdentificationGroup OBJECT-GROUP
44     OBJECTS {
45         ieee8021StreamIdCpeDmacVlanDownDestMac,
46         ieee8021StreamIdCpeDmacVlanDownTagged,
47         ieee8021StreamIdCpeDmacVlanDownVlan,
48         ieee8021StreamIdCpeDmacVlanDownPriority,
49         ieee8021StreamIdCpeDmacVlanUpDestMac,
50         ieee8021StreamIdCpeDmacVlanUpTagged,
51         ieee8021StreamIdCpeDmacVlanUpVlan,
52         ieee8021StreamIdCpeDmacVlanUpPriority
53     }
54     STATUS          current
55     DESCRIPTION
56         "Objects that are part of the Active Destination MAC and
57         VLAN stream identification method."
58     ::= { ieee8021StreamIdGroups 4 }
59

```

```

1 -- =====
2 -- the ieee8021StreamIdIpStreamIdentification group
3 -- =====
4
5 ieee8021StreamIdIpStreamIdentificationGroup OBJECT-GROUP
6     OBJECTS {
7         ieee8021StreamIdCpeIpIdDestMac,
8         ieee8021StreamIdCpeIpIdTagged,
9         ieee8021StreamIdCpeIpIdVlan,
10        ieee8021StreamIdCpeIpIdIpSourceType,
11        ieee8021StreamIdCpeIpIdIpSource,
12        ieee8021StreamIdCpeIpIdIpDestinationType,
13        ieee8021StreamIdCpeIpIdIpDestination,
14        ieee8021StreamIdCpeIpIdDscp,
15        ieee8021StreamIdCpeIpIdNextProtocol,
16        ieee8021StreamIdCpeIpIdSourcePort,
17        ieee8021StreamIdCpeIpIdDestinationPort
18    }
19     STATUS      current
20     DESCRIPTION
21         "Objects that are part of the IP stream identification method."
22     ::= { ieee8021StreamIdGroups 5 }
23
24 -- =====
25 -- the ieee8021StreamIdAutoConfiguration group
26 -- =====
27
28 ieee8021StreamIdAutoConfigurationGroup OBJECT-GROUP
29     OBJECTS {
30         ieee8021StreamIdAutoConfigured,
31         ieee8021StreamIdLanPathId,
32         ieee8021StreamIdStatus
33     }
34     STATUS      current
35     DESCRIPTION
36         "Objects that are used if auto configuration for streams is
37         used."
38     ::= { ieee8021StreamIdGroups 6 }
39
40 -- =====
41 -- the ieee8021StreamIdPerPortPerStreamCounters group
42 -- =====
43
44 ieee8021StreamIdPerPortPerStreamCountersGroup OBJECT-GROUP
45     OBJECTS {
46         ieee8021StreamIdPerPortPerStreamInputPackets,
47         ieee8021StreamIdPerPortPerStreamOutputPackets
48     }
49     STATUS      current
50     DESCRIPTION
51         "Objects that provide information on the count of packets
52         handled by stream identification per port, stream and
53         direction ."
54     ::= { ieee8021StreamIdGroups 7 }
55
56 -- =====
57 -- the ieee8021StreamIdPerPortCounters group
58 -- =====
59

```

```

1 ieee8021StreamIdPerPortCountersGroup OBJECT-GROUP
2   OBJECTS {
3       ieee8021StreamIdPerPortInputPackets,
4       ieee8021StreamIdPerPortOutputPackets
5   }
6   STATUS      current
7   DESCRIPTION
8       "Objects that provide information on the count of packets
9       handled by stream identification per port and direction ."
10  ::= { ieee8021StreamIdGroups 8 }
11
12 -- =====
13 -- the ieee8021StreamIdMaskAndMatchIdentification group
14 -- =====
15
16 ieee8021StreamIdMaskAndMatchIdentificationGroup OBJECT-GROUP
17   OBJECTS {
18       ieee8021StreamIdCpeMmIdDestMacMask,
19       ieee8021StreamIdCpeMmIdDestMacMatch,
20       ieee8021StreamIdCpeMmIdSrcMacMask,
21       ieee8021StreamIdCpeMmIdSrcMacMatch,
22       ieee8021StreamIdCpeMmIdMsduMaskLength,
23       ieee8021StreamIdCpeMmIdMsduMask,
24       ieee8021StreamIdCpeMmIdMsduMatch
25   }
26   STATUS      current
27   DESCRIPTION
28       "Objects that are part of the Mask-and-Match stream
29       identification method."
30  ::= { ieee8021StreamIdGroups 9 }
31
32 -- =====
33 -- the ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLength group
34 -- =====
35
36 ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthGroup OBJECT-GROUP
37   OBJECTS {
38       ieee8021StreamIdMaskAndMatchMsduMaskMaxLength
39   }
40   STATUS      current
41   DESCRIPTION
42       "Objects that provide information on the maximum msdu mask
43       handled by the Mask-and-match stream identification function
44       per port."
45  ::= { ieee8021StreamIdGroups 10 }
46
47 -- =====
48 -- compliance statements
49 -- =====
50
51 ieee8021StreamIdCompliance MODULE-COMPLIANCE
52   STATUS      current
53   DESCRIPTION
54       "The compliance statement for devices supporting
55       stream identification."
56
57   MODULE -- this module
58       MANDATORY-GROUPS {
59           ieee8021StreamIdStreamIdentificationGroup,

```

```

1         ieee8021StreamIdNullStreamIdentificationGroup,
2         ieee8021StreamIdSrcMacVlanIdentificationGroup,
3         ieee8021StreamIdActiveDestMacVlanIdentificationGroup,
4         ieee8021StreamIdIpStreamIdentificationGroup,
5         ieee8021StreamIdPerPortPerStreamCountersGroup,
6         ieee8021StreamIdPerPortCountersGroup,
7         ieee8021StreamIdMaskAndMatchIdentificationGroup,
8         ieee8021StreamIdMaskAndMatchPerPortMsduMaskMaxLengthGroup
9     }
10
11     GROUP ieee8021StreamIdAutoConfigurationGroup
12     DESCRIPTION
13         "Implementation of this group is mandatory if the
14         auto configuration feature is implemented."
15
16     ::= { ieee8021StreamIdCompliances 1 }
17
18 END

```

19 11.5.2 Definitions for the IEEE8021-FRER-MIB

20 IEEE8021-FRER-MIB DEFINITIONS ::= BEGIN

```

-- =====
-- MIB for support of frame replication and elimination of
-- IEEE 802.1CB Frame Replication and Elimination for Reliability
-- =====

```

IMPORTS

```

    OBJECT-GROUP,
    MODULE-COMPLIANCE
        FROM SNMPv2-CONF
    MODULE-IDENTITY,
    OBJECT-TYPE,
    Counter64,
    Unsigned32,
    Integer32
        FROM SNMPv2-SMI
    TEXTUAL-CONVENTION,
    TruthValue,
    RowStatus,
    AutonomousType,
    VariablePointer
        FROM SNMPv2-TC
    ifIndex,
    InterfaceIndex
        FROM IF-MIB
    ieee802dot1mibs
        FROM IEEE8021-TC-MIB
    Ieee8021CBTaggedType,
    Ieee8021CBVlanIdentifier,
    Ieee8021CBLanPathIdType,
    ieee8021StreamIdStreamIdHandle
        FROM IEEE8021-STREAM-IDENTIFICATION-MIB
;

```

ieee8021FrerMib MODULE-IDENTITY

```

    LAST-UPDATED "202112080000Z" -- December 8, 2021
    ORGANIZATION "IEEE 802.1 Working Group"
    CONTACT-INFO
        "WG-URL: http://ieee802.org/1/
        WG-EMail: stds-802-1-l@ieee.org

```

```

    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
            IEEE Standards Association
            445 Hoes Lane

```

1 Piscataway, NJ 08854
USA

E-mail: stds-802-1-chairs@ieee.org"

DESCRIPTION

"The Management Information Base module for IEEE 802.1CB Stream identification. Unless otherwise indicated, the references in this MIB module are to IEEE Std 802.1CB-2017 as amended by IEEE Std 802.1CBcv-2021. Copyright (C) IEEE (2021). This version of this MIB module is part of IEEE Std 802.1CB-2021; see the standard itself for full legal notices."

REVISION "202112080000Z" -- December 8, 2021

DESCRIPTION

"Initial revision published as part of IEEE Std 802.1CBcv-2021."

::= { ieee802dot1mibs 35 }

-- =====
-- Textual Conventions
-- =====

Ieee8021CBSequenceRecoveryAlgorithm ::= TEXTUAL-CONVENTION

STATUS current

DESCRIPTION

"An enumerated value specifying which sequence recovery algorithm is to be used for this instance of the Sequence recovery function. The enumeration uses an OUI or CID as shown in Table 10-1. If the value used is in the range of 0-255, the OUI used is 00-80-C2.
The values 0 and 2-255 are reserved for use by IEEE 802."

REFERENCE "10.4.1.5 and Table 10-1"

SYNTAX INTEGER {
vectorAlgorithm(0),
matchAlgorithm(1),
nonIEEESpecified(256)
}

Ieee8021CBSequenceEncodeDecodeType ::= TEXTUAL-CONVENTION

STATUS current

DESCRIPTION

"An enumerated value indicating the type of encapsulation used for this instance of the Sequence encode/decode function. The enumeration uses an OUI or CID as shown in Table 10-2. If the value used is in the range of 0-255, the OUI used is 00-80-C2.
The values 0 and 4-255 are reserved for use by IEEE 802."

REFERENCE "10.5.1.5 and Table 10-2"

SYNTAX INTEGER {
reserved(0),
rTAG(1),
hsrSequenceTag(2),
prpSequenceTag(3),
nonIEEESpecified(256)
}

-- =====
-- subtrees in the FRER MIB
-- =====

ieee8021FrerNotifications

OBJECT IDENTIFIER ::= { ieee8021FrerMib 0 }

ieee8021FrerObjects

OBJECT IDENTIFIER ::= { ieee8021FrerMib 1 }

ieee8021FrerConformance

OBJECT IDENTIFIER ::= { ieee8021FrerMib 2 }

ieee8021FrerSequenceGeneration

OBJECT IDENTIFIER ::= { ieee8021FrerObjects 1 }

```

1 ieee8021FrerSequenceGenerationHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 2 }

ieee8021FrerSequenceRecovery
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 3 }

ieee8021FrerSequenceRecoveryHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 4 }

ieee8021FrerSequenceRecoveryPortHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 5 }

ieee8021FrerSequenceIdentification
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 6 }

ieee8021FrerSequenceIdentificationHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 7 }

ieee8021FrerStreamSplit
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 8 }

ieee8021FrerStreamSplitInputHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 9 }

ieee8021FrerStreamSplitOutputHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 10 }

ieee8021FrerSequenceAutoconfig
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 11 }

ieee8021FrerSequenceAutoconfigReceivePortHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 12 }

ieee8021FrerSequenceAutoconfigVlanHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 13 }

ieee8021FrerSequenceAutoconfigRecoveryPortHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 14 }

ieee8021FrerOutputAutoconfig
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 15 }

ieee8021FrerOutputAutoconfigPortHandleList
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 16 }

ieee8021FrerPerPortPerStreamCounters
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 17 }

ieee8021FrerPerPortCounters
  OBJECT IDENTIFIER ::= { ieee8021FrerObjects 18 }

-- =====
-- the ieee8021FrerSequenceGeneration table
-- =====

ieee8021FrerSequenceGenerationTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceGenerationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of sequence generation entry objects.
         There is one entry for each sequence generation function."
    REFERENCE   "10.3"
    ::= { ieee8021FrerSequenceGeneration 1 }

ieee8021FrerSequenceGenerationEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceGenerationEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects that manage the Streams and direction
         for which a single instance of the Sequence generation function

```



```

1      is to be placed."
REFERENCE "10.3.1"
INDEX { ieee8021FrerSequenceGenerationIndex }
::= { ieee8021FrerSequenceGenerationTable 1 }

Ieee8021FrerSequenceGenerationEntry ::=
SEQUENCE {
    ieee8021FrerSequenceGenerationIndex
        Unsigned32,
    ieee8021FrerSequenceGenerationStreamList
        AutonomousType,
    ieee8021FrerSequenceGenerationDirection
        TruthValue,
    ieee8021FrerSequenceGenerationReset
        TruthValue
}

ieee8021FrerSequenceGenerationIndex OBJECT-TYPE
SYNTAX      Unsigned32
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "Index for the Sequence Generation table."
::= { ieee8021FrerSequenceGenerationEntry 1 }

ieee8021FrerSequenceGenerationStreamList OBJECT-TYPE
SYNTAX      AutonomousType
MAX-ACCESS  read-write
STATUS      current
DESCRIPTION
    "A list of stream_handle values, corresponding to the values of
    the tsnStreamIdHandle objects in the Stream identity table,
    on which this instance of the Sequence generation function
    is to operate.
    The single instance of the Sequence generation function created
    by this frerSeqGenEntry operates every packet belonging to this
    Stream, regardless of the port on which it is received.
    For each instance of the Sequence generation function present
    in the ieee8021FrerSequenceGenerationTable there exists a list
    of stream_handle values in the
    ieee8021FrerSequenceGenerationHandleListTable."
REFERENCE "10.3.1.1"
::= { ieee8021FrerSequenceGenerationEntry 2 }

ieee8021FrerSequenceGenerationDirection OBJECT-TYPE
SYNTAX      TruthValue
MAX-ACCESS  read-write
STATUS      current
DESCRIPTION
    "This managed object indicates whether the Sequence generation
    function is to be placed on the out-facing (True) or
    in-facing (False) side of the port."
REFERENCE "10.3.1.2"
::= { ieee8021FrerSequenceGenerationEntry 3 }

ieee8021FrerSequenceGenerationReset OBJECT-TYPE
SYNTAX      TruthValue
MAX-ACCESS  read-write
STATUS      current
DESCRIPTION
    "This managed object indicates that the Sequence generation
    function is to be reset by calling its corresponding
    SequenceGenerationReset function. Writing the value True to
    this managed object triggers a reset; writing the value False
    has no effect. When read, this managed object always returns
    the value False."
REFERENCE "10.3.1.3"
::= { ieee8021FrerSequenceGenerationEntry 4 }

-- =====
-- the ieee8021FrerSequenceGenerationHandleList table

```

```

1 -- =====

ieee8021FrerSequenceGenerationHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceGenerationHandleListEntry
    MAX-ACCESS   not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of stream handle values,
        corresponding to the values of the tsnStreamIdHandle objects in
        the Stream identity table, on which the instance of the
        Sequence generation function referenced in
        ieee8021FrerSequenceGenerationIndex is to operate.
        The single instance of the Sequence generation function created
        by the ieee8021FrerSequenceGenerationEntry referenced operates
        every packet belonging to this Stream, regardless of the port
        on which it is received."
    REFERENCE   "10.3.1.1"
    ::= { ieee8021FrerSequenceGenerationHandleList 2 }

ieee8021FrerSequenceGenerationHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceGenerationHandleListEntry
    MAX-ACCESS   not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the stream handle values
        corresponding to the values of the tsnStreamIdHandle objects in
        the Stream identity table, on which the instance of the
        Sequence generation function referenced in
        ieee8021FrerSequenceGenerationIndex is to operate."
    REFERENCE   "10.3.1.1"
    INDEX       { ieee8021FrerSequenceGenerationIndex,
                  ieee8021FrerSequenceGenerationHandleListIndex
                }
    ::= { ieee8021FrerSequenceGenerationHandleListTable 1 }

Ieee8021FrerSequenceGenerationHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceGenerationHandleListIndex
            Unsigned32,
        ieee8021FrerSequenceGenerationStreamHandle
            VariablePointer,
        ieee8021FrerSequenceGenerationHandleListStatus
            RowStatus
    }

ieee8021FrerSequenceGenerationHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS   not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Generation handle list table."
    ::= { ieee8021FrerSequenceGenerationHandleListEntry 1 }

ieee8021FrerSequenceGenerationStreamHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS   read-create
    STATUS      current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
        element of the ieee8021FrerSequenceGenerationStreamList
        instance in the ieee8021StreamIdStreamIdentificationTable."
    REFERENCE   "10.3.1.1"
    ::= { ieee8021FrerSequenceGenerationHandleListEntry 2 }

ieee8021FrerSequenceGenerationHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS   read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Sequence Generation handle list table."
    ::= { ieee8021FrerSequenceGenerationHandleListEntry 3 }

```

```

1 -- =====
-- the ieee8021FrerSequenceRecovery table
-- =====

ieee8021FrerSequenceRecoveryTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceRecoveryEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of managed objects for the single
        instance of a Base recovery function and Latent error detection
        function included in the Sequence recovery function or
        Individual recovery function."
    REFERENCE   "10.4"
    ::= { ieee8021FrerSequenceRecovery 3 }

ieee8021FrerSequenceRecoveryEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceRecoveryEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects listing the Streams, ports, and
        direction for which instances of a Sequence recovery function
        or Individual recovery function are to be instantiated."
    REFERENCE   "10.4.1"
    INDEX       { ieee8021FrerSequenceRecoveryIndex }
    ::= { ieee8021FrerSequenceRecoveryTable 1 }

Ieee8021FrerSequenceRecoveryEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceRecoveryIndex
            Unsigned32,
        ieee8021FrerSequenceRecoveryStreamList
            AutonomousType,
        ieee8021FrerSequenceRecoveryPortList
            AutonomousType,
        ieee8021FrerSequenceRecoveryDirection
            TruthValue,
        ieee8021FrerSequenceRecoveryReset
            TruthValue,
        ieee8021FrerSequenceRecoveryAlgorithm
            Ieee8021CBSequenceRecoveryAlgorithm,
        ieee8021FrerSequenceRecoveryAlgorithmOUI
            OCTET STRING,
        ieee8021FrerSequenceCustomRecoveryAlgorithm
            Integer32,
        ieee8021FrerSequenceCustomRecoveryAlgorithmOUI
            OCTET STRING,
        ieee8021FrerSequenceRecoveryHistoryLength
            Integer32,
        ieee8021FrerSequenceRecoveryResetMSec
            Unsigned32,
        ieee8021FrerSequenceRecoveryInvalidSequenceValue
            Unsigned32,
        ieee8021FrerSequenceRecoveryTakeNoSequence
            TruthValue,
        ieee8021FrerSequenceRecoveryIndividualRecovery
            TruthValue,
        ieee8021FrerSequenceRecoveryLatentErrorDetection
            TruthValue,
        ieee8021FrerSequenceRecoveryLatentErrorDifference
            Integer32,
        ieee8021FrerSequenceRecoveryLatentErrorPeriod
            Integer32,
        ieee8021FrerSequenceRecoveryLatentErrorPaths
            Integer32,
        ieee8021FrerSequenceRecoveryLatentResetPeriod
            Integer32,
        ieee8021FrerSequenceRecoveryStatus
            RowStatus
    }

```

```

1 ieee8021FrerSequenceRecoveryIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Recovery table."
    ::= { ieee8021FrerSequenceRecoveryEntry 1 }

ieee8021FrerSequenceRecoveryStreamList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A list of the stream handle values, corresponding to the values
        of the tsNStreamIdHandle objects in the Stream identity table
        to which the system is to apply the instance of the Sequence
        recovery function or Individual recovery function.
        For each entry in the ieee8021FrerSequenceRecoveryTable
        there exists a list of stream handle values in the
        ieee8021FrerSequenceRecoveryHandleListTable."
    REFERENCE   "10.4.1.1"
    ::= { ieee8021FrerSequenceRecoveryEntry 2 }

ieee8021FrerSequenceRecoveryPortList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "The list of ports on each of which the system is to instantiate
        the Sequence recovery function, or from which received packets
        are to be fed to a single instance of the Individual recovery
        function.
        For each entry in the ieee8021FrerSequenceRecoveryTable
        there exists a list of ports in the
        ieee8021FrerSequenceRecoveryPortHandleListTable."
    REFERENCE   "10.4.1.2"
    ::= { ieee8021FrerSequenceRecoveryEntry 3 }

ieee8021FrerSequenceRecoveryDirection OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object indicates whether the Sequence recovery
        function or Individual recovery function is to be placed on the
        out-facing (True) or in-facing (False) side of the port."
    REFERENCE   "10.4.1.3"
    ::= { ieee8021FrerSequenceRecoveryEntry 4 }

ieee8021FrerSequenceRecoveryReset OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object indicates that the Sequence recovery
        function or Individual recovery function is to be reset by
        calling its corresponding SequenceGenerationReset function.
        Writing the value True to ieee8021FrerSequenceRecoveryReset
        triggers a reset; writing the value False has no effect.
        When read, ieee8021FrerSequenceRecoveryReset always returns
        the value False."
    REFERENCE   "10.4.1.4"
    ::= { ieee8021FrerSequenceRecoveryEntry 5 }

ieee8021FrerSequenceRecoveryAlgorithm OBJECT-TYPE
    SYNTAX      Ieee8021CBSequenceRecoveryAlgorithm
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object is an enumerated value specifying which
        sequence recovery algorithm is to be used for this instance of
        the Sequence recovery function. If the value used is

```

```

1      nonIEEESpecified(256), the managed objects
      ieee8021FrerSequenceCustomRecoveryAlgorithmOUI and
      ieee8021FrerSequenceCustomRecoveryAlgorithm are used to specify
      OUI/CID and the recovery algorithm defined by the entity owning
      the OUI/CID."
      REFERENCE    "10.4.1.5"
      DEFVAL { vectorAlgorithm }
      ::= { ieee8021FrerSequenceRecoveryEntry 6 }

ieee8021FrerSequenceRecoveryAlgorithmOUI OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE (3))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "This managed object specifies the OUI/CID used together with
        the sequence recovery algorithm type specified in
        ieee8021FrerSequenceRecoveryAlgorithm.
        It is used if and only if the value for
        ieee8021FrerSequenceRecoveryAlgorithm is in the range of 0-255.
        In this case it always takes the value 00-80-C2."
    REFERENCE    "10.4.1.5"
    ::= { ieee8021FrerSequenceRecoveryEntry 7 }

ieee8021FrerSequenceCustomRecoveryAlgorithm OBJECT-TYPE
    SYNTAX      Integer32 (256..2147483647)
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object is an enumerated value specifying which
        sequence recovery algorithm is to be used for this instance of
        the Sequence recovery function. This managed object is used if
        and only if the value for ieee8021FrerSequenceRecoveryAlgorithm
        is 256."
    REFERENCE    "10.4.1.5"
    ::= { ieee8021FrerSequenceRecoveryEntry 8 }

ieee8021FrerSequenceCustomRecoveryAlgorithmOUI OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE (3))
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies the OUI/CID used together with
        the sequence recovery algorithm type specified in
        ieee8021FrerSequenceRecoveryAlgorithm. It is used if and only
        if the value for ieee8021FrerSequenceRecoveryAlgorithm is 256."
    REFERENCE    "10.4.1.5"
    ::= { ieee8021FrerSequenceRecoveryEntry 9 }

ieee8021FrerSequenceRecoveryHistoryLength OBJECT-TYPE
    SYNTAX      Integer32
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies how many bits of the
        SequenceHistory variable are to be used. The minimum and the
        default value is 2, maximum is the maximum allowed by the
        implementation. It is not used if the value for
        ieee8021FrerSequenceRecoveryAlgorithm is 1."
    REFERENCE    "10.4.1.6"
    ::= { ieee8021FrerSequenceRecoveryEntry 10 }

ieee8021FrerSequenceRecoveryResetMSec OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies the timeout period in
        milliseconds for the RECOVERY_TIMEOUT event."
    REFERENCE    "10.4.1.7"
    ::= { ieee8021FrerSequenceRecoveryEntry 11 }

ieee8021FrerSequenceRecoveryInvalidSequenceValue OBJECT-TYPE

```

```

1  SYNTAX      Unsigned32
   MAX-ACCESS  read-only
   STATUS      current
   DESCRIPTION
       "This managed object specifies a value that cannot be encoded in
       a packet as a value for the sequence number subparameter,
       i.e., ieee8021FrerSequenceRecoveryInvalidSequenceValue is
       larger than or equal to the value specified in RecovSeqSpace."
   REFERENCE   "10.4.1.8 and 7.4.3.2.1"
   ::= { ieee8021FrerSequenceRecoveryEntry 12 }

ieee8021FrerSequenceRecoveryTakeNoSequence OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies whether packets with no
        sequence number subparameter are to be accepted (True) or
        not (False)."
    REFERENCE   "10.4.1.9"
    DEFVAL { false }
    ::= { ieee8021FrerSequenceRecoveryEntry 13 }

ieee8021FrerSequenceRecoveryIndividualRecovery OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies whether this entry describes a
        Sequence recovery function (False) or Individual recovery
        function (True)."
    REFERENCE   "10.4.1.10"
    ::= { ieee8021FrerSequenceRecoveryEntry 14 }

ieee8021FrerSequenceRecoveryLatentErrorDetection OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object indicates whether an instance of the
        Latent error detection function is to be instantiated along
        with the Base recovery function in this recovery function.
        It cannot be set True if
        ieee8021FrerSequenceRecoveryIndividualRecovery is also True;
        an Individual recovery function does not include a Latent
        error detection function."
    REFERENCE   "10.4.1.11"
    ::= { ieee8021FrerSequenceRecoveryEntry 15 }

ieee8021FrerSequenceRecoveryLatentErrorDifference OBJECT-TYPE
    SYNTAX      Integer32
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies the maximum difference between
        ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets, and the
        product of ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets and
        (ieee8021FrerSequenceRecoveryLatentErrorPaths - 1)
        that is allowed. Any larger difference will trigger the
        detection of a latent error by the LatentErrorTest function.
        It is used if and only if
        ieee8021FrerSequenceRecoveryIndividualRecovery is False."
    REFERENCE   "10.4.1.12.1"
    ::= { ieee8021FrerSequenceRecoveryEntry 16 }

ieee8021FrerSequenceRecoveryLatentErrorPeriod OBJECT-TYPE
    SYNTAX      Integer32
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies the number of milliseconds that
        are to elapse between instances of running the LatentErrorTest

```

```

1      function. An implementation can have a minimum value for
      this managed object, below which it cannot be set, but this
      minimum shall be no larger than 1000 ms (1 s).
      Default value 2000 (2 s).
      It is used if and only if
      ieee8021FrerSequenceRecoveryIndividualRecovery is False."
REFERENCE   "10.4.1.12.2"
DEFVAL     { 2000 }
::= { ieee8021FrerSequenceRecoveryEntry 17 }

ieee8021FrerSequenceRecoveryLatentErrorPaths OBJECT-TYPE
SYNTAX      Integer32
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies the number of paths over which
    FRER is operating for this instance of the Base recovery
    function and Latent error detection function.
    It is used if and only if
    ieee8021FrerSequenceRecoveryIndividualRecovery is False."
REFERENCE   "10.4.1.12.3"
::= { ieee8021FrerSequenceRecoveryEntry 18 }

ieee8021FrerSequenceRecoveryLatentResetPeriod OBJECT-TYPE
SYNTAX      Integer32
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies the number of milliseconds that
    are to elapse between instances of running the LatentErrorReset
    function. An implementation can have a minimum value for
    this managed object, below which it cannot be set, but this
    minimum shall be no larger than 1000 ms (1 s).
    It is used if and only if
    ieee8021FrerSequenceRecoveryIndividualRecovery is False."
REFERENCE   "10.4.1.12.4"
DEFVAL     { 30000 }
::= { ieee8021FrerSequenceRecoveryEntry 19 }

ieee8021FrerSequenceRecoveryStatus OBJECT-TYPE
SYNTAX      RowStatus
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "Row-Status for the Sequence Recovery table."
::= { ieee8021FrerSequenceRecoveryEntry 20 }

-- =====
-- the ieee8021FrerSequenceRecoveryHandleList table
-- =====

ieee8021FrerSequenceRecoveryHandleListTable OBJECT-TYPE
SYNTAX      SEQUENCE OF Ieee8021FrerSequenceRecoveryHandleListEntry
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "A table containing a list of stream_handle values,
    corresponding to the values of the tsNStreamIdHandle objects in
    the Stream identity table, to which the system is to apply the
    instance of the Sequence recovery function or Individual
    recovery function referenced in
    ieee8021FrerSequenceRecoveryIndex."
REFERENCE   "10.4.1.1"
::= { ieee8021FrerSequenceRecoveryHandleList 4 }

ieee8021FrerSequenceRecoveryHandleListEntry OBJECT-TYPE
SYNTAX      Ieee8021FrerSequenceRecoveryHandleListEntry
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "A set of managed objects providing the stream_handle values

```

```

1      corresponding to the values of the tsnStreamIdHandle objects in
      the Stream identity table, to which the system is to apply the
      instance of the Sequence recovery function or Individual
      recovery function referenced in
      ieee8021FrerSequenceRecoveryIndex."
REFERENCE      "10.4.1.1"
INDEX      { ieee8021FrerSequenceRecoveryIndex,
              ieee8021FrerSequenceRecoveryHandleListIndex
            }
      ::= { ieee8021FrerSequenceRecoveryHandleListTable 1 }

Ieee8021FrerSequenceRecoveryHandleListEntry ::=
SEQUENCE {
    ieee8021FrerSequenceRecoveryHandleListIndex
        Unsigned32,
    ieee8021FrerSequenceRecoveryStreamHandle
        VariablePointer,
    ieee8021FrerSequenceRecoveryHandleListStatus
        RowStatus
}

ieee8021FrerSequenceRecoveryHandleListIndex OBJECT-TYPE
SYNTAX      Unsigned32
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "Index for the Sequence Recovery handle list table."
      ::= { ieee8021FrerSequenceRecoveryHandleListEntry 1 }

ieee8021FrerSequenceRecoveryStreamHandle OBJECT-TYPE
SYNTAX      VariablePointer
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "A pointer to the ieee8021StreamIdStreamIdHandle that is an
     element of the ieee8021FrerSequenceRecoveryStreamList instance
     in the ieee8021FrerSequenceRecoveryTable."
REFERENCE      "10.4.1.1"
      ::= { ieee8021FrerSequenceRecoveryHandleListEntry 2 }

ieee8021FrerSequenceRecoveryHandleListStatus OBJECT-TYPE
SYNTAX      RowStatus
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "Row-Status for the Sequence Recovery handle list table."
      ::= { ieee8021FrerSequenceRecoveryHandleListEntry 3 }

-- =====
-- the ieee8021FrerSequenceRecoveryPortHandleList table
-- =====
ieee8021FrerSequenceRecoveryPortHandleListTable OBJECT-TYPE
SYNTAX      SEQUENCE OF Ieee8021FrerSequenceRecoveryPortHandleListEntry
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "A table containing a list of ports on each of which the system
     is to instantiate the Sequence recovery function, or from which
     received packets are to be fed to a single instance of the
     Individual recovery function, referenced in
     ieee8021FrerSequenceRecoveryIndex."
REFERENCE      "10.4.1.2"
      ::= { ieee8021FrerSequenceRecoveryPortHandleList 5 }

ieee8021FrerSequenceRecoveryPortHandleListEntry OBJECT-TYPE
SYNTAX      Ieee8021FrerSequenceRecoveryPortHandleListEntry
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "A set of managed objects providing the ports on each of which the system
     is to instantiate the Sequence recovery function, or from which
     received packets are to be fed to a single instance of the

```



```

1      Individual recovery function, referenced in
      ieee8021FrerSequenceRecoveryIndex."
REFERENCE    "10.4.1.2"
INDEX      { ieee8021FrerSequenceRecoveryIndex,
              ieee8021FrerSequenceRecoveryPortHandleListIndex
            }
      ::= { ieee8021FrerSequenceRecoveryPortHandleListTable 1 }

Ieee8021FrerSequenceRecoveryPortHandleListEntry ::=
SEQUENCE {
    ieee8021FrerSequenceRecoveryPortHandleListIndex
        Unsigned32,
    ieee8021FrerSequenceRecoveryPortHandle
        VariablePointer,
    ieee8021FrerSequenceRecoveryPortHandleListStatus
        RowStatus
}

ieee8021FrerSequenceRecoveryPortHandleListIndex OBJECT-TYPE
SYNTAX      Unsigned32
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "Index for the Sequence Recovery Port handle list table."
    ::= { ieee8021FrerSequenceRecoveryPortHandleListEntry 1 }

ieee8021FrerSequenceRecoveryPortHandle OBJECT-TYPE
SYNTAX      VariablePointer
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "A pointer to an interface that is an element of the
     ieee8021FrerSequenceRecoveryPortList instance in the
     ieee8021FrerSequenceRecoveryTable."
REFERENCE    "10.4.1.2"
    ::= { ieee8021FrerSequenceRecoveryPortHandleListEntry 2 }

ieee8021FrerSequenceRecoveryPortHandleListStatus OBJECT-TYPE
SYNTAX      RowStatus
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "Row-Status for the Sequence Recovery Port handle list table."
    ::= { ieee8021FrerSequenceRecoveryPortHandleListEntry 3 }

-- =====
-- the ieee8021FrerSequenceIdentification table
-- =====

ieee8021FrerSequenceIdentificationTable OBJECT-TYPE
SYNTAX      SEQUENCE OF Ieee8021FrerSequenceIdentificationEntry
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "A table containing a set of sequence identification entry
     objects. There is one entry for each port and direction for
     which an instance of the Sequence encode/decode function is to
     be instantiated for a list of Streams."
REFERENCE    "10.5"
    ::= { ieee8021FrerSequenceIdentification 6 }

ieee8021FrerSequenceIdentificationEntry OBJECT-TYPE
SYNTAX      Ieee8021FrerSequenceIdentificationEntry
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "A set of managed objects that specify a port and direction on
     which an instance of the Sequence encode/decode function is to
     be instantiated for a list of Streams."
REFERENCE    "10.5.1"
INDEX      { ieee8021FrerSequenceIdentificationEncodePort,
              ieee8021FrerSequenceIdentificationEncodeDirection
            }

```

```

1      }
      ::= { ieee8021FrerSequenceIdentificationTable 1 }

Ieee8021FrerSequenceIdentificationEntry ::=
SEQUENCE {
    ieee8021FrerSequenceIdentificationEncodePort
        InterfaceIndex,
    ieee8021FrerSequenceIdentificationEncodeDirection
        TruthValue,
    ieee8021FrerSequenceIdentificationStreamList
        AutonomousType,
    ieee8021FrerSequenceIdentificationEncodeActive
        TruthValue,
    ieee8021FrerSequenceIdentificationEncodeEncapsulationType
        Ieee8021CBSquenceEncodeDecodeType,
    ieee8021FrerSequenceIdentificationEncodeEncapsulationOUI
        OCTET STRING,
    ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType
        Integer32,
    ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI
        OCTET STRING,
    ieee8021FrerSequenceIdentificationEncodePathIdLanId
        Integer32,
    ieee8021FrerSequenceIdentificationStatus
        RowStatus
}

ieee8021FrerSequenceIdentificationEncodePort OBJECT-TYPE
SYNTAX      InterfaceIndex
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "The port on which the system is to place an instance of the
    Sequence encode/decode function."
REFERENCE   "10.5.1.2"
::= { ieee8021FrerSequenceIdentificationEntry 1 }

ieee8021FrerSequenceIdentificationEncodeDirection OBJECT-TYPE
SYNTAX      TruthValue
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "This managed object indicates whether the Sequence
    encode/decode function is to be placed on the out-facing (True)
    or in-facing (False) side of the port."
REFERENCE   "10.5.1.3"
::= { ieee8021FrerSequenceIdentificationEntry 2 }

ieee8021FrerSequenceIdentificationStreamList OBJECT-TYPE
SYNTAX      AutonomousType
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "A list of stream_handles, corresponding to the values of the
    tsNStreamIdHandle objects in the Stream identity table, for
    which the system is to use the same encapsulation for the
    Sequence encode/decode function.
    For each entry in the ieee8021FrerSequenceIdentificationTable
    there exists a list of stream_handle values in the
    ieee8021FrerSequenceIdentificationHandleListTable."
REFERENCE   "10.5.1.1"
::= { ieee8021FrerSequenceIdentificationEntry 3 }

ieee8021FrerSequenceIdentificationEncodeActive OBJECT-TYPE
SYNTAX      TruthValue
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies whether this
    ieee8021FrerSequenceIdentificationEntry is passive (False),
    and therefore is used only to decode (extract information from)
    input packets passing up the protocol stack, or active (True),

```

```

1         and therefore is used both for recognizing input packets and
           for encoding output packets being passed down the protocol
           stack."
REFERENCE   "10.5.1.4"
::= { ieee8021FrerSequenceIdentificationEntry 4 }

ieee8021FrerSequenceIdentificationEncodeEncapsulationType OBJECT-TYPE
SYNTAX      Ieee8021CBSequenceEncodeDecodeType
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "An enumerated value specifying the type of encapsulation used
     for this instance of the Sequence encode/decode function. If
     the value used is nonIEEESpecified(256), the managed objects
     ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI
     and
     ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType
     are used to specify OUI/CID and the Sequence encode/decode
     method defined by the entity owning the OUI/CID."
REFERENCE   "10.5.1.5"
::= { ieee8021FrerSequenceIdentificationEntry 5 }

ieee8021FrerSequenceIdentificationEncodeEncapsulationOUI OBJECT-TYPE
SYNTAX      OCTET STRING (SIZE (3))
MAX-ACCESS  read-only
STATUS      current
DESCRIPTION
    "This managed object specifies the OUI/CID used together with
     the encapsulation type specified in
     ieee8021FrerSequenceIdentificationEncodeEncapsulationType.
     It is used if and only if the value for
     ieee8021FrerSequenceIdentificationEncodeEncapsulationType is
     in the range of 0-255. In this case it always takes the value
     00-80-C2."
REFERENCE   "10.5.1.5"
::= { ieee8021FrerSequenceIdentificationEntry 6 }

ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType OBJECT-TYPE
SYNTAX      Integer32 (256..2147483647)
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "An enumerated value specifying the type of encapsulation used
     for this instance of the Sequence encode/decode function. This
     managed object is used if and only if the value for
     ieee8021FrerSequenceIdentificationEncodeEncapsulationType is
     256."
REFERENCE   "10.5.1.5"
::= { ieee8021FrerSequenceIdentificationEntry 7 }

ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI OBJECT-TYPE
SYNTAX      OCTET STRING (SIZE (3))
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies the OUI/CID used together with
     the encapsulation type specified in
     ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType.
     It is used if and only if the value for
     ieee8021FrerSequenceIdentificationEncodeEncapsulationType is
     256."
REFERENCE   "10.5.1.5"
::= { ieee8021FrerSequenceIdentificationEntry 8 }

ieee8021FrerSequenceIdentificationEncodePathIdLanId OBJECT-TYPE
SYNTAX      Integer32 (0..15)
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "A 4-bit integer value to be placed in the PathId field of an
     HSR sequence tag or the LanId field of a PRP sequence trailer
     added to an output packet."

```

```

1    REFERENCE    "10.5.1.6"
    ::= { ieee8021FrerSequenceIdentificationEntry 9 }

ieee8021FrerSequenceIdentificationStatus OBJECT-TYPE
    SYNTAX        RowStatus
    MAX-ACCESS    read-create
    STATUS        current
    DESCRIPTION
        "Row-Status for the Sequence Identification table."
    ::= { ieee8021FrerSequenceIdentificationEntry 10 }

-- =====
-- the ieee8021FrerSequenceIdentificationHandleList table
-- =====

ieee8021FrerSequenceIdentificationHandleListTable OBJECT-TYPE
    SYNTAX        SEQUENCE OF Ieee8021FrerSequenceIdentificationHandleListEntry
    MAX-ACCESS    not-accessible
    STATUS        current
    DESCRIPTION
        "A table containing a list of stream handle values,
        corresponding to the values of the TsnStreamIdHandle objects in
        the Stream identity table, for which the system is to use the
        same encapsulation for the Sequence encode/decode function
        referenced in ieee8021FrerSequenceIdentificationEncodePort
        and ieee8021FrerSequenceIdentificationEncodeDirection."
    REFERENCE    "10.5.1.1"
    ::= { ieee8021FrerSequenceIdentificationHandleList 7 }

ieee8021FrerSequenceIdentificationHandleListEntry OBJECT-TYPE
    SYNTAX        Ieee8021FrerSequenceIdentificationHandleListEntry
    MAX-ACCESS    not-accessible
    STATUS        current
    DESCRIPTION
        "A set of managed objects providing the stream handle values
        corresponding to the values of the tsnStreamIdHandle objects in
        the Stream identity table, for which the system is to use the
        same encapsulation for the Sequence encode/decode function
        referenced in ieee8021FrerSequenceIdentificationEncodePort
        and ieee8021FrerSequenceIdentificationEncodeDirection."
    REFERENCE    "10.5.1.1"
    INDEX        { ieee8021FrerSequenceIdentificationEncodePort,
                    ieee8021FrerSequenceIdentificationEncodeDirection,
                    ieee8021FrerSequenceIdentificationHandleListIndex
                  }
    ::= { ieee8021FrerSequenceIdentificationHandleListTable 1 }

Ieee8021FrerSequenceIdentificationHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceIdentificationHandleListIndex
            Unsigned32,
        ieee8021FrerSequenceIdentificationStreamHandle
            VariablePointer,
        ieee8021FrerSequenceIdentificationHandleListStatus
            RowStatus
    }

ieee8021FrerSequenceIdentificationHandleListIndex OBJECT-TYPE
    SYNTAX        Unsigned32
    MAX-ACCESS    not-accessible
    STATUS        current
    DESCRIPTION
        "Index for the Sequence Identification handle list table."
    ::= { ieee8021FrerSequenceIdentificationHandleListEntry 1 }

ieee8021FrerSequenceIdentificationStreamHandle OBJECT-TYPE
    SYNTAX        VariablePointer
    MAX-ACCESS    read-create
    STATUS        current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
        element of the ieee8021FrerSequenceIdentificationStreamList

```

```

1      instance in the ieee8021FrerSequenceIdentificationTable."
REFERENCE    "10.5.1.1"
::= { ieee8021FrerSequenceIdentificationHandleListEntry 2 }

ieee8021FrerSequenceIdentificationHandleListStatus OBJECT-TYPE
SYNTAX      RowStatus
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "Row-Status for the Sequence Identification handle list table."
::= { ieee8021FrerSequenceIdentificationHandleListEntry 3 }

-- =====
-- the ieee8021FrerStreamSplit table
-- =====

ieee8021FrerStreamSplitTable OBJECT-TYPE
SYNTAX      SEQUENCE OF Ieee8021FrerStreamSplitEntry
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "A table containing a set of FRER split entry objects. There is
     one entry per Stream splitting function per set of
     stream_handle values."
REFERENCE    "10.6"
::= { ieee8021FrerStreamSplit 8 }

ieee8021FrerStreamSplitEntry OBJECT-TYPE
SYNTAX      Ieee8021FrerStreamSplitEntry
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "A set of managed objects that specify a port and direction
     on which an instance of the Stream splitting function is to be
     instantiated, and the list of stream_handles specifying its
     operation."
REFERENCE    "10.6.1"
INDEX       { ieee8021FrerStreamSplitIndex }
::= { ieee8021FrerStreamSplitTable 1 }

Ieee8021FrerStreamSplitEntry ::=
SEQUENCE {
    ieee8021FrerStreamSplitIndex
        Unsigned32,
    ieee8021FrerStreamSplitPort
        InterfaceIndex,
    ieee8021FrerStreamSplitDirection
        TruthValue,
    ieee8021FrerStreamSplitInputIdList
        AutonomousType,
    ieee8021FrerStreamSplitOutputIdList
        AutonomousType
}

ieee8021FrerStreamSplitIndex OBJECT-TYPE
SYNTAX      Unsigned32
MAX-ACCESS  not-accessible
STATUS      current
DESCRIPTION
    "Index for the Stream Split table."
::= { ieee8021FrerStreamSplitEntry 1 }

ieee8021FrerStreamSplitPort OBJECT-TYPE
SYNTAX      InterfaceIndex
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "The port on which the system is to place an instance of the
     Stream splitting function performing the stream handle
     translations specified by ieee8021FrerStreamSplitInputIdList
     and ieee8021FrerStreamSplitOutputIdList is to be placed."
REFERENCE    "10.6.1.1"

```

```

1 ::= { ieee8021FrerStreamSplitEntry 2 }

ieee8021FrerStreamSplitDirection OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS   read-create
    STATUS       current
    DESCRIPTION
        "This managed object indicates whether the instance of the
        Stream splitting function performing the stream handle
        translations specified by ieee8021FrerStreamSplitInputIdList
        and ieee8021FrerStreamSplitOutputIdList is to be placed on the
        out-facing (True) or in-facing (False) side of the port."
    REFERENCE    "10.6.1.1"
    ::= { ieee8021FrerStreamSplitEntry 3 }

ieee8021FrerStreamSplitInputIdList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION
        "A list of stream handles that are to be split.
        For each entry in the ieee8021FrerStreamSplitTable
        there exists a list of stream handle values in the
        ieee8021FrerStreamSplitInputHandleListTable."
    REFERENCE    "10.6.1.3"
    ::= { ieee8021FrerStreamSplitEntry 4 }

ieee8021FrerStreamSplitOutputIdList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS   read-write
    STATUS       current
    DESCRIPTION
        "A list of stream handles into which the input packet is to be
        split, one copy per item in the
        ieee8021FrerStreamSplitOutputIdList.
        For each entry in the ieee8021FrerStreamSplitTable
        there exists a list of stream handle values in the
        ieee8021FrerStreamSplitOutputHandleListTable."
    REFERENCE    "10.6.1.4"
    ::= { ieee8021FrerStreamSplitEntry 5 }

-- =====
-- the ieee8021FrerStreamSplitInputHandleList table
-- =====
ieee8021FrerStreamSplitInputHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerStreamSplitInputHandleListEntry
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION
        "A table containing a list of stream handle values that are to
        be split, referenced in ieee8021FrerStreamSplitIndex."
    REFERENCE    "10.6.1.3"
    ::= { ieee8021FrerStreamSplitInputHandleList 9 }

ieee8021FrerStreamSplitInputHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerStreamSplitInputHandleListEntry
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION
        "A set of managed objects providing the stream handle values
        corresponding to the values of the tsNStreamIdHandle objects in
        the Stream identity table, that are to be split, referenced in
        ieee8021FrerStreamSplitIndex."
    REFERENCE    "10.6.1.3"
    INDEX        { ieee8021FrerStreamSplitIndex,
                    ieee8021FrerStreamSplitInputHandleListIndex
                  }
    ::= { ieee8021FrerStreamSplitInputHandleListTable 1 }

Ieee8021FrerStreamSplitInputHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerStreamSplitInputHandleListIndex
    }

```

```

1      Unsigned32,
      ieee8021FrerStreamSplitInputIdHandle
      VariablePointer,
      ieee8021FrerStreamSplitInputHandleListStatus
      RowStatus
    }

ieee8021FrerStreamSplitInputHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Stream Split Input handle list table."
    ::= { ieee8021FrerStreamSplitInputHandleListEntry 1 }

ieee8021FrerStreamSplitInputIdHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
        element of the ieee8021FrerStreamSplitInputIdList instance in
        the ieee8021FrerStreamSplitTable."
    REFERENCE   "10.6.1.3"
    ::= { ieee8021FrerStreamSplitInputHandleListEntry 2 }

ieee8021FrerStreamSplitInputHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Stream Split Input handle list table."
    ::= { ieee8021FrerStreamSplitInputHandleListEntry 3 }

-- =====
-- the ieee8021FrerStreamSplitOutputHandleList table
-- =====
ieee8021FrerStreamSplitOutputHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerStreamSplitOutputHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of stream handles values into which
        the input packet is to be split, one copy per item in this
        table, referenced in ieee8021FrerStreamSplitIndex."
    REFERENCE   "10.6.1.4"
    ::= { ieee8021FrerStreamSplitOutputHandleList 10 }

ieee8021FrerStreamSplitOutputHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerStreamSplitOutputHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the stream handle values
        corresponding to the values of the tsNStreamIdHandle objects in
        the Stream identity table, into which the input packet is to be
        split, one copy per item in this table, referenced in
        ieee8021FrerStreamSplitIndex."
    REFERENCE   "10.6.1.4"
    INDEX       { ieee8021FrerStreamSplitIndex,
                  ieee8021FrerStreamSplitOutputHandleListIndex
                }
    ::= { ieee8021FrerStreamSplitOutputHandleListTable 1 }

Ieee8021FrerStreamSplitOutputHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerStreamSplitOutputHandleListIndex
            Unsigned32,
        ieee8021FrerStreamSplitOutputIdHandle
            VariablePointer,
        ieee8021FrerStreamSplitOutputHandleListStatus
            RowStatus
    }

```

```

1      }

ieee8021FrerStreamSplitOutputHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Stream Split Output handle list table."
    ::= { ieee8021FrerStreamSplitOutputHandleListEntry 1 }

ieee8021FrerStreamSplitOutputIdHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
         element of the ieee8021FrerStreamSplitOutputIdList instance in
         the ieee8021FrerStreamSplitTable."
    REFERENCE   "10.6.1.4"
    ::= { ieee8021FrerStreamSplitOutputHandleListEntry 2 }

ieee8021FrerStreamSplitOutputHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Stream Split Output handle list table."
    ::= { ieee8021FrerStreamSplitOutputHandleListEntry 3 }

-- =====
-- the ieee8021FrerSequenceAutoconfig table
-- =====

ieee8021FrerSequenceAutoconfigTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceAutoconfigEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of managed objects that specify how
         entries are automatically created (and destroyed) in the
         Stream identity table, the Sequence recovery table, and the
         Sequence identification table."
    REFERENCE   "10.7.1"
    ::= { ieee8021FrerSequenceAutoconfig 11 }

ieee8021FrerSequenceAutoconfigEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceAutoconfigEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects relating to a single class of Streams,
         and specifying how entries are created (and destroyed) in the
         Stream identity table, the Sequence recovery table, and the
         Sequence identification table."
    REFERENCE   "10.7.1.1"
    INDEX       { ieee8021FrerSequenceAutoconfigIndex }
    ::= { ieee8021FrerSequenceAutoconfigTable 1 }

Ieee8021FrerSequenceAutoconfigEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceAutoconfigIndex
            Unsigned32,
        ieee8021FrerSequenceAutoconfigSequenceEncapsulation
            Ieee8021CBSequenceEncodeDecodeType,
        ieee8021FrerSequenceAutoconfigSequenceEncapsulationOUI
            OCTET STRING,
        ieee8021FrerSequenceAutoconfigCustomSequenceEncapsulation
            Integer32,
        ieee8021FrerSequenceAutoconfigCustomSequenceEncOUI
            OCTET STRING,
        ieee8021FrerSequenceAutoconfigReceivePortList
            AutonomousType,

```



```

1      ieee8021FrerSequenceAutoconfigTagged
        Ieee8021CBTaggedType,
      ieee8021FrerSequenceAutoconfigVlanList
        AutonomousType,
      ieee8021FrerSequenceAutoconfigRecoveryPortList
        AutonomousType,
      ieee8021FrerSequenceAutoconfigDestructMSec
        Integer32,
      ieee8021FrerSequenceAutoconfigResetMSec
        Unsigned32,
      ieee8021FrerSequenceAutoconfigAlgorithm
        Ieee8021CBSequenceRecoveryAlgorithm,
      ieee8021FrerSequenceAutoconfigAlgorithmOUI
        OCTET STRING,
      ieee8021FrerSequenceAutoconfigCustomAlgorithm
        Integer32,
      ieee8021FrerSequenceAutoconfigCustomAlgorithmOUI
        OCTET STRING,
      ieee8021FrerSequenceAutoconfigHistoryLength
        Integer32,
      ieee8021FrerSequenceAutoconfigCreateIndividual
        TruthValue,
      ieee8021FrerSequenceAutoconfigCreateRecovery
        TruthValue,
      ieee8021FrerSequenceAutoconfigLatentErrorDetection
        TruthValue,
      ieee8021FrerSequenceAutoconfigLatentErrorDifference
        Integer32,
      ieee8021FrerSequenceAutoconfigLatentErrorPeriod
        Integer32,
      ieee8021FrerSequenceAutoconfigLatentErrorResetPeriod
        Integer32,
      ieee8021FrerSequenceAutoconfigStatus
        RowStatus
    }

ieee8021FrerSequenceAutoconfigIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Autoconfiguration table."
    ::= { ieee8021FrerSequenceAutoconfigEntry 1 }

ieee8021FrerSequenceAutoconfigSequenceEncapsulation OBJECT-TYPE
    SYNTAX      Ieee8021CBSequenceEncodeDecodeType
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "This managed object specifies which Sequence encode/decode
        function, and therefore, which type sequence_number encoding,
        is to be recognized for the purposes of AutoConfiguration."
    REFERENCE   "10.7.1.1.1"
    ::= { ieee8021FrerSequenceAutoconfigEntry 2 }

ieee8021FrerSequenceAutoconfigSequenceEncapsulationOUI OBJECT-TYPE
    SYNTAX      OCTET STRING (SIZE (3))
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "This managed object specifies which Sequence encode/decode
        function, and therefore, which type sequence_number encoding,
        is to be recognized for the purposes of AutoConfiguration.
        It is used if and only if the value for
        ieee8021FrerSequenceAutoconfigSequenceEncapsulation is
        in the range of 0-255. In this case it always takes the value
        00-80-C2."
    REFERENCE   "10.7.1.1.1"
    ::= { ieee8021FrerSequenceAutoconfigEntry 3 }

ieee8021FrerSequenceAutoconfigCustomSequenceEncapsulation OBJECT-TYPE
    SYNTAX      Integer32 (256..2147483647)

```

```

1  MAX-ACCESS  read-create
   STATUS      current
   DESCRIPTION
       "This managed object specifies which Sequence encode/decode
       function, and therefore, which type sequence_number encoding,
       is to be recognized for the purposes of AutoConfiguration.
       This managed object is used if and only if the value for
       ieee8021FrerSequenceAutoconfigSequenceEncapsulation is
       256."
   REFERENCE   "10.7.1.1.1"
   ::= { ieee8021FrerSequenceAutoconfigEntry 4 }

ieee8021FrerSequenceAutoconfigCustomSequenceEncOUI OBJECT-TYPE
SYNTAX      OCTET STRING (SIZE (3))
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
       "This managed object specifies which Sequence encode/decode
       function, and therefore, which type sequence_number encoding,
       is to be recognized for the purposes of AutoConfiguration.
       This managed object is used if and only if the value for
       ieee8021FrerSequenceAutoconfigSequenceEncapsulation is
       256."
   REFERENCE   "10.7.1.1.1"
   ::= { ieee8021FrerSequenceAutoconfigEntry 5 }

ieee8021FrerSequenceAutoconfigReceivePortList OBJECT-TYPE
SYNTAX      AutonomousType
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
       "The list of ports to which this Sequence Autoconfiguration
       entry applies, and on which Stream identification functions,
       Sequence encode/decode functions, and Individual recovery
       functions are to be autocreated.
       For each entry in the ieee8021FrerSequenceAutoconfigTable
       there exists a list of ports in the
       ieee8021FrerSequenceAutoconfigReceivePortHandleListTable."
   REFERENCE   "10.7.1.1.2"
   ::= { ieee8021FrerSequenceAutoconfigEntry 6 }

ieee8021FrerSequenceAutoconfigTagged OBJECT-TYPE
SYNTAX      Ieee8021CBTaggedType
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
       "An enumerated value indicating whether packets to be matched by
       this Autorecovery entry are permitted to have a VLAN tag."
   REFERENCE   "10.7.1.1.3"
   ::= { ieee8021FrerSequenceAutoconfigEntry 7 }

ieee8021FrerSequenceAutoconfigVlanList OBJECT-TYPE
SYNTAX      AutonomousType
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
       "A list of vlan_identifiers for the packet to match. A null list
       matches all vlan_identifiers.
       For each entry in the ieee8021FrerSequenceAutoconfigTable
       there exists a list of vlan_identifiers in the
       ieee8021FrerSequenceAutoconfigVlanListTable."
   REFERENCE   "10.7.1.1.4"
   ::= { ieee8021FrerSequenceAutoconfigEntry 8 }

ieee8021FrerSequenceAutoconfigRecoveryPortList OBJECT-TYPE
SYNTAX      AutonomousType
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
       "The list of ports on which Sequence recovery functions are to
       be autocreated by this Autorecovery entry.
       For each entry in the ieee8021FrerSequenceAutoconfigTable

```

```

1         there exists a list of ports in the
         ieee8021FrerSequenceAutoconfigRecoveryPortHandleListTable."
REFERENCE   "10.7.1.1.5"
::= { ieee8021FrerSequenceAutoconfigEntry 9 }

ieee8021FrerSequenceAutoconfigDestructMSec OBJECT-TYPE
SYNTAX      Integer32
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies the number of milliseconds after
    which an idle set of functions created by this Autorecovery
    entry can be destroyed. A value of 0 indicates that idle
    autoconfigured functions are not to be destroyed."
REFERENCE   "10.7.1.1.6"
DEFVAL     { 86400000 }
::= { ieee8021FrerSequenceAutoconfigEntry 10 }

ieee8021FrerSequenceAutoconfigResetMSec OBJECT-TYPE
SYNTAX      Unsigned32
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies the value used to fill
    ieee8021FrerSequenceRecoveryResetMSec when autoconfiguring
    entries in the Sequence recovery table."
REFERENCE   "10.7.1.1.7"
::= { ieee8021FrerSequenceAutoconfigEntry 11 }

ieee8021FrerSequenceAutoconfigAlgorithm OBJECT-TYPE
SYNTAX      Ieee8021CBSequenceRecoveryAlgorithm
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies the value used to fill
    ieee8021FrerSequenceRecoveryAlgorithm when autoconfiguring
    entries in the Sequence recovery table."
REFERENCE   "10.7.1.1.8"
::= { ieee8021FrerSequenceAutoconfigEntry 12 }

ieee8021FrerSequenceAutoconfigAlgorithmOUI OBJECT-TYPE
SYNTAX      OCTET STRING (SIZE (3))
MAX-ACCESS  read-only
STATUS      current
DESCRIPTION
    "This managed object specifies the value used to fill
    ieee8021FrerSequenceRecoveryAlgorithmOUI when autoconfiguring
    entries in the Sequence recovery table."
REFERENCE   "10.7.1.1.8"
::= { ieee8021FrerSequenceAutoconfigEntry 13 }

ieee8021FrerSequenceAutoconfigCustomAlgorithm OBJECT-TYPE
SYNTAX      Integer32 (256..2147483647)
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies the value used to fill
    ieee8021FrerSequenceCustomRecoveryAlgorithm when
    autoconfiguring entries in the Sequence recovery table."
REFERENCE   "10.7.1.1.8"
::= { ieee8021FrerSequenceAutoconfigEntry 14 }

ieee8021FrerSequenceAutoconfigCustomAlgorithmOUI OBJECT-TYPE
SYNTAX      OCTET STRING (SIZE (3))
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies the value used to fill
    ieee8021FrerSequenceCustomRecoveryAlgorithmOUI
    when autoconfiguring entries in the Sequence recovery table."
REFERENCE   "10.7.1.1.8"
::= { ieee8021FrerSequenceAutoconfigEntry 15 }

```

```

1
ieee8021FrerSequenceAutoconfigHistoryLength OBJECT-TYPE
    SYNTAX      Integer32
    MAX-ACCESS   read-create
    STATUS       current
    DESCRIPTION
        "This managed object specifies the value used to fill
         ieee8021FrerSequenceRecoveryHistoryLength when autoconfiguring
         entries in the Sequence recovery table."
    REFERENCE    "10.7.1.1.9"
    ::= { ieee8021FrerSequenceAutoconfigEntry 16 }

ieee8021FrerSequenceAutoconfigCreateIndividual OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS   read-create
    STATUS       current
    DESCRIPTION
        "This managed object specifies if the receipt of a packet that
         triggers the autoconfiguration of a new entry in the Stream
         Identity table also triggers the instantiation of a new entry
         in the Sequence Generation table for an Individual recovery
         function (true) or not (false)."

```

```

1  SYNTAX      Integer32
   MAX-ACCESS  read-create
   STATUS      current
   DESCRIPTION
       "This managed object specifies the value used to fill
       ieee8021FrerSequenceRecoveryLatentResetPeriod when
       autoconfiguring entries in the Sequence recovery table."
   REFERENCE   "10.7.1.1.15"
   ::= { ieee8021FrerSequenceAutoconfigEntry 22 }

ieee8021FrerSequenceAutoconfigStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Sequence Autoconfiguration table."
    ::= { ieee8021FrerSequenceAutoconfigEntry 23 }

-- =====
-- the ieee8021FrerSequenceAutoconfigReceivePortHandleList
-- table
-- =====
ieee8021FrerSequenceAutoconfigReceivePortHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF
Ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of ports to which this
        frerAutSeqEntry applies, and on which Stream identification
        functions, Sequence encode/decode functions, and Individual
        recovery functions are to be autocreated, referenced in
        ieee8021FrerSequenceAutoconfigIndex."
    REFERENCE   "10.7.1.1.2"
    ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleList 12 }

ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the ports to which this
        frerAutSeqEntry applies, and on which Stream identification
        functions, Sequence encode/decode functions, and Individual
        recovery functions are to be autocreated, referenced in
        ieee8021FrerSequenceAutoconfigIndex."
    REFERENCE   "10.7.1.1.2"
    INDEX       { ieee8021FrerSequenceAutoconfigIndex,
                  ieee8021FrerSequenceAutoconfigReceivePortHandleListIndex
                }
    ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleListTable 1 }

Ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceAutoconfigReceivePortHandleListIndex
            Unsigned32,
        ieee8021FrerSequenceAutoconfigReceivePortHandle
            VariablePointer,
        ieee8021FrerSequenceAutoconfigReceivePortHandleListStatus
            RowStatus
    }

ieee8021FrerSequenceAutoconfigReceivePortHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Autoconfiguration Receive Port handle
        list table."
    ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry 1 }

ieee8021FrerSequenceAutoconfigReceivePortHandle OBJECT-TYPE

```

```

1  SYNTAX      VariablePointer
   MAX-ACCESS  read-create
   STATUS      current
   DESCRIPTION
       "A pointer to the ieee8021StreamIdStreamIdHandle that is an
       element of the ieee8021FrerSequenceAutoconfigReceivePortList
       instance in the ieee8021FrerSequenceAutoconfigTable."
   REFERENCE   "10.7.1.1.2"
   ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry 2 }

ieee8021FrerSequenceAutoconfigReceivePortHandleListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "Row-Status for the Sequence Autoconfiguration Receive Port
        handle list table."
    ::= { ieee8021FrerSequenceAutoconfigReceivePortHandleListEntry 3 }

-- =====
-- the ieee8021FrerSequenceAutoconfigVlanHandleList table
-- =====
ieee8021FrerSequenceAutoconfigVlanListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerSequenceAutoconfigVlanListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a list of vlan_identifiers for the packet to
        match. A null list matches all vlan_identifiers."
    REFERENCE   "10.7.1.1.4"
    ::= { ieee8021FrerSequenceAutoconfigVlanHandleList 13}

ieee8021FrerSequenceAutoconfigVlanListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceAutoconfigVlanListEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects providing the list of vlan_identifiers
        for the packet to match. A null list matches all
        vlan_identifiers."
    REFERENCE   "10.7.1.1.4"
    INDEX       { ieee8021FrerSequenceAutoconfigIndex,
                  ieee8021FrerSequenceAutoconfigVlanListIndex
                }
    ::= { ieee8021FrerSequenceAutoconfigVlanListTable 1 }

Ieee8021FrerSequenceAutoconfigVlanListEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceAutoconfigVlanListIndex
            Unsigned32,
        ieee8021FrerSequenceAutoconfigVlan
            Ieee8021CBVlanIdentifier,
        ieee8021FrerSequenceAutoconfigVlanListStatus
            RowStatus
    }

ieee8021FrerSequenceAutoconfigVlanListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Sequence Autoconfiguration VLAN list table."
    ::= { ieee8021FrerSequenceAutoconfigVlanListEntry 1 }

ieee8021FrerSequenceAutoconfigVlan OBJECT-TYPE
    SYNTAX      Ieee8021CBVlanIdentifier
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "A vlan_identifier for the packet to match."
    REFERENCE   "10.7.1.1.4"
    ::= { ieee8021FrerSequenceAutoconfigVlanListEntry 2 }

```

```

1
ieee8021FrerSequenceAutoconfigVlanListStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS   read-create
    STATUS       current
    DESCRIPTION
        "Row-Status for the Sequence Autoconfiguration VLAN list
        table."
    ::= { ieee8021FrerSequenceAutoconfigVlanListEntry 3 }

-- =====
-- the ieee8021FrerSequenceAutoconfigRecoveryPortHandleList
-- table
-- =====
ieee8021FrerSequenceAutoconfigRecoveryPortHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF
Ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION
        "A table containing a list of ports on which Sequence recovery
        functions are to be autocreated by this frerAutSeqEntry,
        referenced in ieee8021FrerSequenceAutoconfigIndex."
    REFERENCE    "10.7.1.1.5"
    ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleList 14}

ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION
        "A set of managed objects providing the ports on which Sequence
        recovery functions are to be autocreated by this
        frerAutSeqEntry, referenced in
        ieee8021FrerSequenceAutoconfigIndex."
    REFERENCE    "10.7.1.1.5"
    INDEX        { ieee8021FrerSequenceAutoconfigIndex,
                    ieee8021FrerSequenceAutoconfigRecoveryPortHandleListIndex
                  }
    ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleListTable 1 }

Ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerSequenceAutoconfigRecoveryPortHandleListIndex
            Unsigned32,
        ieee8021FrerSequenceAutoconfigRecoveryPortHandle
            VariablePointer,
        ieee8021FrerSequenceAutoconfigRecoveryPortHandleListStatus
            RowStatus
    }

ieee8021FrerSequenceAutoconfigRecoveryPortHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION
        "Index for the Sequence Autoconfiguration Recovery Port handle
        list table."
    ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry 1 }

ieee8021FrerSequenceAutoconfigRecoveryPortHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS   read-create
    STATUS       current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
        element of the ieee8021FrerSequenceAutoconfigRecoveryPortList
        instance in the ieee8021FrerSequenceAutoconfigTable."
    REFERENCE    "10.7.1.1.5"
    ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry 2 }

ieee8021FrerSequenceAutoconfigRecoveryPortHandleListStatus OBJECT-TYPE

```

```

1  SYNTAX      RowStatus
   MAX-ACCESS  read-create
   STATUS      current
   DESCRIPTION
       "Row-Status for the Sequence Autoconfiguration Recovery Port
       handle list table."
   ::= { ieee8021FrerSequenceAutoconfigRecoveryPortHandleListEntry 3 }

-- =====
-- the ieee8021FrerOutputAutoconfig table
-- =====

ieee8021FrerOutputAutoconfigTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerOutputAutoconfigEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of managed objects that specify how
        active entries are created in the Sequence identification
        table."
    REFERENCE   "10.7.2"
    ::= { ieee8021FrerOutputAutoconfig 15 }

ieee8021FrerOutputAutoconfigEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerOutputAutoconfigEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A set of managed objects relating to a single class of Streams
        specifying how active entries are created in the Sequence
        identification table."
    REFERENCE   "10.7.2.1"
    INDEX { ieee8021FrerOutputAutoconfigIndex }
    ::= { ieee8021FrerOutputAutoconfigTable 1 }

Ieee8021FrerOutputAutoconfigEntry ::=
    SEQUENCE {
        ieee8021FrerOutputAutoconfigIndex
            Unsigned32,
        ieee8021FrerOutputAutoconfigPortList
            AutonomousType,
        ieee8021FrerOutputAutoconfigEncapsulation
            Ieee8021CBSequenceEncodeDecodeType,
        ieee8021FrerOutputAutoconfigEncapsulationOUI
            OCTET STRING,
        ieee8021FrerOutputAutoconfigCustomEncapsulation
            Integer32,
        ieee8021FrerOutputAutoconfigCustomEncapsulationOUI
            OCTET STRING,
        ieee8021FrerOutputAutoconfigLanPathId
            Ieee8021CBLanPathIdType,
        ieee8021FrerOutputAutoconfigStatus
            RowStatus
    }

ieee8021FrerOutputAutoconfigIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "Index for the Output Autoconfiguration table."
    ::= { ieee8021FrerOutputAutoconfigEntry 1 }

ieee8021FrerOutputAutoconfigPortList OBJECT-TYPE
    SYNTAX      AutonomousType
    MAX-ACCESS  read-create
    STATUS      current
    DESCRIPTION
        "The list of ports to which this Output Autoconfiguration entry
        applies, and on which active Sequence encode/decode functions
        are to be autocreated.
        For each entry in the ieee8021FrerOutputAutoconfigTable

```



```

1         there exists a list of ports in the
         ieee8021FrerOutputAutoconfigPortHandleListTable."
REFERENCE   "10.7.2.1.1"
 ::= { ieee8021FrerOutputAutoconfigEntry 2 }

ieee8021FrerOutputAutoconfigEncapsulation OBJECT-TYPE
SYNTAX      Ieee8021CBSequenceEncodeDecodeType
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies which Sequence encode/decode
    function, and therefore, which type sequence number encoding,
    is to be used for autoconfigured Streams on the ports in
    ieee8021FrerSequenceAutoconfigReceivePortList."
REFERENCE   "10.7.2.1.2"
 ::= { ieee8021FrerOutputAutoconfigEntry 3 }

ieee8021FrerOutputAutoconfigEncapsulationOUI OBJECT-TYPE
SYNTAX      OCTET STRING (SIZE (3))
MAX-ACCESS  read-only
STATUS      current
DESCRIPTION
    "This managed object specifies which Sequence encode/decode
    function, and therefore, which type sequence number encoding,
    is to be used for autoconfigured Streams on the ports in
    ieee8021FrerSequenceAutoconfigReceivePortList.
    It is used if and only if the value for
    ieee8021FrerOutputAutoconfigEncapsulation is in the
    range of 0-255. In this case it always takes the value
    00-80-C2."
REFERENCE   "10.7.2.1.2"
 ::= { ieee8021FrerOutputAutoconfigEntry 4 }

ieee8021FrerOutputAutoconfigCustomEncapsulation OBJECT-TYPE
SYNTAX      Integer32 (256..2147483647)
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies which Sequence encode/decode
    function, and therefore, which type sequence number encoding,
    is to be used for autoconfigured Streams on the ports in
    ieee8021FrerSequenceAutoconfigReceivePortList. This
    managed object is used if and only if the value for
    ieee8021FrerOutputAutoconfigEncapsulation is 256."
REFERENCE   "10.7.2.1.2"
 ::= { ieee8021FrerOutputAutoconfigEntry 5 }

ieee8021FrerOutputAutoconfigCustomEncapsulationOUI OBJECT-TYPE
SYNTAX      OCTET STRING (SIZE (3))
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "This managed object specifies which Sequence encode/decode
    function, and therefore, which type sequence number encoding,
    is to be used for autoconfigured Streams on the ports in
    ieee8021FrerSequenceAutoconfigReceivePortList. This
    managed object is used if and only if the value for
    ieee8021FrerOutputAutoconfigEncapsulation is 256."
REFERENCE   "10.7.2.1.2"
 ::= { ieee8021FrerOutputAutoconfigEntry 6 }

ieee8021FrerOutputAutoconfigLanPathId OBJECT-TYPE
SYNTAX      Ieee8021CBLanPathIdType
MAX-ACCESS  read-create
STATUS      current
DESCRIPTION
    "An integer specifying a path or LAN. If and only if
    ieee8021FrerOutputAutoconfigEncapsulation specifies HSR
    or PRP this managed object specifies the LanId or PathId value
    to be inserted into the HSR sequence tag or PRP sequence
    trailer of autoconfigured packets transmitted on the ports
    in ieee8021FrerSequenceAutoconfigReceivePortList."

```

```

1    REFERENCE    "10.7.2.1.3"
    ::= { ieee8021FrerOutputAutoconfigEntry 7 }

ieee8021FrerOutputAutoconfigStatus OBJECT-TYPE
    SYNTAX      RowStatus
    MAX-ACCESS   read-create
    STATUS       current
    DESCRIPTION
        "Row-Status for the Output Autoconfiguration table."
    ::= { ieee8021FrerOutputAutoconfigEntry 8 }

-- =====
-- the ieee8021FrerOutputAutoconfigPortHandleList table
-- =====
ieee8021FrerOutputAutoconfigPortHandleListTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerOutputAutoconfigPortHandleListEntry
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION
        "A table containing a list of ports to which this
         frerAutOutEntry applies, and on which active Sequence
         encode/decode functions are to be autocreated, referenced in
         ieee8021FrerOutputAutoconfigIndex."
    REFERENCE    "10.7.2.1.1"
    ::= { ieee8021FrerOutputAutoconfigPortHandleList 16}

ieee8021FrerOutputAutoconfigPortHandleListEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerOutputAutoconfigPortHandleListEntry
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION
        "A set of managed objects providing the ports to which this
         frerAutOutEntry applies, and on which active Sequence
         encode/decode functions are to be autocreated, referenced in
         ieee8021FrerOutputAutoconfigIndex."
    REFERENCE    "10.7.2.1.1"
    INDEX        { ieee8021FrerOutputAutoconfigIndex,
                   ieee8021FrerOutputAutoconfigPortHandleListIndex
                 }
    ::= { ieee8021FrerOutputAutoconfigPortHandleListTable 1 }

Ieee8021FrerOutputAutoconfigPortHandleListEntry ::=
    SEQUENCE {
        ieee8021FrerOutputAutoconfigPortHandleListIndex
            Unsigned32,
        ieee8021FrerOutputAutoconfigPortHandle
            VariablePointer,
        ieee8021FrerOutputAutoconfigPortHandleListStatus
            RowStatus
    }

ieee8021FrerOutputAutoconfigPortHandleListIndex OBJECT-TYPE
    SYNTAX      Unsigned32
    MAX-ACCESS   not-accessible
    STATUS       current
    DESCRIPTION
        "Index for the Output Autoconfiguration Port handle list
         table."
    ::= { ieee8021FrerOutputAutoconfigPortHandleListEntry 1 }

ieee8021FrerOutputAutoconfigPortHandle OBJECT-TYPE
    SYNTAX      VariablePointer
    MAX-ACCESS   read-create
    STATUS       current
    DESCRIPTION
        "A pointer to the ieee8021StreamIdStreamIdHandle that is an
         element of the ieee8021FrerOutputAutoconfigPortList instance in
         the ieee8021FrerOutputAutoconfigTable."
    REFERENCE    "10.7.2.1.1"
    ::= { ieee8021FrerOutputAutoconfigPortHandleListEntry 2 }

ieee8021FrerOutputAutoconfigPortHandleListStatus OBJECT-TYPE

```

```

1  SYNTAX      RowStatus
   MAX-ACCESS  read-create
   STATUS      current
   DESCRIPTION
       "Row-Status for the Output Autoconfiguration Port handle list
       table."
   ::= { ieee8021FrerOutputAutoconfigPortHandleListEntry 3 }

-- =====
-- the ieee8021FrerPerPortPerStreamCounters table
-- =====

ieee8021FrerPerPortPerStreamCountersTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerPerPortPerStreamCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of counters for frame replication
        and elimination for reliability that are instantiated per
        port, Stream, and direction."
    REFERENCE   "10.8"
    ::= { ieee8021FrerPerPortPerStreamCounters 17 }

ieee8021FrerPerPortPerStreamCountersEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerPerPortPerStreamCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A a set of managed objects for FRER counters that are
        instantiated per port, Stream, and direction."
    REFERENCE   "10.8"
    INDEX { ifIndex,
            ieee8021StreamIdStreamIdHandle,
            ieee8021FrerPerPortPerStreamDirection
          }
    ::= { ieee8021FrerPerPortPerStreamCountersTable 1 }

Ieee8021FrerPerPortPerStreamCountersEntry ::=
    SEQUENCE {
        ieee8021FrerPerPortPerStreamDirection
        TruthValue,
        ieee8021FrerPerPortPerStreamSeqGenResets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqRecoveryOutOfOrderPackets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqRecoveryLostPackets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqRecoveryTaglessPackets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqRecoveryResets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqRecoveryLatentErrorResets
        Counter64,
        ieee8021FrerPerPortPerStreamSeqEncErroredPackets
        Counter64
    }

ieee8021FrerPerPortPerStreamDirection OBJECT-TYPE
    SYNTAX      TruthValue
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A managed object specifying the facing of a given Stream
        on a port. The Stream can be either in-facing (False) or
        out-facing (True)."
    REFERENCE   "10.8"

```

```

1 ::= { ieee8021FrerPerPortPerStreamCountersEntry 1 }

ieee8021FrerPerPortPerStreamSeqGenResets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS   read-only
    STATUS       current
    DESCRIPTION
        "A managed object serving as a counter that is incremented each
        time the SequenceGenerationReset function is called."
    REFERENCE    "10.8.2"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 2 }

ieee8021FrerPerPortPerStreamSeqRecoveryOutOfOrderPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS   read-only
    STATUS       current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        for each packet accepted out-of-order by the
        VectorRecoveryAlgorithm or MatchRecoveryAlgorithm. Out-of-order
        means that the packet's sequence number is not one more than
        the previous packet received."
    REFERENCE    "10.8.3"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 3 }

ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS   read-only
    STATUS       current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        for each packet discarded by the VectorRecoveryAlgorithm
        because its sequence number subparameter is more than
        ieee8021FrerSequenceRecoveryHistoryLength from RecovSeqNum."
    REFERENCE    "10.8.4"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 4 }

ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS   read-only
    STATUS       current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        for each packet passed up the stack by the
        VectorRecoveryAlgorithm or MatchRecoveryAlgorithm."
    REFERENCE    "10.8.5"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 5 }

ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS   read-only
    STATUS       current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        for each packet discarded due to a duplicate sequence number by
        the VectorRecoveryAlgorithm or MatchRecoveryAlgorithm."
    REFERENCE    "10.8.6"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 6 }

ieee8021FrerPerPortPerStreamSeqRecoveryLostPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS   read-only
    STATUS       current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        for each packet lost by the VectorRecoveryAlgorithm. A packet
        is counted as lost if its sequence number is not received on
        any ingress port"
    REFERENCE    "10.8.7"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 7 }

ieee8021FrerPerPortPerStreamSeqRecoveryTaglessPackets OBJECT-TYPE

```

```

1  SYNTAX      Counter64
   MAX-ACCESS  read-only
   STATUS      current
   DESCRIPTION
       "A managed object serving as a counter that is incremented once
       for each packet received by the VectorRecoveryAlgorithm that
       has no sequence_number subparameter."
   REFERENCE   "10.8.8"
   ::= { ieee8021FrerPerPortPerStreamCountersEntry 8 }

ieee8021FrerPerPortPerStreamSeqRecoveryResets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        each time the SequenceRecoveryReset function is called."
    REFERENCE   "10.8.9"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 9 }

ieee8021FrerPerPortPerStreamSeqRecoveryLatentErrorResets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        each time the LatentErrorReset function is called."
    REFERENCE   "10.8.10"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 10 }

ieee8021FrerPerPortPerStreamSeqEncErroredPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        each time the Sequence encode/decode function receives a packet
        that it is unable to decode successfully."
    REFERENCE   "10.8.11"
    ::= { ieee8021FrerPerPortPerStreamCountersEntry 11 }

-- =====
-- the ieee8021FrerPerPortCounters table
-- =====

ieee8021FrerPerPortCountersTable OBJECT-TYPE
    SYNTAX      SEQUENCE OF Ieee8021FrerPerPortCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A table containing a set of counters for frame replication
        and elimination for reliability that are instantiated per
        port and direction."
    REFERENCE   "10.9"
    ::= { ieee8021FrerPerPortCounters 18 }

ieee8021FrerPerPortCountersEntry OBJECT-TYPE
    SYNTAX      Ieee8021FrerPerPortCountersEntry
    MAX-ACCESS  not-accessible
    STATUS      current
    DESCRIPTION
        "A a set of managed objects for FRER counters that are
        instantiated per port and direction."
    REFERENCE   "10.9"
    INDEX { ifIndex }
    ::= { ieee8021FrerPerPortCountersTable 1 }

Ieee8021FrerPerPortCountersEntry ::=
    SEQUENCE {
        ieee8021FrerPerPortSeqRecoveryPassedPackets
            Counter64,
        ieee8021FrerPerPortfrerCpSeqRecoveryDiscardPackets

```

```

1      Counter64,
      ieee8021FrerPerPortFrerCpSeqEncErroredPackets
      Counter64
    }

ieee8021FrerPerPortSeqRecoveryPassedPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        for each packet passed up the stack by the
        VectorRecoveryAlgorithm or MatchRecoveryAlgorithm. Its value
        equals the sum (modulo the size of the counters) of all of the
        ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets counters on
        this same port."
    REFERENCE   "10.9.1"
    ::= { ieee8021FrerPerPortCountersEntry 1 }

ieee8021FrerPerPortFrerCpSeqRecoveryDiscardPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        for each packet discarded due to a duplicate sequence number or
        for being a rogue packet by any VectorRecoveryAlgorithm or
        MatchRecoveryAlgorithm on this port. Its value equals the sum
        (modulo the size of the counters) of all of the
        ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets and
        ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets counters on
        this same port."
    REFERENCE   "10.9.2"
    ::= { ieee8021FrerPerPortCountersEntry 2 }

ieee8021FrerPerPortFrerCpSeqEncErroredPackets OBJECT-TYPE
    SYNTAX      Counter64
    MAX-ACCESS  read-only
    STATUS      current
    DESCRIPTION
        "A managed object serving as a counter that is incremented once
        each time the Sequence encode/decode function receives a packet
        that it is unable to decode successfully. Its value equals the
        sum (modulo the size of the counters) of all of the
        ieee8021FrerPerPortPerStreamSeqEncErroredPackets counters on
        this same port."
    REFERENCE   "10.9.3"
    ::= { ieee8021FrerPerPortCountersEntry 3 }

-- =====
-- IEEE802 FRER MIB - Conformance Information
-- =====

ieee8021FrerCompliances
    OBJECT IDENTIFIER ::= { ieee8021FrerConformance 1 }
ieee8021FrerGroups
    OBJECT IDENTIFIER ::= { ieee8021FrerConformance 2 }

-- =====
-- units of conformance
-- =====

-- =====
-- the ieee8021FrerSequenceGeneration group
-- =====

ieee8021FrerSequenceGenerationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021FrerSequenceGenerationStreamList,
        ieee8021FrerSequenceGenerationDirection,
        ieee8021FrerSequenceGenerationReset,
        ieee8021FrerSequenceGenerationStreamHandle,
    }

```

```

1      ieee8021FrerSequenceGenerationHandleListStatus
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the FRER sequence generation
        functionality."
    ::= { ieee8021FrerGroups 1 }

-- =====
-- the ieee8021FrerSequenceRecovery group
-- =====

ieee8021FrerSequenceRecoveryGroup OBJECT-GROUP
    OBJECTS {
        ieee8021FrerSequenceRecoveryStreamList,
        ieee8021FrerSequenceRecoveryPortList,
        ieee8021FrerSequenceRecoveryDirection,
        ieee8021FrerSequenceRecoveryReset,
        ieee8021FrerSequenceRecoveryAlgorithm,
        ieee8021FrerSequenceRecoveryAlgorithmOUI,
        ieee8021FrerSequenceCustomRecoveryAlgorithm,
        ieee8021FrerSequenceCustomRecoveryAlgorithmOUI,
        ieee8021FrerSequenceRecoveryHistoryLength,
        ieee8021FrerSequenceRecoveryResetMSec,
        ieee8021FrerSequenceRecoveryInvalidSequenceValue,
        ieee8021FrerSequenceRecoveryTakeNoSequence,
        ieee8021FrerSequenceRecoveryIndividualRecovery,
        ieee8021FrerSequenceRecoveryLatentErrorDetection,
        ieee8021FrerSequenceRecoveryLatentErrorDifference,
        ieee8021FrerSequenceRecoveryLatentErrorPeriod,
        ieee8021FrerSequenceRecoveryLatentErrorPaths,
        ieee8021FrerSequenceRecoveryLatentResetPeriod,
        ieee8021FrerSequenceRecoveryStatus,
        ieee8021FrerSequenceRecoveryStreamHandle,
        ieee8021FrerSequenceRecoveryHandleListStatus,
        ieee8021FrerSequenceRecoveryPortHandle,
        ieee8021FrerSequenceRecoveryPortHandleListStatus
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the FRER sequence recovery
        functionality."
    ::= { ieee8021FrerGroups 2 }

-- =====
-- the ieee8021FrerSequenceIdentification group
-- =====

ieee8021FrerSequenceIdentificationGroup OBJECT-GROUP
    OBJECTS {
        ieee8021FrerSequenceIdentificationStreamList,
        ieee8021FrerSequenceIdentificationEncodeActive,
        ieee8021FrerSequenceIdentificationEncodeEncapsulationType,
        ieee8021FrerSequenceIdentificationEncodeEncapsulationOUI,
        ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationType,
        ieee8021FrerSequenceIdentificationCustomEncodeEncapsulationOUI,
        ieee8021FrerSequenceIdentificationEncodePathIdLanId,
        ieee8021FrerSequenceIdentificationStatus,
        ieee8021FrerSequenceIdentificationStreamHandle,
        ieee8021FrerSequenceIdentificationHandleListStatus
    }
    STATUS      current
    DESCRIPTION
        "Objects that are part of the FRER sequence identification
        functionality."
    ::= { ieee8021FrerGroups 3 }

-- =====
-- the ieee8021FrerStreamSplit group
-- =====

ieee8021FrerStreamSplitGroup OBJECT-GROUP

```

```

1  OBJECTS {
    ieee8021FrerStreamSplitPort,
    ieee8021FrerStreamSplitDirection,
    ieee8021FrerStreamSplitInputIdList,
    ieee8021FrerStreamSplitOutputIdList,
    ieee8021FrerStreamSplitInputIdHandle,
    ieee8021FrerStreamSplitInputHandleListStatus,
    ieee8021FrerStreamSplitOutputIdHandle,
    ieee8021FrerStreamSplitOutputHandleListStatus
  }
  STATUS      current
  DESCRIPTION
    "Objects that are part of the FRER Stream split functionality."
  ::= { ieee8021FrerGroups 4 }

-- =====
-- the ieee8021FrerSequenceAutoconfig group
-- =====

ieee8021FrerSequenceAutoconfigGroup OBJECT-GROUP
  OBJECTS {
    ieee8021FrerSequenceAutoconfigSequenceEncapsulation,
    ieee8021FrerSequenceAutoconfigSequenceEncapsulationOUI,
    ieee8021FrerSequenceAutoconfigCustomSequenceEncapsulation,
    ieee8021FrerSequenceAutoconfigCustomSequenceEncOUI,
    ieee8021FrerSequenceAutoconfigReceivePortList,
    ieee8021FrerSequenceAutoconfigTagged,
    ieee8021FrerSequenceAutoconfigVlanList,
    ieee8021FrerSequenceAutoconfigRecoveryPortList,
    ieee8021FrerSequenceAutoconfigDestructMSec,
    ieee8021FrerSequenceAutoconfigResetMSec,
    ieee8021FrerSequenceAutoconfigAlgorithm,
    ieee8021FrerSequenceAutoconfigAlgorithmOUI,
    ieee8021FrerSequenceAutoconfigCustomAlgorithm,
    ieee8021FrerSequenceAutoconfigCustomAlgorithmOUI,
    ieee8021FrerSequenceAutoconfigHistoryLength,
    ieee8021FrerSequenceAutoconfigCreateIndividual,
    ieee8021FrerSequenceAutoconfigCreateRecovery,
    ieee8021FrerSequenceAutoconfigLatentErrorDetection,
    ieee8021FrerSequenceAutoconfigLatentErrorDifference,
    ieee8021FrerSequenceAutoconfigLatentErrorPeriod,
    ieee8021FrerSequenceAutoconfigLatentErrorResetPeriod,
    ieee8021FrerSequenceAutoconfigStatus,
    ieee8021FrerSequenceAutoconfigReceivePortHandle,
    ieee8021FrerSequenceAutoconfigReceivePortHandleListStatus,
    ieee8021FrerSequenceAutoconfigVlan,
    ieee8021FrerSequenceAutoconfigVlanListStatus,
    ieee8021FrerSequenceAutoconfigRecoveryPortHandle,
    ieee8021FrerSequenceAutoconfigRecoveryPortHandleListStatus
  }
  STATUS      current
  DESCRIPTION
    "Objects that are part of the FRER sequence autoconfiguration
    functionality."
  ::= { ieee8021FrerGroups 5 }

-- =====
-- the ieee8021FrerOutputAutoconfig group
-- =====

ieee8021FrerOutputAutoconfigGroup OBJECT-GROUP
  OBJECTS {
    ieee8021FrerOutputAutoconfigPortList,
    ieee8021FrerOutputAutoconfigEncapsulation,
    ieee8021FrerOutputAutoconfigEncapsulationOUI,
    ieee8021FrerOutputAutoconfigCustomEncapsulation,
    ieee8021FrerOutputAutoconfigCustomEncapsulationOUI,
    ieee8021FrerOutputAutoconfigLanPathId,
    ieee8021FrerOutputAutoconfigStatus,
    ieee8021FrerOutputAutoconfigPortHandle,
    ieee8021FrerOutputAutoconfigPortHandleListStatus
  }

```



```

1  STATUS      current
   DESCRIPTION
      "Objects that are part of the FRER output autoconfiguration
      functionality."
      ::= { ieee8021FrerGroups 6 }

-- =====
-- the ieee8021FrerPerPortPerStreamCounters group
-- =====

ieee8021FrerPerPortPerStreamCountersGroup OBJECT-GROUP
    OBJECTS {
        ieee8021FrerPerPortPerStreamSeqGenResets,
        ieee8021FrerPerPortPerStreamSeqRecoveryOutOfOrderPackets,
        ieee8021FrerPerPortPerStreamSeqRecoveryRoguePackets,
        ieee8021FrerPerPortPerStreamSeqRecoveryPassedPackets,
        ieee8021FrerPerPortPerStreamSeqRecoveryDiscardedPackets,
        ieee8021FrerPerPortPerStreamSeqRecoveryLostPackets,
        ieee8021FrerPerPortPerStreamSeqRecoveryTaglessPackets,
        ieee8021FrerPerPortPerStreamSeqRecoveryResets,
        ieee8021FrerPerPortPerStreamSeqRecoveryLatentErrorResets,
        ieee8021FrerPerPortPerStreamSeqEncErroredPackets
    }
    STATUS      current
    DESCRIPTION
        "Objects that provide information on the count of packets
        handled or actions taken by FRER algorithms per port, Stream,
        and direction."
        ::= { ieee8021FrerGroups 7 }

-- =====
-- the ieee8021FrerPerPortCounters group
-- =====

ieee8021FrerPerPortCountersGroup OBJECT-GROUP
    OBJECTS {
        ieee8021FrerPerPortSeqRecoveryPassedPackets,
        ieee8021FrerPerPortFrerCpSeqRecoveryDiscardPackets,
        ieee8021FrerPerPortFrerCpSeqEncErroredPackets
    }
    STATUS      current
    DESCRIPTION
        "Objects that provide information on the count of packets
        handled by FRER per port and direction."
        ::= { ieee8021FrerGroups 8 }

-- =====
-- compliance statements
-- =====

ieee8021FrerCompliance MODULE-COMPLIANCE
    STATUS      current
    DESCRIPTION
        "The compliance statement for devices supporting
        Stream identification."

    MODULE -- this module
        MANDATORY-GROUPS {
            ieee8021FrerSequenceGenerationGroup,
            ieee8021FrerSequenceRecoveryGroup,
            ieee8021FrerSequenceIdentificationGroup,
            ieee8021FrerStreamSplitGroup,
            ieee8021FrerPerPortPerStreamCountersGroup,
            ieee8021FrerPerPortCountersGroup
        }

        GROUP ieee8021FrerSequenceAutoconfigGroup
        DESCRIPTION
            "Implementation of this group is mandatory if the
            auto configuration feature is implemented."

        GROUP ieee8021FrerOutputAutoconfigGroup

```

1 DESCRIPTION
 "Implementation of this group is mandatory if the
 auto configuration feature is implemented."

 ::= { ieee8021FrerCompliances 1 }

END

1 **12. YANG Data Model**

2 This clause specifies YANG data models that provide control and monitoring for all features of this
3 standard. Specifically:

- 4 a) Stream Identification
- 5 b) Frame Replication and Elimination for Reliability

6 The YANG data management models are derived from UML models specified in 12.2. The UML models
7 are based on Clause 9 and Clause 10.

8 NOTE 1—OMG UML 2.5 [B14] conventions are used in this clause as a representation to convey model structure and
9 relationships.

10 NOTE 2—The MIB modules specified in Clause 11 were also derived from Clause 9 and Clause 10. Consequently, the
11 capabilities and structure of the YANG data models are closely aligned with their MIB representations. However, the
12 YANG data models have not been derived from the MIB modules, and no attempt has been made to include data or
13 modeling constructs that might appear in the MIB modules but not in the information models.

14 **12.1 YANG Framework**

15 This clause has been developed according to the YANG guidelines published in IETF RFC 6087 [B7] as
16 applicable to IEEE standards.

17 The YANG framework applies hierarchy in the following areas:

- 18 a) The uniform resource name (URN), as specified in IEEE Std 802d [B3]. ~~The structure of the URN is~~
19 ~~such that ieee is the root (i.e., name-space identifier), followed by the standard, then the working~~
20 ~~group developing the standard.~~
- 21 b) The YANG objects form a hierarchy of configuration and operational data structures that define the
22 YANG model. These hierarchical relationships are described in 12.2.

23 The general YANG framework hierarchy is illustrated in Figure 12-1.

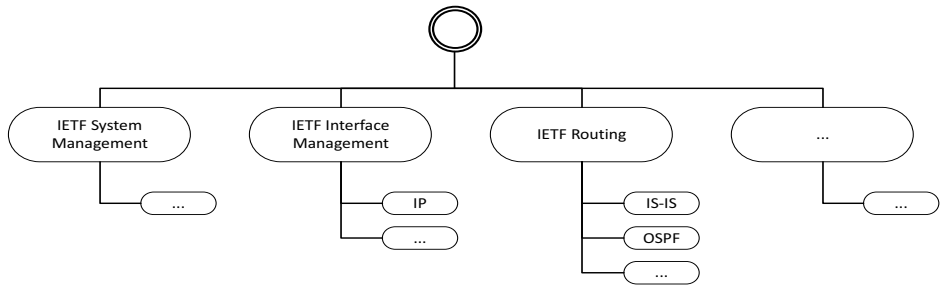


Figure 12-1—General YANG hierarchy

24 Network interfaces are central to the management of protocols supported over said interface. Thus, it is
25 important to establish a common data model for how interfaces are identified, configured, and monitored.
26 The IETF Interface Management model (IETF RFC 8343) defines a generic YANG data model for the
27 management of network interfaces. Consequently, IEEE Std 802.1CB™ augments the generic interfaces
28 data model defined by the IETF Interface Management model (IETF RFC 8343).

1 In addition, two high level YANG objects are defined to support IEEE Std 802.1CB™ features that exist at
2 device level.

3 The YANG hierarchical structure that incorporates the IEEE Std 802.1CB™ YANG models supported by
4 this standard is represented in Figure 12-2. Objects that are added by this standard are indicated by a darker
5 shading.

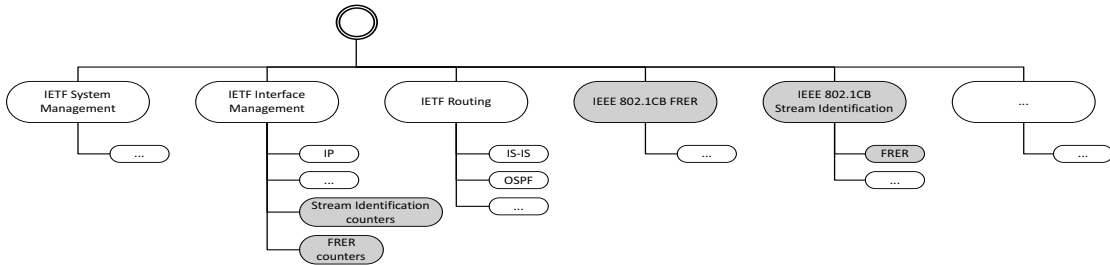


Figure 12-2—YANG root hierarchy with IEEE Std 802.1CB™ YANG modules

6 12.2 IEEE Std 802.1CB™ YANG model

7 The YANG data models are based on the management models outlined in Clause 9 and Clause 10. A UML
8 representation of the management model is provided in the following subclauses.

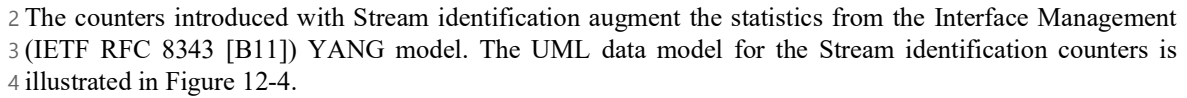
9 The purpose of a UML-like¹⁹ diagram is to express the model design on a single piece of paper. The
10 structure of the UML-like representation shows the name of the object followed by a list of properties for the
11 object. The properties indicate their type and accessibility. It should be noted that the UML-like
12 representation is meant to express simplified semantics for the properties. It is not meant to provide the
13 specific datatype as used to encode the object in either MIB or YANG. In the UML-like representation, a
14 box with a white background represents information that comes from sources outside of the IEEE. A box
15 with a gray background represents objects that are defined by this IEEE Standard.

16 12.2.1 Stream Identification model

17 The UML data model for Stream identification is illustrated in Figure 12-3.

18 NOTE—The {XOR} UML constraint denotes that the stream-identity parameters can be either one of the identification-
19 types described on the right-hand side of the diagram.

¹⁹A description of the UML-like diagrams used in this clause is provided at <https://1.ieee802.org/uml-like-diagrams>



1

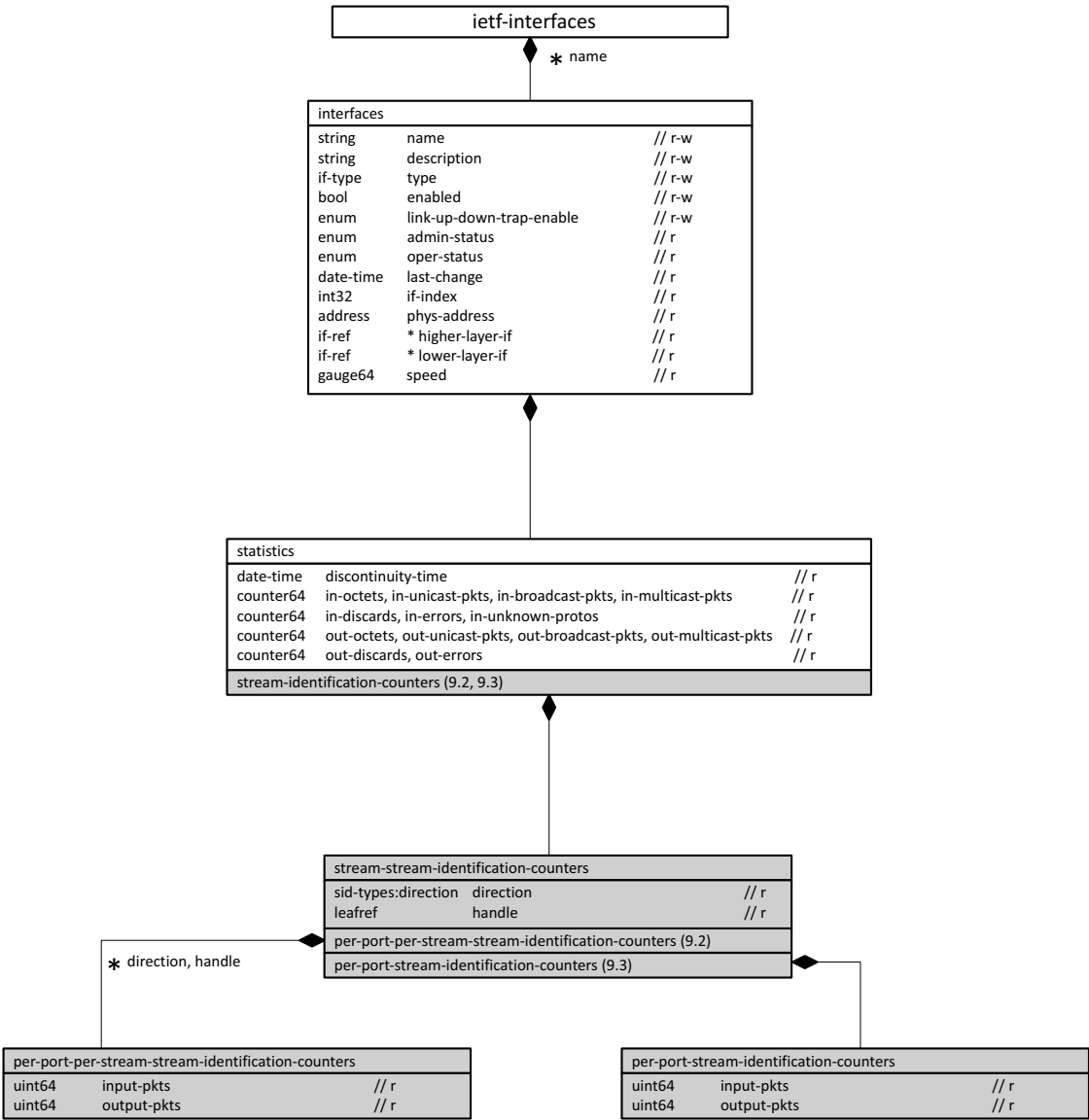


Figure 12-4—Stream identification counters model

2 12.2.2 Frame Replication and Elimination for Reliability model

3 The UML data model for FRER is illustrated in Figure 12-5. Additionally, FRER augments the Stream
4 identification YANG model as illustrated in Figure 12-6.

1

sequence-generation (10.3)		
uint32	index	// r-w
handle-ref	* stream	// r-w
boolean	direction-out-facing	// r-w
boolean	reset	// r-w

sequence-recovery (10.4)		
uint32	index	// r-w
handle-ref	* stream	// r-w
if-ref	* port	// r-w
boolean	direction-out-facing	// r-w
boolean	reset	// r-w
sequence-recovery-algorithm	algorithm	// r-w
sequence-history-length	history-length	// r-w
uint32	reset-timeout	// r-w
uint32	invalid-sequence-value	// r
boolean	take-no-sequence	// r-w
boolean	individual-recovery	// r-w
boolean	latent-error-detection	// r-w
int32	latent-error-difference	// r-w
uint32	latent-error-period	// r-w
uint16	latent-error-paths	// r-w
uint32	latent-error-reset-period	// r-w

sequence-identification (10.5)		
handle-ref	* stream	// r-w
if-ref	port	// r-w
boolean	direction-out-facing	// r-w
boolean	active	// r-w
enum	encapsulation	// r-w
int8	path-id-lan-id	// r-w

stream-split (10.6)		
uint32	index	// r-w
if-ref	port	// r-w
boolean	direction-out-facing	// r-w
leaf-ref	* input-id	// r-w
leaf-ref	* output-id	// r-w

sequence-autoconfiguration (10.7.1)		
uint32	index	// r-w
enum	sequence-encapsulation	// r-w
if-ref	* receive-port	// r-w
enum	tagged	// r-w
dot1qtypes:vlanid	* vlan	// r-w
if-ref	* recovery-port	// r-w
uint64	destruction-interval	// r-w
uint64	reset-interval	// r-w
sequence-recovery-algorithm	algorithm	// r-w
sequence-history-length	history-length	// r-w
boolean	create-individual	// r-w
boolean	create-recovery	// r-w
boolean	latent-error-detection	// r-w
int32	latent-error-difference	// r-w
uint32	latent-error-period	// r-w
uint32	latent-error-reset-period	// r-w

output-autoconfiguration (10.7.2)		
uint32	index	// r-w
if-ref	* port	// r-w
enum	encapsulation	// r-w
int8	lan-path-id	// r-w

Figure 12-5—FRER model

1

stream-identity		
uint32	index	// not accessible
uint32	handle	// r-w
if-name	* in-facing-input-port	// r-w
if-name	* in-facing-output-port	// r-w
if-name	* out-facing-input-port	// r-w
if-name	* out-facing-output-port	// r-w
enum	identification-type	// r-w
parameters		
boolean	auto-configured;	// r
dot1cb-frer-types:lan-path-id	lan-path-id;	// r

Figure 12-6—FRER Stream identification augmentation

2 The counters introduced with FRER augment the statistics from the Interface Management (IETF RFC 8343

3 [B11]) YANG model. The UML data model for the FRER counters is illustrated in Figure 12-7.

1

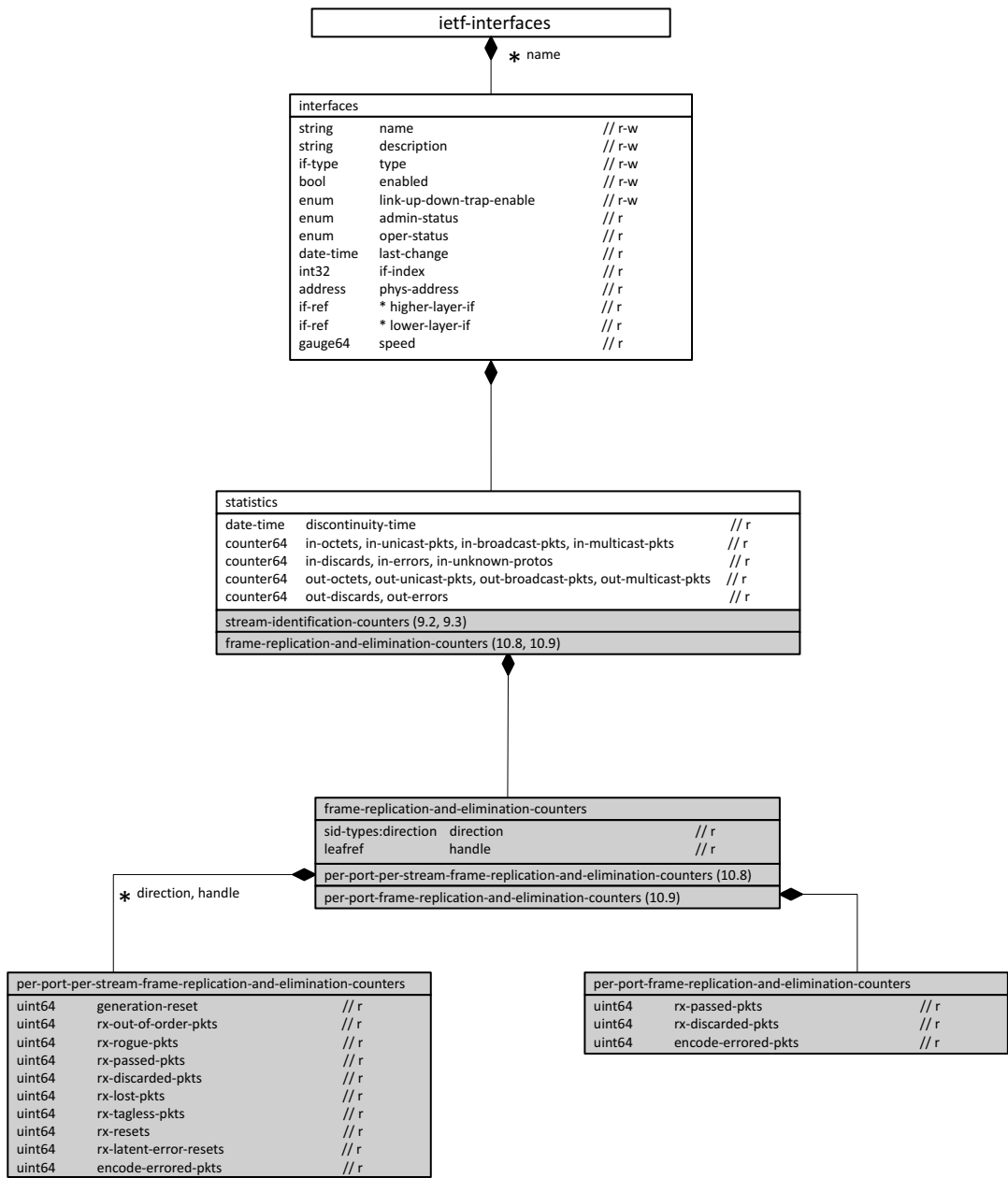


Figure 12-7—FRER counters model

2 12.3 Structure of the YANG model

3 IEEE Std 802.1CB™ YANG models are divided into a number of YANG modules. A summary of the
4 modules contained in this clause is represented in Table 12-1.

5 The modules in Table 12-1 can be used to create a Stream Identification model or an FRER model.

Table 12-1—Description of the YANG modules

Module	References	Notes
ieee802-dot1cb-stream-identification-types	12.6.2.1	General type definitions used by IEEE Std 802.1CB™ Stream identification.
ieee802-dot1cb-stream-identification	12.6.2.2	YANG model for Stream identification.
ieee802-dot1cb-mask-and-match	12.6.2.3	YANG model for the extended stream identification.
ieee802-dot1cb-frer-types	12.6.2.4	General type definitions used by IEEE Std 802.1CB™ frame replication and elimination for reliability.
ieee802-dot1cb-frer	12.6.2.5	YANG model for FRER.

1 12.3.1 Structure of the ieee802-dot1cb-stream-identification YANG module

2 The Stream Identification model consists of the *ieee802-dot1cb-stream-identification* YANG module along
3 with all the dependencies (YANG imports) that the module uses. Clause 12.6.1.1 contains the YANG data
4 schema tree for the *ieee802-dot1cb-stream-identification* module.

5 The high-level structure of the *ieee802-dot1cb-stream-identification* YANG module is found in Table 12-2.

6 The list of YANG modules directly imported by the *ieee802-dot1cb-stream-identification* YANG module is
7 found in Table 12-3.

Table 12-2—ieee802-dot1cb-stream-identification structure and relationship to this standard

Module	References	Notes
ieee802-dot1cb-stream-identification	9	—
stream-identity	9.1	Stream identity management within a system.
per-port-counters	9.3	Per-port counters for Stream identification.
per-port-per-stream-counters	9.2	Per-port-per-stream counters for Stream identification.

Table 12-3—YANG module dependencies for the Stream identification model

YANG module
ieee802-types
ieee802-dot1q-types
ietf-inet-types
ietf-interfaces
ieee802-dot1cb-stream-identification-types

To complete the model, all the dependencies from the imported modules must also be identified. The process to determine all the dependencies can be done through tooling. For example, if the pyang [B15] or yanglint tool [B16] is used on the *ieee802-dot1cb-stream-identification* YANG module, the tooling will try to include all the imports and produce an error message if an import is missing. The YANG Catalog [B17] search tools and/or the YANG Catalog's github repository [B18] can be used to find the missing imports.

12.3.2 Structure of the *ieee802-dot1cb-mask-and-match* YANG module

The Extended Stream Identification model consists of the *ieee802-dot1cb-mask-and-match* YANG module along with all the dependencies (YANG imports) that the module uses. 12.6.1.2 contains the YANG data schema tree for the *ieee802-dot1cb-mask-and-match* module.

The high-level structure of the *ieee802-dot1cb-mask-and-match* YANG module is found in Table 12-4.

The list of YANG modules directly imported by the *ieee802-dot1cb-mask-and-match* YANG module is found in Table 12-5.

Table 12-4—*ieee802-dot1cb-mask-and-match* structure and relationship to this standard

Module	References	Notes
<i>ieee802-dot1cb-mask-and-match</i>		—
stream-identity parameters	9.1.6	Mask-and-match stream identification parameters.
per-port-msdu-mask-max-length	9.4	Per-port stream identification parameter.

Table 12-5—YANG module dependencies for the Extended Stream identification model

YANG module
<i>ieee802-types</i>
<i>ietf-interfaces</i>
<i>ieee802-dot1cb-stream-identification-types</i>
<i>ieee802-dot1cb-stream-identification</i>

To complete the model, all the dependencies from the imported modules must also be identified. The process to determine all the dependencies can be done through tooling. For example, if the pyang [B15] or yanglint tool [B16] is used on the *ieee802-dot1cb-mask-and-match* YANG module, the tooling will try to include all the imports and produce an error message if an import is missing. The YANG Catalog [B17] search tools and/or the YANG Catalog repository [B18] can be used to find the missing imports.

12.3.3 Structure of the *ieee802-dot1cb-frer* YANG module

The FRER model consists of the *ieee802-dot1cb-frer* YANG module along with all the dependencies (YANG imports) that the module uses. Clause 12.6.1.3 contains the YANG data schema tree for the *ieee802-dot1cb-frer* module.

The high-level structure of the *ieee802-dot1cb-frer* YANG module is found in Table 12-6.

The list of YANG modules directly imported by the *ieee802-dot1cb-frer* YANG module is found in Table 12-7.

Table 12-6—*ieee802-dot1cb-frer* structure and relationship to this standard

Module	References	Notes
ieee802-dot1cb-frer	10	—
sequence-generation	10.3	Sequence generation management within a system.
sequence-recovery	10.4	Sequence recovery management within a system.
sequence-identification	10.5	Sequence identification management within a system.
stream-split	10.6	Stream splitting management within a system.
autoconfiguration	10.7	Autoconfiguration management within a system.
per-port-counters	10.9	Per-port counters for FRER.
per-port-per-stream-counters	10.8	Per-port-per-stream counters for FRER.

8

Table 12-7—YANG module dependencies for the FRER model

YANG module
ieee802-dot1q-types
ietf-interfaces
ieee802-dot1cb-stream-identification-types
ieee802-dot1cb-stream-identification
ieee802-dot1cb-frer-types
ieee802-dot1cb-frer

To complete the model, all the dependencies from the imported modules must also be identified. The process to determine all the dependencies can be done through tooling. For example, if the pyang [B15] or yanglint tool [B16] is used on the *ieee802-dot1cb-frer* YANG module, the tooling will try to include all the

1 imports and produce an error message if an import is missing. The YANG Catalog [B17] search tools and/or
2 the YANG Catalog's github repository [B18] can be used to find the missing imports.

3 **12.4 Relationship to other YANG modules**

4 In order to facilitate interoperable management of the features defined in this standard by remote means,
5 12.6 contains a complete set of YANG modules. This clause describes how the YANG modules defined in
6 this standard are related to the YANG modules that are imported.

7 **12.4.1 IEEE 802 Types Module**

8 The *ieee802-types* YANG module provides reusable types that are used in IEEE 802 standards.

9 The type for mac-addresses defined in *ieee802-types* has a pattern that allows upper and lower case letters.
10 To avoid issues with string comparison, it is suggested to only use upper case for the letters in the
11 hexadecimal numbers. Implementers using code comparing MAC addresses should note that there is still an
12 issue with a difference between the IETF mac-address definition and the IEEE mac-address definition.

13 **12.4.2 IEEE 802.1Q Types Module**

14 The *ieee802-dot1q-types* YANG module provides reusable types that are used in IEEE 802.1 standards.

15 **12.4.3 IETF Inet Types Module**

16 The *ietf-inet-types* YANG module (IETF RFC 6021) provides reusable data types to be used with the YANG
17 data modeling language.

18 **12.4.4 IETF Interfaces YANG Module**

19 The *ietf-interfaces* YANG module (IETF RFC 8343) contains a set of YANG definitions for managing
20 network interfaces. This document models a port as an interface. Counters in the YANG modules defined in
21 this document augment the interface statistics provided by *ietf-interfaces*.

22 **12.4.5 IEEE 802.1CB Stream Identification Types YANG module**

23 The *ieee802-dot1cb-stream-identification-types* YANG module provides reusable types that are used for
24 IEEE 802.1CB Stream Identification.

25 **12.4.6 IEEE 802.1CB Stream Identification YANG module**

26 The *ieee802-dot1cb-stream-identification* YANG module defined in this standard provides Stream
27 identification functionality and a Stream identification handle. The *ieee802-dot1cb-frer* YANG module that
28 is also defined in this standard uses the Stream identification handle and also augments the Stream
29 identification functionality.

30 **12.4.7 IEEE 802.1CB Extended Stream identification module**

31 The *ieee802-dot1cb-mask-and-match* YANG module defined in this standard provides an additional Stream
32 identification functionality by augmenting the Stream identification functionality provided by the *ieee802-*
33 *dot1cb-stream-identification* module.

12.4.8 IEEE 802.1CB FRER Types YANG module

The *ieee802-dot1cb-frer-types* YANG module provides reusable types that are used for IEEE 802.1CB frame replication and elimination for reliability.

12.5 Security Considerations

The YANG modules defined in this clause are designed to be accessed via a network configuration protocol, e.g., NETCONF protocol (IETF RFC 6242 [B9]). In the case of NETCONF, the lowest NETCONF layer is the secure transport layer and the mandatory to implement secure transport is SSH (IETF RFC 6242 [B9]). The NETCONF access control model (IETF RFC 6536 [B10]) provides the means to restrict access for particular NETCONF users to a preconfigured subset of all available NETCONF protocol operations and content.

It is the responsibility of a system's implementor and administrator to ensure that the protocol entities in the system that support NETCONF, and any other remote configuration protocols that make use of these YANG modules, are properly configured to allow access only to those principals (users) that have legitimate rights to read or write data nodes. This standard does not specify how the credentials of those users are to be stored or validated.

12.5.1 Security Considerations of the *ieee802-dot1cb-stream-identification* YANG module

There are a number of management objects defined in the *ieee802-dot1cb-stream-identification* YANG module that are configurable (i.e., read-write) and/or operational (i.e., read-only). Such objects may be considered sensitive or vulnerable in some network environments. A network configuration protocol, such as NETCONF (IETF RFC 6241 [B8]), can support protocol operations that can edit or delete YANG module configuration data (e.g., edit-config, delete-config, copy-config). If this is done in a non-secure environment without proper protection, then negative effects on the network operation is possible.

12.5.2 Security Considerations of the *ieee802-dot1cb-frer* YANG module

There are a number of management objects defined in the *ieee802-dot1cb-frer* YANG module that are configurable (i.e., read-write) and/or operational (i.e., read-only). Such objects may be considered sensitive or vulnerable in some network environments. A network configuration protocol, such as NETCONF (IETF RFC 6241 [B8]), can support protocol operations that can edit or delete YANG module configuration data (e.g., edit-config, delete-config, copy-config). If this is done in a non-secure environment without proper protection, then negative effects on the network operation are possible.

12.6 Definition of IEEE Std 802.1CB™ YANG modules^{20 21}

The structure of the YANG modules related to this standard is described in 12.3. In the following YANG module definitions, if any discrepancy between the DESCRIPTION text and the corresponding definition in any other part of this standard occurs, the definitions outside this subclause take precedence.

²⁰Copyright release for YANG modules: Users of this standard may freely reproduce the YANG modules contained in this subclause so that they can be used for their intended purpose.

²¹An ASCII version of the YANG modules are attached to the PDF version of this standard, and can be obtained by Web browser from the IEEE 802.1 Website at <https://1.ieee802.org/yang-modules/>.

12.6.1 YANG data scheme tree definitions

A simplified graphical representation of the data model is used in this document. The meaning of the symbols in these diagrams is as follows:

- Brackets “[” and “]” enclose list keys.
- Abbreviations before data node names: “rw” means configuration (read-write), and “ro” means state data (read-only).
- Symbols after data node names: “?” means an optional node, “!” means a presence container, and “*” denotes a list and leaf-list.
- Parentheses enclose choice and case nodes, and case nodes are also marked with a colon (“:”).
- Ellipsis (“...”) stands for contents of subtrees that are not shown.

12.6.1.1 YANG data scheme definition for ieee802-dot1cb-stream-identification YANG module

```

module: ieee802-dot1cb-stream-identification
  +--rw stream-identity* [index]
    +--rw index                               uint32
    +--rw handle                             uint32
    +--rw in-facing
      | +--rw input-port*   if:interface-ref
      | +--rw output-port*  if:interface-ref
    +--rw out-facing
      | +--rw input-port*   if:interface-ref
      | +--rw output-port*  if:interface-ref
    +--rw (parameters)
      +--:(null-stream-identification)
        | +--rw null-stream-identification
        | | +--ro identification-type
        | | | +--ro type-number?  dot1cb-sid-types:stream-id-function
        | | | +--ro oui-cid?      string
        | | +--rw destination-mac? ieee:mac-address
        | | +--rw tagged?          vlan-tag-identification-type
        | | +--rw vlan?            vlan-identifier-type
      +--:(smac-vlan-stream-identification)
        | +--rw smac-vlan-stream-identification
        | | +--ro identification-type
        | | | +--ro type-number?  dot1cb-sid-types:stream-id-function
        | | | +--ro oui-cid?      string
        | | +--rw source-mac?     ieee:mac-address
        | | +--rw tagged?          vlan-tag-identification-type
        | | +--rw vlan?            vlan-identifier-type
      +--:(dmac-vlan-stream-identification)
        | +--rw dmac-vlan-stream-identification
        | | +--ro identification-type
        | | | +--ro type-number?  dot1cb-sid-types:stream-id-function
        | | | +--ro oui-cid?      string
        | | +--rw down
        | | | +--rw destination-mac? ieee:mac-address
        | | | +--rw tagged?          vlan-tag-identification-type
        | | | +--rw vlan?            vlan-identifier-type
        | | | +--rw priority?        dot1qtypes:priority-type
        | | +--rw up
        | | | +--rw destination-mac? ieee:mac-address
        | | | +--rw tagged?          vlan-tag-identification-type
        | | | +--rw vlan?            vlan-identifier-type
        | | | +--rw priority?        dot1qtypes:priority-type
      +--:(ip-stream-identification)
        | +--rw ip-stream-identification
        | | +--ro identification-type
        | | | +--ro type-number?  dot1cb-sid-types:stream-id-function
        | | | +--ro oui-cid?      string
        | | +--rw destination-mac? ieee:mac-address
        | | +--rw tagged?          vlan-tag-identification-type
        | | +--rw vlan?            vlan-identifier-type

```

```

1      |      +--rw ip-source?          inet:ip-address
      |      +--rw ip-destination?     inet:ip-address
      |      +--rw dscp?               inet:dscp
      |      +--rw next-protocol?      enumeration
      |      +--rw source-port?        inet:port-number
      |      +--rw destination-port?   inet:port-number
      +---:(organization-specific)
          +--rw organization-specific
              +--rw identification-type
                  +--rw type-number?   int32
                  +--rw oui-cid?       string

augment /if:interfaces/if:interface/if:statistics:
    +--ro stream-id
        +--ro per-port-counters
            | +--ro input-pkts?   uint64
            | +--ro output-pkts?  uint64
        +--ro per-port-per-stream-counters* [direction-out-facing handle]
            +--ro direction-out-facing  dot1cb-sid-types:direction
            +--ro handle                  -> /stream-identity/handle
            +--ro input-pkts?            uint64
            +--ro output-pkts?          uint64

```

23

24 12.6.1.2 YANG data scheme definition for ieee802-dot1cb-mask-and-match YANG module

```

25 module: ieee802-dot1cb-mask-and-match
26   +--rw per-port-msdu-mask-max-length* [port-name]
27     +--rw port-name                    if:interface-ref
28     +--ro port-msdu-mask-max-length?   msdu-mask-length-type
29
30 augment /dot1cb-sid:stream-identity/dot1cb-sid:parameters:
31   +--rw mask-and-match-stream-identification
32     +--ro identification-type
33       | +--ro type-number?   dot1cb-sid-types:stream-id-function
34       | +--ro oui-cid?       string
35     +--rw destination-mac-mask?   ieee802:mac-address
36     +--rw destination-mac-match?  ieee802:mac-address
37     +--rw source-mac-mask?        ieee802:mac-address
38     +--rw source-mac-match?       ieee802:mac-address
39     +--rw msdu-mask-length?       msdu-mask-length-type
40     +--rw msdu-mask?              msdu-mask-match-hex-string
41     +--rw msdu-match?             msdu-mask-match-hex-string
42

```

43 12.6.1.3 YANG data scheme definition for ieee802-dot1cb-frer YANG module

```

44 module: ieee802-dot1cb-frer
45   +--rw frer
46     +--rw sequence-generation* [index]
47   | +--rw index                    uint32
48   | | +--rw stream*                -> /dot1cb-sid:stream-identity/handle
49   | | +--rw direction-out-facing?  dot1cb-sid-types:direction
50   | | +--rw reset?                 boolean
51   +--rw sequence-recovery* [index]
52   | +--rw index                    uint32
53   | +--rw stream*
54   | | -> /dot1cb-sid:stream-identity/handle
55   | +--rw port*                    if:interface-ref
56   | +--rw direction-out-facing?    dot1cb-sid-types:direction
57   | +--rw reset?                   boolean
58   | +--rw algorithm

```



```

1 | | +---rw (algorithm)?
2 | | +---:(vector)
3 | | | +---rw vector!
4 | | | +---ro type-number? dotlcb-frer-types:seq-rcvy-algorithm
5 | | | +---ro oui-cid? string
6 | | +---:(match)
7 | | | +---rw match!
8 | | | +---ro type-number? dotlcb-frer-types:seq-rcvy-algorithm
9 | | | +---ro oui-cid? string
10 | | +---:(organization-specific)
11 | | +---rw organization-specific!
12 | | +---rw type-number? int32
13 | | +---rw oui-cid? string
14 | +---rw history-length? sequence-history-length
15 | +---rw reset-timeout? uint32
16 | +---ro invalid-sequence-value? uint32
17 | +---rw take-no-sequence? boolean
18 | +---rw individual-recovery? boolean
19 | +---rw latent-error-detection? boolean
20 | +---rw latent-error-detection-parameters
21 | | +---rw difference? int32
22 | | +---rw period? uint32
23 | | +---rw paths? uint16
24 | | +---rw reset-period? uint32
25 +---rw sequence-identification* [port direction-out-facing]
26 | +---rw stream* -> /dotlcb-sid:stream-identity/handle
27 | +---rw port if:interface-ref
28 | +---rw direction-out-facing dotlcb-sid-types:direction
29 | +---rw active? boolean
30 | +---rw encapsulation
31 | | +---rw (encapsulation)?
32 | | +---:(r-tag)
33 | | | +---rw r-tag!
34 | | | +---ro type-number? dotlcb-frer-types:seq-encaps-method
35 | | | +---ro oui-cid? string
36 | | +---:(hsr-sequence-tag)
37 | | | +---rw hsr-sequence-tag!
38 | | | +---ro type-number? dotlcb-frer-types:seq-encaps-method
39 | | | +---ro oui-cid? string
40 | | +---:(prp-sequence-tag)
41 | | | +---rw prp-sequence-tag!
42 | | | +---ro type-number? dotlcb-frer-types:seq-encaps-method
43 | | | +---ro oui-cid? string
44 | | +---:(organization-specific)
45 | | +---rw organization-specific!
46 | | +---rw type-number? int32
47 | | +---rw oui-cid? string
48 | +---rw path-id-lan-id? lan-path-id
49 +---rw stream-split* [index]
50 | +---rw index uint32
51 | +---rw port if:interface-ref
52 | +---rw direction-out-facing dotlcb-sid-types:direction
53 | +---rw input-id* -> /dotlcb-sid:stream-identity/handle
54 | +---rw output-id* -> /dotlcb-sid:stream-identity/handle
55 +---rw autoconfiguration {auto-configuration}?
56 | +---rw sequence* [index]
57 | | +---rw index uint32
58 | | +---rw sequence-encapsulation
59 | | | +---rw (encapsulation)?

```

```

1      | |      +---: (r-tag)
2      | |      | +---rw r-tag!
3      | |      | +---ro type-number?
4      | |      | |      dot1cb-frer-types:seq-encaps-method
5      | |      | +---ro oui-cid?      string
6      | |      +---: (hsr-sequence-tag)
7      | |      | +---rw hsr-sequence-tag!
8      | |      | +---ro type-number?
9      | |      | |      dot1cb-frer-types:seq-encaps-method
10     | |      | +---ro oui-cid?      string
11     | |      +---: (prp-sequence-tag)
12     | |      | +---rw prp-sequence-tag!
13     | |      | +---ro type-number?
14     | |      | |      dot1cb-frer-types:seq-encaps-method
15     | |      | +---ro oui-cid?      string
16     | |      +---: (organization-specific)
17     | |      +---rw organization-specific!
18     | |      +---rw type-number?    int32
19     | |      +---rw oui-cid?      string
20     | +---rw receive-port*          if:interface-ref
21     | +---rw tagged?                enumeration
22     | +---rw vlan*                  dot1qt-types:vlanid
23     | +---rw recovery-port*         if:interface-ref
24     | +---rw destruction-interval?  uint64
25     | +---rw reset-interval?        uint64
26     | +---rw algorithm
27     | | +---rw (algorithm)?
28     | | +---: (vector)
29     | | | +---rw vector!
30     | | | +---ro type-number?
31     | | | |      dot1cb-frer-types:seq-rcvy-algorithm
32     | | | +---ro oui-cid?      string
33     | | +---: (match)
34     | | | +---rw match!
35     | | | +---ro type-number?
36     | | | |      dot1cb-frer-types:seq-rcvy-algorithm
37     | | | +---ro oui-cid?      string
38     | | +---: (organization-specific)
39     | | +---rw organization-specific!
40     | | +---rw type-number?    int32
41     | | +---rw oui-cid?      string
42     | +---rw history-length?      sequence-history-length
43     | +---rw create-individual?    boolean
44     | +---rw create-recovery?      boolean
45     | +---rw latent-error-detection? boolean
46     | +---rw latent-error-difference? int32
47     | +---rw latent-error-period?  uint32
48     | +---rw latent-error-reset-period? uint32
49     +---rw output* [index]
50     +---rw index                  uint32
51     +---rw port*                  if:interface-ref
52     +---rw encapsulation
53     | +---rw (encapsulation)?
54     | +---: (r-tag)
55     | | +---rw r-tag!
56     | | +---ro type-number?
57     | | |      dot1cb-frer-types:seq-encaps-method
58     | | +---ro oui-cid?      string
59     | +---: (hsr-sequence-tag)

```

```

1      |      |      +---rw hsr-sequence-tag!
2      |      |      +---ro type-number?
3      |      |      |      dotlcb-frer-types:seq-encaps-method
4      |      |      +---ro oui-cid?      string
5      |      +---: (prp-sequence-tag)
6      |      |      +---rw prp-sequence-tag!
7      |      |      +---ro type-number?
8      |      |      |      dotlcb-frer-types:seq-encaps-method
9      |      |      +---ro oui-cid?      string
10     |      +---: (organization-specific)
11     |      +---rw organization-specific!
12     |      +---rw type-number?      int32
13     |      +---rw oui-cid?      string
14     +---rw lan-path-id?      lan-path-id
15
16 augment /dotlcb-sid:stream-identity:
17   +---ro auto-configured?      boolean {auto-configuration}?
18   +---rw lan-path-id?      lan-path-id {auto-configuration}?
19 augment /if:interfaces/if:interface/if:statistics:
20   +---ro frer
21     +---ro per-port-counters
22       | +---ro rx-passed-pkts?      uint64
23       | +---ro rx-discarded-pkts?   uint64
24       | +---ro encode-errored-pkts?  uint64
25     +---ro per-port-per-stream-counters* [direction-out-facing handle]
26       +---ro direction-out-facing      dotlcb-sid-types:direction
27       +---ro handle
28       |      -> /dotlcb-sid:stream-identity/handle
29       +---ro generation-reset?      uint64
30       +---ro rx-out-of-order-pkts?   uint64
31       +---ro rx-rogue-pkts?          uint64
32       +---ro rx-passed-pkts?         uint64
33       +---ro rx-discarded-pkts?      uint64
34       +---ro rx-lost-pkts?           uint64
35       +---ro rx-tagless-pkts?        uint64
36       +---ro rx-resets?              uint64
37       +---ro rx-latent-error-resets?  uint64
38       +---ro encode-errored-pkts?    uint64
39

```

40 12.6.2 YANG data module definitions

41 12.6.2.1 Definition for the ieee802-dot1cb-stream-identification-types YANG module

```

42 module ieee802-dot1cb-stream-identification-types {
43   yang-version "1.1";
44   namespace
45     urn:ieee:std:802.1Q:yang:ieee802-dot1cb-stream-identification-types;
46   prefix dotlcb-sid-types;
47   organization
48     "Institute of Electrical and Electronics Engineers";
49   contact
50     "WG-URL: http://ieee802.org/1/
51     WG-EMail: stds-802-1-1@ieee.org
52
53     Contact: IEEE 802.1 Working Group Chair
54     Postal: C/O IEEE 802.1 Working Group
55     IEEE Standards Association

```

```

1      445 Hoes Lane
2      Piscataway, NJ 08854
3      USA
4
5      E-mail: stds-802-1-chairs@ieee.org";
6  description
7      "Management objects that control the stream identification from IEEE
8      Std 802.1CB-2017. This YANG data model conforms to the Network
9      Management Datastore Architecture defined in RFC 8342."; Copyright
10     (C)IEEE (2021). This version of this YANG module is part of IEEE Std
11     802.1CBdb-2021; see the draft itself for full legal notices.
12 revision 2021-06-14 {
13     description
14         "Published as part of IEEE Std 802.1CBdb-2021.
15         Added the stream identification type used for the Mask-and-match
16         identification method ";
17     reference
18         "IEEE Std 802.1CBdb-2021, Frame Replication and Elimination for
19         Reliability - Extended Stream identification functions.";
20 }
21 revision 2021-05-06 {
22     description
23         "Published as part of IEEE Std 802.1CBcv-2021. Initial version.";
24     reference
25         "IEEE Std 802.1CBcv-2021, Frame Replication and Elimination for
26         Reliability - FRER YANG Data Model and Management Information Base
27         Module.";
28 }
29 identity strid-idty {
30     description
31         "Root identity for all stream identification types";
32 }
33 typedef direction {
34     type boolean;
35     description
36         "A boolean object indicating whether the direction is out-facing
37         (True) or in-facing (False).";
38     reference
39         "10.4.1.3 of IEEE Std 802.1CB-2017";
40 }
41 typedef stream-id-function {
42     type enumeration {
43         enum reserved {
44             value 0;
45             description
46                 "Reserved value.";
47         }
48         enum null-stream {
49             value 1;
50             description
51                 "The stream identification type used for the Null Stream
52                 identification method.";
53         }
54         enum smac-vlan {
55             value 2;
56             description
57                 "The stream identification type used for the Source MAC and
58                 VLAN Stream identification method.";
59         }
60     }

```

```

1      enum dmac-vlan {
2          value 3;
3          description
4              "The stream identification type used for the Active Destination
5              MAC and VLAN Stream identification method.";
6      }
7      enum ip {
8          value 4;
9          description
10         "The stream identification type used for the IP Stream
11         identification method.";
12     }
13     enum mask-and-match {
14         value 5;
15         description
16             "The stream identification type used for the Mask-and-match
17             identification method.";
18     }
19 }
20 description
21     "An enumerated value indicating the method used to identify packets
22     belonging to a Stream.";
23 reference
24     "9.1.1.6 of IEEE Std 802.1CBdb-2021";
25 }
26 }
27

```

28 12.6.2.2 Definition for the ieee802-dot1cb-stream-identification YANG module

```

29 module ieee802-dot1cb-stream-identification {
30     yang-version "1.1";
31     namespace urn:ieee:std:802.1Q:yang:ieee802-dot1cb-stream-identification;
32     prefix dot1cb-sid;
33     import ieee802-types {
34         prefix ieee;
35     }
36     import ieee802-dot1q-types {
37         prefix dot1qtypes;
38     }
39     import ietf-inet-types {
40         prefix inet;
41     }
42     import ietf-interfaces {
43         prefix if;
44     }
45     import ieee802-dot1cb-stream-identification-types {
46         prefix dot1cb-sid-types;
47     }
48     organization
49         "Institute of Electrical and Electronics Engineers";
50     contact
51         "WG-URL: http://ieee802.org/1/
52         WG-EMail: stds-802-1-1@ieee.org
53
54         Contact: IEEE 802.1 Working Group Chair
55         Postal: C/O IEEE 802.1 Working Group
56             IEEE Standards Association

```

```

1          445 Hoes Lane
2          Piscataway, NJ 08854
3          USA
4
5      E-mail: stds-802-1-chairs@ieee.org";
6  description
7      "Management objects that control the Stream identification from IEEE
8      Std 802.1CB-2017. This YANG data model conforms to the Network
9      Management Datastore Architecture defined in RFC 8342. Copyright (C)
10     IEEE (2021). This version of this YANG module is part of IEEE Std
11     802.1CBcv; see the standard itself for full legal notices.";
12 revision 2025-10-16 {
13     description
14     "Published as part of IEEE Std 802.1CB-2026.
15     Introduced a wildcard to ignore the destination address and IP source
16     address in the IP Stream identification method.";
17     reference
18     "IEEE Std 802.1CB-2026, Frame Replication and Elimination for
19     Reliability - Extended Stream identification functions.";
20 }
21 revision 2021-12-08 {
22     description
23         "Published as part of IEEE Std 802.1CBcv-2021. Initial version.";
24     reference
25         "IEEE Std 802.1CBcv-2021, Frame Replication and Elimination for
26         Reliability - FRER YANG Data Model and Management Information Base
27         Module.";
28 }
29 identity null-stream-identification {
30     base dot1cb-sid-types:strid-idty;
31     description
32         "Null Stream identification";
33 }
34 identity smac-vlan-stream-identification {
35     base dot1cb-sid-types:strid-idty;
36     description
37         "Source MAC and VLAN Stream identification";
38 }
39 identity dmac-vlan-stream-identification {
40     base dot1cb-sid-types:strid-idty;
41     description
42         "Active Destination MAC and VLAN Stream identification";
43 }
44 identity ip-stream-identification {
45     base dot1cb-sid-types:strid-idty;
46     description
47         "IP Stream identification";
48 }
49 typedef vlan-tag-identification-type {
50     type enumeration {
51         enum tagged {
52             value 1;
53             description
54                 "A frame must have a VLAN tag to be recognized as belonging to
55                 the Stream.";
56         }
57         enum priority {
58             value 2;
59             description

```

```
1      "A frame must be untagged, or have a VLAN tag with a VLAN ID =
2      0 to be recognized as belonging to the Stream.";
3  }
4  enum all {
5      value 3;
6      description
7          "A frame is recognized as belonging to the Stream whether
8          tagged or not.";
9  }
10 }
11 description
12     "Enumeration describing how a Stream can be identified using the
13     VLAN tag.";
14 }
15 typedef vlan-identifier-type {
16     type uint16 {
17         range "0 .. 4095";
18     }
19     description
20         "Specifies the vlan_identifier. A value of 0 indicates that the
21         vlan_identifier carries a special meaning.";
22 }
23 list stream-identity {
24     key "index";
25     description
26         "The Stream identity table consists of a set of tsnStreamIdEntry
27         objects, each relating to a single Stream, specifying the points in
28         the system where Stream identification functions are to be
29         instantiated. Each entry in the Stream identity table has a
30         tsnStreamIdHandle object specifying a stream_handle value and one
31         or more tsnStreamIdEntry objects describing one identification
32         method for that Stream. If a single Stream has multiple
33         identification methods, perhaps (but not necessarily) on different
34         ports, then there can be multiple tsnStreamIdEntry objects with the
35         same value for the tsnStreamIdHandle. If the HSR or PRP method or
36         the Sequence encode/decode function is applied to a packet, then
37         the LanId or PathId fields are also used to identify the Stream to
38         which the packet belongs.";
39     reference
40         "9.1. of IEEE Std 802.1CB-2017";
41     leaf index {
42         type uint32;
43         description
44             "If a single Stream has multiple identification methods, perhaps
45             (but not necessarily) on different ports, then there can be
46             multiple tsnStreamIdEntry objects with the same value for the
47             tsnStreamIdHandle";
48     }
49     leaf handle {
50         type uint32;
51         mandatory true;
52         description
53             "The objects in a given entry of the Stream identity table are
54             used to control packets whose stream_handle subparameter is equal
55             to the entry's tsnStreamIdHandle object. The specific values used
56             in the tsnStreamIdHandle object are not necessarily used in the
57             system; they are used only to relate the various management
58             objects in Clause 9 and Clause 10.";
59         reference
```

```
1      "9.1.1.1 of IEEE Std 802.1CB-2017";
2  }
3  container in-facing {
4      description
5          "Container for in-facing Stream identification functions.";
6      leaf-list input-port {
7          type if:interface-ref;
8          description
9              "The list of ports on which an in-facing Stream identification
10             function using this identification method is to be placed for
11             this Stream in the input (coming from the system forwarding
12             function) direction. Any number of tsnStreamIdEntry objects can
13             list the same port for the same tsnStreamIdHandle in its
14             tsnStreamIdInFacInputPortList.";
15          reference
16              "9.1.1.4 of IEEE Std 802.1CB-2017";
17      }
18      leaf-list output-port {
19          type if:interface-ref;
20          description
21              "The list of ports on which an in-facing Stream identification
22             function using this identification method is to be placed for
23             this Stream in the output (towards the system forwarding
24             function) direction. At most one tsnStreamIdEntry can list a
25             given port for a given tsnStreamIdHandle in its
26             tsnStreamIdInFacOutputPortList.";
27          reference
28              "9.1.1.2 of IEEE Std 802.1CB-2017";
29      }
30  }
31  container out-facing {
32      description
33          "Container for out-facing Stream identification functions.";
34      leaf-list input-port {
35          type if:interface-ref;
36          description
37              "The list of ports on which an out-facing Stream identification
38             function using this identification method is to be placed for
39             this Stream in the input (coming from the physical interface)
40             direction. Any number of tsnStreamIdEntry objects can list the
41             same port for the same tsnStreamIdHandle in its
42             tsnStreamIdOutFacInputPortList.";
43          reference
44              "9.1.1.5 of IEEE Std 802.1CB-2017";
45      }
46      leaf-list output-port {
47          type if:interface-ref;
48          description
49              "The list of ports on which an out-facing Stream identification
50             function using this identification method is to be placed for
51             this Stream in the output (towards the physical interface)
52             direction. At most one tsnStreamIdEntry can list a given port
53             for a given tsnStreamIdHandle in its
54             tsnStreamIdOutFacOutputPortList.";
55          reference
56              "9.1.1.3 of IEEE Std 802.1CB-2017";
57      }
58  }
59  choice parameters {
```



```

1      mandatory true;
2      description
3          "The number of controlling parameters for a Stream identification
4          method, their types and values, are specific to the
5          tsnStreamIdIdentificationType.";
6      reference
7          "9.1.1.7 of IEEE Std 802.1CB-2017";
8      container null-stream-identification {
9          description
10             "When instantiating an instance of the Null Stream
11             identification function for a particular input Stream, the
12             managed objects in this container serve as the
13             tsnStreamIdParameters managed object.";
14         reference
15             "9.1.2 of IEEE Std 802.1CB-2017";
16         container identification-type {
17             config false;
18             description
19                 "The identification type indicating the method used to
20                 identify packets belonging to the Stream. The identification
21                 type contains a type number and an Organizationally Unique
22                 Identifier (OUI) or Company ID (CID) to identify the
23                 organization defining the identification method.";
24             reference
25                 "9.1.1.6 of IEEE Std 802.1CB-2017";
26             leaf type-number {
27                 type dot1cb-sid-types:stream-id-function;
28                 default "null-stream";
29                 description
30                     "The Stream identification type used for the Null Stream
31                     identification method.";
32                 reference
33                     "9.1.1.6 of IEEE Std 802.1CB-2017";
34             }
35             leaf oui-cid {
36                 type string {
37                     pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
38                 }
39                 default "00-80-C2";
40                 description
41                     "The Organizationally Unique Identifier (OUI) or Company ID
42                     (CID) to identify the organization defining the
43                     identification method. For identification methods defined
44                     in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
45                 reference
46                     "9.1.1.6 of IEEE Std 802.1CB-2017";
47             }
48         }
49         leaf destination-mac {
50             type ieee:mac-address;
51             description
52                 "Specifies the destination_address that identifies a packet
53                 in an EISS indication primitive, to the Null Stream
54                 identification function. The ieee:mac-address type has a
55                 pattern that allows upper and lower case letters. To avoid
56                 issues with string comparison, it is suggested to only use
57                 upper case for the letters in the hexadecimal numbers. There
58                 is still an issue with a difference between the IETF
59                 mac-address definition and the IEEE mac-address definition,

```

```

1         so consider that if implementing code that compares
2         mac-addresses.";
3     reference
4         "9.1.2.1 of IEEE Std 802.1CB-2017";
5 }
6 leaf tagged {
7     type vlan-tag-identification-type;
8     description
9         "An enumerated value indicating whether a packet in an EISS
10        indication primitive to the Null Stream identification
11        function is permitted to have a VLAN tag.";
12    reference
13        "9.1.2.2 of IEEE Std 802.1CB-2017";
14 }
15 leaf vlan {
16     type vlan-identifier-type;
17     description
18         "Specifies the vlan_identifier parameter that identifies a
19        packet in an EISS indication primitive to the Null Stream
20        identification function. A value of 0 indicates that the
21        vlan_identifier parameter is ignored on EISS indication
22        primitives.";
23    reference
24        "9.1.2.3 of IEEE Std 802.1CB-2017";
25 }
26 }
27 container smac-vlan-stream-identification {
28     description
29         "When instantiating an instance of the Source MAC and VLAN
30        Stream identification function for a particular input Stream,
31        the managed objects in the following subclauses serve as the
32        tsnStreamIdParameters managed object.";
33    reference
34        "9.1.3 of IEEE Std 802.1CB-2017";
35    container identification-type {
36        config false;
37        description
38            "The identification type indicating the method used to
39            identify packets belonging to the Stream. The identification
40            type contains a type number and an Organizationally Unique
41            Identifier (OUI) or Company ID (CID) to identify the
42            organization defining the identification method.";
43        reference
44            "9.1.1.6 of IEEE Std 802.1CB-2017";
45        leaf type-number {
46            type dot1cb-sid-types:stream-id-function;
47            default "smac-vlan";
48            description
49                "The Stream identification type used for the Source MAC and
50                VLAN Stream identification method.";
51            reference
52                "9.1.1.6 of IEEE Std 802.1CB-2017";
53        }
54        leaf oui-cid {
55            type string {
56                pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
57            }
58            default "00-80-C2";
59            description

```

```

1         "The Organizationally Unique Identifier (OUI) or Company ID
2         (CID) to identify the organization defining the
3         identification method. For identification methods defined
4         in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
5     reference
6         "9.1.1.6 of IEEE Std 802.1CB-2017";
7 }
8 }
9 leaf source-mac {
10     type ieee:mac-address;
11     description
12         "Specifies the source_address that identifies a packet in an
13         EISS indication primitive, to the Source MAC and VLAN Stream
14         identification function. The ieee:mac-address type has a
15         pattern that allows upper and lower case letters. To avoid
16         issues with string comparison, it is suggested to only use
17         upper case for the letters in the hexadecimal numbers. There
18         is still an issue with a difference between the IETF
19         mac-address definition and the IEEE mac-address definition,
20         so consider that if implementing code that compares
21         mac-addresses.";
22     reference
23         "9.1.3.1 of IEEE Std 802.1CB-2017";
24 }
25 leaf tagged {
26     type vlan-tag-identification-type;
27     description
28         "An enumerated value indicating whether a packet in an EISS
29         indication primitive to the Source MAC and VLAN Stream
30         identification function is permitted to have a VLAN tag.";
31     reference
32         "9.1.3.2 of IEEE Std 802.1CB-2017";
33 }
34 leaf vlan {
35     type vlan-identifier-type;
36     description
37         "Specifies the vlan_identifier parameter that identifies a
38         packet in an EISS indication primitive to the Source MAC and
39         VLAN Stream identification function. A value of 0 indicates
40         that the vlan_identifier parameter is ignored on EISS
41         indication primitives.";
42     reference
43         "9.1.3.3 of IEEE Std 802.1CB-2017";
44 }
45 }
46 container dmac-vlan-stream-identification {
47     description
48         "When instantiating an instance of the Active Destination MAC
49         and VLAN Stream identification function for a particular output
50         Stream, the managed objects in the following subclauses, along
51         with those listed in 9.1.2, serve as the tsnStreamIdParameters
52         managed object.";
53     reference
54         "9.1.4 of IEEE Std 802.1CB-2017";
55     container identification-type {
56         config false;
57         description
58             "The identification type indicating the method used to
59             identify packets belonging to the Stream. The identification

```

```

1         type contains a type number and an Organizationally Unique
2         Identifier (OUI) or Company ID (CID) to identify the
3         organization defining the identification method.";
4     reference
5         "9.1.1.6 of IEEE Std 802.1CB-2017";
6     leaf type-number {
7         type dot1cb-sid-types:stream-id-function;
8         default "dmac-vlan";
9         description
10            "The Stream identification type used for the Active
11            Destination MAC and VLAN Stream identification method.";
12        reference
13            "9.1.1.6 of IEEE Std 802.1CB-2017";
14    }
15    leaf oui-cid {
16        type string {
17            pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
18        }
19        default "00-80-C2";
20        description
21            "The Organizationally Unique Identifier (OUI) or Company ID
22            (CID) to identify the organization defining the
23            identification method. For identification methods defined
24            in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
25        reference
26            "9.1.1.6 of IEEE Std 802.1CB-2017";
27    }
28 }
29 container down {
30     description
31         "Container for all parameters which are sent to lower layers.";
32     leaf destination-mac {
33         type ieee:mac-address;
34         description
35             "Specifies the destination_address parameter to use in the
36             EISS request primitive for output packets sent to lower
37             layers by the Active Destination MAC and VLAN Stream
38             identification function, and the destination_address that
39             identifies an input packet in an EISS indication primitive
40             to the Active Destination MAC and VLAN Stream
41             identification function. The ieee:mac-address type has a
42             pattern that allows upper and lower case letters. To avoid
43             issues with string comparison, it is suggested to only use
44             upper case for the letters in the hexadecimal numbers.
45             There is still an issue with a difference between the IETF
46             mac-address definition and the IEEE mac-address definition,
47             so consider that if implementing code that compares
48             mac-addresses.";
49         reference
50             "9.1.4.1 of IEEE Std 802.1CB-2017";
51     }
52     leaf tagged {
53         type vlan-tag-identification-type;
54         description
55             "An enumerated value indicating whether a packet in an EISS
56             indication or request primitive between the Active
57             Destination MAC and VLAN Stream identification function and
58             the lower layers is to have a VLAN tag. This variable is
59             not used in an FRER C-component. See 8.4.";

```

```

1         reference
2         "9.1.4.2 of IEEE Std 802.1CB-2017";
3     }
4     leaf vlan {
5         type vlan-identifier-type;
6         description
7             "Specifies the vlan_identifier parameter to use in the EISS
8             request primitive for output packets sent to lower layers
9             by the Active Destination MAC and VLAN Stream
10            identification function, and the vlan_identifier that
11            identifies an input packet in an EISS indication primitive
12            to the Active Destination MAC and VLAN Stream
13            identification function. A value of 0 indicates that the
14            vlan_identifier parameter is ignored on EISS indication
15            primitives.";
16         reference
17         "9.1.4.3 of IEEE Std 802.1CB-2017";
18     }
19     leaf priority {
20         type dot1qtypes:priority-type;
21         description
22             "Specifies the priority parameter to use in the EISS
23             request primitive for output packets sent to lower layers
24             by the Active Destination MAC and VLAN Stream
25             identification function for all packets in a particular
26             Stream.";
27         reference
28         "9.1.4.4 of IEEE Std 802.1CB-2017";
29     }
30 }
31 container up {
32     description
33         "Container for all parameters which are offered to higher
34         layers.";
35     leaf destination-mac {
36         type ieee:mac-address;
37         description
38             "Specifies the destination_address parameter to use in the
39             EISS indication primitive for input packets offered to
40             upper layers by the Active Destination MAC and VLAN Stream
41             identification layer. This address replaces the address
42             that was used to identify the packet
43             (tsnCpeDmacVlanDownDestMac). The ieee:mac-address type has
44             a pattern that allows upper and lower case letters. To
45             avoid issues with string comparison, it is suggested to
46             only use upper case for the letters in the hexadecimal
47             numbers. There is still an issue with a difference between
48             the IETF mac-address definition and the IEEE mac-address
49             definition, so consider that if implementing code that
50             compares mac-addresses.";
51         reference
52         "9.1.4.5 of IEEE Std 802.1CB-2017";
53     }
54     leaf tagged {
55         type vlan-tag-identification-type;
56         description
57             "An enumerated value indicating whether a packet in an EISS
58             indication or request primitive between the Active
59             Destination MAC and VLAN Stream identification function and

```

```

1         the upper layers is to have a VLAN tag. This variable is
2         used only by an end system and not by a relay system.";
3     reference
4         "9.1.4.6 of IEEE Std 802.1CB-2017";
5 }
6 leaf vlan {
7     type vlan-identifier-type;
8     description
9         "Specifies the vlan_identifier parameter to use in the EISS
10        indication primitive for packets offered to upper layers,
11        or the VLAN ID field for an IEEE 802.1Q tag in an ISS
12        mac_service_data_unit. This address replaces the VLAN ID
13        that was used to identify the packet
14        (tsnCpeDmacVlanDownVlan).";
15     reference
16         "9.1.4.7 of IEEE Std 802.1CB-2017";
17 }
18 leaf priority {
19     type dot1qtypes:priority-type;
20     description
21         "Specifies the priority parameter to use in the EISS
22        indication primitive for packets offered to upper layers.";
23     reference
24         "9.1.4.8 of IEEE Std 802.1CB-2017";
25 }
26 }
27 }
28 container ip-stream-identification {
29     description
30         "When instantiating an instance of the IP Stream identification
31        function, the parameters in the following subclauses replace
32        the tsnStreamIdParameters managed object.";
33     reference
34         "9.1.5 of IEEE Std 802.1CB-2017";
35     container identification-type {
36         config false;
37         description
38             "The identification type indicating the method used to
39            identify packets belonging to the Stream. The identification
40            type contains a type number and an Organizationally Unique
41            Identifier (OUI) or Company ID (CID) to identify the
42            organization defining the identification method.";
43         reference
44             "9.1.1.6 of IEEE Std 802.1CB-2017";
45         leaf type-number {
46             type dot1cb-sid-types:stream-id-function;
47             default "ip";
48             description
49                 "The Stream identification type used for the IP Stream
50                identification method.";
51             reference
52                 "9.1.1.6 of IEEE Std 802.1CB-2017";
53         }
54         leaf oui-cid {
55             type string {
56                 pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
57             }
58             default "00-80-C2";
59             description

```

```

1         "The Organizationally Unique Identifier (OUI) or Company ID
2         (CID) to identify the organization defining the
3         identification method. For identification methods defined
4         in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
5     reference
6         "9.1.1.6 of IEEE Std 802.1CB-2017";
7     }
8 }
9     choice destination-mac {
10         case mac-address {
11             leaf mac-address {
12                 type ieee:mac-address;
13                 description
14                     "Specifies the destination address parameter that identifies
15                     a packet in an EISS indication primitive. The
16                     ieee:mac-address type has a pattern that allows upper and
17                     lower case letters. To avoid issues with string comparison,
18                     it is suggested to only use upper case for the letters in the
19                     hexadecimal numbers. There is still an issue with a
20                     difference between the IETF mac-address definition and the
21                     IEEE mac-address definition, so consider that if implementing
22                     code that compares mac-addresses.";
23                 reference
24                     "9.1.5.1 of IEEE Std 802.1CB-2017";
25             }
26         }
27         case wildcard {
28             leaf wildcard {
29                 type empty;
30                 description
31                     "wildcard";
32             }
33         }
34     }
35     leaf tagged {
36         type vlan-tag-identification-type;
37         description
38             "An enumerated value indicating whether a packet in an EISS
39             indication or request primitive to the IP Stream
40             identification function is to have a VLAN tag.";
41         reference
42             "9.1.5.2 of IEEE Std 802.1CB-2017";
43     }
44     leaf vlan {
45         type vlan-identifier-type;
46         description
47             "Specifies the vlan_identifier parameter that identifies a
48             packet in an EISS indication primitive. A value of 0
49             indicates that the frame is not to have a VLAN tag.";
50         reference
51             "9.1.5.3 of IEEE Std 802.1CB-2017";
52     }
53     choice ip-source {
54         case leaf ip-source {
55             leaf ip-source {
56                 type inet:ip-address;
57                 description
58                     "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) source
59                     address parameter that must be matched to identify packets

```

```

1         coming up from lower layers. An address of all 0 indicates
2         that the IP source address is to be ignored on packets
3         received from lower layers.";
4         reference
5         "9.1.5.4 of IEEE Std 802.1CB-2017";
6     }
7 }
8     case wildcard {
9         leaf wildcard {
10            type empty;
11            description
12            "wildcard";
13        }
14    }
15 }
16     leaf ip-destination {
17         type inet:ip-address;
18         description
19         "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460) destination
20         address parameter that must be matched to identify packets
21         coming up from lower layers.";
22         reference
23         "9.1.5.5 of IEEE Std 802.1CB-2017";
24     }
25     leaf dscp {
26         type inet:dscp;
27         description
28         "Specifies the IPv4 (RFC 791) or IPv6 (RFC 2460)
29         differentiated services codepoint (DSCP, RFC 2474) that must
30         be matched to identify packets coming up from the lower
31         layers. A value of 64 decimal indicates that the DSCP is to
32         be ignored on packets received from lower layers.";
33         reference
34         "9.1.5.6 of IEEE Std 802.1CB-2017";
35     }
36     leaf next-protocol {
37         type enumeration {
38             enum none {
39                 description
40                 "No protocol is specified";
41             }
42             enum udp {
43                 description
44                 "UDP is specified as the next protocol.";
45                 reference
46                 "RFC 768";
47             }
48             enum tcp {
49                 description
50                 "TCP is specified as the next protocol.";
51                 reference
52                 "RFC 793";
53             }
54             enum sctp {
55                 description
56                 "SCTP is specified as the next protocol.";
57                 reference
58                 "RFC 4960";
59             }
60         }
61     }

```



```
1      }
2      description
3          "Specifies the IP next protocol parameter that must be
4          matched to identify packets coming up from lower layers. The
5          value of this parameter must specify either none, UDP (RFC
6          768), TCP (RFC 793), or SCTP (RFC 4960). If "none," then the
7          tsnCpeIpIdSourcePort and tsnCpeIpIdDestinationPort managed
8          objects are not used.";
9      reference
10         "9.1.5.7 of IEEE Std 802.1CB-2017";
11    }
12    leaf source-port {
13        type inet:port-number;
14        description
15            "Specifies the TCP or UDP Source Port parameter that must be
16            matched to identify packets coming up from lower layers. A
17            value of 0 indicates that the Source Port number of the
18            packet is to be ignored on packets received from lower
19            layers.";
20        reference
21            "9.1.5.8 of IEEE Std 802.1CB-2017";
22    }
23    leaf destination-port {
24        type inet:port-number;
25        description
26            "Specifies the TCP or UDP Destination Port parameter that
27            must be matched to identify packets coming up from lower
28            layers. A value of 0 indicates that the Destination Port
29            number of the packet is to be ignored on packets received
30            from lower layers.";
31        reference
32            "9.1.5.9 of IEEE Std 802.1CB-2017";
33    }
34 }
35 container organization-specific {
36     description
37         "This container allows to select Stream identification methods
38         that are defined by entities outside of IEEE 802.1.";
39     reference
40         "9.1.1.6 of IEEE Std 802.1CB-2017";
41     container identification-type {
42         description
43             "The identification type indicating the method used to
44             identify packets belonging to the Stream. The identification
45             type contains a type number and an Organizationally Unique
46             Identifier (OUI) or Company ID (CID) to identify the
47             organization defining the identification method.";
48         reference
49             "9.1.1.6 of IEEE Std 802.1CB-2017";
50         leaf type-number {
51             type int32 {
52                 range "256..max";
53             }
54             description
55                 "The type number used for an identification method defined
56                 by an entity owning the OUI or CID for this identification
57                 type.";
58             reference
59                 "9.1.1.6 of IEEE Std 802.1CB-2017";
```

```
1      }
2      leaf oui-cid {
3          type string {
4              pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
5          }
6          description
7              "The Organizationally Unique Identifier (OUI) or Company ID
8              (CID) to identify the organization defining the
9              identification method.";
10         reference
11             "9.1.1.6 of IEEE Std 802.1CB-2017";
12     }
13 }
14 }
15 }
16 }
17 augment "/if:interfaces/if:interface/if:statistics" {
18     description
19         "The following counters are the counters for Stream identification.
20         All counters are unsigned integers. If used on links faster than
21         650 000 000 bits per second, they shall be 64 bits in length to
22         ensure against excessively short wrap times.";
23     reference
24         "9.2 of IEEE Std 802.1CB-2017
25         9.3 of IEEE Std 802.1CB-2017";
26     container stream-id {
27         description
28             "This container contains the per-port as well as the
29             per-port-per-stream counters for Stream identification.";
30         reference
31             "9.2 of IEEE Std 802.1CB-2017
32             9.3 of IEEE Std 802.1CB-2017";
33         container per-port-counters {
34             config false;
35             description
36                 "Contains the per-port counters for Stream identification. The
37                 following counters are instantiated for each port on which the
38                 Stream identification function is configured. The counters are
39                 indexed by port number.";
40             reference
41                 "9.3 of IEEE Std 802.1CB-2017";
42             leaf input-pkts {
43                 type uint64;
44                 config false;
45                 description
46                     "The tsnCpSidInputPackets counter is incremented once for
47                     each packet identified by any Stream identification function
48                     on this port. Its value equals the sum (modulo the size of
49                     the counters) of all of the tsnCpsSidInputPackets counters on
50                     this same port.";
51                 reference
52                     "9.3.1 of IEEE Std 802.1CB-2017";
53             }
54             leaf output-pkts {
55                 type uint64;
56                 config false;
57                 description
58                     "The tsnCpSidOutputPackets counter is incremented once for
59                     each packet passed down the stack by any Stream
```

```

1         identification function on this port. Its value equals the
2         sum (modulo the size of the counters) of all of the
3         tsnCpsSidOutputPackets counters on this same port.";
4     reference
5         "9.3.2 of IEEE Std 802.1CB-2017";
6 }
7 }
8 list per-port-per-stream-counters {
9     key "direction-out-facing handle";
10    config false;
11    description
12        "Contains the per-port-per-stream counters for Stream
13        identification. The following counters are instantiated for
14        each port on which the Stream identification function is
15        configured. The counters are indexed by port number, facing
16        (in-facing or out-facing), and stream_handle value
17        (tsnStreamIdHandle).";
18    reference
19        "9.2 of IEEE Std 802.1CB-2017";
20    leaf direction-out-facing {
21        type dot1cb-sid-types:direction;
22        description
23            "An object indicating whether the counters apply to
24            out-facing (True) or in-facing (False).";
25    }
26    leaf handle {
27        type leafref {
28            path '/stream-identity/handle';
29        }
30        description
31            "The according tsnStreamIdHandle for these counters.";
32    }
33    leaf input-pkts {
34        type uint64;
35        description
36            "The tsnCpsSidInputPackets counter is incremented once for
37            each packet identified by the Stream identification function.";
38        reference
39            "9.2.1 of IEEE Std 802.1CB-2017";
40    }
41    leaf output-pkts {
42        type uint64;
43        description
44            "The tsnCpsSidOutputPackets counter is incremented once for
45            each packet passed down the stack by the Stream
46            identification function.";
47        reference
48            "9.2.2 of IEEE Std 802.1CB-2017";
49    }
50 }
51 }
52 }
53 }
54

```

55 12.6.2.3 Definition for the ieee802-dot1cb-mask-and-match module

```

56 module ieee802-dot1cb-mask-and-match {

```

```
1 yang-version 1.1;
2 namespace "urn:ieee:std:802.1CB:yang:ieee802-dot1cb-mask-and-match";
3 prefix dot1cb-mm-sid;
4
5 import ieee802-dot1cb-stream-identification {
6     prefix dot1cb-sid;
7 }
8
9 import ieee802-dot1cb-stream-identification-types {
10     prefix dot1cb-sid-types;
11 }
12
13 import ieee802-types {
14     prefix ieee802;
15 }
16
17 import ietf-interfaces {
18     prefix if;
19 }
20
21 organization
22     "Institute of Electrical and Electronics Engineers";
23 contact
24     "WG-URL: http://ieee802.org/1/
25     WG-EMail: stds-802-1-1@ieee.org
26
27     Contact: IEEE 802.1 Working Group Chair
28     Postal: C/O IEEE 802.1 Working Group
29             IEEE Standards Association
30             445 Hoes Lane
31             Piscataway, NJ 08854
32             USA
33
34     E-mail: stds-802-1-chairs@ieee.org";
35
36 description
37     "Management objects that control the Mask-and-match
38     stream identification function from IEEE Std 802.1CBdb-2021.
39     Augmentation to the ieee802-dot1cb-stream-identification module.      This
40 YANG data model conforms to the Network Management Datastore
41 Architecture defined in RFC 8342. Copyright (C) IEEE (2021). This
42 version of this YANG module is part of IEEE Std 802.1CBdb-2021;
43 see the draft itself for full legal notices.";
44
45 revision 2021-06-14 {
46     description
47         "Published as part of IEEE Std 802.1CBdb-2021.
48         Initial version.";
49     reference
50         "IEEE Std 802.1CBdb-2021, Frame Replication and Elimination for
51         Reliability - Extended Stream identification functions.";
52 }
53
54 typedef msdu-mask-match-hex-string {
55     type string {
56         pattern '[0-9a-fA-F]{2}(-[0-9a-fA-F]{2}){1,1983}';
57     }
58     description
59         "Hex string representing a series of 2 to 1984 octets, each
```

```
1     separated by a '-' separator";
2 }
3
4 typedef msdu-mask-length-type {
5     type uint16 {
6         range "2..1984";
7     }
8     description
9         "unsigned integer between 2 and 1984";
10 }
11
12 identity mask-and-match-stream-identification {
13     base dot1cb-sid-types:strid-idty;
14     description
15         "Mask-and-match Stream Identification";
16 }
17
18 augment "/dot1cb-sid:stream-identity/dot1cb-sid:parameters" {
19     description
20         "Controlling parameters for the Mask-and-match Stream
21         identification method";
22     container mask-and-match-stream-identification {
23         description
24             "When instantiating an instance of the Mask-and-match Stream
25             identification function (6.8), the parameters in the following
26             subclauses replace the tsnStreamIdParameters managed object
27             (9.1.1.7).";
28         reference
29             "9.1.6 of IEEE Std 802.1CBdb-2021";
30         container identification-type {
31             config false;
32             description
33                 "The identification type indicating the method used to
34                 identify packets belonging to the Stream. The
35                 identification type contains a type number and an
36                 Organizationally Unique Identifier (OUI) or Company ID
37                 (CID) to identify the organization defining the
38                 identification method.";
39             reference
40                 "9.1.1.6 of IEEE Std 802.1CB-2017";
41             leaf type-number {
42                 type dot1cb-sid-types:stream-id-function;
43                 default "mask-and-match";
44                 description
45                     "The stream identification type used for the Mask-and-match
46                     Stream identification method.";
47                 reference
48                     "9.1.1.6 of IEEE Std 802.1CBdb-2021";
49             }
50             leaf oui-cid {
51                 type string {
52                     pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
53                 }
54                 default "00-80-C2";
55                 description
56                     "The Organizationally Unique Identifier (OUI) or Company ID
57                     (CID) to identify the organization defining the
58                     identification method. For identification methods defined
59                     in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
```

```
1         reference
2         "9.1.1.6 of IEEE Std 802.1CB-2017";
3     }
4 }
5 leaf destination-mac-mask {
6     type ieee802:mac-address;
7     description
8         "Specifies a 48-bit mask. A bitwise AND operation is performed
9         between destination-mac-mask and the
10        destination_address_parameter passed by the ISS indication
11        primitive to the Mask-and-match Stream identification
12        function. The resulting 48-bit information is the masked
13        destination_address that is used as input for the instance of
14        the Mask-and-match Stream identification function. If
15        destination-mac-mask has a value of 0, the destination-address
16        parameter is ignored.";
17    reference
18        "9.1.6.1 of IEEE Std 802.1CBdb-2021";
19 }
20 leaf destination-mac-match {
21     when '../destination-mac-mask';
22     type ieee802:mac-address;
23     description
24         "Specifies the 48-bit value of the masked destination_address,
25         to be matched by the instance of the Mask-and-match Stream
26         identification function.";
27     reference
28         "9.1.6.2 of IEEE Std 802.1CBdb-2021";
29 }
30 leaf source-mac-mask {
31     type ieee802:mac-address;
32     description
33         "Specifies a 48-bit mask. A bitwise AND operation is performed
34         between source-mac-mask and the source_address_parameter
35         passed by the ISS indication primitive to the Mask-and-match
36         Stream identification function. The resulting 48-bit
37         information is the masked source_address that is used as input
38         for the instance of the Mask-and-match Stream identification
39         function. If source-mac-mask has a value of 0, the
40         source-address parameter is ignored.";
41     reference
42         "9.1.6.3 of IEEE Std 802.1CBdb-2021";
43 }
44 leaf source-mac-match {
45     when '../source-mac-mask';
46     type ieee802:mac-address;
47     description
48         "Specifies the 48-bit value of the masked source_address, to be
49         matched by the instance of the Mask-and-match Stream
50         identification function.";
51     reference
52         "9.1.6.4 of IEEE Std 802.1CBdb-2021";
53 }
54 leaf msdu-mask-length {
55     type msdu-mask-length-type;
56     description
57         "Integer value that specifies the length, in octets, of
58         tsnCpeMmIdMsduMask (9.1.6.6) and tsnCpeMmIdMsduMatch
59         (9.1.6.7). The behavior of the Mask-and-match Stream
```

```

1      identification function is undefined on any port where the
2      value of tsnCpeMmIdMsduMaskLength exceeds the per-port
3      mac_service_data_unit mask maximum length (9.4.1).
4      tsnCpeMmIdMsduMaskLength has a minimum value of 2. If the
5      length of the frame's mac_service_data_unit is shorter than
6      tsnCpeMmIdMsduMaskLength, then the identification fails.";
7  reference
8      "9.1.6.5 of IEEE Std 802.1CBdb-2021";
9  }
10 leaf msdu-mask {
11     type msdu-mask-match-hex-string;
12     description
13         "Specifies a mask of msdu-mask-length bytes. A bitwise AND
14         operation is performed between the msdu-mask-length first
15         bytes of the mac_service_data_unit parameter passed by the ISS
16         indication primitive to the Mask-and-match Stream
17         identification function. The resulting 'msdu-mask-length'-byte
18         information is the masked mac_service_data_unit that is used
19         as input for the instance of the Mask-and-match Stream
20         identification function. If msdu-mask has a value of 0, the
21         mac-service-data-unit parameter is ignored.";
22     reference
23         "9.1.6.6 of IEEE Std 802.1CBdb-2021";
24 }
25 leaf msdu-match {
26     when '../msdu-mask';
27     type msdu-mask-match-hex-string;
28     description
29         "Specifies the 'msdu-mask-length'-byte value of the masked
30         mac_service_data_unit to be matched by the instance of the
31         Mask-and-match Stream identification function";
32     reference
33         "9.1.6.7 of IEEE Std 802.1CBdb-2021";
34 }
35 }
36 }
37
38 list per-port-msdu-mask-max-length {
39     key "port-name";
40     description
41         "Table of the per-port maximum msdu mask lengths";
42     reference
43         "9.4.1 of IEEE Std 802.1CBdb-2021";
44     leaf port-name {
45         type if:interface-ref;
46         description
47             "Identification of the port";
48         reference
49             "9.4.1 of IEEE Std 802.1CBdb-2021";
50     }
51     leaf port-msdu-mask-max-length {
52         type msdu-mask-length-type;
53         config false;
54         description
55             "tsnPpMmIdMsduMaskMaxLength is a read-only variable defined for
56             each port the Mask-and-match Stream identification function is
57             to be ran on. It defines the maximum length a
58             mac_service_data_unit mask can have. It is an integer value,
59             in octets, with a minimum value of 2.";

```

```

1     reference
2     "9.4.1 of IEEE Std 802.1CBdb-2021";
3 }
4 }
5 }

```

6 12.6.2.4 Definition for the ieee802-dot1cb-frer-types YANG module

```

7 module ieee802-dot1cb-frer-types {
  yang-version "1.1";
  namespace urn:ieee:std:802.1Q:yang:ieee802-dot1cb-frer-types;
  prefix dot1cb-frer-types;
  organization
    "Institute of Electrical and Electronics Engineers";
  contact
    "WG-URL: http://ieee802.org/1/
    WG-Email: stds-802-1-1@ieee.org

    Contact: IEEE 802.1 Working Group Chair
    Postal: C/O IEEE 802.1 Working Group
            IEEE Standards Association
            445 Hoes Lane
            Piscataway, NJ 08854
            USA

    E-mail: stds-802-1-chairs@ieee.org";
  description
    "Management objects that control the frame replication and
    elimination from IEEE Std 802.1CB-2017. This YANG data model conforms
    to the Network Management Datastore Architecture defined in RFC 8342.
    Copyright (C) IEEE (2021). This version of this YANG module is part
    of IEEE Std 802.1CBcv; see the standard itself for full legal
    notices.";
  revision 2021-12-08 {
    description
      "Published as part of IEEE Std 802.1CBcv-2021. Initial version.";
    reference
      "IEEE Std 802.1CBcv-2021, Frame Replication and Elimination for
      Reliability - FRER YANG Data Model and Management Information Base
      Module.";
  }
  typedef seq-rcvy-algorithm {
    type enumeration {
      enum vector {
        value 0;
        description
          "The sequence recovery type used for the Vector Recovery
          Algorithm.";
      }
      enum match {
        value 1;
        description
          "The sequence recovery type used for the Match Recovery
          Algorithm.";
      }
    }
    description
      "An enumerated value specifying which sequence recovery algorithm
      is to be used for an instance of the Sequence recovery function.";
    reference
      "10.4.1.5 of IEEE Std 802.1CB-2017";
  }
  typedef seq-encaps-method {
    type enumeration {
      enum reserved {
        value 0;
        description
          "Reserved value.";
      }
    }
  }
}

```



```

1      enum r-tag {
        value 1;
        description
            "The sequence encode decode type used for the R_TAG
            encode/decode method.";
    }
    enum hsr-seq-tag {
        value 2;
        description
            "The sequence encode decode type used for the HSR encode/decode
            method.";
    }
    enum prp-seq-trailer {
        value 3;
        description
            "The sequence encode decode type used for the PRP encode/decode
            method.";
    }
    }
    description
        "An enumerated value indicating the type of encapsulation used for
        an instance of the Sequence encode/decode function.";
    reference
        "10.5.1.5 of IEEE Std 802.1CB-2017";
    }
}
27

```

28 12.6.2.5 Definition for the ieee802-dot1cb-frer YANG module

```

29 module ieee802-dot1cb-frer {
30     yang-version "1.1";
31     namespace urn:ieee:std:802.1Q:yang:ieee802-dot1cb-frer;
32     prefix dot1cb-frer;
33     import ieee802-dot1cb-stream-identification {
34         prefix dot1cb-sid;
35     }
36     import ieee802-dot1cb-stream-identification-types {
37         prefix dot1cb-sid-types;
38     }
39     import ieee802-dot1cb-frer-types {
40         prefix dot1cb-frer-types;
41     }
42     import ieee802-dot1q-types {
43         prefix dot1qtypes;
44     }
45     import ietf-interfaces {
46         prefix if;
47     }
48     organization
49         "Institute of Electrical and Electronics Engineers";
50     contact
51         "WG-URL: http://ieee802.org/1/
52         WG-EMail: stds-802-1-1@ieee.org
53
54         Contact: IEEE 802.1 Working Group Chair
55         Postal: C/O IEEE 802.1 Working Group
56                 IEEE Standards Association
57                 445 Hoes Lane
58                 Piscataway, NJ 08854
59                 USA
60
61         E-mail: stds-802-1-chairs@ieee.org";

```

```

1  description
2    "Management objects that control the frame replication and
3    elimination from IEEE Std 802.1CB-2017. This YANG data model conforms
4    to the Network Management Datastore Architecture defined in RFC 8342.
5    Copyright (C) IEEE (2021). This version of this YANG module is part
6    of IEEE Std 802.1CBcv; see the standard itself for full legal
7    notices.";
8  revision 2024-08-22 {
9    description
10     "Published as part of IEEE Std 802.1CB-2017/Cor1-2025.";
11    reference
12     "IEEE Std 802.1CB-2017/Cor1-2025, Frame Replication and Elimination for
13     Reliability - Corrigendum 1.";
14  }
15  revision 2021-12-08 {
16    description
17     "Published as part of IEEE Std 802.1CBcv-2021. Initial version.";
18    reference
19     "IEEE Std 802.1CBcv-2021, Frame Replication and Elimination for
20     Reliability - FRER YANG Data Model and Management Information Base
21     Module.";
22  }
23  feature auto-configuration {
24    description
25     "Autoconfiguration of entries in the tables for Stream identity
26     table, sequence recovery table, and sequence identification table.";
27    reference
28     "7.11 of IEEE Std 802.1CB-2017
29     10.7 of IEEE Std 802.1CB-2017";
30  }
31  typedef lan-path-id {
32    type int8;
33    description
34     "An integer specifying a path or LAN. If and only if a packet
35     matches an entry in the Sequence identification table that
36     specifies HSR or PRP in its frerSeqEncEncapsType object,
37     tsnStreamIdLanPathId specifies the LanId or PathId value that must
38     be matched for this tsnStreamIdEntry to apply. A value of -1
39     indicates that the LanId or PathId are to be ignored.";
40    reference
41     "10.22 of IEEE Std 802.1CB-2017";
42  }
43  typedef sequence-history-length {
44    type uint32 {
45      range "2..max";
46    }
47    description
48     "An integer specifying how many bits of the SequenceHistory
49     variable are to be used.";
50    reference
51     "7.4.3.2.2 of IEEE Std 802.1CB-2017";
52  }
53  grouping sequence-encode-decode {
54    description
55     "The sequence-encode-decode grouping indicates the type of
56     encapsulation used for this instance of the Sequence encode/decode
57     function.";
58    reference
59     "10.5.1.5 of IEEE Std 802.1CB-2017";

```

```

1  choice encapsulation {
2      description
3          "This choice indicates the type of encapsulation used for this
4          instance of the Sequence encode/decode function. The
5          encapsulation includes an Organizationally Unique Identifier
6          (OUI) or Company ID (CID) to identify the organization defining
7          the Sequence encode/decode method.";
8      reference
9          "10.5.1.5 of IEEE Std 802.1CB-2017";
10     container r-tag {
11         presence "true";
12         description
13             "R-TAG";
14         reference
15             "7.8 of IEEE Std 802.1CB-2017";
16         leaf type-number {
17             type dot1cb-frer-types:seq-encaps-method;
18             default "r-tag";
19             config false;
20             description
21                 "The type number used for the R-TAG encode/decode method.";
22             reference
23                 "10.5.1.5 of IEEE Std 802.1CB-2017";
24         }
25         leaf oui-cid {
26             type string {
27                 pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
28             }
29             default "00-80-C2";
30             config false;
31             description
32                 "The Organizationally Unique Identifier (OUI) or Company ID
33                 (CID) to identify the organization defining the encode/decode
34                 method. For encode/decode methods defined in IEEE Std
35                 802.1CB-2017 the OUI/CID is always 00-80-C2.";
36             reference
37                 "10.5.1.5 of IEEE Std 802.1CB-2017";
38         }
39     }
40     container hsr-sequence-tag {
41         presence "true";
42         description
43             "HSR sequence tag";
44         reference
45             "7.9 of IEEE Std 802.1CB-2017";
46         leaf type-number {
47             type dot1cb-frer-types:seq-encaps-method;
48             default "hsr-seq-tag";
49             config false;
50             description
51                 "The type number used for the HSR sequence tag encode/decode
52                 method.";
53             reference
54                 "10.5.1.5 of IEEE Std 802.1CB-2017";
55         }
56         leaf oui-cid {
57             type string {
58                 pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
59             }

```

```

1      default "00-80-C2";
2      config false;
3      description
4          "The Organizationally Unique Identifier (OUI) or Company ID
5          (CID) to identify the organization defining the encode/decode
6          method. For encode/decode methods defined in IEEE Std
7          802.1CB-2017 the OUI/CID is always 00-80-C2.";
8      reference
9          "10.5.1.5 of IEEE Std 802.1CB-2017";
10     }
11 }
12 container prp-sequence-tag {
13     presence "true";
14     description
15         "PRP sequence trailer";
16     reference
17         "7.10 of IEEE Std 802.1CB-2017";
18     leaf type-number {
19         type dot1cb-frer-types:seq-encaps-method;
20         default "prp-seq-trailer";
21         config false;
22         description
23             "The type number used for the PRP sequence trailer
24             encode/decode method.";
25         reference
26             "10.5.1.5 of IEEE Std 802.1CB-2017";
27     }
28     leaf oui-cid {
29         type string {
30             pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
31         }
32         default "00-80-C2";
33         config false;
34         description
35             "The Organizationally Unique Identifier (OUI) or Company ID
36             (CID) to identify the organization defining the encode/decode
37             method. For encode/decode methods defined in IEEE Std
38             802.1CB-2017 the OUI/CID is always 00-80-C2.";
39         reference
40             "10.5.1.5 of IEEE Std 802.1CB-2017";
41     }
42 }
43 container organization-specific {
44     presence "true";
45     description
46         "This container allows to select Sequence encode/decode types
47         that are defined by entities outside of IEEE 802.1.";
48     reference
49         "10.5.1.5 of IEEE Std 802.1CB-2017";
50     leaf type-number {
51         type int32 {
52             range "256..max";
53         }
54         description
55             "The type number used for an encode/decode method defined by
56             an entity owning the OUI or CID for this encapsulation type.";
57         reference
58             "10.5.1.5 of IEEE Std 802.1CB-2017";
59     }

```

```

1      leaf oui-cid {
2          type string {
3              pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
4          }
5          description
6              "The Organizationally Unique Identifier (OUI) or Company ID
7              (CID) to identify the organization defining the encode/decode
8              method.";
9          reference
10             "10.5.1.5 of IEEE Std 802.1CB-2017";
11     }
12 }
13 }
14 }
15 grouping sequence-recovery-algorithm {
16     description
17         "The sequence-recovery-algorithm grouping specifies which sequence
18         recovery algorithm is to be used for this instance of the Sequence
19         recovery function.";
20     reference
21         "10.4.1.5 of IEEE Std 802.1CB-2017";
22     choice algorithm {
23         description
24             "This choice indicates the sequence recovery algorithm used for
25             this instance of the Sequence recovery function. It includes an
26             Organizationally Unique Identifier (OUI) or Company ID (CID) to
27             identify the organization defining the sequence recovery
28             algorithm.";
29         reference
30             "10.4.1.5 of IEEE Std 802.1CB-2017";
31         container vector {
32             presence "true";
33             description
34                 "Vector Recovery Algorithm.";
35             reference
36                 "10.4.1.5 of IEEE Std 802.1CB-2017";
37             leaf type-number {
38                 type dot1cb-frer-types:seq-rcvy-algorithm;
39                 default "vector";
40                 config false;
41                 description
42                     "The type number used for the VectorRecoveryAlgorithm.";
43                 reference
44                     "10.4.1.5 of IEEE Std 802.1CB-2017";
45             }
46             leaf oui-cid {
47                 type string {
48                     pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
49                 }
50                 default "00-80-C2";
51                 config false;
52                 description
53                     "The Organizationally Unique Identifier (OUI) or Company ID
54                     (CID) to identify the organization defining the sequence
55                     recovery algorithm. For sequence recovery algorithms defined
56                     in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
57                 reference
58                     "10.4.1.5 of IEEE Std 802.1CB-2017";
59             }

```

```

1      }
2      container match {
3          presence "true";
4          description
5              "Match Recovery Algorithm.";
6          reference
7              "10.4.1.5 of IEEE Std 802.1CB-2017";
8          leaf type-number {
9              type dot1cb-frer-types:seq-rcvy-algorithm;
10             default "match";
11             config false;
12             description
13                 "The type number used for the MatchRecoveryAlgorithm.";
14             reference
15                 "10.4.1.5 of IEEE Std 802.1CB-2017";
16         }
17         leaf oui-cid {
18             type string {
19                 pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
20             }
21             default "00-80-C2";
22             config false;
23             description
24                 "The Organizationally Unique Identifier (OUI) or Company ID
25                 (CID) to identify the organization defining the sequence
26                 recovery algorithm. For sequence recovery algorithms defined
27                 in IEEE Std 802.1CB-2017 the OUI/CID is always 00-80-C2.";
28             reference
29                 "10.4.1.5 of IEEE Std 802.1CB-2017";
30         }
31     }
32     container organization-specific {
33         presence "true";
34         description
35             "This container allows to select sequence recovery algorithms
36             that are defined by entities outside of IEEE 802.1.";
37         reference
38             "10.4.1.5 of IEEE Std 802.1CB-2017";
39         leaf type-number {
40             type int32 {
41                 range "256..max";
42             }
43             description
44                 "The type number used for a sequence recovery algorithm
45                 defined by an entity owning the OUI or CID for this algorithm
46                 type.";
47             reference
48                 "10.4.1.5 of IEEE Std 802.1CB-2017";
49         }
50         leaf oui-cid {
51             type string {
52                 pattern "[0-9A-F]{2}(-[0-9A-F]{2}){2}";
53             }
54             description
55                 "The Organizationally Unique Identifier (OUI) or Company ID
56                 (CID) to identify the organization defining the sequence
57                 recovery algorithm.";
58             reference
59                 "10.4.1.5 of IEEE Std 802.1CB-2017";

```

```

1      }
2    }
3  }
4 }
5 container frer {
6   description
7     "The managed objects that control Stream identification are
8     described in Clause 9 of IEEE Std 802.1CB-2017. The managed objects
9     that control FRER are described in Clause 10 of IEEE Std
10    802.1CB-2017 as follows:
11
12    a) General requirements on the behavior of counters are in 10.1.
13    b) The various tables of managed objects that can manage, in
14       detail, each individual Stream, are described in five
15       subclauses, including:
16       1) Additions to the Stream identity table required for
17          Autoconfiguration.
18       2) The Sequence generation table that configures instances of
19          the Sequence generation function;
20       3) The Sequence recovery table that configures instances of the
21          Individual recovery function, the Sequence recovery function,
22          and the Latent error detection function;
23       4) The Sequence identification table that configures instances
24          of the Sequence encode/decode function; and
25       5) The Stream split table that configures instances of the
26          Stream splitting function.
27    c) The managed objects that support the automatic configuration,
28       upon receipt of a packet, of entries in the first four of the
29       preceding tables, are described in the subclause on
30       Autoconfiguration.
31    d) The per-port, per-Stream packet counters that are kept by FRER
32       functions for inspection by network management entities are
33       described in 10.8, and the per-port (totaled over all Streams)
34       counters in 10.9.
35
36    The managed objects in the subclauses under 9.1 make it possible to
37    configure more than one encapsulation for the same stream_handle
38    subparameter on the same port. Similarly, the managed objects in
39    the subclauses under 10.3 and 10.4 make it possible to configure
40    more than one Sequence encode/decode function or more than one
41    Sequence generation function for the same stream_handle
42    subparameter. [The same value of stream_handle can be in the
43    frerSeqGenStreamList of more than one frerSeqGenEntry or in the
44    frerSeqRcvyStreamList of more than one frerSeqRcvyEntry.] A system
45    shall return an error if an attempt is made to configure
46    conflicting requirements upon that system.";
47  reference
48    "Clause 10 of IEEE Std 802.1CB-2017";
49  list sequence-generation {
50    key "index";
51    description
52      "There is one Sequence generation table in a system, and one
53      entry in the Sequence generation table for each Sequence
54      generation function. Each frerSeqGenEntry lists the Streams and
55      direction for which a single instance of the Sequence generation
56      function is to be placed.";
57    reference
58      "10.3 of IEEE Std 802.1CB-2017";
59    leaf index {

```

```

1      type uint32;
2      description
3          "Clause 10 of IEEE Std 802.1CB-2017 states that the same Stream
4          handle can be present multiple times in the sequence generation
5          table. Therefore this index leaf is being used to uniquely
6          identify an entry in the sequence generation table.";
7  }
8  leaf-list stream {
9      type leafref {
10         path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
11     }
12     min-elements 1;
13     description
14         "A list of stream_handle values, corresponding to the values of
15         the tsNStreamIdHandle objects in the Stream identity table, on
16         which this instance of the Sequence generation function is to
17         operate. The single instance of the Sequence generation
18         function created by this frerSeqGenEntry operates every packet
19         belonging to this Stream, regardless of the port on which it is
20         received.";
21     reference
22         "10.3.1.1 of IEEE Std 802.1CB-2017";
23 }
24 leaf direction-out-facing {
25     type dot1cb-sid-types:direction;
26     description
27         "An object indicating whether the Sequence generation function
28         is to be placed on the out-facing (True) or in-facing (False)
29         side of the port.";
30     reference
31         "10.3.1.2 of IEEE Std 802.1CB-2017";
32 }
33 leaf reset {
34     type boolean;
35     description
36         "A Boolean object indicating that the Sequence generation
37         function is to be reset by calling its corresponding
38         SequenceGenerationReset function. Writing the value True to
39         frerSeqGenReset triggers a reset; writing the value False has
40         no effect. When read, frerSeqGenReset always returns the value
41         False.";
42     reference
43         "10.3.1.3 of IEEE Std 802.1CB-2017";
44 }
45 }
46 list sequence-recovery {
47     key "index";
48     description
49         "There is one Sequence recovery table in a system, and one entry
50         in the Sequence recovery table for each Sequence recovery
51         function or Individual recovery function that can also be
52         present. The entry describes a set of managed objects for the
53         single instance of a Base recovery function and Latent error
54         detection function included in the Sequence recovery function or
55         Individual recovery function. Each frerSeqRcvyEntry lists the
56         Streams, ports, and direction for which instances of a Sequence
57         recovery function or Individual recovery function are to be
58         instantiated.";
59     reference

```



```
1      "10.4 of IEEE Std 802.1CB-2017";
2  leaf index {
3      type uint32;
4      description
5          "Clause 10 of IEEE Std 802.1CB-2017 states that the same Stream
6           handle can be present multiple times in the sequence recovery
7           table. Therefore this index leaf is being used to uniquely
8           identify an entry in the sequence recovery table.";
9  }
10 leaf-list stream {
11     type leafref {
12         path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
13     }
14     min-elements 1;
15     description
16         "A list of the stream_handle values, corresponding to the
17         values of the tsnStreamIdHandle objects in the Stream identity
18         table, to which the system is to apply the instance of the
19         Sequence recovery function or Individual recovery function.";
20     reference
21         "10.4.1.1 of IEEE Std 802.1CB-2017";
22 }
23 leaf-list port {
24     type if:interface-ref;
25     min-elements 1;
26     description
27         "The list of ports on each of which the system is to
28         instantiate the Sequence recovery function, or from which
29         received packets are to be fed to a single instance of the
30         Individual recovery function.";
31     reference
32         "10.4.1.2 of IEEE Std 802.1CB-2017";
33 }
34 leaf direction-out-facing {
35     type dot1cb-sid-types:direction;
36     description
37         "An object indicating whether the Sequence recovery function or
38         Individual recovery function is to be placed on the out-facing
39         (True) or in-facing (False) side of the port.";
40     reference
41         "10.4.1.3 of IEEE Std 802.1CB-2017";
42 }
43 leaf reset {
44     type boolean;
45     default "false";
46     description
47         "A Boolean object indicating that the Sequence recovery
48         function or Individual recovery function is to be reset by
49         calling its corresponding SequenceGenerationReset function.
50         Writing the value True to frerSeqRcvyReset triggers a reset;
51         writing the value False has no effect. When read,
52         frerSeqRcvyReset always returns the value False.";
53     reference
54         "10.4.1.4 of IEEE Std 802.1CB-2017";
55 }
56 container algorithm {
57     description
58         "This object is an enumerated value specifying which sequence
59         recovery algorithm is to be used for this instance of the
```

```

1      Sequence recovery function. The enumeration uses an OUI or CID
2      as shown in Table 10-1. The default value for
3      frerSeqRcvyAlgorithm is Vector_Alg (00-80-C2, 0).";
4      reference
5      "10.4.1.5 of IEEE Std 802.1CB-2017";
6      uses sequence-recovery-algorithm;
7  }
8  leaf history-length {
9      type sequence-history-length;
10     default "2";
11     description
12         "An integer specifying how many bits of the SequenceHistory
13         variable are to be used. The minimum and the default value is
14         2, maximum is the maximum allowed by the implementation. [Not
15         used if frerSeqRcvyAlgorithm = Match_Alg (00-80-C2, 1).]";
16     reference
17         "10.4.1.6 of IEEE Std 802.1CB-2017";
18 }
19 leaf reset-timeout {
20     type uint32;
21     units "ms";
22     description
23         "An unsigned integer specifying the timeout period in
24         milliseconds for the RECOVERY_TIMEOUT event.";
25     reference
26         "10.4.1.7 of IEEE Std 802.1CB-2017";
27 }
28 leaf invalid-sequence-value {
29     type uint32;
30     config false;
31     description
32         "A read-only unsigned integer value that cannot be encoded in a
33         packet as a value for the sequence_number subparameter, i.e.,
34         frerSeqRcvyInvalidSequenceValue is larger than or equal to
35         RecovSeqSpace.";
36     reference
37         "10.4.1.8 of IEEE Std 802.1CB-2017";
38 }
39 leaf take-no-sequence {
40     type boolean;
41     default "false";
42     description
43         "A Boolean value specifying whether packets with no
44         sequence_number subparameter are to be accepted (True) or not
45         (False). Default value False.";
46     reference
47         "10.4.1.9 of IEEE Std 802.1CB-2017";
48 }
49 leaf individual-recovery {
50     type boolean;
51     description
52         "A Boolean value specifying whether this entry describes a
53         Sequence recovery function or Individual recovery function.
54         a) True: The entry describes an Individual recovery function.
55             Packets discarded by the SequenceGenerationAlgorithm will
56             cause the variable RemainingTicks to be reset. There is no
57             Latent error detection function associated with this entry,
58             so frerSeqRcvyLatentErrorDetection cannot also be True.
59         b) False: The entry describes a Sequence recovery function.

```

```

1           Packets discarded by the SequenceGenerationAlgorithm will
2           not cause the variable RemainingTicks to be reset.";
3       reference
4           "10.4.1.10 of IEEE Std 802.1CB-2017";
5   }
6   leaf latent-error-detection {
7       type boolean;
8       description
9           "A Boolean value indicating whether an instance of the Latent
10          error detection function is to be instantiated along with the
11          Base recovery function in this Sequence recovery function or
12          Individual recovery function. frerSeqRcvyLatentErrorDetection
13          cannot be set True if frerSeqRcvyIndividualRecovery is also
14          True; an Individual recovery function does not include a Latent
15          error detection function.";
16       reference
17           "10.4.1.11 of IEEE Std 802.1CB-2017";
18   }
19   container latent-error-detection-parameters {
20       description
21           "The objects in the following subclauses are present if and
22           only if frerSeqRcvyIndividualRecovery is False.";
23       reference
24           "10.4.1.12 of IEEE Std 802.1CB-2017";
25       leaf difference {
26           type int32;
27           description
28               "An integer specifying the maximum difference between
29               frerCpsSeqRcvyDiscardedPackets, and the product of
30               frerCpsSeqRcvyPassedPackets and (frerSeqRcvyLatentErrorPaths
31               - 1) that is allowed. Any larger difference will trigger the
32               detection of a latent error by the LatentErrorTest function.";
33           reference
34               "10.4.1.12.1 of IEEE Std 802.1CB-2017";
35       }
36       leaf period {
37           type uint32;
38           units "ms";
39           default "2000";
40           description
41               "The integer number of milliseconds that are to elapse
42               between instances of running the LatentErrorTest function. An
43               implementation can have a minimum value for
44               frerSeqRcvyLatentErrorPeriod, below which it cannot be set,
45               but this minimum shall be no larger than 1000 ms (1 s).
46               Default value 2000 (2 s).";
47           reference
48               "10.4.1.12.2 of IEEE Std 802.1CB-2017";
49       }
50       leaf paths {
51           type uint16;
52           description
53               "The integer number of paths over which FRER is operating for
54               this instance of the Base recovery function and Latent error
55               detection function.";
56           reference
57               "10.4.1.12.3 of IEEE Std 802.1CB-2017";
58       }
59       leaf reset-period {

```

```

1      type uint32;
2      units "ms";
3      default "30000";
4      description
5          "The integer number of milliseconds that are to elapse
6          between instances of running the LatentErrorReset function.
7          An implementation can have a minimum value for
8          LatentErrorReset, below which it cannot be set, but this
9          minimum shall be no larger than 1000 ms (1 s). Default value
10         30000 (30 s).";
11      reference
12         "10.4.1.12.4 of IEEE Std 802.1CB-2017";
13    }
14 }
15 }
16 list sequence-identification {
17     key "port direction-out-facing";
18     description
19         "There is one Sequence identification table per system, and one
20         entry in the Sequence identification table for each port and
21         direction for which an instance of the Sequence encode/decode
22         function is to be created. Each entry in the Sequence
23         identification table specifies a port and direction on which an
24         instance of the Sequence encode/decode function is to be
25         instantiated for a list of Streams.";
26     reference
27         "10.5 of IEEE Std 802.1CB-2017";
28     leaf-list stream {
29         type leafref {
30             path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
31         }
32         min-elements 1;
33         description
34             "A list of stream_handles, corresponding to the values of the
35             tsNStreamIdHandle objects in the Stream identity table, for
36             which the system is to use the same encapsulation for the
37             Sequence encode/decode function.";
38         reference
39             "10.5.1.1 of IEEE Std 802.1CB-2017";
40     }
41     leaf port {
42         type if:interface-ref;
43         description
44             "The port on which the system is to place an instance of the
45             Sequence encode/decode function.";
46         reference
47             "10.5.1.2 of IEEE Std 802.1CB-2017";
48     }
49     leaf direction-out-facing {
50         type dot1cb-sid-types:direction;
51         description
52             "An object indicating whether the Sequence encode/decode
53             function is to be placed on the out-facing (True) or in-facing
54             (False) side of the port.";
55         reference
56             "10.5.1.3 of IEEE Std 802.1CB-2017";
57     }
58     leaf active {
59         type boolean;

```

```

1      description
2          "A Boolean value specifying whether this frerSeqEncEntry is
3          passive (False), and therefore is used only to decode (extract
4          information from) input packets passing up the protocol stack,
5          or active (True), and therefore is used both for recognizing
6          input packets and for encoding output packets being passed down
7          the protocol stack.";
8      reference
9          "10.5.1.4 of IEEE Std 802.1CB-2017";
10  }
11  container encapsulation {
12      description
13          "An enumerated value indicating the type of encapsulation used
14          for this instance of the Sequence encode/decode function. The
15          type includes an OUI or CID.";
16      reference
17          "10.5.1.5 of IEEE Std 802.1CB-2017";
18      uses sequence-encode-decode;
19  }
20  leaf path-id-lan-id {
21      type lan-path-id;
22      description
23          "A 4-bit integer value to be placed in the PathId field of an
24          HSR sequence tag or the LanId field of a PRP sequence trailer
25          added to an output packet. This managed object is used only if:
26          a) The HSR sequence tag or the PRP sequence trailer is
27             selected by the frerSeqEncEncapsType object; and
28          b) frerSeqEncActive is False (passive)";
29      reference
30          "10.5.1.6 of IEEE Std 802.1CB-2017";
31  }
32  }
33  list stream-split {
34      key "index";
35      description
36          "There is one Stream split table per system, with one
37          frerSplitEntry per Stream splitting function per set of
38          stream_handle values. Each entry in the Stream split table
39          specifies a port and direction on which an instance of the Stream
40          splitting function is to be instantiated, and the list of
41          stream_handles specifying its operation.";
42      reference
43          "10.6 of IEEE Std 802.1CB-2017";
44      leaf index {
45          type uint32;
46          description
47              "Entry in the Stream split table.";
48          reference
49              "10.6 of IEEE Std 802.1CB-2017";
50      }
51      leaf port {
52          type if:interface-ref;
53          description
54              "The port on which the system is to place an instance of the
55              Stream splitting function performing the stream_handle
56              translations specified by frerSplitInputIdList and
57              frerSplitOutputIdList is to be placed.";
58          reference
59              "10.6.1.1 of IEEE Std 802.1CB-2017";

```

```

1      }
2      leaf direction-out-facing {
3          type dot1cb-sid-types:direction;
4          description
5              "An object indicating whether the instance of the Stream
6              splitting function performing the stream_handle translations
7              specified by frerSplitInputIdList and frerSplitOutputIdList is
8              to be placed on the out-facing (True) or in-facing (False) side
9              of the port.";
10         reference
11             "10.6.1.2 of IEEE Std 802.1CB-2017";
12     }
13     leaf-list input-id {
14         type leafref {
15             path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
16         }
17         min-elements 1;
18         description
19             "A list of stream_handles (tsnStreamIdHandle values) that are
20             to be split.";
21         reference
22             "10.6.1.3 of IEEE Std 802.1CB-2017";
23     }
24     leaf-list output-id {
25         type leafref {
26             path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
27         }
28         min-elements 1;
29         description
30             "A list of stream_handles (tsnStreamIdHandle values) into which
31             the input packet is to be split, one copy per item in the
32             frerSplitOutputIdList.";
33         reference
34             "10.6.1.4 of IEEE Std 802.1CB-2017";
35     }
36 }
37 container autoconfiguration {
38     if-feature "auto-configuration";
39     description
40         "Container for autoconfiguration managed objects.";
41     reference
42         "10.7 of IEEE Std 802.1CB-2017";
43     list sequence {
44         key "index";
45         description
46             "There is one Sequence autoconfiguration table per system. It
47             contains any number of table entries. No two (or more) entries
48             in the Sequence autoconfiguration table can have the same
49             values for frerAutSeqSeqEncaps, frerAutSeqTagged, and
50             frerAutSeqVlan on any given port. Each frerAutSeqEntry object
51             relates to a single class of Streams, and specifies how entries
52             are created (and destroyed) in the Stream identity table, the
53             Sequence recovery table, and the Sequence identification table.";
54         reference
55             "10.7.1 of IEEE Std 802.1CB-2017";
56         leaf index {
57             type uint32;
58             description
59                 "Entry in the sequence list referencing to a single class of

```

```

1      Streams.";
2  }
3  container sequence-encapsulation {
4      description
5          "An enumerated value, specifying which Sequence encode/decode
6          function, and therefore, which type sequence_number encoding,
7          is to be recognized for the purposes of Autoconfiguration.";
8      reference
9          "10.7.1.1.1 of IEEE Std 802.1CB-2017";
10     uses sequence-encode-decode;
11 }
12 leaf-list receive-port {
13     type if:interface-ref;
14     min-elements 1;
15     description
16         "The list of ports to which this frerAutSeqEntry applies, and
17         on which Stream identification functions, Sequence
18         encode/decode functions, and Individual recovery functions
19         are to be autocreated.";
20     reference
21         "10.7.1.1.2 of IEEE Std 802.1CB-2017";
22 }
23 leaf tagged {
24     type enumeration {
25         enum tagged {
26             value 1;
27             description
28                 "A frame must have a VLAN tag to be matched.";
29         }
30         enum priority {
31             value 2;
32             description
33                 "A frame must be untagged, or have a VLAN tag with a VLAN
34                 ID = 0 to be matched.";
35         }
36         enum all {
37             value 3;
38             description
39                 "A frame is matched whether tagged or not.";
40         }
41     }
42     description
43         "An enumerated value indicating whether packets to be matched
44         by this frerAutSeqEntry are permitted to have a VLAN tag.";
45     reference
46         "10.7.1.1.3 of IEEE Std 802.1CB-2017";
47 }
48 leaf-list vlan {
49     type dot1qtys:vlanid;
50     description
51         "A list of vlan_identifiers for the packet to match. A null
52         list matches all vlan_identifiers.";
53     reference
54         "10.7.1.1.4 of IEEE Std 802.1CB-2017";
55 }
56 leaf-list recovery-port {
57     type if:interface-ref;
58     min-elements 1;
59     description

```

```

1         "The list of ports on which Sequence recovery functions are
2         to be autocreated by this frerAutSeqEntry.";
3     reference
4         "10.7.1.1.5 of IEEE Std 802.1CB-2017";
5 }
6 leaf destruction-interval {
7     type uint64;
8     units "ms";
9     default "86400000";
10    description
11        "An integer number of milliseconds after which an idle set of
12        functions created by this frerAutSeqEntry can be destroyed. A
13        value of 0 indicates that idle autoconfigured functions are
14        not to be destroyed. Default value is 86 400 000 decimal (one
15        day).";
16    reference
17        "10.7.1.1.6 of IEEE Std 802.1CB-2017";
18 }
19 leaf reset-interval {
20     type uint64;
21     units "ms";
22     description
23         "The value used to fill frerSeqRcvyResetMSec when
24         autoconfiguring entries in the Sequence recovery table.";
25     reference
26         "10.7.1.1.7 of IEEE Std 802.1CB-2017";
27 }
28 container algorithm {
29     description
30         "The value used to fill frerSeqRcvyAlgorithm when
31         autoconfiguring entries in the Sequence recovery table.";
32     reference
33         "10.7.1.1.8 of IEEE Std 802.1CB-2017";
34     uses sequence-recovery-algorithm;
35 }
36 leaf history-length {
37     type sequence-history-length;
38     default "2";
39     description
40         "The value used to fill frerSeqRcvyHistoryLength when
41         autoconfiguring entries in the Sequence recovery table.";
42     reference
43         "10.7.1.1.9 of IEEE Std 802.1CB-2017";
44 }
45 leaf create-individual {
46     type boolean;
47     description
48         "A Boolean value. If True, the receipt of a packet that
49         triggers the autoconfiguration of a new tsNStreamIdEntry also
50         triggers the instantiation of a frerSeqRcvyEntry for an
51         Individual recovery function.";
52     reference
53         "10.7.1.1.10 of IEEE Std 802.1CB-2017";
54 }
55 leaf create-recovery {
56     type boolean;
57     description
58         "A Boolean value. If True, the receipt of a packet that
59         triggers the autoconfiguration of a new tsNStreamIdEntry can

```



```

1         also trigger the instantiation of a frerSeqRcvyEntry for a
2         Sequence recovery function.";
3     reference
4         "10.7.1.1.11 of IEEE Std 802.1CB-2017";
5 }
6 leaf latent-error-detection {
7     type boolean;
8     description
9         "A Boolean value. If True, the autoconfiguration of a new
10        Sequence recovery function also creates an associated Latent
11        Error Detection function.";
12    reference
13        "10.7.1.1.12 of IEEE Std 802.1CB-2017";
14 }
15 leaf latent-error-difference {
16     type int32;
17     description
18         "The value used to fill frerSeqRcvyLatentErrorDifference when
19        autoconfiguring entries in the Sequence recovery table.";
20    reference
21        "10.7.1.1.13 of IEEE Std 802.1CB-2017";
22 }
23 leaf latent-error-period {
24     type uint32;
25     units "ms";
26     default "2000";
27     description
28         "The value used to fill frerSeqRcvyLatentErrorPeriod when
29        autoconfiguring entries in the Sequence recovery table.";
30    reference
31        "10.7.1.1.14 of IEEE Std 802.1CB-2017";
32 }
33 leaf latent-error-reset-period {
34     type uint32;
35     units "ms";
36     default "30000";
37     description
38         "The value used to fill frerSeqRcvyLatentResetPeriod when
39        autoconfiguring entries in the Sequence recovery table.";
40    reference
41        "10.7.1.1.15 of IEEE Std 802.1CB-2017";
42 }
43 }
44 list output {
45     key "index";
46     description
47         "There is one Output autoconfiguration table per system. It
48        contains any number of frerAutOutEntry objects, each relating
49        to a single class of Streams specifying how active entries are
50        created in the Sequence identification table. No two (or more)
51        entries in the Output autoconfiguration table can include the
52        same port in their frerAutSeqReceivePortList objects.";
53    reference
54        "10.7.2 of IEEE Std 802.1CB-2017";
55    leaf index {
56        type uint32;
57        description
58            "Entry in the output list referencing to a single class of
59            Streams.";

```

```

1      }
2      leaf-list port {
3          type if:interface-ref;
4          min-elements 1;
5          description
6              "The list of ports to which this frerAutOutEntry applies, and
7              on which active Sequence encode/decode functions are to be
8              autocreated.";
9          reference
10             "10.7.2.1.1 of IEEE Std 802.1CB-2017";
11     }
12     container encapsulation {
13         description
14             "An enumerated value, specifying which Sequence encode/decode
15             function, and therefore, which type sequence_number encoding,
16             is to be used for autoconfigured Streams on the ports in
17             frerAutSeqReceivePortList.";
18         reference
19             "10.7.2.1.2 of IEEE Std 802.1CB-2017";
20         uses sequence-encode-decode;
21     }
22     leaf lan-path-id {
23         type lan-path-id;
24         description
25             "An integer specifying a path or LAN. If and only if
26             frerAutOutEncaps specifies HSR or PRP frerAutOutLanPathId
27             specifies the LanId or PathId value to be inserted into the
28             HSR sequence tag or PRP sequence trailer of autoconfigured
29             packets transmitted on the ports in
30             frerAutSeqReceivePortList.";
31         reference
32             "10.7.2.1.3 of IEEE Std 802.1CB-2017";
33     }
34 }
35 }
36 }
37 augment "/dot1cb-sid:stream-identity" {
38     if-feature "auto-configuration";
39     description
40         "Two managed objects augment each tsNStreamIdEntry in the Stream
41         identity table when Managed objects for autoconfiguration is
42         implemented.";
43     reference
44         "10.2 of IEEE Std 802.1CB-2017";
45     leaf auto-configured {
46         type boolean;
47         config false;
48         description
49             "A read-only Boolean value, supplied by the system, specifying
50             whether this entry was created explicitly (False) or via the
51             Sequence autoconfiguration table.";
52         reference
53             "10.2.1 of IEEE Std 802.1CB-2017";
54     }
55     leaf lan-path-id {
56         type lan-path-id;
57         description
58             "An integer specifying a path or LAN. If and only if a packet
59             matches an entry in the Sequence identification table that

```

```
1      specifies HSR or PRP in its frerSeqEncEncapsType object,
2      tsnStreamIdLanPathId specifies the LanId or PathId value that
3      must be matched for this tsnStreamIdEntry to apply. A value of -1
4      indicates that the LanId or PathId are to be ignored.";
5      reference
6      "10.2.2 of IEEE Std 802.1CB-2017";
7  }
8 }
9 augment "/if:interfaces/if:interface/if:statistics" {
10 description
11     "The following counters are the counters for frame replication and
12     elimination for reliability. All counters are unsigned integers. If
13     used on links faster than 650 000 000 bits per second, they shall
14     be 64 bits in length to ensure against excessively short wrap
15     times.
16
17     A Stream identification component shall implement the first two
18     counters provided in the stream-identification YANG module per-port
19     counters, input-packets and output-packets; the remainder of the
20     counters in the frer YANG module module are optional for such a
21     system.";
22 reference
23     "10.8 of IEEE Std 802.1CB-2017
24     10.9 of IEEE Std 802.1CB-2017";
25 container frer {
26     description
27         "This container contains the per-port as well as the
28         per-port-per-stream counters for frame replication and
29         elimination for reliability.";
30     reference
31         "10.8 of IEEE Std 802.1CB-2017
32         10.9 of IEEE Std 802.1CB-2017";
33     container per-port-counters {
34         config false;
35         description
36             "Contains the per-port counters for frame replication and
37             elimination for reliability. The following counters are
38             instantiated for each port on which any of the Stream
39             identification function, Sequencing function, or Sequence
40             encode/decode function is configured. The counters are indexed
41             by port number.";
42         leaf rx-passed-pkts {
43             type uint64;
44             config false;
45             description
46                 "The frerCpSeqRcvyPassedPackets counter is incremented once
47                 for each packet passed up the stack by the
48                 VectorRecoveryAlgorithm or MatchRecoveryAlgorithm. Its value
49                 equals the sum (modulo the size of the counters) of all of
50                 the frerCpsSeqRcvyPassedPackets counters on this same port.";
51             reference
52                 "10.9.1 of IEEE Std 802.1CB-2017";
53         }
54         leaf rx-discarded-pkts {
55             type uint64;
56             config false;
57             description
58                 "The frerCpSeqRcvyDiscardPackets counter is incremented once
59                 for each packet discarded due to a duplicate sequence number
```

```

1         or for being a rogue packet by any VectorRecoveryAlgorithm or
2         MatchRecoveryAlgorithm on this port. Its value equals the sum
3         (modulo the size of the counters) of all of the
4         frerCpsSeqRcvyRoguePackets and frerCpsSeqRcvyDiscardedPackets
5         counters on this same port.";
6     reference
7         "10.9.2 of IEEE Std 802.1CB-2017";
8 }
9 leaf encode-errored-pkts {
10     type uint64;
11     config false;
12     description
13         "The frerCpSeqEncErroredPackets counter is incremented once
14         each time the Sequence encode/decode function receives a
15         packet that it is unable to decode successfully. Its value
16         equals the sum (modulo the size of the counters) of all of
17         the frerCpsSeqEncErroredPackets counters on this same port.";
18     reference
19         "10.9.2 of IEEE Std 802.1CB-2017";
20 }
21 }
22 list per-port-per-stream-counters {
23     key "direction-out-facing handle";
24     config false;
25     description
26         "Contains the per-port-per-stream counters for frame
27         replication and elimination for reliability. The following
28         counters are instantiated for each port on which any of the
29         Stream identification function, Sequencing function, or
30         Sequence encode/decode function is configured. The counters are
31         indexed by port number, facing (in-facing or out-facing), and
32         stream_handle value.";
33     reference
34         "10.8 of IEEE Std 802.1CB-2017";
35     leaf direction-out-facing {
36         type dot1cb-sid-types:direction;
37         description
38             "An object indicating whether the counters apply to
39             out-facing (True) or in-facing (False).";
40     }
41     leaf handle {
42         type leafref {
43             path '/dot1cb-sid:stream-identity/dot1cb-sid:handle';
44         }
45         description
46             "The according tsNStreamIdHandle for the counters.";
47     }
48     leaf generation-reset {
49         type uint64;
50         config false;
51         description
52             "The frerCpsSeqGenResets counter is incremented each time the
53             SequenceGenerationReset function is called.";
54         reference
55             "10.8.2 of IEEE Std 802.1CB-2017";
56     }
57     leaf rx-out-of-order-pkts {
58         type uint64;
59         config false;

```

```

1      description
2          "The frerCpsSeqRcvyOutOfOrderPackets counter is incremented
3          once for each packet accepted out-of-order by the
4          VectorRecoveryAlgorithm or MatchRecoveryAlgorithm.
5          Out-of-order means that the packet's sequence number is not
6          one more than the previous packet received.";
7      reference
8          "10.8.3 of IEEE Std 802.1CB-2017";
9  }
10 leaf rx-rogue-pkts {
11     type uint64;
12     config false;
13     description
14         "The frerCpsSeqRcvyRoguePackets counter is incremented once
15         for each packet discarded by the VectorRecoveryAlgorithm
16         because its sequence_number subparameter is more than
17         frerSeqRcvyHistoryLength from RecovSeqNum.";
18     reference
19         "10.8.4 of IEEE Std 802.1CB-2017";
20 }
21 leaf rx-passed-pkts {
22     type uint64;
23     config false;
24     description
25         "The frerCpsSeqRcvyPassedPackets counter is incremented once
26         for each packet passed up the stack by the
27         VectorRecoveryAlgorithm or MatchRecoveryAlgorithm.";
28     reference
29         "10.8.5 of IEEE Std 802.1CB-2017";
30 }
31 leaf rx-discarded-pkts {
32     type uint64;
33     config false;
34     description
35         "The frerCpsSeqRcvyDiscardedPackets counter is incremented
36         once for each packet discarded due to a duplicate sequence
37         number by the VectorRecoveryAlgorithm or
38         MatchRecoveryAlgorithm.";
39     reference
40         "10.8.6 of IEEE Std 802.1CB-2017";
41 }
42 leaf rx-lost-pkts {
43     type uint64;
44     config false;
45     description
46         "The frerCpsSeqRcvyLostPackets counter is incremented once
47         for each packet lost by the VectorRecoveryAlgorithm. A packet
48         is counted as lost if its sequence number is not received on
49         any ingress port.
50
51         NOTE—If per-source sequence numbering is used,
52         frerCpsSeqRcvyLostPackets can count, as lost, packets that
53         were sent to another destination, but not lost.";
54     reference
55         "10.8.7 of IEEE Std 802.1CB-2017";
56 }
57 leaf rx-tagless-pkts {
58     type uint64;
59     config false;

```

```

1      description
2          "The frerCpsSeqRcvyTaglessPackets counter is incremented once
3          for each packet received by the VectorRecoveryAlgorithm that
4          has no sequence_number subparameter.";
5      reference
6          "10.8.8 of IEEE Std 802.1CB-2017";
7  }
8  leaf rx-resets {
9      type uint64;
10     config false;
11     description
12         "The frerCpsSeqRcvyResets counter is incremented once each
13         time the SequenceRecoveryReset function is called.";
14     reference
15         "10.8.9 of IEEE Std 802.1CB-2017";
16 }
17 leaf rx-latent-error-resets {
18     type uint64;
19     config false;
20     description
21         "The frerCpsSeqRcvyLatentErrorResets counter is incremented
22         once each time the LatentErrorReset function is called.";
23     reference
24         "10.8.10 of IEEE Std 802.1CB-2017";
25 }
26 leaf encode-errored-pkts {
27     type uint64;
28     config false;
29     description
30         "The frerCpsSeqEncErroredPackets counter is incremented once
31         each time the Sequence encode/decode function receives a
32         packet that it is unable to decode successfully.";
33     reference
34         "10.8.11 of IEEE Std 802.1CB-2017";
35 }
36 }
37 }
38 }
39 }
40
41
42

```

1 Annex A

2 (normative)

3 Protocol Implementation Conformance Statement (PICS) 4 proforma

5 A.1 Introduction²²

6 The supplier of an implementation that is claimed to conform to Clause 7 shall complete the following
7 protocol implementation conformance statement (PICS) proforma.

8 A completed PICS proforma is the PICS for the implementation in question. The PICS is a statement of
9 which capabilities and options of the protocol have been implemented. A PICS is included at the end of each
10 clause as appropriate. The PICS can be used for a variety of purposes by various parties, including the
11 following:

- 12 a) As a checklist by the protocol implementor, to reduce the risk of failure to conform to the standard
13 through oversight;
- 14 b) As a detailed indication of the capabilities of the implementation, stated relative to the common
15 basis for understanding provided by the standard PICS proforma, by the supplier and acquirer, or
16 potential acquirer, of the implementation;
- 17 c) As a basis for initially checking the possibility of interworking with another implementation by the
18 user, or potential user, of the implementation (note that, while interworking can never be guaranteed,
19 failure to interwork can often be predicted from incompatible PICS);
- 20 d) As the basis for selecting appropriate tests against which to assess the claim for conformance of the
21 implementation, by a protocol tester.

22 A.1.1 Abbreviations and special symbols

23 The following symbols are used in the PICS proforma:

24	M	mandatory field/function
25	!	negation
26	O	optional field/function
27	O,<n>	optional field/function, but at least one of the group of options labeled by
28		the same numeral <n> is required
29	O/<n>	optional field/function, but one and only one of the group of options
30		labeled by the same numeral <n> is required
31	X	prohibited field/function
32	<item>:	simple-predicate condition, dependent on the support marked for <item>
33	<item1>*<item2>:	AND-predicate condition, the requirement must be met if both optional
34		items are implemented
35	<item1>+<item2>:	OR-predicate condition, the requirement must be met if either of the
36		optional items are implemented

²²Copyright release for PICS proformas: Users of this standard may freely reproduce the PICS proforma in this subclause so that it can be used for its intended purpose and may further publish the completed PICS.

1 A.1.2 Instructions for completing the PICS proforma

2 The first part of the PICS proforma, Implementation Identification and Protocol Summary, is to be
3 completed as indicated with the information necessary to identify fully both the supplier and the
4 implementation.

5 The main part of the PICS proforma is a fixed-format questionnaire divided into subclauses, each containing
6 a group of items. Answers to the questionnaire items are to be provided in the right-most column, either by
7 simply marking an answer to indicate a restricted choice (usually Yes, No, or Not Applicable), or by
8 entering a value or a set or range of values. (Note that there are some items where two or more choices from
9 a set of possible answers can apply; all relevant choices are to be marked.)

10 Each item is identified by an item reference in the first column; the second column contains the question to
11 be answered; the third column contains the reference or references to the material that specifies the item in
12 the main body of the standard; the sixth column contains values and/or comments pertaining to the question
13 to be answered. The remaining columns record the status of the items—whether the support is mandatory,
14 optional, or conditional—and provide the space for the answers.

15 The supplier may also provide, or be required to provide, further information, categorized as either
16 Additional Information or Exception Information. When present, each kind of further information is to be
17 provided in a further subclause of items labeled A<i> or X<i>, respectively, for cross-referencing purposes,
18 where <i> is any unambiguous identification for the item (e.g., simply a numeral); there are no other
19 restrictions on its format or presentation.

20 A completed PICS proforma, including any Additional Information and Exception Information, is the
21 protocol implementation conformance statement for the implementation in question.

22 Note that where an implementation is capable of being configured in more than one way, according to the
23 items listed under Major Capabilities/Options, a single PICS may be able to describe all such configurations.
24 However, the supplier has the choice of providing more than one PICS, each covering some subset of the
25 implementation's configuration capabilities, if that would make presentation of the information easier and
26 clearer.

27 A.1.3 Additional information

28 Items of Additional Information allow a supplier to provide further information intended to assist the
29 interpretation of the PICS. It is not intended or expected that a large quantity will be supplied, and the PICS
30 can be considered complete without any such information. Examples might be an outline of the ways in
31 which a (single) implementation can be set up to operate in a variety of environments and configurations; or
32 a brief rationale, based perhaps upon specific application needs, for the exclusion of features that, although
33 optional, are nonetheless commonly present in implementations.

34 References to items of Additional Information may be entered next to any answer in the questionnaire, and
35 may be included in items of Exception Information.

36 A.1.4 Exceptional information

37 It may occasionally happen that a supplier will wish to answer an item with mandatory or prohibited status
38 (after any conditions have been applied) in a way that conflicts with the indicated requirement. No
39 preprinted answer will be found in the Support column for this; instead, the supplier is required to write into
40 the Support column an X<i> reference to an item of Exception Information, and to provide the appropriate
41 rationale in the Exception item itself.

- 1 An implementation for which an Exception item is required in this way does not conform to this standard.
- 2 Note that a possible reason for the situation described above is that a defect in the standard has been
3 reported, a correction for which is expected to change the requirement not met by the implementation.

4 **A.1.5 Conditional items**

- 5 The PICS proforma contains a number of conditional items. These are items for which both the applicability
6 of the item itself, and its status if it does apply—mandatory, optional, or prohibited—are dependent upon
7 whether or not certain other items are supported.
- 8 Individual conditional items are indicated by a conditional symbol of the form “<item>:<s>” in the Status
9 column, where “<item>” is an item reference that appears in the first column of the table for some other
10 item, and “<s>” is a status symbol, M (Mandatory), O (Optional), or X (Not Applicable).
- 11 If the item referred to by the conditional symbol is marked as supported, then 1) the conditional item is
12 applicable, 2) its status is given by “<s>”, and 3) the support column is to be completed in the usual way.
13 Otherwise, the conditional item is not relevant and the Not Applicable (N/A) answer is to be marked.
- 14 Each item whose reference is used in a conditional symbol is indicated by an asterisk in the Item column.

A.1.6 Identification

A.1.6.1 Implementation identification

Supplier (Note 1)	
Contact point for queries about the PICS (Note 1)	
Implementation Name(s) and Version(s) (Notes 1 and 3)	
Other information necessary for full identification— e.g., name(s) and version(s) of machines and/or operating system names (Note 2)	
NOTE 1—Required for all implementations. NOTE 2—May be completed as appropriate in meeting the requirements for the identification. NOTE 3—The terms Name and Version should be interpreted appropriately to correspond with a supplier’s terminology (e.g., Type, Series, Model).	

A.1.6.2 Protocol summary

Identification of protocol specification	IEEE Std 802.1CB™, IEEE Standard for Frame Replication and Elimination for Reliability
Identification of amendments and corrigenda to the PICS proforma that have been completed as part of the PICS	Amd : _____ Cor: _____ Amd : _____ Cor: _____
Have any exceptions been noted? (See A.1.4. The answer, “Yes” means that the implementation does not conform to IEEE Std 802.1CB™.)	Yes [] No []

1 A.2 PICS proforma for Frame Replication and Elimination for Reliability²³

2 A.2.1 Major capabilities/options

Item	Feature	Subclause	Value/Comment	Status	Support
BG	FRER C-component implemented?	5.15		O	Yes [] No []
IS	Stream identification system implemented?	5.3, 5.4, 5.5	One or more of IS, TE, LE, and/or RS must be answered "Yes."	O.1	Yes [] No []
TE	Talker end system implemented?	5.6, 5.7, 5.8		O.1	Yes [] No []
LE	Listener end system implemented?	5.9, 5.10, 5.11		O.1	Yes [] No []
RS	Relay system implemented?	5.12, 5.13, 5.14, 5.15:b, 5.15:c		BG:M + O.1	Yes [] No []
MIB	Does the system implementation support management operations using SMlv2 MIB modules?	5.8, 5.11, 5.14		O	Yes [] No []
YANG	Does the implementation support management operations using the YANG modules?	5.8, 5.11, 5.14		O	Yes [] No []

3 A.2.2 Stream identification component

Item	Feature	Subclause	Value/Comment	Status	Support
IS1	Can the system identify frames using the Null Stream identification function?	5.3:b, 6.4		IS: M	Yes []
IS2	Does the system implement the required managed objects of Clause 9?	5.3:c, 9		IS: M	Yes []
IS3	Can the system identify frames using the Source MAC and VLAN Stream identification function ?	5.4:a, 6.5		IS: O	Yes [] No []
IS4	Can the system encode frames using the Active Destination MAC and VLAN Stream identification?	5.4:b, 6.6		IS: O	Yes [] No [] — ^a
IS5	Can the system identify packets using the IP Stream identification?	5.5:c, 6.7		IS: O	Yes [] No []

²³ Copyright release for PICS proformas: Users of this standard may freely reproduce the PICS proforma in this annex so that it can be used for its intended purpose and may further publish the completed PICS.

Item	Feature	Subclause	Value/Comment	Status	Support
IS6	Can the system identify packets using the Mask-and-match Stream identification ?	5.5:d, 6.8		IS: O	Yes [] No []
IS7	For what additional Stream decodings can the system be configured?	5.5:e		IS: O	—
IS8	Explain the limits on which ports the above features can be configured.	5.5:a		IS: O	—
IS9	Explain the limits on the number of Streams for which the above features can be configured.	5.5:b		IS: O	—

^aIf “No,” supply a reason why.

1 A.2.3 Talker end system

Item	Feature	Subclause	Value/Comment	Status	Support
TE1	Can the system identify frames using the Null Stream identification function?	5.6:b, 6.4		TE: M	Yes []
TE2	Can the system be configured with a Sequence generation function?	5.6:c, 7.4.1		TE: M	Yes []
TE3	Can the system be configured with a Sequence encode/decode function?	5.6:d, 7.8		TE: M	Yes []
TE4	Does the system implement the managed objects of Clause 9 and Clause 10 (10.7 not required)?	5.6:e, 9, 10		TE: M	Yes []
TE5	Can the system identify frames using the Source MAC and VLAN Stream identification function ?	5.7:a, 6.5		TE: O	Yes [] No []
TE6	Can the system encode frames using the Active Destination MAC and VLAN Stream identification?	5.7:b, 6.6		TE: O	Yes [] No [] — ^a
TE7	Can the system be configured with a Stream splitting function?	5.7:c, 7.7		TE: M	Yes [] No [] — ^a
TE8	Can the system identify packets using the IP Stream identification?	5.8:c, 6.7		TE: O	Yes [] No []
TE9	Can the system identify packets using the Mask-and-match Stream identification ?	5.8:d, 6.8		TE: O	Yes [] No []

Item	Feature	Subclause	Value/Comment	Status	Support
TE10	For what additional Stream decodings can the system be configured?	5.8:e		TE: O	—
TE11	Can the system encode frames using HSR sequence tag?	5.8:f, 7.9		TE: O	Yes [] No []
TE12	Can the system encode frames using PRP sequence trailer?	5.8:g, 7.10		TE: O	Yes [] No []
TE13	For what additional Sequence encode/decode functions can the system be configured?	5.8:h		TE: O	—
TE14	Explain the limits on which ports the above features can be configured.	5.8:a		TE: O	—
TE15	Explain the limits on the number of Streams for which the above features can be configured.	5.8:b		TE: O	—

^aIf “No,” supply a reason why.

1 A.2.4 Listener end system

Item	Feature	Subclause	Value/Comment	Status	Support
LE1	Can the system identify frames using the Null Stream identification?	5.9:b, 6.4		LE: M	Yes []
LE2	Can the system be configured with at least two Individual recovery functions?	5.9:c, 7.5		LE: M	Yes []
LE3	Can the system be configured with at least one Sequence recovery function using the MatchRecoveryAlgorithm?	5.9:c, 7.4.2, 7.4.3.5		LE: M	Yes []
LE4	Does the system support the Sequence recovery function using the VectorRecoveryAlgorithm with a value of $\text{frerSeqRcvyHistoryLength} \geq 2$?	5.9:c, 7.4.2, 7.4.3.4		LE: M	Yes []
LE5	Can the system be configured with at least two Individual recovery functions using the MatchRecoveryAlgorithm?	5.9:d, 7.5, 7.4.3.5		LE: M	Yes []
LE6	Can the system be configured with a Sequence decoding function?	5.9:e, 7.8		LE: M	Yes []
LE7	Does the system implement the managed objects of Clause 9 and Clause 10 (10.7 not required)?	5.9:f, 9, 10		LE: M	Yes []

Item	Feature	Subclause	Value/Comment	Status	Support
LE8	Does the Base recovery function process a frame before its FCS has been verified?	7.4.3		LE: M	No []
LE9	Can the system identify frames using the Source MAC and VLAN Stream identification function ?	5.10:a, 6.5		LE: O	Yes [] No []
LE10	Can the system decode frames using the Active Destination MAC and VLAN Stream identification?	5.10:b, 6.6		LE: O	Yes [] No [] — ^a
LE11	Can the system decode packets using the IP Stream identification?	5.11:c, 6.7		LE: O	Yes [] No []
LE12	Can the system identify packets using the Mask-and-match Stream identification ?	5.11:d, 6.8		LE: O	Yes [] No []
LE13	For what additional Stream decodings can the system be configured?	5.11:e		LE: O	—
LE14	Can the system decode frames using HSR sequence tag?	5.11:f, 7.9		LE: O	Yes [] No []
LE15	Can the system decode frames using PRP sequence trailer?	5.11:g, 7.10		LE: O	Yes [] No []
LE16	For what additional Sequence decodings can the system be configured?	5.11:h		LE: O	—
LE17	Can the system be configured with at least two Individual recovery functions using the VectorRecoveryAlgorithm?	5.11:i, 7.5, 7.4.3.4		LE: O	Yes [] No []
LE18	Explain the limits on which ports the above features can be configured.	5.11:a		LE: O	—
LE19	Explain the limits on the number of Streams for which the above features can be configured.	5.11:b		LE: O	—

^aIf “No,” supply a reason why.

1 A.2.5 Relay system

Item	Feature	Subclause	Value/Comment	Status	Support
RS1	Can the system identify frames using the Null Stream identification function?	5.12:b, 6.4		RS: M	Yes []
RS2	Can the system be configured with a Sequence generation function?	5.12:c, 7.4.1		RS: M	Yes []
RS3	Can the system be configured with at least two Individual recovery functions?	5.12:e, 7.5		RS: M	Yes []
RS4	Can the system be configured with at least one Sequence recovery function using the MatchRecoveryAlgorithm?	5.12:e, 7.4.2, 7.4.3.5		RS: M	Yes []
RS5	Does the system support the Sequence recovery function using the VectorRecoveryAlgorithm with a value of $\text{frerSeqRcvyHistoryLength} \geq 2$?	5.12:e, 7.4.2, 7.4.3.4		RS: M	Yes []
RS6	Can the system be configured with at least two Individual recovery functions using the MatchRecoveryAlgorithm?	5.12:f, 7.5, 7.4.3.5		RS: M	Yes []
RS7	Can the system be configured with a Sequence encode/decode function?	5.12:d, 7.8		RS: M	Yes []
RS8	Does the system implement the managed objects of Clause 9 and Clause 10 (including 10.7)?	5.12:g, 9, 10		RS: M	Yes []
RS9	Does the Base recovery function process a frame before its FCS has been verified?	7.4.3		RS: M	No []
RS10	Can the system identify frames using the Source MAC and VLAN Stream identification function ?	5.13:a, 6.5		RS: O	Yes [] No []
RS11	Can the system encode/decode frames using the Active Destination MAC and VLAN Stream identification?	5.13:b, 6.6		RS: O	Yes [] No [] — ^a
RS12	Can the system identify packets using the IP Stream identification?	5.13:c:1, 6.7		RS: O	Yes [] No [] — ^a
RS13	Can the system identify packets using the Mask-and-match Stream identification ?	5.13:c:2, 6.8		RS: O	Yes [] No [] — ^a

Item	Feature	Subclause	Value/Comment	Status	Support
RS14	For what additional Stream identification functions can the system be configured?	5.14:c		RS: O	—
RS15	Can the Stream splitting function be configured on the system?	5.14:d, 7.7		RS: O	Yes [] No []
RS16	Can the system encode/decode frames using HSR sequence tag?	5.14:e, 7.9		RS: O	Yes [] No []
RS17	Can the system encode/decode frames using PRP sequence trailer?	5.14:f, 7.10		RS: O	Yes [] No []
RS18	For what additional Sequence encode/decode functions can the system be configured?	5.14:g		RS: O	—
RS19	Can the system be configured with at least two Individual recovery functions using the VectorRecoveryAlgorithm?	5.14:i, 7.5, 7.4.3.4		RS: O	Yes [] No []
RS20	Can the system be configured for Autoconfiguration via the Managed objects for autoconfiguration?	5.14:j, 7.11, 10.7		RS: O	Yes [] No []
RS21	Explain the limits on which ports the above features can be configured.	5.14:a		RS: O	—
RS22	Explain the limits on the number of Streams for which the above features can be configured.	5.14:b		RS: O	—
RS23	Explain the limits on whether the above features can be configured at in-facing or out-facing positions.	5.14:h		RS: O	—

^aIf “No,” supply a reason why.

1 A.2.6 FRER 802.1Q C-component

Item	Feature	Subclause	Value/Comment	Status	Support
CB1	Does the FRER 802.1Q C-component conform to the required and optional behaviors required of an IEEE 802.1Q C-VLAN component?	5.15:a		BG:M	Yes []
CB1	Does the FRER 802.1Q C-component conform to the placement of the FRER functions of Clause 8?	5.15:d, 8		BG:M	Yes []
CB2	Does the FRER 802.1Q C-component implement all required managed objects?	8.4		BG:M	Yes []

2 A.2.7 Common requirements

Item	Feature	Subclause	Value/Comment	Status	Support
COM1	Does the R-TAG use the specified EtherType?	7.8.1		M	Yes []
COM1	Is the Reserved field of the R-TAG transmitted as 0 and ignored on receipt?	7.1.1:d		M	Yes []
COM2	Do all managed object counters roll over to 0 from their maximum value?	10.1		M	Yes []
COM3	Can the system be configured with a latent error detection function?	7.4.4	Latent error detection required if sequence recovery supported	LE+RS: M	N/A [] Y []
COM4	Is the minimum value for frerSeqRcvyLatentErrorPeriod no larger than 1 s?	10.4.1.12.2		LE+RS: M	N/A [] Y []
COM5	Is the RemainingTicks decremented at least 100 ticks/s?	7.4.3.2.5		LE+RS: M	N/A [] Y []
COM6	Does the system return an error if an attempt is made to configure conflicting requirements?	10		M	Yes []
COM7	Is the minimum supported value of frerSeqRcvyLatentResetPeriod no larger than 1 s?	10.4.1.12.4		LE+RS: M	N/A [] Y []
COM8	Are all counters 64 bits in length on links faster than 650 000 000 bits/s?	10.8, 10.9		M	N/A [] Y []

1 A.2.8 Management Information Base (MIB)

Item	Feature	Subclause	Value/Comment	Status	Support
	If item MIB is not supported, mark N/A.				N/A []
MIB1	Is the IEEE8021-STREAM-IDENTIFICATION-MIB module fully supported (per its MODULECOMPLIANCE)?	11.5.1		MIB:M	Yes [] No []
MIB2	Is the IEEE8021-FRER-MIB module fully supported (per its MODULECOMPLIANCE)?	11.5.2		MIB:O	Yes [] No []

2 A.2.9 YANG

Item	Feature	Subclause	Value/Comment	Status	Support
	If item YANG is not supported, mark N/A.				N/A []
YANG1	Is the <i>ieee802-dot1cb-stream-identification-types</i> module supported?	12.6.2.1		YANG:M	Yes [] No []
YANG2	Is the <i>ieee802-dot1cb-stream-identification</i> module supported?	12.6.2.2		YANG:M	Yes [] No []
YANG3	Is the <i>ieee802-dot1cb-frer-types</i> module supported?	12.6.2.4		YANG:O YANG*YANG4:M	Yes [] No []
YANG4	Is the <i>ieee802-dot1cb-frer</i> module supported?	12.6.2.5		YANG:O	Yes [] No []
YANG5	Is the <i>ieee802-dot1cb-mask-and-match</i> module supported?	12.6.2.3		YANG:O	Yes [] No []

3

Annex B

(informative)

Interoperability with other standards

IEC 62439-3 defines High-availability Seamless Redundancy (HSR) and the Parallel Redundancy Protocol (PRP). IEEE Std 802.1CB™ Frame Replication and Elimination for Reliability (FRER) has features that enable, though the procedures are not specified by this standard, to achieve a degree of interoperation between systems employing IEEE Std 802.1CB™, and systems employing some other, similar protocol, including HSR and PRP, and RFC 3985 pseudowires (see [B6]).

B.1 Sequence number size

HSR, PRP, and pseudowires all use 16-bit sequence numbers. This standard supports the same sized sequence_number subparameter, in order to be maximally compatible.

B.2 Per-Stream versus per-source sequencing

IEC 62439-3 HSR/PRP uses one variable per source MAC address to sequence Ethernet frames. That is, frames belonging to different flows, and sent to different destinations, all share a single sequence number space. FRER, as defined in this standard, can be configured in the same way, or can be configured to generate a separate sequence number space for each Stream. For interoperability with HSR or PRP, per-source sequence numbering should be configured by the user.

A problem needs to be considered if one configures a VectorRecoveryAlgorithm (7.4.3.4) to receive Intermittent Streams (item c in 7.1.1) transmitted by an HSR/PRP end system. The HSR/PRP end system uses a single sequence number, not one per Stream. If that end system is transmitting more than one Stream, and some Listener is receiving less than all of those Streams, then that Listener can observe gaps in the sequence_number subparameters of the received packets, which will be counted in frerCpsSeqRcvyLostPackets (10.8.7). For both Intermittent Streams and Bulk Streams, the Base recovery function's frerSeqRcvyHistoryLength managed object (10.4.1.6) has to be large enough to accommodate both the delivery time difference and the maximum possible gap due to packets sent to other destinations, because packets received outside the window defined by frerSeqRcvyHistoryLength are discarded. If frerSeqRcvyHistoryLength is not large enough, duplicate packets will be delivered.

1 Annex C

2 (informative)

3 Frame Replication and Elimination for Reliability in systems

4 The building blocks described in Clause 6 and Clause 7 can be put together in many ways to achieve
5 particular goals. A few examples are given in this annex. These examples are chosen, rather arbitrarily, to
6 illustrate the range of applicability of FRER; they are not meant to constrain its use or to describe the only
7 way to accomplish any given goal.

8 C.1 Example 1: End-to-end FRER

9 Illustrated in Figure C-1 is a simple network utilizing FRER for a single Compound Stream. In this example,
10 all FRER functions are confined to the two end systems B and G. “Split” indicates that the Stream splitting
11 function (7.7) is acting on transmitted packets. “Seq.” and “Rec.” indicate whether the Sequencing function
12 is acting on transmitted packets (Sequence generation function, 7.4.1) or received packets (Sequence
13 recovery function, 7.4.1, 7.4.2).

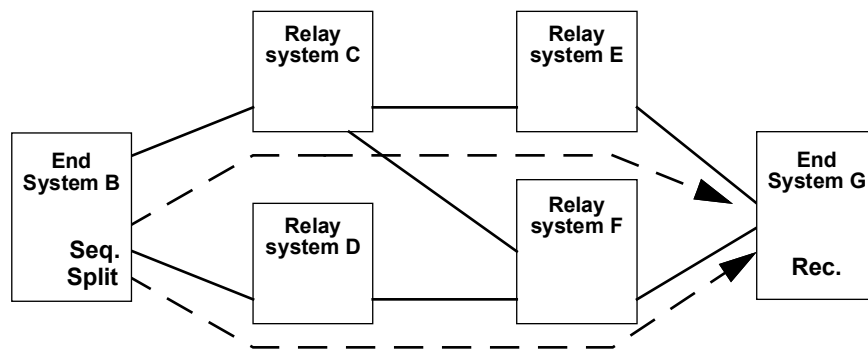


Figure C-1—Dual-homed end systems using Link Aggregation

14 In this example, both end systems utilize IEEE 802.1AX Link Aggregation along with FRER in order to
15 connect their two ports (each) to a single protocol stack, as shown in Figure C-2 (end system B) and
16 Figure C-3 (end system G). In end system B, each packet is replicated and given two different
17 stream_handle subparameter values. The two stream_handles result in the two packets being assigned two
18 different VLAN IDs. Based on those VLAN IDs, the two packets are both transmitted, on different physical
19 ports, by Link Aggregation. The Distributed Resilient Network Interconnect (DRNI) feature of
20 IEEE Std 802.1AX-2014 allows each end system’s aggregation to terminate in two relay systems.

21 In this example, the two-port end systems never relay packets from one port to the other; they are strictly end
22 systems, and never act as relay systems. See C.2 and C.6 for similar examples that use alternative methods
23 for building an end system.

24

25

26

1

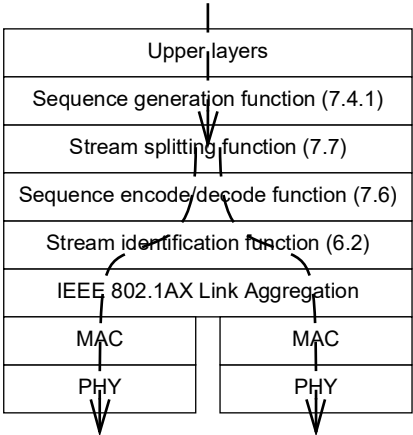


Figure C-2—Protocol stack for End System B in Figure C-1

2

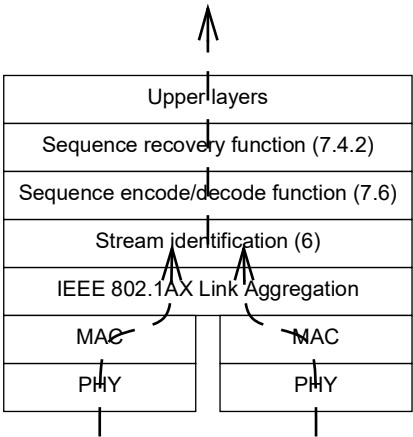


Figure C-3—Protocol stack for End System G in Figure C-1 and Figure C-4

3 C.2 Example 2: Various stack positions

4 Illustrated in Figure C-4 is a simple network utilizing FRER for a single Compound Stream, in which a relay
5 system serves as a proxy for an end system that has no FRER capability (see item h in 7.1.1). In this figure,
6 the numbers (1, 2, 3) are points in item e in 7.1.1. “Split” indicates that the Compound Stream is split into
7 two streams by the ordinary bridge multicast mechanism, with the {VLAN, destination address} altered by
8 Stream identification functions on output. [No Stream splitting function (7.7) is used.] “Seq.” and “Rec.”
9 indicate whether the Sequencing function is acting on transmitted packets (sequence generation) or received
10 packets (sequence recovery).

1

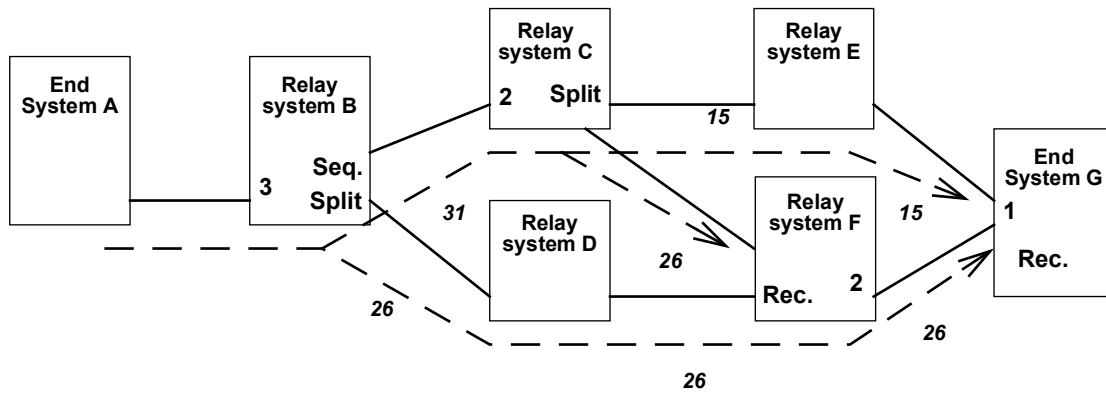


Figure C-4—IEEE Std 802.1CB™ flexible positioning

2 In Figure C-4, End System A is transmitting a Stream, but has no FRER functions. Relay system B
3 transforms that Stream into a Compound Stream by sequencing the packets and splitting them into Member
4 Streams 26 and 31 to go to relay systems C and D. Relay system C further splits the Stream into Streams 15
5 and 26 to go to relay systems E and F. (No sequencing is required; the packets were sequenced by relay
6 system B.) Relay system F combines the two Member Streams 26 from relay systems C and D, outputting a
7 single copy of each Packet on Stream 26 to End System G. End System G merges the two remaining
8 Member Streams 15 and 26, and discards the extras.

9 NOTE—In this example, stream_handle values are shown on the wires, being relayed from system to system. This is for
10 the convenience of the example. In an actual network, the stream_handle is an arbitrary integer that has meaning only
11 within a system, and is mapped to or from an external representation by the Stream identification functions (6.2) in the
12 various systems. That external representation can, but does not necessarily, include an explicit field corresponding to a
13 stream_handle subparameter.

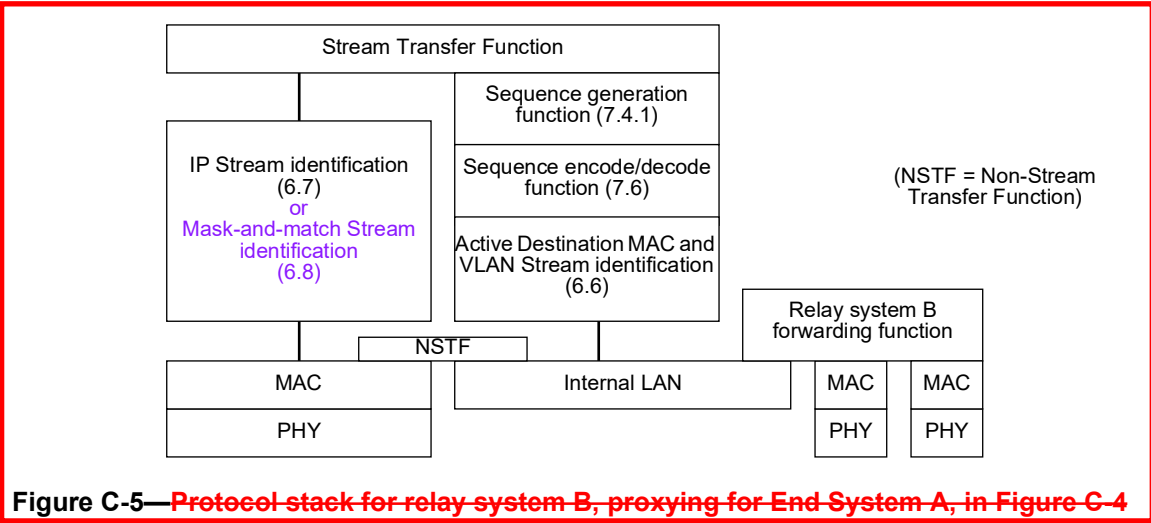
14 *Change the text as shown :*

15 Figure C-5 illustrates relay system B in Figure C-4. As the packets enter from the left, from End System A,
16 they pass first through a Stream identification function [IP Stream identification (6.7) or Mask-and-match
17 Stream identification (6.8)], which identifies the Stream. The Stream Transfer Function delivers the packet
18 with all TSN parameters, including the stream_handle subparameter, to the Sequence generation function
19 (7.4.1, marked “Seq.” in Figure C-4), which adds a sequence_number subparameter with a steadily-
20 increasing integer sequence value (modulo the size of the packet field carrying the sequence_number). The
21 sequence_number subparameter is encapsulated into the packet by the Sequence encode/decode function
22 (7.6). A Stream identification function [this time, In-facing Active Destination MAC and VLAN Stream
23 identification (6.6)] modifies the two packets’ destination MAC addresses and VLANs for identification
24 through the bridged network. Relay system B’s forwarding function then outputs the two packets on two
25 different ports. The external form of the packets are labeled differently, as indicated by the italic numbers 26
26 and 31 in Figure C-4.

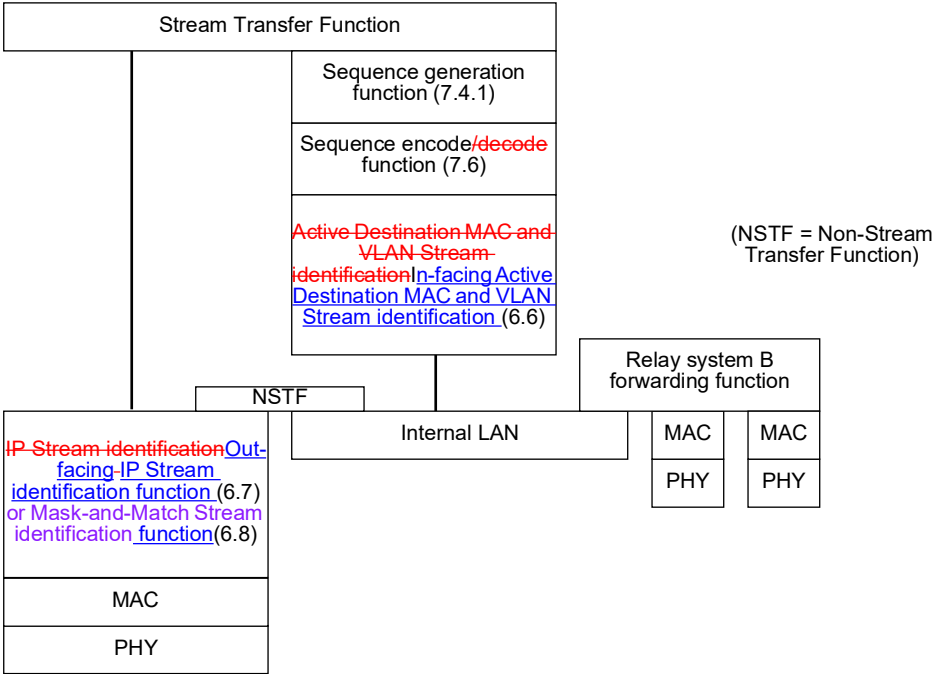
27 Figure C-6 illustrates relay system C in Figure C-4. The packets on Stream 31 enter from the left, from relay
28 system B. On output, they pass first through a passive Stream identification (Clause 6) function, which
29 identifies the Stream. The Stream Transfer Function delivers the packet with all TSN parameters, including
30 the stream_handle subparameter, to an active Stream identification (Clause 6) function. No Sequencing
31 function, Sequence generation function, or Sequence recovery functions are needed, because the packets
32 have already been sequenced (by relay system B) and none are being discarded. Stream identification
33 encapsulates the two packets differently on the two output ports (only one output port is shown in Figure C-6),
34 marking them as belonging to Streams 15 and 26, and relay system C’s forwarding function outputs the two
35 packets on two different ports.

1 *Change Figure C-5 as shown :*

2



3



4 *Change Figure C-6 as show :*

5

6

7

Figure C-6—Protocol stack for relay system C in Figure C-4

The diagram illustrates the Stream Transfer Function (STF) architecture. It shows the flow of data between Relay system C and the Internal LAN. The components and their connections are as follows:

- Relay system C forwarding function**: This block is connected to the **Internal LAN** and the **Stream Transfer Function**.
- Internal LAN**: This block is connected to the **Relay system C forwarding function** and the **Stream Transfer Function**.
- Stream Transfer Function**: This block is connected to the **Relay system C forwarding function**, the **Internal LAN**, and the **NSTF**.
- NSTF**: This block is connected to the **Stream Transfer Function** and the **MAC** and **PHY** blocks.
- MAC**: This block is connected to the **NSTF** and the **PHY** block.
- PHY**: This block is connected to the **MAC** block.

The diagram also includes labels for **In-facing** and **Out-facing** active and passive stream identification (Clause 6).

1

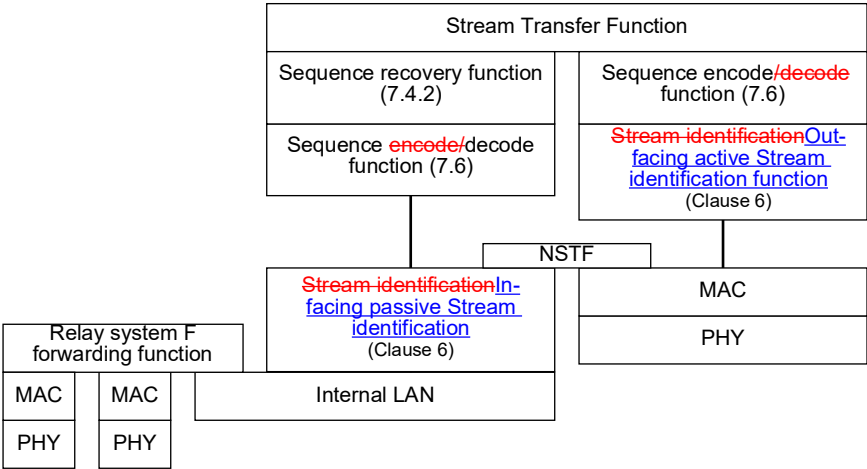
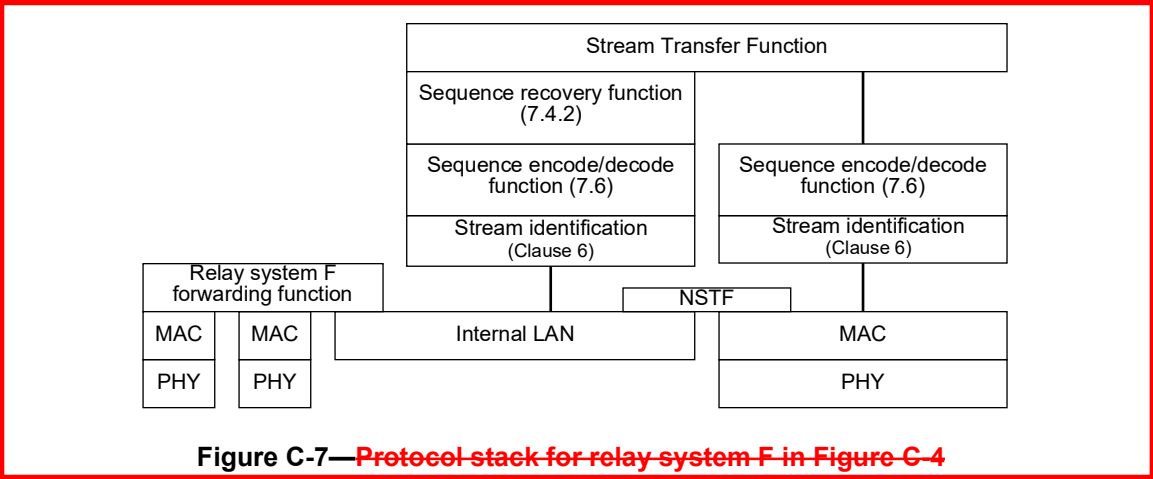
2 Change the text as shown :

3 Figure C-7 illustrates relay system F in Figure C-4. In this case, the TSN layers are on the output port, rather
4 than the input port, as was the case for relay system B. Data packets from both Streams 26 are brought
5 together simply by the fact that they are both output to the same port. On that port, the [In-facing passive](#)
6 Stream identification function gives them both the same stream_handle subparameter, effectively merging
7 them into a single Stream. A Sequence encode/decode function (7.6) extracts the sequence_number
8 subparameters from the packets, so that the Sequence recovery function (7.4.2) can discard the replicates.
9 The Stream Transfer Function delivers all of these parameters to a Sequencing function and an [Out-facing](#)
10 [active](#) Stream identification function that re-encapsulate the packets, marking them all as belonging to
11 Stream 26, the same as when they were received. In this particular example, unlike Figure C-5, no
12 translation of Stream identification functions is being performed.

13

14 Finally, the protocol stack in the right-hand End System G of Figure C-4 is shown in Figure C-3. We assume
15 in this example that relay systems E and F use IEEE Std 802.1AX™ Distributed Resilient Network Interface
16 (DRNI), and that End System G uses IEEE 802.1AX Link Aggregation, in order to make End System G's
17 two physical ports appear, to the network, to be a single port, even though they are connected to two
18 different relay systems. The packets on Streams 15 and 26 are brought together by Link Aggregation. The
19 Stream identification function assigns the same stream_handle subparameter to packets of both Streams,
20 effectively merging them into a single Stream. They then pass through the Sequence recovery function,
21 which deletes the replicates for delivery to the upper layers. See Figure C-2 for a similar example of a
22 Talker, and C.6 for another type of two-port end system.

23 ***Change Figure C-7 as shown :***



1 C.3 Example 3: Ladder redundancy

2 Illustrated in Figure C-8 is a network implementing “ladder redundancy.” This network will continue to
3 function in the face of multiple failures, because the Stream is repeatedly split and remerged.

4 In Figure C-8, the Talker end system sequences Stream 31, then splits it into two Streams 26 and 31, sending
5 one on each of its two ports. The network “ladder” has two “rails,” an upper (relay systems B and D) and a
6 lower (relay systems C and E). The “rungs” are the connections B–C and D–E. Each relay system sends the
7 Stream received on the rail from the left both to the right, along the rail, and up or down, over the rung. It
8 forwards the packets received from the rung only to the right, along the rail. At each output port to the rail,
9 the end systems have a Sequence recovery function (7.4.2) to eliminate duplicates.

10 Each of the relay systems in Figure C-8 uses the protocol stack illustrated in Figure C-7.

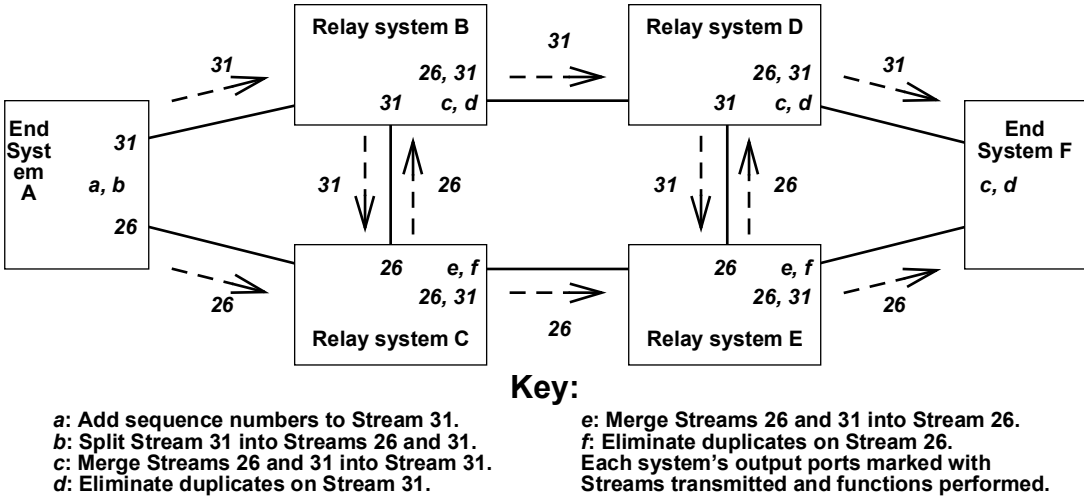


Figure C-8—Ladder redundancy

1 C.4 Example 4: Multicast trees

2 Illustrated in Figure C-9 is a multicast FRER application. Each Stream is a multicast Stream. There are two
3 paths to get from the Talker to each Listener; no single failure can disrupt both paths to any relay system.
4 Presumably, each relay system has a Sequence recovery function (7.4.2) on each port to each Listener, so
5 that each Listener receives only one copy of each packet of the Stream.

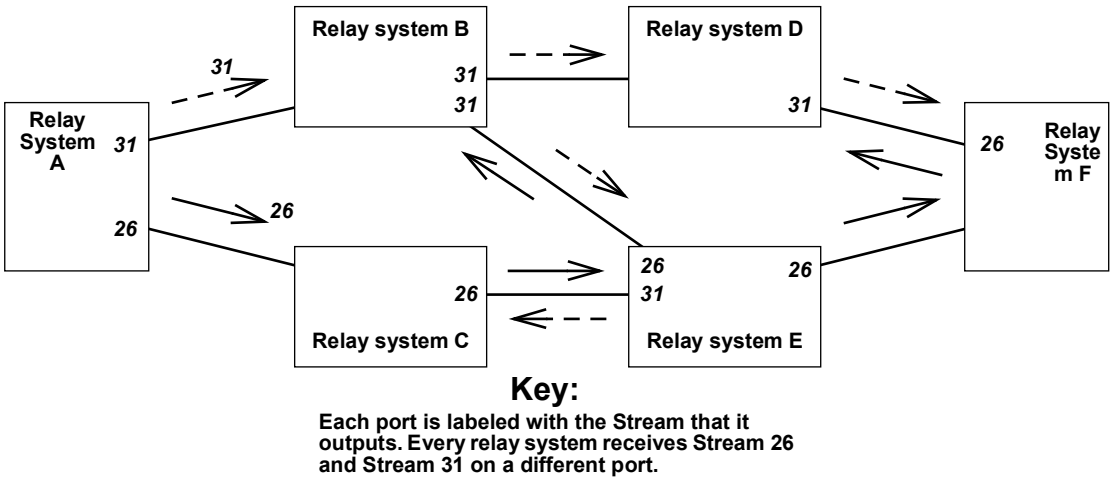


Figure C-9—Multicast trees

6 NOTE—The reader may find it informative to see that not all packets are replicated on all links in Figure C-9. It is not
7 necessary to use each link twice; it is only necessary to get each packet to each relay system twice.

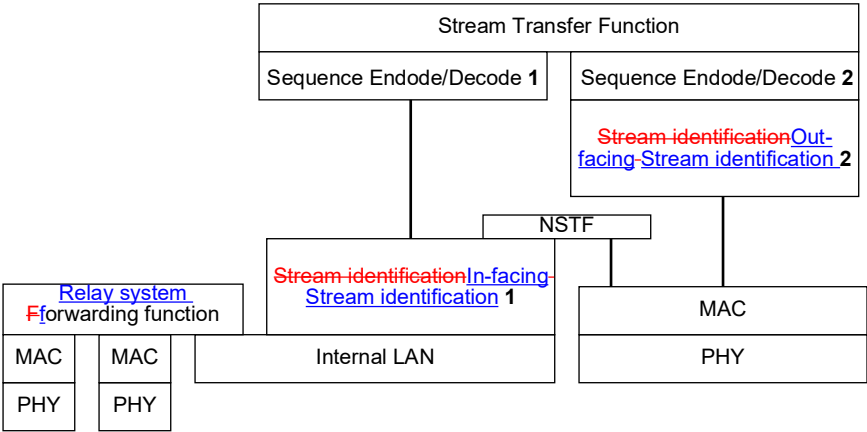
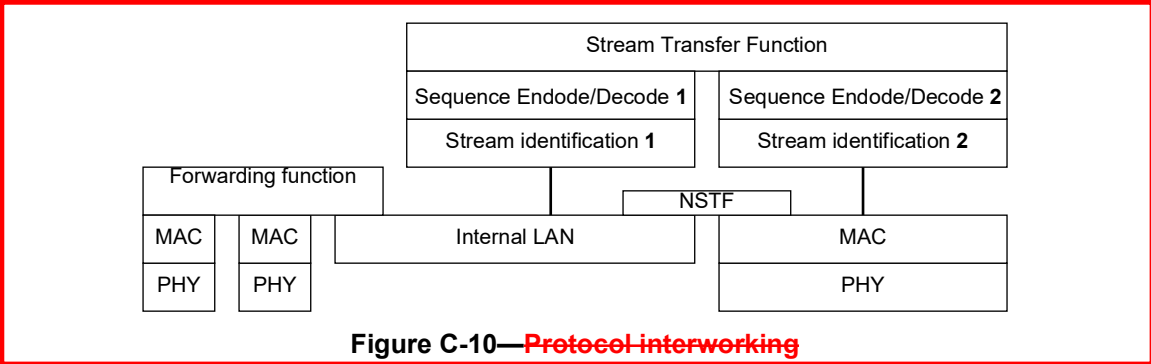
8 C.5 Example 5: Protocol interworking

9 Figure C-10 illustrates a simple protocol interworking function in one port of a relay system. In this
10 example, two different encapsulation schemes **1** and **2** are used for the two legs of the Stream Transfer
11 Function, so that packets are transformed from using one encapsulation to using the other encapsulation as

1 they pass through the port. No additional functions, e.g., a Sequence recovery function (7.4.2) are shown,
2 although they would be perfectly admissible. If this were a port of a bridge attached to an end system,
3 encapsulation **1** could be the Active Destination MAC and VLAN Stream identification (6.6), and
4 encapsulation **2** could be the IP Stream identification (6.7) or the Mask-and-match Stream identification
5 (6.8). The net result for the end system could be to make a specific unicast IP Stream use a specific multicast
6 destination address and VLAN, in order to direct the packet through a specific path through the bridged
7 network. Presumably, a similar interworking pair at the other end of the Stream would restore the packet to
8 its original destination MAC address and VLAN.

9 *Change Figure C-10 as shown:*

10



11 **C.6 Example 6: Chained two-port end systems**

12 Illustrated in Figure C-11 is a simple network utilizing FRER for a single Compound Stream. In this
13 example, all FRER functions are confined to the two end systems B and G. “Seq.” and “Rec.” indicate
14 whether the Sequencing function is acting on transmitted packets (sequence generation) or received packets
15 (sequence recovery). In this example, unlike that in C.1, both end systems are composite systems; each uses
16 a separate one-port end system (B1, G1) attached to a three-port FRER C-component (B2, G2) in order to
17 connect their two ports (each) to a single protocol stack, as shown in Figure C-12.

18

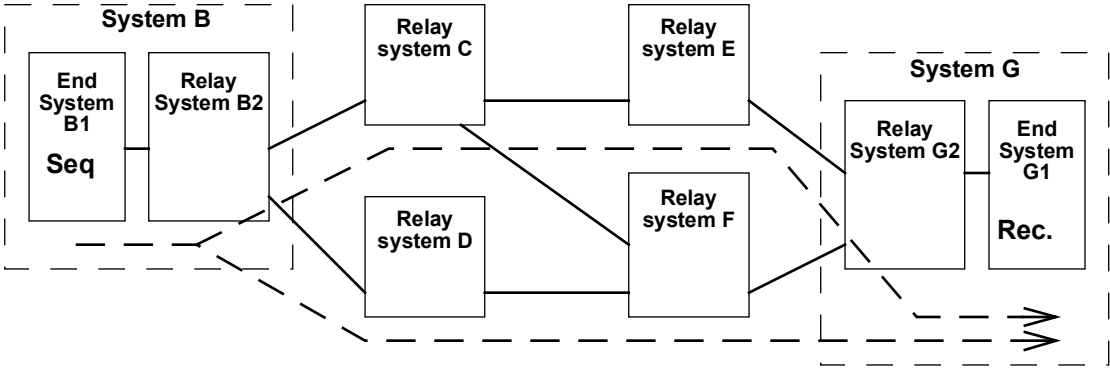


Figure C-11—Dual-homed end systems using 3-port bridge

1

2 *Change Figure C-12 as shown*

3

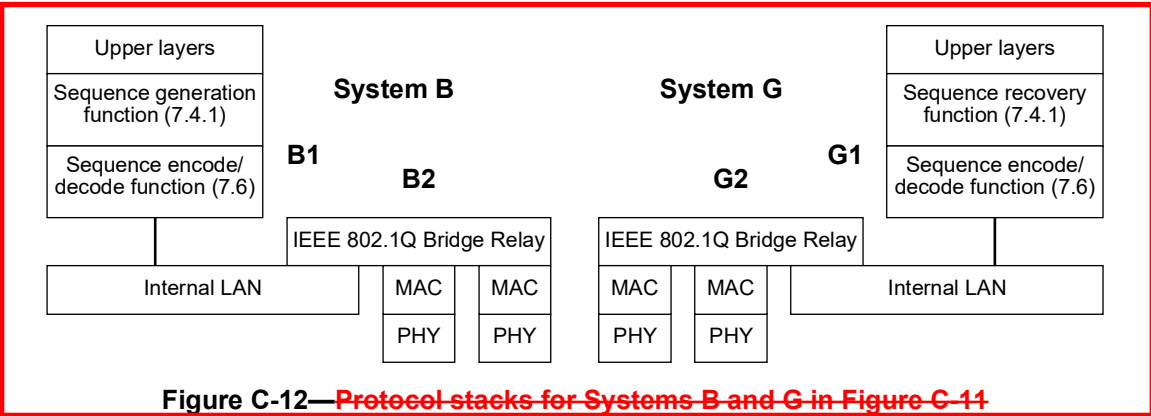


Figure C-12—Protocol stacks for Systems B and G in Figure C-11

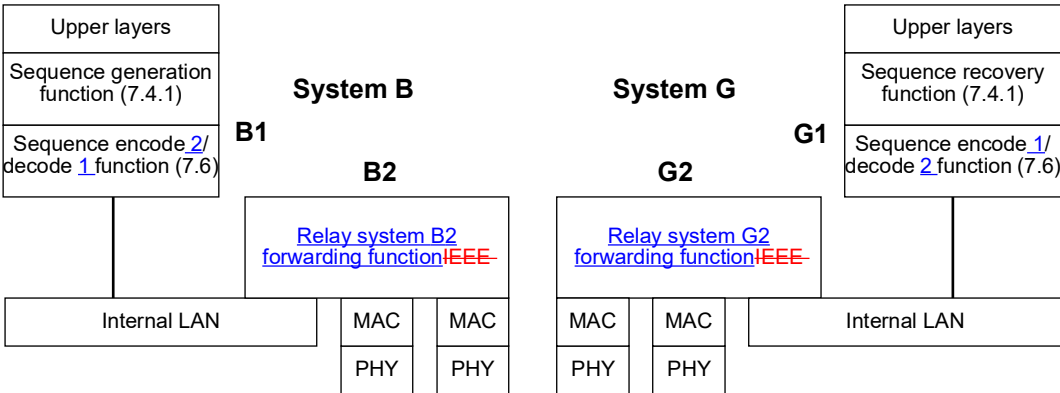


Figure C-12—Protocol stacks for Systems B and G in Figure C-11

4 In this example, we assume that the relay systems are all FRER C-components, and that the network has
5 been configured so that:

- 1 a) The Stream packets carry a multicast destination MAC address;
- 2 b) Bridge B2 has been configured to transmit the (multicast) Member Stream on both ports; and
- 3 c) The other Bridges, especially G2, have been configured to allow both Member Streams to enter
- 4 Bridge G2, but that Bridge G2 passes the Stream to end system G1 only (that is, the normal rules for
- 5 a multicast tree are broken).

6 Given this configuration, no Stream splitting function (7.7) is necessary for end system B1; packet
7 duplication is handled by the multicast forwarding functions performed in its associated relay system B2.

8 Unlike the example in C.1, the two-port composite end systems' Bridge functions (B2, G2) are required to
9 play their part in the network as Bridges, operating protocols and forwarding packets.

10 C.7 Cautions

11 Explicit path creation carries a clear danger when combined with automatic network forwarding, e.g., as
12 defined by IEEE Std 802.1Q™. Figure C-13 shows an explicit tree (the dashed arrows) established in relay
13 system D that sends a particular Stream to relay system B and end system F. (Perhaps, at one time, the
14 Talker was attached to relay system D and the Listeners were towards B and F.) Now, end system A starts
15 transmitting that Stream along an IEEE 802.1Q spanning tree. When the Stream hits relay system D, it is
16 directed back to relay system B, and the Stream loops, multiplying the packets until the bandwidth is
17 saturated. Note also that, if this is a bridged network, relay system B (a bridge) learns conflicting
18 information about the path to the Stream's source; System A's source MAC address is being received both
19 from end system A and relay system D.

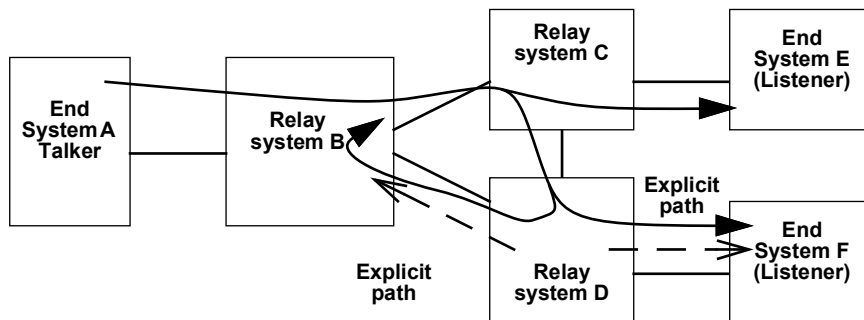


Figure C-13—Explicit path causing a loop

20 In order to avoid this situation in a bridged network, the application or individual that sets up the explicit
21 paths can configure them so that they are kept, end-to-end, on separate VLANs from traffic that is not
22 following explicit paths. On the explicit path VLANs, source learning is not performed, and destination
23 addresses not found in the filtering database are discarded, not flooded.

24 C.8 Balancing tag insertion and removal

25 A Sequence encode/decode function will often add something to a packet. The R-TAG described in 7.8, for
26 example, adds six octets. In a network in which not all end systems contain a Sequence encode/decode
27 function (7.6), instances of that function can be configured in relay systems adjacent to the end systems
28 lacking the layer, in order to remove the extra information (e.g., the R-TAG) from the packets before
29 delivery to the end system. If the tag is not removed, and the end system has no Sequence encode/decode
30 function, the end system will be unable to parse the received packet. It is up to the network administrator
31 and/or configuration and management software to make sure that this does not happen.

1 C.9 FRER and reserved bandwidth

2 Satisfying the Zero congestion loss goal (item k in 7.1.1) can require the forwarding of packets to be
3 delayed. The problem is that the different paths taken by packets in a Compound Stream can take different
4 times to reach various Sequence recovery functions (7.4.2). If these times are very different, and if the faster
5 path fails and then recovers, the receiving Sequence recovery function can be presented with a double-rate
6 load of packets that all have to be delivered.

7

8 As an example, Figure C-14 illustrates a network with two Member Streams being merged into a single
9 Stream by a Sequence recovery function. The two paths are of different lengths such that, typically, there are
10 40 more packets in flight on one path than on the other.

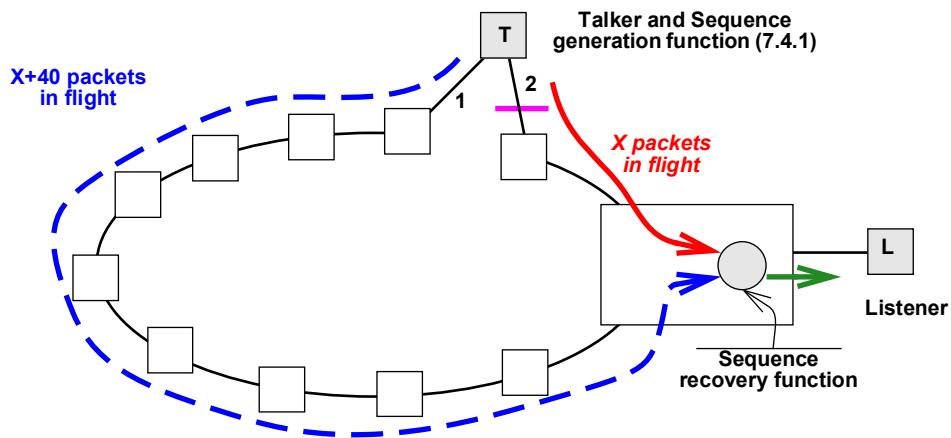


Figure C-14—Example of Long and short paths

11 Let us examine a hypothetical sequence of events as Link 2 in Figure C-14 fails, and then recovers. In the
12 following list, the sequence numbers of packets taking the path through Link 2 (the short path) are in *italics*,
13 and packets that are discarded by the Sequence recovery function are marked with asterisks (*).

- 14 a) The network is in a steady state. The Sequence recovery function receives packets:
15 *1040*, 1000*, *1041*, 1001*, *1042*, 1002*, ...
16 That is, excepting the occasional random packet loss (not shown), the packets taking the *short path*
17 (Link 2) are passed on, and those taking the long path (Link 1) are discarded.
- 18 b) Link 2 fails sometime later (4000 packets later). The Sequence recovery function receives:
19 *5040*, 5000* (failure occurs), *5001**, 5002*, ...
20 That is, for a while at least, no packets are passed on; those in the slow path have already been seen.
- 21 c) The discards continue with no packets passed until the backlog on the slow path is emptied:
22 5038*, 5039*, 5040*, *5041*, 5042, 5043, ...
23 Normal output rates then resume, with all packets coming from the long path (Link 1).
- 24 d) When the *short path* (Link 2) heals (3000 packets later), the Sequence recovery function receives:
25 7998, 7999, (link heals) *8040*, 8000, *8041*, 8001, *8042*, 8002, ...
26 Note that the Sequence recovery function is now passing packets at twice the normal rate, until the
27 40-packet backlog on the long (Link 1) path has been passed along.
- 28 e) Finally, the backlog is exhausted, and the Sequence recovery function resumes the normal output
29 rate, passing packets from the *short path* (Link 2) and discarding those from the long path (Link 1):
30 *8078*, 8038, *8079*, 8039, *8080*, 8040*, *8081*, 8041*, *8082*, 8042*, ...

1 Zero congestion loss goal (item k in 7.1.1) can require that packets be output to the Listener at a fixed
2 maximum rate over some observation interval smaller than the transmission time of this 80-packet burst.
3 (See, for example, Clauses 34 and 35 of IEEE Std 802.1Q™-2022.) In that case, some means of buffering,
4 and thus delaying the packets in this burst, is required at or near the Sequence recovery function.

5 This buffering and/or delaying is beyond the scope of this standard. We can observe, however, that the extra
6 buffering required by FRER in order to prevent congestion loss when bandwidth reservation is employed,
7 due to this problem, can be calculated separately from the buffering required for the non-FRER case, based
8 on the bandwidth of the Compound Stream and the difference in worst-case delivery latency along the two
9 (or more) Member Streams' paths.

10 C.10 Use of the Individual recovery function

11 The Individual recovery function is described in 7.5, and a sample network employing the Individual
12 recovery function is illustrated in Figure 7-3. The primary use of the Individual recovery function is to
13 prevent stale data from being transmitted in the event that one of the Member Streams of a Compound
14 Stream fails such that the same packet is sent over and over again. Imagine, in Figure 7-3, that the leftmost
15 system fails on its upper port, resending the packet with sequence_number 5 over and over again on Member
16 Stream 1. The [VectorRecoveryAlgorithm \(7.4.3.4\)](#) of the Sequence recovery functions (7.4.2) in the two
17 systems receiving the Member Streams 1 and 2 will discard the repeated packets until the sequence_number
18 subparameter on the good Member Stream 2 wraps around after 65 536 packets. [When sequence number 5](#)
19 [gets accepted due to the history window covering it again, the next packet 5 of Stream 1 is accepted and](#)
20 [relayed to the next stage. The desired Member Stream 2's packet 5 is discarded when it arrives a number of](#)
21 [packets later, depending on the window size and the number of member streams.](#) If Individual recovery
22 functions are configured for Member Stream 1 as shown by the triangles in the two systems receiving that
23 Stream, then all of the packet 5s after the first will be discarded there, and never reach the Sequence
24 recovery functions.

25 C.11 Use of autoconfiguration

26 Every last detail of the operation of FRER on each individual Stream can be configured by using the
27 managed objects in Clause 9 and Clause 10. Often, however, there is enough regularity in the use of FRER
28 across various Streams in a network that Autoconfiguration (7.11) can be used. There are four steps to using
29 Autoconfiguration in a given relay system or end system, as follows:

- 30 a) Deciding on which port(s) packets belonging to Member Streams will be received and transmitted,
31 using what translations are required that do not use active Stream identification functions (C.11.1).
- 32 b) Deciding which packets can trigger Autoconfiguration (C.11.2).
- 33 c) Deciding which method for encoding the sequence_number subparameter will be used for input and
34 output on each port (C.11.3).
- 35 d) Deciding on whether Individual recovery functions, Sequence recovery functions, or both, are to be
36 automatically instantiated, and what controlling parameters are to be used for each instantiation
37 (C.11.4).

38 NOTE—Although the following examples describe interoperability between HSR/PRP and the R-TAG, such
39 interoperability is not the primary reason for defining autoconfiguration. Autoconfiguration greatly simplifies the
40 amount of configuration required for networks that use only the R-TAG.

41 C.11.1 Routing and labeling Member Streams

42 Figure C-15 illustrates an example network with four Member Streams constituting a Compound Stream.
43 This example is similar to that illustrated in Figure 7-3.

1 In general, the selection of paths taken by Member Streams through the network is not determined by FRER,
2 but by other standards. Autoconfiguration does not provide a means of establishing active Stream
3 identification functions to perform per-Stream identification transformations. In Figure C-15, let us assume
4 for the sake of example that Member Stream 1 and Member Stream 2 both have the same destination MAC
5 address, but are on two special VLANs 1000 and 1001, used exclusively in this network for fully managed
6 pinned-down paths. That is, these two VLANs are not controlled by the topology protocols defined in
7 IEEE Std 802.1Q. Let us further suppose that Member Stream 3 and Member Stream 4 are similar, but both
8 use the same VLANs 2000. (Presumably, source address learning is disabled on this VLAN.)

9

1

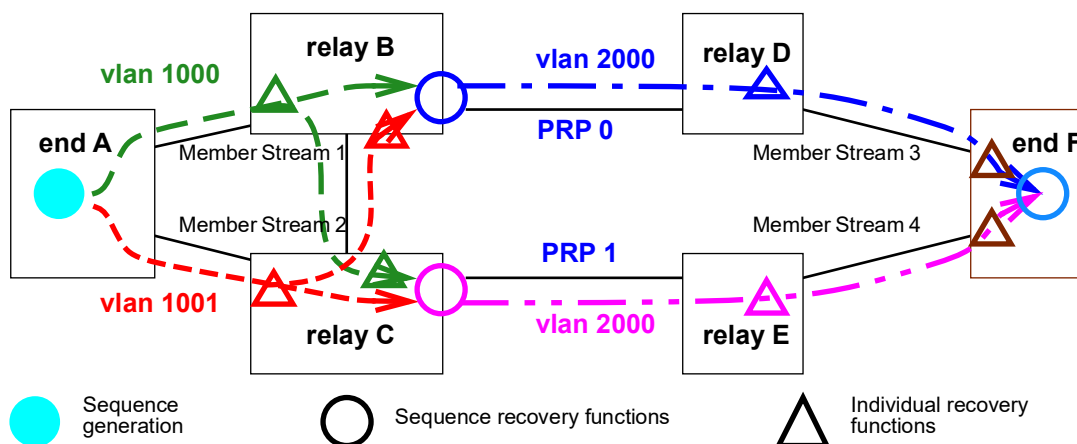


Figure C-15—Autoconfiguration example

2 Looking at relay system B in Figure C-15, we can see that all packets in Member Stream 3 are to be output
 3 on VLAN 2000 with a PRP sequence trailer specifying LanId 0. That means that packets from Member
 4 Streams 1 and 2 that are accepted by the Sequence recovery function in relay system B must be translated
 5 from VLAN 1000 or 1001 to VLAN 2000. The Autoconfiguration facility of 7.11 does not automatically
 6 setup or change `vlan_identifiers`. Relay system B must be configured to perform a VLAN ID translation on
 7 its right-hand port, translating both VLAN 1000 and VLAN 1001 to VLAN 2000 on output. If there are a
 8 great many Compound Streams, then presumably they would share the same VLAN assignments and thus
 9 make use of the same VLAN translations. IEEE 802.1Q bridges have such a VLAN ID translation
 10 capability.

11 Although not shown in Figure C-15, we can imagine that there are Streams with Talkers in the right side of
 12 the figure and listeners on the left. Additional VLAN mapping would then be required on the left and bottom
 13 ports of relay system B to map VLAN 2000 to VLAN 1000 and VLAN 1001. If the VLAN situation were
 14 significantly more complex, then the network might not be suitable for Autoconfiguration.

15 C.11.2 Recognizing packets that trigger autoconfiguration

16 In the example in Figure C-15, let us consider only relay system B, and only packets that follow the same
 17 routes as Member Streams 1 and 2. In that case, we expect relay system B to receive packets suitable for
 18 Autoconfiguration only on its left and bottom ports, all encoded with the R-TAG (7.8). Thus, an
 19 administrator would construct a `frerAutSeqEntry` (10.7.1.1) in the Sequence autoconfiguration table
 20 specifying that tag on those ports, for VLANs 1000 and 1001. This entry will, when a matching packet is
 21 received, create a `tsnStreamIdEntry` (9.1.1) in the Sequence identification table (10.5) using Source MAC
 22 and VLAN Stream identification (6.5). Since the R-TAG contains no indication as to which Member Stream
 23 a packet belongs, the entry is tied to the port and VLAN on which the packet was received. In this way, all of
 24 the Member Streams from end system A that take the path of Member Stream 1 are assigned one
 25 `stream_handle`, and all those following the path of Member Stream 2 get a second `stream_handle`.

26 Again, we can imagine that there are Streams with Talkers in the right side of Figure C-15 and listeners on
 27 the left. A second `frerAutSeqEntry` is required in the Sequence autoconfiguration table is required to
 28 recognize, for Autoconfiguration, the packets coming in on relay system B's right-hand port, and create
 29 entries in the Sequence identification table. This single entry specifies packets encoded with the PRP
 30 sequence trailer (see a further explanation of this in C.11.3), and includes all PRP ports—in this example,
 31 just the one. A `tsnStreamIdEntry` created for HSR or PRP by this entry that assigns packets a `stream_handle`
 32 would not be tied to the port and VLAN on which the packet was received, but to all ports covered by the

1 tsnStreamIdEntry, and to the LanId or PathId contained in the packet. That is, the contents of the HSR
2 sequence tag or the PRP sequence trailer override the port number.

3 **C.11.3 Per-port packet decoding and encoding**

4 The example in Figure C-15 also illustrates a second use of Autoconfiguration. In this network, everything
5 to the right of relay systems B and C, including the rightmost ports on relay systems B and C, use the PRP
6 sequence trailer (7.10), whereas all ports on relay systems A, B, and C to the left use the R-TAG (7.8).
7 Packets transmitted to the right by relay system B, and all packets transmitted by system D and the upper
8 port of system F are marked as being on LanId 0, and by other ports on the right side of the figure as being
9 on LanId 1. Packets transmitted by any other port on relay systems B and C, or by system A, use the
10 R-TAG. These distinctions are made using the Output autoconfiguration table (10.7.2). Using this table,
11 each port is assigned an output encapsulation, including the LanId or PathId for PRP or HSR, respectively.
12 Again, if the usage is too complex (e.g., one Stream is LanId 0 on a port, while another is LanId 1 on that
13 same port), Autoconfiguration cannot be used.

14 Note that the administrator could configure the Sequence autoconfiguration tables and Output
15 autoconfiguration tables in relay systems D and E to translate among the R-TAG, the HSR sequence tag, and
16 the PRP sequence trailer for the purpose of interworking, even if no Individual recovery functions or
17 Sequence recovery functions were desired in those relay systems.

18 **C.11.4 Individual and Sequence recovery functions**

19 Given the explanations in C.11.1, C.11.2, and C.11.3, the reader can consult 7.11.2 to understand how
20 Individual recovery functions and Sequence recovery functions are configured as a result of
21 Autoconfiguration. They are set up, along with the input (passive) Stream identification functions, by entries
22 in the Sequence autoconfiguration table (10.7.1). The reader may find the following facts helpful:

- 23 a) At most one instance of the Individual recovery function is instantiated for each stream_handle
24 associated with an autocreated tsnStreamIdEntry (9.1.1).
- 25 a) One instance of the Sequence recovery function is instantiated on each expected output port
26 (frerAutSeqRecoveryPortList, 10.7.1.1.5) for each Compound Stream.
- 27 b) Compound Streams are inferred by associating Member Streams with the same Source MAC
28 address and VLAN ID.

1 Annex D

2 (informative)

3 Mask-and-match Stream identification function

4 D.1 Example 1: Generic Stream identification for industrial communication networks

6 This example illustrates a way to identify a stream of an industrial communication network protocol,
7 described in IEC 61158-1:2019 [B1], whose frames are characterized by:

- 8 — being untagged
- 9 — the 12 LSBs of their destination MAC address having a value of 0xFAB
- 10 — carrying an EtherType of value 0x88B6
- 11 — having a payload starting with a 2-octet application-specific field of value 0xBABE

12 The tsnStreamIdParameters of the tsnStreamIdEntry (9.1.1) corresponding to the instance of the Mask-and-
13 match Stream identification function that identifies this particular Stream are shown in Table D-1

Table D-1—Managed objects values for example 1

Managed object	Value
tsnCpeMmIdDestMacMask	00-00-00-00-0F-FF
tsnCpeMmIdDestMacMatch	0F-AB
tsnCpeMmIdSrcMacMask	00-00-00-00-00-00
tsnCpeMmIdSrcMacMatch	N/A
tsnCpeMmIdMsduMask	FF-FF-FF-FF
tsnCpeMmIdMsduMaskLength	4
tsnCpeMmIdMsduMatch	88-B6-BA-BE
tsnPpMmIdMsduMaskMaxLength	8

14 D.2 Example 2: MPLS DetNet flow identification

15 IETF Deterministic Networking (DetNet) operates at the IP layer and delivers service over lower layer
16 technologies such as MPLS. DetNet accomplishes these goals by dedicating network resources such as link
17 bandwidth and buffer space to DetNet flows and/or classes of DetNet flows, and by replicating packets
18 along multiple paths.

19 MPLS encapsulated DetNet flows can be carried over network technologies that can provide DetNet the
20 required level of service, such as IEEE 802.1 TSN sub-networks.

21 In a TSN sub network TSN supporting an MPLS DetNet network (See section 6 in [B13]), the identification
22 of DetNet flows can be necessary. In such a case, one or more DetNet flows are supported by a TSN Stream.
23 Therefore, the identification of Streams supporting DetNet flows is needed.

1 This example illustrates a way to identify packets of a TSN Stream supporting an MPLS encapsulated
2 DetNet flow. Frames of such Stream are characterized by:

- 3 — their destination MAC address having a value of 02-00-B0-C1-19-71 (MAC address of the MPLS
4 next-hop node)
- 5 — being C-tagged (EtherType: 81-00), with VLAN-ID of 0x042
- 6 — carrying MPLS unicast payload (EtherType: 88-48)
- 7 — having a payload starting with a label stack comprising three labels, of which the 2 bottom ones have
8 the value of 0x0900D (F-label) and 0x0FACE (A-label). (For DetNet label definitions see section
9 4.2 in [B12].)

10 The tsnStreamIdParameters of the tsnStreamIdEntry (9.1.1) corresponding to the instance of the Mask-and-
11 match Stream identification function that identifies this particular Stream are shown in Table D-2

Table D-2—Managed objects values for example 2

Managed object	Value
tsnCpeMmIdDestMacMask	FF-FF-FF-FF-FF-FF
tsnCpeMmIdDestMacMatch	02-00-b0-c1-19-71
tsnCpeMmIdSrcMacMask	00-00-00-00-00-00
tsnCpeMmIdSrcMacMatch	N/A
tsnCpeMmIdMsduMask	FF-FF-0F-FF-FF-FF 00-80-00-00-00-8F FF-FF-00-8F-FF-FF
tsnCpeMmIdMsduMaskLength	18
tsnCpeMmIdMsduMatch	81-00-00-42-88-48 00-00-00-00-00-00 90-0D-00-80-FA-CE
tsnPpMmIdMsduMaskMaxLength	20

1 Annex E

2 (informative)

3 Bibliography

4 Bibliographical references are resources that provide additional or helpful material but do not need to be
5 understood or used to implement this standard. Reference to these resources is made for informational use
6 only.

7 [B1] IEC 61158-1:2019, Industrial communication networks—Fieldbus specifications—Part 1: Overview
8 and guidance for the IEC 61158 and IEC 61784 series.

9 [B2] IEEE Std 802.1BA™-2021, IEEE Standard for Local and Metropolitan Area Networks—Audio Video
10 Bridging (AVB) Systems.^{24, 25}

11 [B3] IEEE Std 802d-2017, IEEE Standard for Local and Metropolitan Area Networks: Overview and
12 Architecture—Allocation of Uniform Resource Name (URN) Values in IEEE 802® Standards.^{26, 27}

13 [B4] IEEE Std 802.3™, IEEE Standard for Ethernet.

14 [B5] IEEE Std 1722™-2016, IEEE Standard for a Transport Protocol for Time-Sensitive Applications in
15 Bridged Local Area Networks.

16 [B6] IETF RFC 3985, Bryant, S., Ed., and P. Pate, Ed., Pseudo Wire Emulation Edge-to-Edge (PWE3)
17 Architecture, DOI 10.17487/RFC3985, March 2005.²⁸

18 [B7] IETF RFC 6087, Guidelines for Authors and Reviewers of YANG Data Model Documents,
19 Bierman, A., Jan. 2011.²⁹

20 [B8] IETF RFC 6241, Network Configuration Protocol (NETCONF), Enns, R., M. Bjorklund,
21 J. Schoenwaelder, and A. Bierman, Jun. 2011.

22 [B9] IETF RFC 6242, Using the NETCONF Protocol over Secure Shell (SSH), Wasserman, M., Jun. 2011.

23 [B10] IETF RFC 6536, Network Configuration Protocol (NETCONF) Access Control Model, Bierman, A.,
24 and M. Bjorklund, Mar. 2012.

25 [B11] IETF RFC 8343, A YANG Data Model for Interface Management, Bjorklund, M., Mar. 2018.

26 [B12] IETF RFC 8964, B. Varga, Ed., J. Farkas, L. Berger, A. Malis, S. Bryant, J. Korhonen, Deterministic
27 Networking (DetNet) Data Plane: MPLS, January 2021.

²⁴The IEEE standards or products referred to in Annex E are trademarks owned by The Institute of Electrical and Electronics Engineers, Incorporated.

²⁵IEEE publications are available from The Institute of Electrical and Electronics Engineers (<http://standards.ieee.org>).

²⁶IEEE publications are available from the Institute of Electrical and Electronics Engineers (<https://standards.ieee.org/>).

²⁷The IEEE standards or products referred to in Annex E are trademarks of the Institute of Electrical and Electronics Engineers, Incorporated.

²⁸ETF documents (i.e., RFCs) are available for download at <http://www.rfc-archive.org/>.

²⁹ETF RFCs are available from the Internet Engineering Task Force (<https://www.rfc-archive.org/>).

- 1 [B13] [B12] IETF RFC 9037, B. Varga, Ed., J. Farkas, A. Malis, S. Bryant, Deterministic Networking
- 2 (DetNet) Data Plane: MPLS over IEEE 802.1 Time-Sensitive Networking (TSN), June 2021.
- 3 [B14] OMG Unified Modeling Language (OMG UML), Version 2.5, March 2015.³⁰
- 4 [B15] pyang, pyang GitHub repository, <https://github.com/mbj4668/pyang>
- 5 [B16] yanglint, libyang GitHub repository, <https://github.com/CESNET/libyang>
- 6 [B17] YANG Catalog, YANG catalog website, <https://yangcatalog.org/>
- 7 [B18] YANG Catalog, YANG Catalog GitHub repository, <https://github.com/YangModels/yang>

³⁰ OMG documents are available from the Object Management Group (<https://www.omg.org/>).